Unity Pro
Communication
Block Library

09/2017

33002527.19

www.schneider-electric.com

Schneider Electric

The information provided in this documentation contains general descriptions and/or technical characteristics of the performance of the products contained herein. This documentation is not intended as a substitute for and is not to be used for determining suitability or reliability of these products for specific user applications. It is the duty of any such user or integrator to perform the appropriate and complete risk analysis, evaluation and testing of the products with respect to the relevant specific application or use thereof. Neither Schneider Electric nor any of its affiliates or subsidiaries shall be responsible or liable for misuse of the information contained herein. If you have any suggestions for improvements or amendments or have found errors in this publication, please notify us.

You agree not to reproduce, other than for your own personal, noncommercial use, all or part of this document on any medium whatsoever without permission of Schneider Electric, given in writing. You also agree not to establish any hypertext links to this document or its content. Schneider Electric does not grant any right or license for the personal and noncommercial use of the document or its content, except for a non-exclusive license to consult it on an "as is" basis, at your own risk. All other rights are reserved.

All pertinent state, regional, and local safety regulations must be observed when installing and using this product. For reasons of safety and to help ensure compliance with documented system data, only the manufacturer should perform repairs to components.

When devices are used for applications with technical safety requirements, the relevant instructions must be followed.

Failure to use Schneider Electric software or approved software with our hardware products may result in injury, harm, or improper operating results.

Failure to observe this information can result in injury or equipment damage.

© 2017 Schneider Electric. All Rights Reserved.

Table of Contents

	Safety Information	11
	About the Book	15
Part I	General Information	17
Chapter 1	Block Types and their Applications	19
	Block Types	20
	FFB Structure	22
	EN and ENO	26
Chapter 2	Block Availability on the Various Hardware Platforms	29
	Block Availability on Various Hardware Platforms	29
Chapter 3	Operation of the Communication EFs	33
3.1	Structure and Management of Communication Functions	34
	Communication Functions Management	35
	Structure of Communication Functions	38
	Destination Address	39
	Structure of the Management Parameters	40
	Management Parameters: Communication and Operation Reports	43
	Management Parameters: Length and Timeout	46
3.2	General Information on Premium and Atrium Communication Functions	48
	Rules for Using the Communication Functions of Premium and Atrium PLCs	49
	Communication Functions on Premium and Atrium PLCs	50
	Server Function	52
3.3	General Information on Quantum Communication Functions	54
	Rules for Using the Communication Functions of Quantum PLCs	55
	Communication Functions on Quantum PLCs	57
3.4	General Information on M340 Communication Functions	60
	Communication Functions on M340 CPUs	60
3.5	General Information on M580 Communication Functions	62
	Rules for using the Communication Functions of M580 CPUs	63
	Communication Functions on M580 CPUs	65
3.6	General Information on Momentum Communication Functions	67
	Communication Functions on Momentum PLCs	67

Part II Extended 69
Chapter 4 ADDM: Address Conversion. 71
Description 71
Chapter 5 ADDMX: Address Conversion 77
Description 77
Chapter 6 ADDR: Address Conversion. 83
Description 83
Chapter 7 CANCEL: Stopping an Exchange in Progress 85
Description 86
Example of How to Cancel an Exchange. 88
Chapter 8 CREAD_REG: Continuous Register Reading 89
Description 90
Derived Data Types 93
Function Mode 95
Parameter Description 96
Chapter 9 CWRITE_REG: Continuous Register Writing. 97
Description 98
Derived Data Types 101
Function Mode 103
Parameter Description 104
Chapter 10 DATA_EXCH: Exchanging Data between Applications . . 105
Description 106
Assisted Entry Screen 111
Example of How to Use a Fipway Network 113
Example of How to Enable/Disable HTTP or FTP/TFTP Services. . . . 115
Chapter 11 ETH_PORT_CTRL: Activating or Deactivating a Protocol 119
ETH_PORT_CTRL: Executing a Security Command in an Application . 119
Chapter 12 EthPort_Control_MX: Activating or Deactivating a Protocol 123
EthPort_Control_MX: Deactivating or Activating a Protocol. 123
Chapter 13 EXCH_QX: Exchanging Data between Applications on EIO Bus 125
Description 125
Chapter 14 INPUT_BYTE: Receiving Character Strings. 131
Description 131

Chapter 15 INPUT_CHAR: Receiving Character Strings 137
Description 138
Assisted Entry Screen 143
Example of Reading Character Strings via Fipway Network 145
Example of Reading Character Strings via Serial Link of Modicon M340 Processors 147
Chapter 16 INPUT_CHAR_QX: Receiving Character Strings on EIO Bus 149
Description 149
Chapter 17 MBP_MSTR: Modbus Plus Master 153
Block Description 154
Operational Function Codes 157
Network Control Block Structures 158
Read Data 161
Write Data 163
Get Local Statistics 165
Clear Local Statistics 166
Write Global Data 167
Read Global Data 168
Get Remote Statistics 169
Clear Remote Statistics 171
Peer Cop Health 172
Reset Optional Module 173
Read CTE 174
Write CTE 176
Send Email 178
Send Modbus Request 180
Close Connection Request 185
Read/Write Data 186
Enable / Disable HTTP or FTP/TFTP Services 187
Peer Cop Communications Health Status 189
Modbus Plus Network Statistics 191
TCP/IP Ethernet Network Statistics 196
Chapter 18 ModbusP_ADDR: Modbus Plus Address 199
Description 200
Detailed Description 203

Chapter 19 OUT_IN_CHAR: Sending/Receiving Character Strings . . 205
Description . . . 206
Assisted Entry Screen . . . 209
Example of How to Send/Receive a Character String . . . 211
Chapter 20 OUT_IN_MBUS: Modbus Communication Function 213
20.1 General Presentation of the OUT_IN_MBUS Communication Block . . . 214
Function Description . . . 215
Sample Use . . . 216
Functionalities . . . 218
20.2 Description of the OUT_IN_MBUS Communication Block . . . 219
Representations and Parameters . . . 220
The MbusCmd Parameter . . . 223
The RetryLmt Parameter . . . 225
The DataBits Parameter . . . 226
The RespTout Parameter . . . 227
The MasterDataArea Parameter . . . 228
The Status Parameter . . . 229
20.3 Installing the OUT_IN_MBUS Communication Block . . . 230
Configuration of the Serial Link . . . 231
Procedure for Programming . . . 234
Using a Modem . . . 236
20.4 Sample Use of a OUT_IN_MBUS Communication Block . . . 238
Example Description . . . 239
Programming Structure . . . 240
Declaration of Variables . . . 242
Programming . . . 243
Chapter 21 PRINT_CHAR: Sending character strings 251
Description . . . 252
Assisted entry screen . . . 257
Example of Sending Character Strings via Fipway Network . . . 258
Example of Sending Character Strings via Serial Link of Modicon M340 Processors . . . 260
Chapter 22 PRINT_CHAR_QX: Sending Character Strings on EIO Bus . . . 263
Description . . . 263
Chapter 23 RCV_TLG: Receiving telegrams 267
Description . . . 268
Example of how to receive a telegram . . . 271

Chapter 24 READ_ASYN: Reading data asynchronously 273
Description 273
Chapter 25 READ_GDATA: Reading Modbus Plus Global Data . . . 277
Description 277
Chapter 26 READ_REG: Read Register 279
Description 280
Derived Data Types 283
Function mode 285
Parameter description 286
Chapter 27 READ_REG_QX: Read Register on EIO Bus 287
Description 287
Chapter 28 READ_SDO: Reading Service Data Object 291
READ_SDO: Reading Service Data Object 291
Chapter 29 READ_VAR: Reading variables 295
Description 296
Assisted entry screen 303
Example of use on a Uni-Telway bus 305
Example of Reading Bits 307
Example of use in a network 309
Example of Reading Words via Serial Link of Modicon M340 Processors 311
Example including execution check 313
Chapter 30 SEND_EMAIL: Sending Email 315
Send Email 315
Chapter 31 SEND_REQ: Sending requests 319
Description 320
List of UNI-TE Requests 324
Assisted entry screen 330
Example of how to send a UNI-TE request 332
Changing IP Parameters with SEND_REQ (Example) 334
Using the SEND_REQ function 335
Chapter 32 SEND_TLG: Sending telegrams 337
Description 338
Example of how to send a telegram 341
Chapter 33 SYMAX_IP_ADDR: SY/MAX IP Address 343
Description 344
Detailed Description 346

Chapter 34 TCP_IP_ADDR: TCP/IP Address 349
Description 350
Detailed Description 353
Chapter 35 UNITE_SERVER: Immediate server 355
Description 356
Example of immediate server 359
Chapter 36 WRITE_ASYN: Writing data asynchronously 361
Description 361
Chapter 37 WRITE_GDATA: Writing the Global Data Modbus Plus . . 365
Description 365
Chapter 38 WRITE_REG: Write Register 367
Description 368
Derived Data Types 371
Function mode 373
Parameter description 374
Chapter 39 WRITE_REG_QX: Write Register on EIO Bus 375
Description 375
Chapter 40 WRITE_SDO: Writing Service Data Object 379
WRITE_SDO: Writing Service Data Object 379
Chapter 41 WRITE_VAR: Writing variables 383
Description 384
Assisted entry screen 390
Example of how to write words on a network 392
Example of Writing Words via Serial Link of Modicon M340 PLCs . . . 394
Example including execution check 396
Chapter 42 XMIT: Transmit 399
Brief Description 400
Representation 401
Parameter Description 404
Description of Data Structure XMIT_SET 406
Description of Data Structure XMIT_CFG 407
Application Example 413
Chapter 43 XXMIT: Transmit 419
43.1 Introduction to XXMIT 420
XXMIT Functionality 420

43.2 XXMIT:Functions 421
Brief Description 422
Representation 423
Detailed Parameter Description 426
XXMIT Communication Functions 436
XXMIT ASCII Functions 437
XXMIT Modem Functions 442
XXMIT Modbus Functions 444
FIFO and Flow Control 450
Application Examples 453
43.3 XXMIT: Programming Rules 464
XXMIT Programming Rules 464
43.4 Technical References for XXMIT 466
Modbus Query/Response Parameter Limits 467
XXMIT Configuration using Hayes Compatible Dial-Up Modems (Only) 468
Hayes Application Example 472
43.5 Cabling Information 477
Cable Pinouts 478
Cable Adapter Kits 489
Appendices 493
Appendix A EFB Error Codes and Values 495
Tables of Error Codes for the Communication Library 496
Common Floating Point Errors 497
Error Codes of EFBs with STATUS parameter 498
Details of STATUS error code from 31ss to 37ss 501
Details of EFB TCP/IP Ethernet Error Codes 5mss 508
Details of EFB Modbus Plus Error Codes 6mss 512
Quantum EFB SY/MAX Specific Error Codes 513
EtherNet/IP Detected Error Codes 515
CANopen SDO Abort Code 518
Glossary 521
Index 525

Safety Information

Important Information

NOTICE

Read these instructions carefully, and look at the equipment to become familiar with the device before trying to install, operate, service, or maintain it. The following special messages may appear throughout this documentation or on the equipment to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.



The addition of this symbol to a “Danger” or “Warning” safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed.



This is the safety alert symbol. It is used to alert you to potential personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death.

⚠ DANGER

DANGER indicates a hazardous situation which, if not avoided, **will result in** death or serious injury.

⚠ WARNING

WARNING indicates a hazardous situation which, if not avoided, **could result in** death or serious injury.

⚠ CAUTION

CAUTION indicates a hazardous situation which, if not avoided, **could result** in minor or moderate injury.

NOTICE

NOTICE is used to address practices not related to physical injury.

PLEASE NOTE

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material.

A qualified person is one who has skills and knowledge related to the construction and operation of electrical equipment and its installation, and has received safety training to recognize and avoid the hazards involved.

BEFORE YOU BEGIN

Do not use this product on machinery lacking effective point-of-operation guarding. Lack of effective point-of-operation guarding on a machine can result in serious injury to the operator of that machine.

⚠ WARNING

UNGUARDED EQUIPMENT

- Do not use this software and related automation equipment on equipment which does not have point-of-operation protection.
- Do not reach into machinery during operation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

This automation equipment and related software is used to control a variety of industrial processes. The type or model of automation equipment suitable for each application will vary depending on factors such as the control function required, degree of protection required, production methods, unusual conditions, government regulations, etc. In some applications, more than one processor may be required, as when backup redundancy is needed.

Only you, the user, machine builder or system integrator can be aware of all the conditions and factors present during setup, operation, and maintenance of the machine and, therefore, can determine the automation equipment and the related safeties and interlocks which can be properly used. When selecting automation and control equipment and related software for a particular application, you should refer to the applicable local and national standards and regulations. The National Safety Council's Accident Prevention Manual (nationally recognized in the United States of America) also provides much useful information.

In some applications, such as packaging machinery, additional operator protection such as point-of-operation guarding must be provided. This is necessary if the operator's hands and other parts of the body are free to enter the pinch points or other hazardous areas and serious injury can occur. Software products alone cannot protect an operator from injury. For this reason the software cannot be substituted for or take the place of point-of-operation protection.

Ensure that appropriate safeties and mechanical/electrical interlocks related to point-of-operation protection have been installed and are operational before placing the equipment into service. All interlocks and safeties related to point-of-operation protection must be coordinated with the related automation equipment and software programming.

NOTE: Coordination of safeties and mechanical/electrical interlocks for point-of-operation protection is outside the scope of the Function Block Library, System User Guide, or other implementation referenced in this documentation.

START-UP AND TEST

Before using electrical control and automation equipment for regular operation after installation, the system should be given a start-up test by qualified personnel to verify correct operation of the equipment. It is important that arrangements for such a check be made and that enough time is allowed to perform complete and satisfactory testing.

⚠ WARNING

EQUIPMENT OPERATION HAZARD

- Verify that all installation and set up procedures have been completed.
- Before operational tests are performed, remove all blocks or other temporary holding means used for shipment from all component devices.
- Remove tools, meters, and debris from equipment.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Follow all start-up tests recommended in the equipment documentation. Store all equipment documentation for future references.

Software testing must be done in both simulated and real environments.

Verify that the completed system is free from all short circuits and temporary grounds that are not installed according to local regulations (according to the National Electrical Code in the U.S.A, for instance). If high-potential voltage testing is necessary, follow recommendations in equipment documentation to prevent accidental equipment damage.

Before energizing equipment:

- Remove tools, meters, and debris from equipment.
- Close the equipment enclosure door.
- Remove all temporary grounds from incoming power lines.
- Perform all start-up tests recommended by the manufacturer.

OPERATION AND ADJUSTMENTS

The following precautions are from the NEMA Standards Publication ICS 7.1-1995 (English version prevails):

- Regardless of the care exercised in the design and manufacture of equipment or in the selection and ratings of components, there are hazards that can be encountered if such equipment is improperly operated.
- It is sometimes possible to misadjust the equipment and thus produce unsatisfactory or unsafe operation. Always use the manufacturer's instructions as a guide for functional adjustments. Personnel who have access to these adjustments should be familiar with the equipment manufacturer's instructions and the machinery used with the electrical equipment.
- Only those operational adjustments actually required by the operator should be accessible to the operator. Access to other controls should be restricted to prevent unauthorized changes in operating characteristics.

About the Book

At a Glance

Document Scope

This document describes the functions and function blocks of the communication library.

Validity Note

This documentation is valid for Unity Pro 13.0 or later.

Related Documents

Title of Documentation	Reference Number
Modicon Modbus Plus Network, Planning and Installation Guide	31003525
Quantum with Unity Pro, TCP/IP Configuration, User Manual	33002467 (English), 33002468 (French), 33002469 (German), 31008078 (Italian), 33002470 (Spanish), 31007110 (Chinese)
Unity Pro, Program Languages and Structure, Reference Manual	35006144 (English), 35006145 (French), 35006146 (German), 35013361 (Italian), 35006147 (Spanish), 35013362 (Chinese)
Unity Pro, System Bits and Words, Reference Manual	EIO0000002135 (English), EIO0000002136 (French), EIO0000002137 (German), EIO0000002138 (Italian), EIO0000002139 (Spanish), EIO0000002140 (Chinese)

You can download these technical publications and other technical information from our website at http://www.schneider-electric.com/en/download

Part I
General Information

Overview

This section contains general information about the communication library.

NOTE: For a detailed description of system objects (%S and %SW), refer to *Unity Pro, System Bits and Words, Reference Manual*.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
1	Block Types and their Applications	19
2	Block Availability on the Various Hardware Platforms	29
3	Operation of the Communication EFs	33

Chapter 1
Block Types and their Applications

Overview

This chapter describes the different block types and their applications.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Block Types	20
FFB Structure	22
EN and ENO	26

Block Types

Block Types

Different block types are used in Unity Pro. The general term for the block types is FFB.

There are the following types of block:

- Elementary Function (EF)
- Elementary Function Block (EFB)
- Derived Function Block (DFB)
- Procedure

NOTE: Motion Function Blocks are not available on the Quantum platform.

Elementary Function

Elementary functions (EF) have no internal status and one output only. If the input values are the same, the output value is the same for the executions of the function, for example the addition of two values gives the same result at every execution.

An elementary function is represented in the graphical languages (FBD and LD) as a block frame with inputs and an output. The inputs are represented on the left and the outputs on the right of the block frame. The name of the function, that is the function type, is shown in the center of the block frame.

The number of inputs can be increased with some elementary functions.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF EQUIPMENT

For Unity Pro V4.0 and earlier versions, do not use links to connect function blocks outputs, when your application relies on persistent output data of an EF.

Failure to follow these instructions can result in injury or equipment damage.

NOTE: With Unity Pro V4.0 and earlier versions the deactivation of an EF (EN=0) causes links connected to its Input/Output to be reset. To transfer the state of the signal do not use a link. A variable must be connected to the EF's output and must be used to connect the input of the element. With Unity Pro V4.1 and later versions you can maintain the output links even if an EF is deactivated by activating the option **Maintain output links on disabled EF (EN=0)** via the menu **Tools → Program → Languages → Common**.

Elementary Function Block

Elementary function blocks (EFB) have an internal status. If the inputs have the same values, the value on the outputs can have another value during the individual executions. For example, with a counter, the value on the output is incremented.

An elementary function block is represented in the graphical languages (FBD and LD) as a block frame with inputs and outputs. The inputs are represented on the left and the outputs on the right of the block frame. The name of the function block, that is the function block type, is shown in the center of the block frame. The instance name is displayed above the block frame.

Derived Function Block

Derived function blocks (DFBs) have the same properties as elementary function blocks. They are created by the user in the programming languages FBD, LD, IL and/or ST.

Procedure

Procedures are functions with several outputs. They have no internal state.

The only difference from elementary functions is that procedures can have more than one output and they support variables of the `VAR_IN_OUT` data type.

Procedures do not return a value.

Procedures are a supplement to IEC 61131-3 and must be enabled explicitly.

There is no visual difference between procedures and elementary functions.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF EQUIPMENT

For Unity Pro V4.0 and earlier versions, do not use links to connect function blocks outputs, when your application relies on persistent output data of an EF.

Failure to follow these instructions can result in injury or equipment damage.

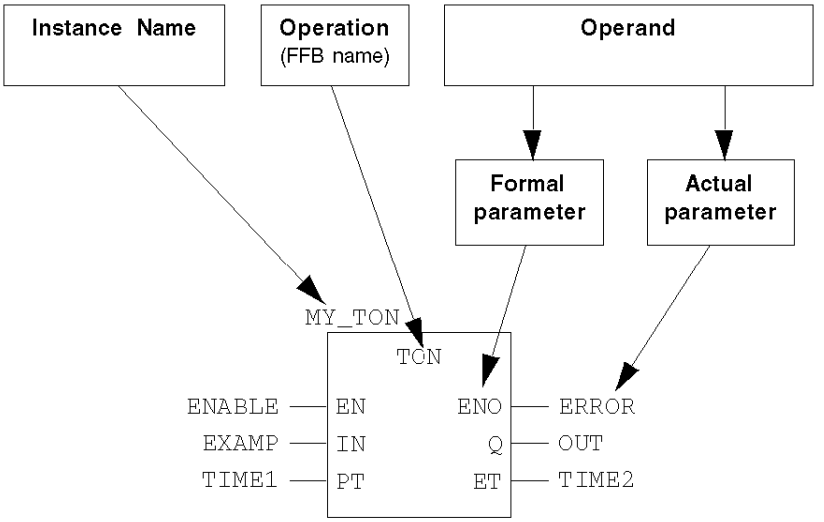
NOTE: With Unity Pro V4.0 and earlier versions the deactivation of an EF (EN=0) causes links connected to its Input/Output to be reset. To transfer the state of the signal do not use a link. A variable must be connected to the EF's output and must be used to connect the input of the element. With Unity Pro V4.1 and later versions you can maintain the output links even if an EF is deactivated by activating the option **Maintain output links on disabled EF (EN=0)** via the menu **Tools → Program → Languages → Common**.

FFB Structure

Structure

Each FFB is made up of an operation (name of the FFB), the operands are required for the operation (formal and actual parameters) and an instance name for elementary/derived function blocks.

Call of a function block in the FBD programming language:



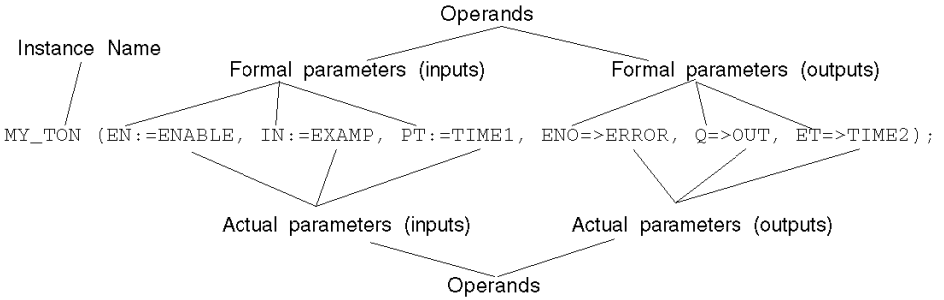
⚠ CAUTION

UNEXPECTED APPLICATION BEHAVIOR

Do not call several times the same block instance within a PLC cycle

Failure to follow these instructions can result in injury or equipment damage.

Formal call of a function block in the ST programming language:



Operation

The operation determines which function is to be executed with the FFB, e.g. shift register, conversion operations.

Operand

The operand specifies what the operation is to be executed with. With FFBs, this consists of formal and actual parameters.

Formal/actual parameters

Inputs and outputs are required to transfer values to or from an FFB. These are called formal parameters.

Objects are linked to formal parameters; these objects contain the current process states. They are called actual parameters.

At program runtime, the values from the process are transferred to the FFB via the actual parameters and then output again after processing.

The data type of the actual parameters must match the data type of the input/output (formal parameters). The only exceptions are generic inputs/outputs whose data type is determined by the actual parameter. If the actual parameters consist of literals, a suitable data type is selected for the function block.

FFB Call in IL/ST

In text languages IL and ST, FFBs can be called in formal and in informal form. For detail, refer to chapter *Programming Language (see Unity Pro, Program Languages and Structure, Reference Manual)*.

Example of a formal function call:

```
out:=LIMIT (MN:=0, IN:=var1, MX:=5);
```

Example of an informal function call:

```
out:=LIMIT (0, var1, 5);
```

NOTE: The use of `EN` and `ENO` is only possible for formal calls.

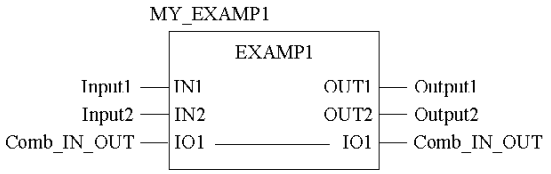
`VAR_IN_OUT` variable

FFBs are often used to read a variable at an input (input variables), to process it and to output the altered values of the **same** variable (output variables).

This special type of input/output variable is also called a `VAR_IN_OUT` variable.

The input and output variable are linked in the graphic languages (FBD and LD) using a line showing that they belong together.

Function block with `VAR_IN_OUT` variable in FBD:



Function block with `VAR_IN_OUT` variable in ST:

```
MY_EXAMP1 (IN1:=Input1, IN2:=Input2, IO1:=Comb_IN_OUT,
           OUT1=>Output1, OUT2=>Output2);
```

The following points must be considered when using FFBs with `VAR_IN_OUT` variables:

- The `VAR_IN_OUT` inputs must be assigned a variable.
- Literals or constants cannot be assigned to `VAR_IN_OUT` inputs/outputs.

The following additional limitations apply to the graphic languages (FBD and LD):

- When using graphic connections, VAR_IN_OUT outputs can only be connected with VAR_IN_OUT inputs.
- Only one graphical link can be connected to a VAR_IN_OUT input/output.
- Different variables/variable components can be connected to the VAR_IN_OUT input and the VAR_IN_OUT output. In this case the value of the variables/variable component on the input is copied to the output variables/variable component.
- No negations can be used on VAR_IN_OUT inputs/outputs.
- A combination of variable/address and graphic connections is not possible for VAR_IN_OUT outputs.

EN and ENO

Description

An EN input and an ENO output can be configured for the FFBs.

If the value of EN is equal to "0" when the FFB is invoked, the algorithms defined by the FFB are not executed and ENO is set to "0".

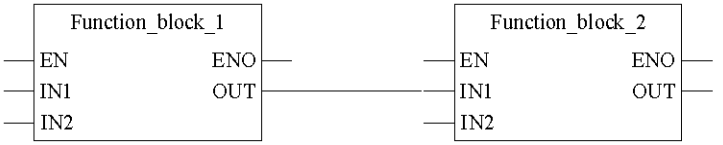
If the value of EN is equal to "1" when the FFB is invoked, the algorithms defined by the FFB will be executed. After the algorithms have been executed successfully, the value of ENO is set to "1". If certain error conditions are detected when executing these algorithms, ENO is set to "0".

If the EN pin is not assigned a value, when the FFB is invoked, the algorithm defined by the FFB is executed (same as if EN equals to "1"), Please refer to *Maintain output links on disabled EF (see Unity Pro, Operating Modes)*.

If the algorithms are executed successfully, then value of ENO is set to "1", else ENO is set to "0".

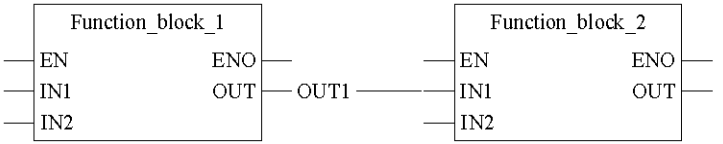
If ENO is set to "0" (caused by EN=0 or a detected error condition during execution or unsuccessful algorithm execution):

- Function blocks
  - EN/ENO handling with function blocks that (only) have one link as an output parameter:



If EN from FunctionBlock_1 is set to "0", the output connection OUT from Function-Block_1 retains the status it had in the last correctly executed cycle.

  - EN/ENO handling with function blocks that have one variable and one link as output parameters:



If EN from FunctionBlock_1 is set to "0", the output connection OUT from Function-Block_1 retains the status it had in the last correctly executed cycle. The variable OUT1 on the same pin, either retains its previous status or can be changed externally without influencing the connection. The variable and the link are saved independently of each other.

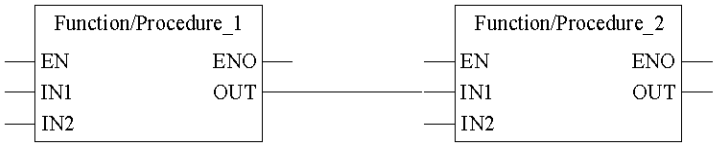
- Functions/Procedures

⚠ CAUTION
UNEXPECTED BEHAVIOR OF EQUIPMENT For Unity Pro V4.0 and earlier versions, do not use links to connect function blocks outputs, when your application relies on persistent output data of an EF. **Failure to follow these instructions can result in injury or equipment damage.**

NOTE: With Unity Pro V4.0 and earlier versions the deactivation of an EF (EN=0) causes links connected to its Input/Output to be reset. To transfer the state of the signal do not use a link. A variable must be connected to the EF's output and must be used to connect the input of the element. With Unity Pro V4.1 and later versions you can maintain the output links even if an EF is deactivated by activating the option **Maintain output links on disabled EF (EN=0)** via the menu **Tools → Program → Languages → Common**.

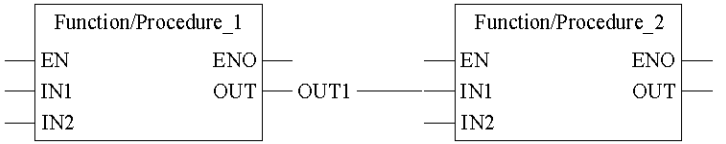
As defined in IEC61131-3, the outputs from deactivated functions (`EN`-input set to "0") is undefined. (The same applies to procedures.)
Here is an explanation of the output status in this case:

- EN/ENO handling with functions/procedures that (only) have one link as an output parameter:



If `EN` from `Function/Procedure_1` is set to "0", the output connection `OUT` from `Function/Procedure_1` is also set to "0".

- EN/ENO handling with function blocks that have one variable and one link as output parameters:



If `EN` from `Function/Procedure_1` is set to "0", the output connection `OUT` from `Function/Procedure_1` is also set to "0". The variable `OUT1` on the same pin, either retains its previous status or can be changed externally without influencing the connection. The variable and the link are saved independently of each other.

The output behavior of the FFBs does not depend on whether the FFBs are called up without `EN`/`ENO` or with `EN`=1.

Conditional/Unconditional FFB Call

"Unconditional" or "conditional" calls are possible with each FFB. The condition is realized by pre-linking the input EN.

- EN connected
  conditional calls (the FFB is only processed if EN = 1)
- EN shown, hidden, and marked TRUE, or shown and not occupied
  unconditional calls (FFB is processed independent from EN)

NOTE: For disabled function blocks (EN = 0) with an internal time function (e.g. DELAY), time seems to keep running, since it is calculated with the help of a system clock and is therefore independent of the program cycle and the release of the block.

⚠ CAUTION
UNEXPECTED APPLICATION EQUIPMENT
Do not disable function blocks with internal time function during their operation.
Failure to follow these instructions can result in injury or equipment damage.

Note for IL and ST

The use of EN and ENO is only possible in the text languages for a formal FFB call, e.g.

```
MY_BLOCK (EN:=enable, IN1:=var1, IN2:=var2,
ENO=>error, OUT1=>result1, OUT2=>result2);
```

Assigning the variables to ENO must be done with the operator =>.

With an informal call, EN and ENO cannot be used.

Chapter 2
Block Availability on the Various Hardware Platforms

Block Availability on Various Hardware Platforms

Introduction

Not all blocks are available on all hardware platforms. Block availability by hardware platform can be found in the following table.

NOTE: The functions, procedures, and function blocks in this library are not defined in IEC 61131–3.

NOTE: It is not recommended to use the communication functions in FAST task. To avoid dysfunctions, the best solution is to use them in MAST task.

Extended

Availability of the blocks:

Block name	Block type	Premium	Quantum	M340	M580	Momentum
ADDM	EF	-	-	+	+	-
ADDMX	EF	-	+(1)	-	+	-
ADDR	EF	+	-	-	-	-
CANCEL	Procedure	+	-	+	+	-
CREAD_REG	EFB	-	+	-	-	-
CWRITE_REG	EFB	-	+	-	-	-
DATA_EXCH	Procedure	+	-	+	+	-
ETH_PORT_CTRL	EFB	+	-	+	+	-
EthPort_Control_MX	EF	-	-	-	+	-
EXCH_QX	EFB	-	+(1)	-	-	-

+ Yes
- No
(1) You can only use these EFBs to go through the following Quantum communication modules: 140 CRP 312 00, 140 NOC 780 00, 140 NOC 781 00. If used with other communication modules, they remain in active state until the `Abort` parameter is set.
(2) Supported operation codes: 1 = Write data, 2 = Read data, 23 = Read / Write data.
(3) Momentum supports the same implementation as Quantum. **NOTE:** The supported implementation presents a different bit enumeration than Concept/ProWORX.
(4) Supported operation codes: 1 = Write data, 2 = Read data, 14 = CIP Explicit Message, 15 = Send Modbus Request, 23 = Read / Write data.

Block name	Block type	Premium	Quantum	M340	M580	Momentum
INPUT_BYTE	Procedure	+	-	+	+	-
INPUT_CHAR	Procedure	+	-	+	+	-
INPUT_CHAR_QX	EFB	-	+(1)	-	-	-
MBP_MSTR	EFB	-	+	-	+(4)	+(2)
MODBUSP_ADDR	EFB	-	+	-	-	-
OUT_IN_CHAR	Procedure	+	-	-	-	-
OUT_IN_MBUS	DFB	+	-	-	-	-
PRINT_CHAR	Procedure	+	-	+	+	-
PRINT_CHAR_QX	EFB	-	+(1)	-	-	-
RCV_TLG	Procedure	+	-	-	-	-
READ_ASYN	Procedure	+	-	-	-	-
READ_GDATA	Procedure	+	-	-	-	-
READ_REG	EFB	-	+	-	-	-
READ_REG_QX	EFB	-	+(1)	-	-	-
READ_VAR	Procedure	+	-	+	+	-
SEND_REQ	Procedure	+	-	-	-	-
SEND_EMAIL	Procedure	-	-	+	-	-
SEND_TLG	Procedure	+	-	-	-	-
SYMAX_IP_ADDR	EFB	-	+	-	-	-
TCP_IP_ADDR	EFB	-	+	-	-	-
UNITE_SERVER	Procedure	+	-	-	-	-
WRITE_ASYN	Procedure	+	-	-	-	-
WRITE_GDATA	Procedure	+	-	-	-	-
WRITE_REG	EFB	-	+	-	-	-
WRITE_REG_QX	EFB	-	+(1)	-	-	-
WRITE_VAR	Procedure	+	-	+	+	-

+ Yes
- No
(1) You can only use these EFBs to go through the following Quantum communication modules: 140 CRP 312 00, 140 NOC 780 00, 140 NOC 781 00. If used with other communication modules, they remain in active state until the `Abort` parameter is set.
(2) Supported operation codes: 1 = Write data, 2 = Read data, 23 = Read / Write data.
(3) Momentum supports the same implementation as Quantum. **NOTE:** The supported implementation presents a different bit enumeration than Concept/ProWORX.
(4) Supported operation codes: 1 = Write data, 2 = Read data, 14 = CIP Explicit Message, 15 = Send Modbus Request, 23 = Read / Write data.

Block name	Block type	Premium	Quantum	M340	M580	Momentum
XMIT	EFB	-	-	-	-	+
XXMIT	EFB	-	+	-	-	+(3)

+ Yes
- No
(1) You can only use these EFBs to go through the following Quantum communication modules: 140 CRP 312 00, 140 NOC 780 00, 140 NOC 781 00. If used with other communication modules, they remain in active state until the Abort parameter is set.
(2) Supported operation codes: 1 = Write data, 2 = Read data, 23 = Read / Write data.
(3) Momentum supports the same implementation as Quantum. **NOTE:** The supported implementation presents a different bit enumeration than Concept/ProWORX.
(4) Supported operation codes: 1 = Write data, 2 = Read data, 14 = CIP Explicit Message, 15 = Send Modbus Request, 23 = Read / Write data.

Chapter 3
Operation of the Communication EFs

Overview

This chapter describes the operation and management of the communication functions.

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
3.1	Structure and Management of Communication Functions	34
3.2	General Information on Premium and Atrium Communication Functions	48
3.3	General Information on Quantum Communication Functions	54
3.4	General Information on M340 Communication Functions	60
3.5	General Information on M580 Communication Functions	62
3.6	General Information on Momentum Communication Functions	67

Section 3.1
Structure and Management of Communication Functions

Overview

This section describes the management of the two main communication function types. It also provides general information about the structure of communication functions.

What Is in This Section?

This section contains the following topics:

Topic	Page
Communication Functions Management	35
Structure of Communication Functions	38
Destination Address	39
Structure of the Management Parameters	40
Management Parameters: Communication and Operation Reports	43
Management Parameters: Length and Timeout	46

Communication Functions Management

At a Glance

The communication functions allow one device to communicate with another device.

Some of these functions are common to several types of communication channels, others may be specific to a single communication channel.

Communication functions are processed asynchronously regarding the application task that is used to activate them (except SEND_TLG and RCV_TLG). A communication function is asynchronous when it is executed during one or several cycles after the cycle in which it has been activated.

Several communication function blocks can be programmed on the same communication port. If the maximum number of communication blocks is exceeded on a port, the exceeding blocks are not serviced until a transaction path is released. When the transaction path resource becomes free, the next block on the same port becomes active and uses the released path.

The purpose of this topic is to describe the management of the communication functions according to their type:

- EF and procedure type functions
- EFB type functions

EF and Procedure Type Communication Functions

The EF and procedure type communication functions are managed as follows:

Launching a communication: The function is called inside the task. Calling the function launches the communication if EN input is set to 1.

Monitoring a communication: The CPU operating system handles the communication autonomously. It is processed asynchronously regarding the application task that activated the function. It is not required to execute the function again once it has been activated. User monitors the communication process through management parameters updated by the operating system.

While the communication is ongoing, the activity bit *(see page 40)* parameter is set to 1. Once the communication is completed (with or without detected error), the activity bit parameter is set to 0. As the communication may require multiple task cycles to be completed, testing the activity bit parameter allows to know when the communication has ended.

For a communication requiring multiple task cycles to be completed: to execute the function once, reset EN input to 0 after the communication function launch and before next task cycle.

To maintain a continuous communication activity, let the EN input set to 1.

Launching a new communication: Two situations are faced:

- The function is called once when a communication is required (edge triggering): call the function with EN input set to 1 to launch the communication, and reset EN (0) before next task cycle.
- The function is cyclically called in a task: if the activity bit parameter is set to 0 (previous communication has ended, tested by the function itself when it is called) the function internal process launches a new communication. This situation leads to a continuous communication.
  To avoid launching a communication in the next task cycle, reset EN input to 0.

EFB Type Communication Functions

The EFB type communication functions are managed as follows:

Launching a communication: The function is called inside the task, but depends on ENABLE input state. To launch the communication, the ENABLE and EN inputs must be set to 1 when the function is called.

Monitoring a communication: The function is processed during the task. If the communication requires many task cycles to be completed, the function must be called and enabled in the task at each cycle.

While the communication is ongoing, ACTIVE output is set to 1. Once the communication is completed (with or without detected error), the ACTIVE output is set to 0.

For a communication requiring multiple task cycles to be completed:

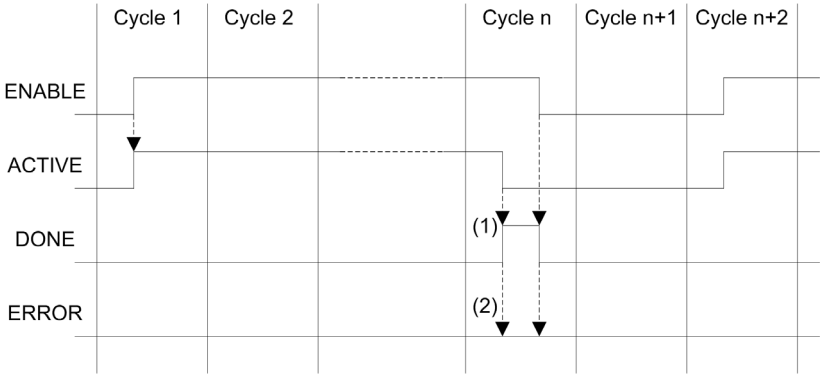
- Once the communication function has been launched, the ENABLE and EN inputs need to be set to 1 during the ongoing communication process. The ACTIVE output value can be used as a condition to manage the ENABLE input of the function. An example is provided in the section on M580 communication functions description *(see page 64)*.
- If the ENABLE or EN input is set to 0 during a task, the function activity is disabled and frozen. The function activity resumes in a following cycle if the ENABLE and EN input are set to 1.

Launching a new communication: The function is cyclically called in a task for monitoring purposes.

When the function is called, the activity is tested by the function itself. If the function is not active (previous communication has ended), the ENABLE and EN inputs must be set to 1 to launch a new communication.

To launch the communication function only once, the ENABLE input needs to be set to 0 when the ACTIVE output is read at 0.

The following figure shows in detail the role and operation of ENABLE, ACTIVE, DONE (or SUCCESS) and ERROR parameters:



1 DONE = 1 if no detected error, DONE = 0 if detected error

2 ERROR = 0 if no detected error, ERROR = 1 if detected error

The ENABLE parameter is written by the application. The ACTIVE, DONE, and ERROR parameters are read by the application.

Structure of Communication Functions

At a Glance

A communication function uses:

- an address parameter
- parameters specific to a communication operation
- management parameters

Syntax

The syntax of a communication function is as follows:

```
Function (Destination address, Specific parameters, Management
parameters)
```

The following table describes the different entities that make up a function:

Entity	Description
Function	Corresponds to the type of communication function.
Destination address	Indicates the address of the exchange destination.
Specific parameters	Depend on the type of communication function. A description of these is provided for each communication function.
Management parameters	Management parameters are common to all asynchronous communication functions. These are made up of: ● a parameter providing data on the activity of the function ● a parameter specifying the exchange number that identifies the transaction in progress ● a parameter containing the exchange report (communication report and operation report) ● a timeout parameter that can be used to check if there is no response ● a length parameter that can be used to store the number of bytes to be sent or the number of bytes received

Destination Address

At a Glance

This parameter indicates the exchange's destination device address *(see Communication Services and Architectures, Reference Manual)*.

It can be located:

- either using internal words (%MW) or internal constants (%KW)
- or by being written directly as an immediate value

In order to facilitate the preparation phase of the exchange, there is the `ADDR` function (only Unity Pro M or higher), which converts an address-type immediate value (character string) into a table which is always comprised of six internal words (%MW).

Example

```
%MWi:6:=ADDR('{2.4}SYS');
```

Structure of the Management Parameters

At a Glance

Management parameters are grouped together in the form of an array of four integers. The values contained in this array can be used to manage communication functions.

NOTE: In technical documentation, these management parameters are also called a management table or report.

NOTE: The first two words are managed by the system. You are responsible for managing the last two words.

NOTE:
For M580 Hot Standby systems, make the following edits to the management parameters in the **Data Editor**:

- Uncheck the **Exchange On STBY** attribute for each of the four management parameters.
- Initialize the Length parameter each time the function is called.
- Set the Timeout parameter accordingly to your application:
  - If the communication function is send through the CPU, the typical timeout value is 500 ms.
  - If the communication function is send through a NOC module, the typical timeout value is 2 s.

Structure

The following table describes the structure of the data in the communication management table:

	Order of the Word	Most Significant Byte	Least Significant Byte
Data managed by the system	1	Exchange number	Activity bit *(see page 40)*: rank 0 Cancel bit *(see page 41)*: rank 1 Immediate acknowledge bit *(see page 41)*: rank 2
	2	operation report *(see page 44)*	communication report *(see page 43)*
Data managed by the user	3	Timeout *(see page 46)*	
	4	Length *(see page 46)*	

Activity Bit

This bit indicates the execution status of the communication function.

It is set to 1 when launched and returns to 0 when its execution is complete.

This is the first bit of the first element of the table.

Declaration: If the management table has been declared as follows:

`Tab_Gest ARRAY [1..4] OF INT`, the activity bit is the bit with the notation `Tab_Gest[1].0`.

NOTE: the notation previously used requires configuration of the project properties in such a way as to authorize the extraction of bits on integer types. If this is not the case `Tab_Gest[1].0` cannot be accessed in this manner.

Cancel Bit

To cancel the communication function (for Modicon M340 and Modicon M580 PLCs only) set this bit to 1 and relaunch the EF. This is another way than using the CANCEL *(see page 86)* EF.

Declaration: If the management table has been declared as follows:

Tab_Gest ARRAY [1..4] OF INT, the cancel bit is the bit with the notation Tab_Gest[1].1.

Programming example in ST: cancelling a READ_VAR function

```
IF (%MW40.0) THEN
 SET(%MW40.1);
 READ_VAR(ADDM('0.0.0.6'), '%MW', 100, 10, %MW40:4, %MW10:10);
END_IF;
```

%MW40 is the GEST parameter (management table).%MW40.0 corresponds to the activity bit of the READ_VAR function and is set to 1 when the communication function is active. If this bit is set to 1, the program sets the %MW40.1 bit, the function cancel bit, to 1. This stops communication of the READ_VAR function.

NOTE: When using the communication function cancel bit contained in the function exchange management word (%MW40 in this example), the function (READ_VAR in this example) must be called in order to activate the cancellation of the exchange.

NOTE: When using the communication function cancel bit, it is possible to cancel a communication from an animation table. This can be done by simply setting the function cancel bit to 1 (%MW40.1 in this example) and then start again the communication function.

Immediate Acknowledge Bit

Used only for the INPUT_CHAR *(see page 138)* EF, this bit forces an immediate execution of the EF. This feature is useful when working with NOM modules on remote drops (the connection is automatically closed by the CRP module if no answer of the CRA module arrives in the two minutes). The function INPUT_CHAR can indeed remain infinitely active in the NOM module waiting for characters coming on the serial connection. When this feature is activated, the NOM module immediately answers to the input character request even if no character is available (of course with receive characters length set to 0).

Declaration: If the management table has been declared as follows:

Tab_Gest ARRAY [1..4] OF INT, the immediate acknowledge bit is the bit with the notation Tab_Gest[1].2.

Exchange Number

When a communication function is sent, the system automatically allocates it a number, enabling the exchange to be identified.

This number can be used where necessary to stop the exchange in progress (using the `CANCEL` *(see page 86)* function).

Management Parameters: Communication and Operation Reports

At a Glance

Communication and operation reports are part of the management parameters.

NOTE: It is recommended that communication function reports always be tested at the end of their execution and before the next activation. On cold start-up, it is imperative that all communication function management parameters be checked and reset to 0.

Communication Report

This report is common to all functions. It is significant when the value of the activity bit switches from 1 to 0.

The reports with a value between `16#01` and `16#FE` concern errors detected by the processor that executed the function.

The different values of this report are indicated in the following table:

Value	Communication Report (Least Significant Byte)
16#00	Correct exchange
16#01	Exchange stop on timeout **NOTE:** OK value returned when a Modicon M340 CPU sends a `MODBUS BROADCAST` request.
16#02	Exchange stop on user request (`CANCEL`)
16#03	Incorrect address format
16#04	Incorrect destination address
16#05	Incorrect management parameter format
16#06	Incorrect specific parameters
16#07	Problem in sending to the destination
16#08	Reserved
16#09	Insufficient receive buffer size
16#0A	Insufficient send buffer size
16#0B	No system resources: the number of simultaneous communication EFs exceeds the maximum that can be managed by the processor.
16#0C	Incorrect exchange number
16#0D	No telegram received
16#0E	Incorrect length
16#0F	Telegram service not configured
16#10	Network module missing
16#11	Request missing
16#12	Application server already active

Value	Communication Report (Least Significant Byte)
16#13	UNI-TE V2 transaction number incorrect
16#FF	Message refused **NOTE:** OK value returned when a TSX SCP ••• or TSX SCY ••• sends a `MODBUS BROADCAST` request.

NOTE: The function can detect a parameter error before activating the exchange. In this case the activity bit remains at 0, and the report is initialized with values corresponding to the error.

NOTE: 16#FF value is returned to indicate a correct exchange using `WRITE_VAR` function in a Modbus broadcast request. This report value is implemented in TSX SCY 21601 from V2.8 IE46, in TSX SCY 11601 from V1.2 IE11 and in TSX SCP 111/114 from V3.2 IR25.

Operation Report

This report byte is specific to each function, and specifies the result of the operation on the remote application.

It is significant only if the communication report has the following values:

- `16#00` (correct exchange),
- `16#FF` (message refused).

If the value of the communication report is 16#00, the operation report will have the following values:

Value	Operation Report (Most Significant Byte)
16#00	Positive result
16#01	Request not processed
16#02	Incorrect response
16#03	Reserved **NOTE:** For Premium this report signals a bad reception buffer size (the reception buffer is too small to contain the response).
Request code + 16#30	Upon positive reply for certain requests
16#FB	Upon reply to minor request
16#FD	Operational error
16#FE	Upon positive reply for certain request

This list is not exhaustive and depends on the communication function used. Refer to the corresponding communication function for specific values:

- Operation report when using SEND_REQ to exchange an UNI-TE request *(see page 324)*.
- Operation report when using SEND_REQ to exchange a Modbus request *(see Premium and Atrium Using Unity Pro, Asynchronous Serial Link, User Manual)*.

If the value of the communication report is 16#FF, the operation report will have the following values:

Value	Operation Report (Most Significant Byte)
16#01	No resources towards the processor
16#02	No line resources
16#03	No device or device without resources **(*)**
16#04	Line error
16#05	Length error
16#06	Faulty communication channel
16#07	Addressing error
16#08	Application error
16#0B	No system resources: the number of simultaneous communication EFs exceeds the maximum that can be managed by the processor.
16#0C	Communication function not active
16#0D	Destination missing
16#0F	Intra-station routing problem or channel not configured
16#11	Address format not managed
16#12	No destination resources
16#14	Non-operational connection (example: Ethernet TCP/IP)
16#15	No resource on the local channel
16#16	Access not authorized (example: Ethernet TCP/IP)
16#17	Inconsistent network configuration (example: Ethernet TCP/IP)
16#18	Connection temporarily unavailable
16#21	Application server stopped
16#30	Transmission error
Legend:	
(*)	Code only managed by PCMCIA cards: TSX FPP20 and TSX FPP10

Management Parameters: Length and Timeout

At a Glance

You are responsible for these two parameters.

Length

The length parameter is used both to specify the number of characters (in bytes) to be sent during transmission, but also to store the number of characters (in bytes) received after reception of a message.

Before certain communication functions are launched (`SEND_REQ`, `DATA_EXCH`, `PRINT_CHAR`, `SEND_TLG`), it is compulsory for some of these functions, and advisable for others, to update the length parameter.

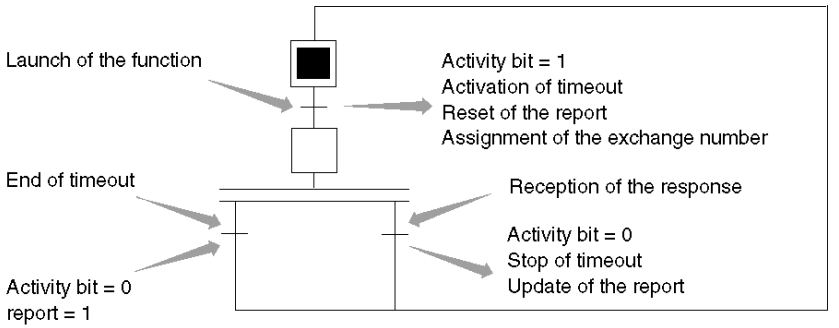
NOTE: With the `PRINT_CHAR` function, for example, if another function in the application is using the same report table where the number of bytes to send is different to the previous function, it is imperative to initialize the length parameter with the new number of bytes to transmit. Otherwise it keeps the same number of bytes sent by the previous function.

Timeout

Timeout determines the maximum waiting time for the response. The time base for this parameter is 100 ms (the value 0 corresponds to an infinite waiting value).

When the timeout elapses, the exchange is ended with an error report. Also, the system does not accept any responses after the end of the timeout.

Example



NOTE: The value of a communication function's timeout must be sufficient to ensure that the response to the question asked is received (use of an external modem on a protocol-based link, for example).

NOTE: For Modbus master communication, application timeout set under the communication functions must be greater than the configuration screen timeout multiplied by the number of retries (hardware timeout).

NOTE: Communication EFs timeout (as `WRITE_VAR` or `READ_VAR`) must be greater than communication master equipment timeout (answer delay).

Section 3.2
General Information on Premium and Atrium Communication Functions

Overview

This section describes the operation and management of the communication functions for Premium and Atrium PLCs.

What Is in This Section?

This section contains the following topics:

Topic	Page
Rules for Using the Communication Functions of Premium and Atrium PLCs	49
Communication Functions on Premium and Atrium PLCs	50
Server Function	52

Rules for Using the Communication Functions of Premium and Atrium PLCs

At a Glance

The communication functions used with Premium and Atrium PLCs have certain unique characteristics which set them apart from the other functions of the library. This documentation complies with the documentation charter concerning the function library, but also contains additional information relating to the particularities of the communication specific-application.

Located Variables

All communication functions that do not require programming in the server PLC (READ_VAR, WRITE_VAR, etc.) provide access to the located variables of remote PLCs. Unlocated variables are not accessible.

NOTE: To perform inter-PLC transfers of unlocated variables, it is necessary to use the DATA_EXCH function. Another solution is to make local copies in the located variable zones.

Apart from the first example of the `WRITE_VAR` function (only Unity Pro M and higher), all of the other examples are made using direct addressing *(see Unity Pro, Program Languages and Structure, Reference Manual)* variables (use of addresses, located variables).

Programming Language

The most concise programming language for creating communication applications is Structured Text language (ST). All of the examples, except that of the `READ_VAR` *(see page 295)* function, are therefore written in ST.

Communication Functions on Premium and Atrium PLCs

At a Glance

These functions allow one device to communicate with another. Certain of these are common to several types of communication channels, others may be specific to a single communication channel.

NOTE: Communication functions are processed asynchronously regarding the application task that was used to activate them. The only exceptions are the telegram send/receive and operation stop functions, which are executed in total synchronization with the execution of the activation task.

Asynchronous Communication Functions

A communication function is asynchronous when it is executed during one or several cycles after the one that activated it.

The following table provides an overview of the communication functions with asynchronous execution:

Function	Role
DATA_EXCH	Send/request receipt of data.
ETH_PORT_CTRL	Activate or deactivate a protocol.
INPUT_BYTE	Read an array of bytes.
INPUT_CHAR	Read a character string.
OUT_IN_CHAR	Send a character string and wait for a response.
OUT_IN_MBUS	Emulate a Modbus master communication from a serial link configured in character mode.
PRINT_CHAR	Write a character string.
READ_ASYN	Read 1 Kbyte of messaging.
READ_GDATA	Read common Modbus Plus data.
READ_VAR	Read standard language objects: internal words and bits, system words and bits, timers, monostables, drums, registers, counters.
SEND_REQ	Send UNI-TE requests.
UNITE_SERVER	Process immediately READ_VAR and WRITE_VAR requests on Modbus (Immediate server).
WRITE_ASYN	Write 1 Kbyte of messaging.
WRITE_GDATA	Write common Modbus Plus data.
WRITE_VAR	Write standard language objects: internal words and bits, system words and bits.

NOTE: It is recommended that asynchronous functions be triggered on edge and not on state to avoid saturating the communication buffers by sending multiple requests. You are also advised, for the same reason, to manage the activity bit *(see page 40)* and report words *(see page 43)* during the execution of each communication function.

Synchronous Communication Functions

A communication function is said to be synchronous when it is wholly executed during the PLC task which activated it.

The following table provides an overview of the communication functions with synchronous execution:

Function	Role
CANCEL	Stop an exchange in progress.
RCV_TLG	Receive a telegram.
SEND_TLG	Send a telegram.

Utility Communication Functions

These functions are executed synchronously. A communication function is said to be synchronous when it is wholly executed during the PLC task which activated it.

The following table provides an overview of the utility communication function:

Function	Role
ADDR	Convert a character string into an address (table of 6 integers) that can be directly exploited by the communication function.

Server Function

At a Glance

The server function can be used to respond to requests from client devices.

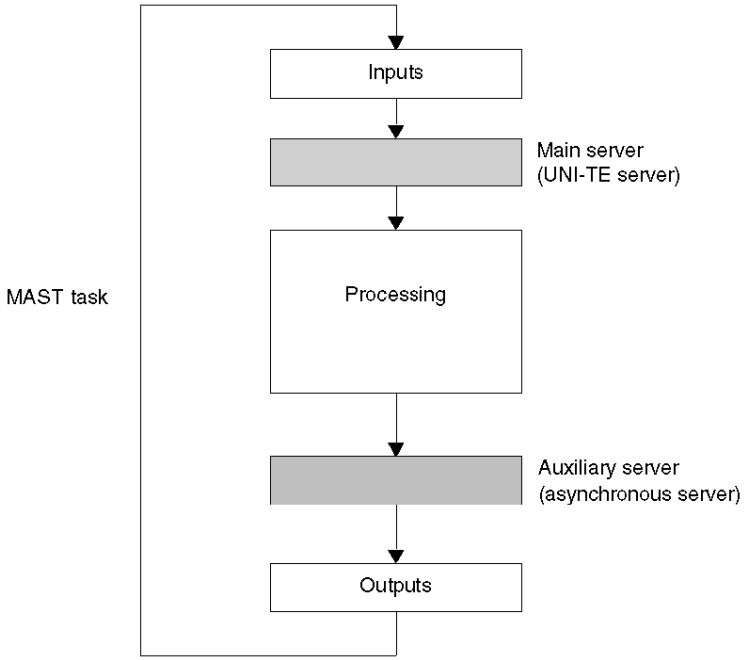
The TSX 57 35• and PCX 57 35• processors provide two request servers:

- a main server (recommended for requests of less than 256 bytes)
- an auxiliary server (recommended for requests of up to 1024 bytes)

Both of these servers can be activated simultaneously.

Illustration

The following diagram shows the requests sent to the servers in the PLC cycle:



Main Server

This server corresponds to port 0 (UNI-TE server). It is activated at the start of the PLC's MAST cycle.

The response time of the client PLC depends on the cycle time of the server PLC. This can be used to process up to 4 simultaneous requests per PLC cycle.

All UNI-TE requests are supported. The size of the request must be less than 256 bytes.

This entity can be addressed at the topological address `SYS` or `{network.station}SYS`.

Auxiliary Server

This server corresponds to port 7 (asynchronous server). It is activated only for periodical tasks at the end of the PLC cycle, after the MAST task has been processed whilst awaiting the start of the next cycle.

The start of the next, higher priority cycle could interrupt a request in progress. Access to this server is therefore reserved for applications requiring no consistency in their read/write data.

The application response time will essentially depend on the PLC cycle time. The size of the request may be up to 1024 bytes. It cannot be accessed from a communication function; the server processes object (bit or word) READ/WRITE requests, etc.

Section 3.3
General Information on Quantum Communication Functions

Overview

This section describes the operation and management of the communication functions for Quantum PLCs.

What Is in This Section?

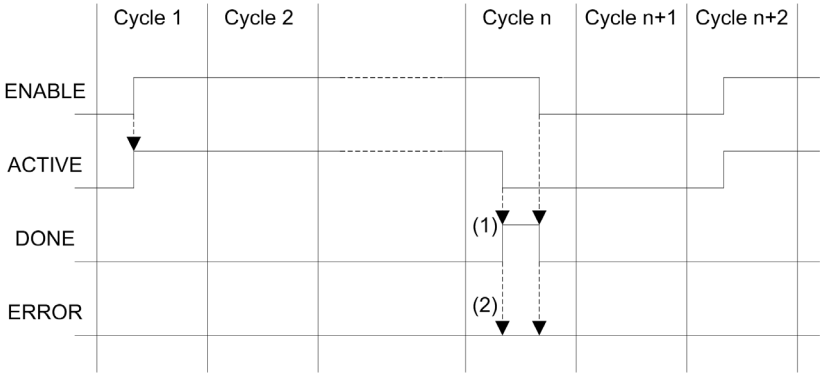
This section contains the following topics:

Topic	Page
Rules for Using the Communication Functions of Quantum PLCs	55
Communication Functions on Quantum PLCs	57

Rules for Using the Communication Functions of Quantum PLCs

Operating Mode of Enable, Active, Done and Error Parameters

The ENABLE, ACTIVE, DONE (or SUCCESS) and ERROR parameters operate as follows:



(1) DONE = 1 if no error, DONE = 0 if error
(2) ERROR = 0 if no error, ERROR = 1 if error

The ENABLE parameter is written by the application.

The ACTIVE, DONE and ERROR parameters are read by the application.

To launch the communication function only once, the ENABLE signal needs to be reset to 0 once the ACTIVE parameter is set to 0. If the ENABLE parameter is maintained to 1 once the ACTIVE parameter is set to 0, the communication function is launched again and the ACTIVE parameter will be set to 1 on the next cycle.

Communicating with Remote Ethernet Drops

When a xxx_QX communication function is used to perform communication exchanges with Ethernet drops, it is highly advisable to test the communication health status of the Ethernet drop before launching the communication function.

A communication function addressed to a faulty drop may take up to 2 minutes to complete, ending with an error status due to the transaction timeout delay (the remote participant has not answered within the timeout delay).

NOTE: The execution of communication functions may be slowed down if all communication ports are already used by communication functions.

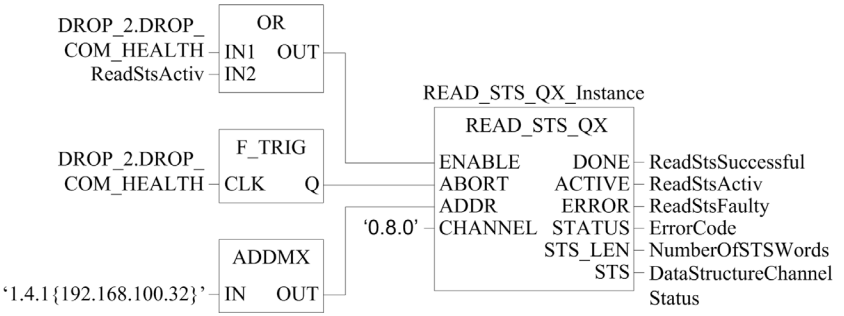
Communication health status information:

- %SW172 to %SW173 *(see Unity Pro, System Bits and Words, Reference Manual)*. Ethernet RIO drop detected communication error status. A bit in those status words is set to 0 when the corresponding connection between the PLC and the drop is not operating properly.
- DROP_COM_HEALTH *(see Quantum EIO, Remote I/O Modules, Installation and Configuration Guide)*. Field in the DDT structure associated with a drop.

Example of Parameters Use in FBD

Below is a FBD diagram example implemented to continuously get the channel 0 status information of:

- a Modicon X80 module located in rack 0, slot 8 of the drop instance #2 at IP address: 192.168.100.32
- via a 140 CRP 312 00 I/O head module in slot 4 of the Quantum local rack



Communication sequence description:

Step	Action and Result
1	When the `READ_STS_QX_Instance` block is not active, it is started if the drop communication health status is set to 1.
2	`READ_STS_QX_Instance` is called every following scan until operation is completed either with: ● a success (`DONE` is set to 1) ● or a failure (`ERROR` is set to 1)
3	If the operation is completed with: ● success, the Modicon X80 module status word can be used in the variable linked to the block `STS` parameter. ● failure, the error code can be read in the `STATUS` parameter. If the communication link with the drop is broken, `DROP_COM_HEALTH` falls to 0 and the current active operation is aborted (`STATUS` = 16#1001). **NOTE:** If `ABORT` parameter is not triggered, the block remains active until the transaction timeout delay is elapsed and the communication ends with an error (`STATUS` = 16#5303).

Communication Functions on Quantum PLCs

Communication Blocks and Transaction Paths

The communication function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

Available Communication Function on Low End CPUs

Communication function roles:

Function	Role
CREAD_REG	Continuously read a register area from a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
CWRITE_REG	Continuously write a register area to a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
MBP_MSTR	Perform various network communication operations on Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
READ_REG (1)	Read a register area from a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
WRITE_REG (1)	Write a register area to a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
XXMIT	Modbus messages from master PLC and ASCII input/output strings.
(1) READ_REG and WRITE_REG communication functions are launched when an edge appears on the **REG** input. The communication is launched when **REG** is set to 1; then **REG** needs to be reset (0) and set to 1 again to launch the communication again.	

Available Communication Function on High End CPUs

Communication function roles:

Function	Role
CREAD_REG	Continuously read a register area from a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
CWRITE_REG	Continuously write a register area to a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
EXCH_QX	Perform data transfers through an EIO bus to and from Modbus slaves connected to a Modicon X80 rack
GET_TS_EVT_Q (2)	Get the time stamped data in a Modicon BMX ERT 1604T or BMX CRA ••••• module in an EIO drop.
INPUT_CHAR_QX	Receive a character string from a Modicon X80 serial communication module through an EIO bus
MBP_MSTR	Perform various network communication operations on Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
PRINT_CHAR_QX	Send a character string of 1000 bytes maximum from a Modicon X80 Modbus master through an EIO bus
READ_REG (1)	Read a register area from a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
READ_REG_QX	Read registers in a Modbus slave connected to a Modicon X80 Modbus master through an EIO bus
READ_STS_QX (*)	Read the status words of a Modicon X80 Ethernet I/O module by performing an explicit exchange with the processor memory
WRITE_CMD_QX (*)	Send a command to a Modicon X80 Ethernet I/O module with a command word by performing an explicit exchange
WRITE_REG (1)	Write a register area to a slave addressed via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet
WRITE_REG_QX	Write registers in a Modbus slave connected to a Modicon X80 Modbus master through an EIO bus
XXMIT	Modbus messages from master PLC and ASCII input/output strings.

***:** READ_STS_QX and WRITE_CMD_QX are part of the I/O Management Library (Explicit Exchange Family) *(see Unity Pro, I/O Management, Block Library)*

1: READ_REG and WRITE_REG communication functions are launched when an edge appears on the **REG** input. The communication is launched when **REG** is set to 1; then **REG** needs to be reset (0) and set to 1 again to launch the communication again.

2: GET_TS_EVT_Q is part of the System Library (SysClock family *(see Unity Pro, System, Block Library)*).

Utility Functions

Those functions are common to low end and high end CPUs and are synchronous functions. A function is said to be synchronous when it is completed during the CPU task that has activated it.

Communication function roles:

Function	Role
ADDMX	Convert a character string (respecting the Unity Pro device address syntax) into an array of integers in order to manage Modicon X80 serial link modules in an Ethernet I/O drop
ModbusP_ADDR	Convert a Modbus Plus address into an address that can be directly used by the communication functions
SYMAX_IP_ADDR	Convert a SYMAX/IP address into an address that can be directly used by the communication functions
TCP_IP_ADDR	Convert a TCP/IP address into an address that can be directly used by the communication functions

Section 3.4
General Information on M340 Communication Functions

Communication Functions on M340 CPUs

Rules for Using Communication Functions on M340 CPUs

M340 CPUs use EF and procedure type communication functions *(see page 35)*.

Legacy Communication Functions Available on M340 CPUs

Legacy communication functions role:

Name	Role
DATA_EXCH	Transmit or receive data.
INPUT_BYTE	Receive an array of bytes on a character mode link of a BMX NOM module in a local rack or linked to a CPU embedded communication channel.
INPUT_CHAR	Receive a character string on a character mode link of a BMX NOM module in a local rack or linked to a CPU embedded communication channel.
PRINT_CHAR	Send a character string on a character mode link of a BMX NOM module in a local rack or linked to a CPU embedded communication channel.
READ_VAR	Read the value of one or more language objects via a communication module in a local rack or linked to a CPU embedded communication channel.
SEND_EMAIL	Send an email over an Ethernet port of a communication module plugged in a local rack.
WRITE_VAR	Write the value of one or more language objects via a communication module in a local rack or linked to a CPU embedded communication channel.

Other Communication Functions Available on M340 CPUs

Other communication function role:

Name	Role
ETH_PORT_CTRL	Activate or deactivate a protocol.
GET_TS_EVT_M (1)	Get the time stamped data in a Modicon BMX ERT 1604T module plugged in a local rack.
1: GET_TS_EVT_M is part of the System Library (SysClock family *(see Unity Pro, System, Block Library)*).	

Utility Functions Available on M340 CPUs

These functions are executed synchronously. A function is synchronous when it is completely executed during the CPU task that activated this function.

Utility functions role:

Name	Role
ADDM	Convert a character string into an address that can be used directly by the communication functions READ_VAR, WRITE_VAR, DATA_EXCH and PRINT_CHAR.
CANCEL	Interrupt an asynchronous communication function in progress. **NOTE:** Another way to cancel an asynchronous communication function in progress is to use the cancel bit *(see page 41)* of the management parameters.

Section 3.5
General Information on M580 Communication Functions

Overview

This section describes the operation and management of the communication functions for **M580** CPUs.

What Is in This Section?

This section contains the following topics:

Topic	Page
Rules for using the Communication Functions of M580 CPUs	63
Communication Functions on M580 CPUs	65

Rules for using the Communication Functions of M580 CPUs

Overview

M580 CPUs use both types of communication functions:

- EF and procedure type functions
- EFB type functions

For general description of the management of the communication functions according to their type, refer to Communication Functions Management *(see page 36)*.

Communicating with Remote Ethernet Drops

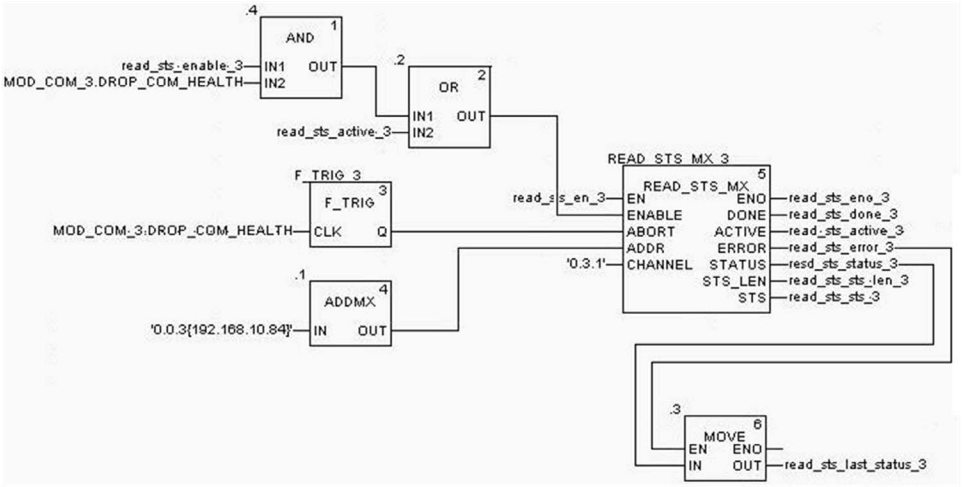
When a ***_MX communication function is used to perform communication exchanges with Ethernet drops, it is highly advisable to test the communication health status of the Ethernet drop before launching the communication function.

A communication function addressed to a faulty drop may take up to 2 minutes to complete, ending with an error status due to the transaction timeout delay (the remote participant has not answered within the timeout delay).

NOTE: The execution of other communication functions may be slowed down if all communication ports are already used by communication functions. Communication health status information is provided in the DROP_COM_HEALTH *(see Modicon M580, RIO Modules, Installation and Configuration Guide)* field of the drop associated device DDT.

Example of Parameters Use in FBD

Following diagram in FBD illustrates how to get the channel 1 status information of a BMX NOM 0200 module located in rack 0, slot 3 of the drop instance number 3 (associated DDT could be for instance: MOD_COM_3 or if we keep the default naming EIO2_d3_r0_s3_NOM0200_3) at IP address 192.168.10.84.



`read_sts_en_3` should be set to 1.

When the `READ_STS_MX_3` block is not active, it is started if the variable `read_sts_enable_3` is set to 1 by the user and the drop communication health status is OK. Every following scan until operation ends up with success (`DONE` set to 1) or with detected failure (`ERROR` set to 1).

In case of success, status words of the module may be exploited in the variable linked to the `STS` parameter of the block. If the communication link with the drop is broken, `MOD_COM_3.DROP_COM_HEALTH` falls to 0 and current active operation is aborted with error code 1001 hex. If `ABORT` pin is not connected, the block executes until the transaction timeout elapses (15 s for ***_MX functions) and ends up in error with status code 3401 hex.

Communication Functions on M580 CPUs

Legacy Communication Functions Available on M580 CPUs

Legacy communication functions role:

Name	Role
DATA_EXCH	End any requests (Modbus, Umas …) to any Modbus slave via a communication module plugged in a local rack or in an EIO drop.
INPUT_BYTE	Receive an array of bytes on a character mode link of a BMX NOM module plugged in a local rack or in an EIO drop.
INPUT_CHAR	Receive a character string on a character mode link of a BMX NOM module plugged in a local rack or in an EIO drop.
PRINT_CHAR	Send a character string on a character mode link of a BMX NOM module plugged in a local rack or in an EIO drop.
READ_VAR	Read the value of one or more language objects via a communication module plugged in a local rack or in an EIO drop.
SEND_EMAIL	Send an email over an Ethernet port of a communication module plugged in a local rack (Ethernet modules cannot be plugged in an EIO drop).
WRITE_VAR	Write the value of one or more language objects via a communication module plugged in a local rack or in an EIO drop.

Other Communication Functions Available on M580 CPUs

Other communication functions role:

Name	Role
ETH_PORT_CTRL	Activate or deactivate a protocol.
EthPort_Control_MX	Enable or disable the configured Ethernet services.
GET_TS_EVT_M (1)	Get the time stamped data in a Modicon BMX ERT 1604T or BMX CRA ••••• module plugged in a local rack or in an EIO drop.
READ_PARAM_MX (2) (3)	Read the parameter words of a Modicon X80 I/O module plugged in a local rack by performing an explicit exchange.
READ_STS_MX (2)	Read the status words of a Modicon X80 I/O module plugged in a local rack or in an EIO drop by performing an explicit exchange.
1: GET_TS_EVT_M is part of the System Library (SysClock family *(see Unity Pro, System, Block Library)*). **2:** READ_STS_MX, WRITE_CMD_MX, READ_PARAM_MX, WRITE_PARAM_MX, SAVE_PARAM_MX, and RESTORE_PARAM_MX are part of the I/O Management Library (Explicit Exchange family *(see Unity Pro, I/O Management, Block Library)*). **3:** READ_PARAM_MX, WRITE_PARAM_MX, SAVE_PARAM_MX, and RESTORE_PARAM_MX can only be used with I/O modules plugged in a local rack. When sent to a module on an EIO drop, error code 16#2007 (Unauthorized network field on slave) is returned.	

Name	Role
RESTORE_PARAM_MX (2) (3)	Restore the parameter words of a Modicon X80 I/O module plugged in a local rack by performing an explicit exchange.
SAVE_PARAM_MX (2) (3)	Save the parameter words of a Modicon X80 I/O module plugged in a local rack by performing an explicit exchange.
WRITE_CMD_MX (2)	Send a command to a Modicon X80 I/O module plugged in a local rack or in an EIO drop by performing an explicit exchange.
WRITE_PARAM_MX (2) (3)	Write the parameter words of a Modicon X80 I/O module plugged in a local rack by performing an explicit exchange.
1: GET_TS_EVT_M is part of the System Library (SysClock family *(see Unity Pro, System, Block Library)*). **2:** READ_STS_MX, WRITE_CMD_MX, READ_PARAM_MX, WRITE_PARAM_MX, SAVE_PARAM_MX, and RESTORE_PARAM_MX are part of the I/O Management Library (Explicit Exchange family *(see Unity Pro, I/O Management, Block Library)*). **3:** READ_PARAM_MX, WRITE_PARAM_MX, SAVE_PARAM_MX, and RESTORE_PARAM_MX can only be used with I/O modules plugged in a local rack. When sent to a module on an EIO drop, error code 16#2007 (Unauthorized network field on slave) is returned.	

Utility Functions Available on M580 CPUs

These functions are executed synchronously. A function is synchronous when it is completely executed during the CPU task that activated this function.

Utility functions role:

Name	Role
ADDM	Convert a character string into an address that can be used directly by the communication functions.
ADDMX	Convert a character string respecting the Unity Pro device address syntax into an array of integers.
CANCEL	Interrupt an asynchronous communication function in progress. **NOTE:** Another way to cancel an asynchronous communication function in progress is to use the cancel bit *(see page 41)* of the management parameters.

NOTE:

- To address a module on an EIO drop, address string to be used with ADDMX needs to be 0.0.3 {xx.xx.xx.xx} (3: channel 3 of CPU correspond to the embedded Ethernet port of the CPU, xx.xx.xx.xx corresponds to the IP address of the drop).
- To address a module in a local rack with ***_MX functions, address string needs to be 0.0.10 (address of the CPU main server).

Section 3.6
General Information on Momentum Communication Functions

Communication Functions on Momentum PLCs

Rules for Using EFB or DFB Type Communication Functions on Momentum PLCs

Momentum PLCs use EFB type communication functions. For more details on those communication functions managements, refer to section Communication Function Management *(see page 34)*.

Communication Functions available on Momentum PLCs

Communication function roles:

Function	Role
MBP_MSTR	Perform various network communication operations on Modbus Plus, TCP/IP-Ethernet.
XMIT	Modbus messages from master PLC and ASCII input/output strings.
XXMIT	Modbus messages from master PLC and ASCII input/output strings.

Part II
Extended

Overview

This section describes the elementary functions and elementary function blocks of the `Extended` family.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
4	ADDM: Address Conversion	71
5	ADDMX: Address Conversion	77
6	ADDR: Address Conversion	83
7	CANCEL: Stopping an Exchange in Progress	85
8	CREAD_REG: Continuous Register Reading	89
9	CWRITE_REG: Continuous Register Writing	97
10	DATA_EXCH: Exchanging Data between Applications	105
11	ETH_PORT_CTRL: Activating or Deactivating a Protocol	119
12	EthPort_Control_MX: Activating or Deactivating a Protocol	123
13	EXCH_QX: Exchanging Data between Applications on EIO Bus	125
14	INPUT_BYTE: Receiving Character Strings	131
15	INPUT_CHAR: Receiving Character Strings	137
16	INPUT_CHAR_QX: Receiving Character Strings on EIO Bus	149
17	MBP_MSTR: Modbus Plus Master	153
18	ModbusP_ADDR: Modbus Plus Address	199
19	OUT_IN_CHAR: Sending/Receiving Character Strings	205
20	OUT_IN_MBUS: Modbus Communication Function	213
21	PRINT_CHAR: Sending character strings	251
22	PRINT_CHAR_QX: Sending Character Strings on EIO Bus	263
23	RCV_TLG: Receiving telegrams	267
24	READ_ASYN: Reading data asynchronously	273
25	READ_GDATA: Reading Modbus Plus Global Data	277
26	READ_REG: Read Register	279
27	READ_REG_QX: Read Register on EIO Bus	287

Chapter	Chapter Name	Page
28	READ_SDO: Reading Service Data Object	291
29	READ_VAR: Reading variables	295
30	SEND_EMAIL: Sending Email	315
31	SEND_REQ: Sending requests	319
32	SEND_TLG: Sending telegrams	337
33	SYMAX_IP_ADDR: SY/MAX IP Address	343
34	TCP_IP_ADDR: TCP/IP Address	349
35	UNITE_SERVER: Immediate server	355
36	WRITE_ASYN: Writing data asynchronously	361
37	WRITE_GDATA: Writing the Global Data Modbus Plus	365
38	WRITE_REG: Write Register	367
39	WRITE_REG_QX: Write Register on EIO Bus	375
40	WRITE_SDO: Writing Service Data Object	379
41	WRITE_VAR: Writing variables	383
42	XMIT: Transmit	399
43	XXMIT: Transmit	419

Chapter 4
ADDM: Address Conversion

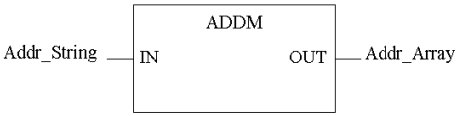
Description

Function Description

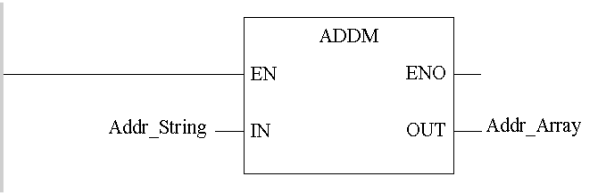
The ADDM function is used to convert a character string into an address that can be used directly by the following communication functions: READ_VAR, WRITE_VAR, INPUT_CHAR, PRINT_CHAR, DATA_EXCH, SEND_EMAIL.

The additional parameters EN and ENO can be configured.

FBD Representation



LD Representation



IL Representation

```
LD Addr_String
ADDM
ST Addr_Array
```

ST Representation

```
Addr_Array := ADDM(Addr_String);
```

Input Parameter Description

The following table describes the input parameter:

Parameter	Type	Comment
IN	Character string	Device address on bus or network.

The input is the concatenation of up to three elements that can take various values. Each element is optional:

Output port used locally to reach the target equipment:	Target equipment	Service to reach in the target equipment:
• r.m.c • Netlink **NOTE:** in the first case, the channel number c is optional	• {hostAddr} • node • {hostAddr}node	• SYS • MBS • SMTP • TCP.MBS • CON.CIP • UNC.CIP

Definitions:

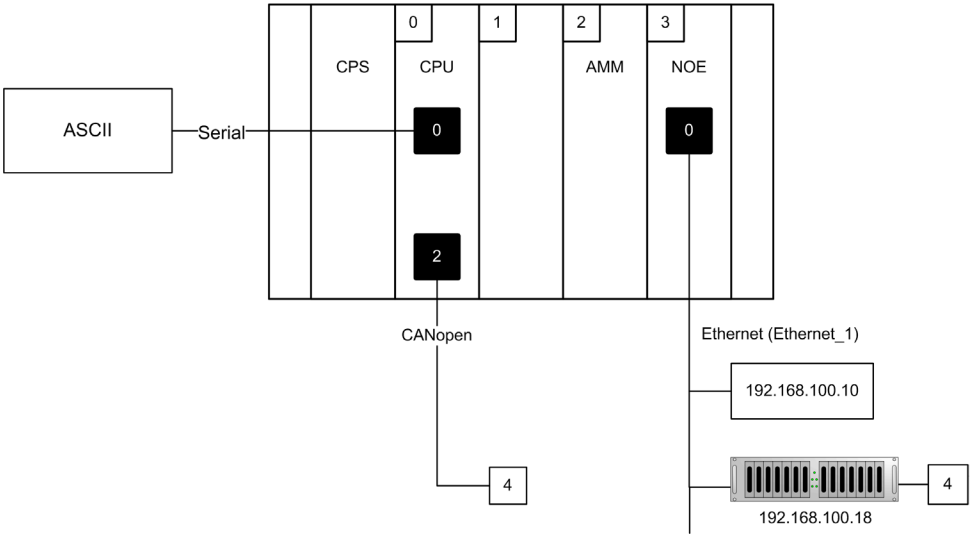
- Netlink: network name set in the Net Link field of Ethernet channel
- hostAddr: IP address of the host we want to reach.
- r: rack number of the communication channel
- m: module position of the communication channel
- c: communication channel number
- node: Modbus or CANopen node behind a gateway (gateway identified with hostAddr)
- SYS: for addressing the system server
- MBS: for addressing the Modbus system server (same as SYS)
- SMTP: for addressing the SMTP server
- TCP.MBS: for addressing a TCP Modbus server
- CON.CIP: for addressing a connected equipment through Ethernet/IP
- UNC.CIP: for addressing an unconnected equipment through Ethernet/IP

NOTE: For Modicon M580 CPU, to configure a communication through the CPU Ethernet port, r=0, m=0 and c=3.

List of different use cases:

Address syntaxes	Comment
'{hostAddr}' '{hostAddr}SYS' '{hostAddr}TCP.MBS'	To access the hostAddr by the nearest configured Netlink from the CPU. Example: ADDM('{192.168.2.3}TCP.MBS').
'Netlink{hostAddr}' 'Netlink{hostAddr}SYS' 'Netlink{hostAddr}TCP.MBS'	To access the hostAddr by the Netlink Ethernet network defined in the Unity Pro Project Browser **Communication → Networks**. Example: ADDM('Ethernet_1{192.168.2.3}SYS').
'Netlink{hostAddr}node'	To access the node behind the 'Netlink{hostAddr}' gateway, it can be a Modbus TCP server ID or a serial Modbus slave managed by the gateway. Example: ADDM('Ethernet_1{192.168.2.3}255').
'r.m.c{hostAddr}' 'r.m.c{hostAddr}SYS' 'r.m.c{hostAddr}TCP.MBS'	To access the hostAddr by giving the rack, module and channel of the Ethernet module. Example: ADDM('0.0.3{192.168.2.3}').
'r.m.c{hostAddr}node'	To access the node behind the 'r.m.c{hostAddr}' gateway, it can be a Modbus TCP server ID or a serial Modbus slave managed by the gateway. Example: ADDM('0.0.3{192.168.2.3}255').
'r.m.c{hostAddr}CON.CIP' 'r.m.c{hostAddr}UNC.CIP'	To access an equipment through an Ethernet/IP network, the suffix CON.CIP is used if the Equipment is connnected (a link is still established with it) and UCN.CIP if it is not connected. Example 1: ADDM('0.0.3{192.168.20.34}CON.CIP') for a Modicon M580 Ethernet CPU. Example 2: ADDM('0.4.0{192.168.20.34}CON.CIP') for a Modicon M340 with a BMX NOC in the slot 4 of rack 0.
'r.m.c' 'r.m.c.SYS'	To access the channel system of a Modbus channel or to send data through a Character Mode Link. **NOTE:** c= 0 or 1. Example: ADDM('1.5.0.SYS').
'r.m.c.e' 'r.m.c.e.MBS'	To access the Modbus serial slave equipment server. **NOTE:** c= 0 or 1, e=1 to 247. Example: ADDM('1.5.1.54.MBS').
Specific case for M340 CPU with CANopen embedded:'0.0.2.e'	To access the CANopen slave e server. **NOTE:** e=1 to 127. Example: ADDM('10.0.2.67').

Examples with Modicon M340:



In the following examples, the third syntax element is not used:

Device to address	ADDM syntax
Device #4 on CANopen link	`ADDM('0.0.2.4')`
ASCII printer on CPU serial link	`ADDM('0.0.0')`
Device whose address is 192.168.100.10 on a NOE Ethernet link	`ADDM('0.3.0{192.168.100.10}')` `ADDM('Ethernet_1{192.168.100.10}')`
Modbus device #4 behind Ethernet gateway whose address is 192.168.100.18 on NOE Ethernet link	`ADDM('0.3.0{192.168.100.18}4')` `ADDM('Ethernet_1{192.168.100.18}4')`

For further information, please refer to M340 PLCs addressing *(see Communication Services and Architectures, Reference Manual)* overview of user manual Communication Architectures.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF APPLICATION

Do not use wrong address parameters. For instance, do not set an address parameter that does not correspond to the targeted equipment.

Failure to follow these instructions can result in injury or equipment damage.

Output Parameter Description

The following table describes the output parameter:

Parameter	Type	Comment
OUT	ADDM_TYPE Array of 8 integers	Array representing the address of a device. This parameter can be used as an input parameter for several communication functions.

The ADDM block analyses the addressing string syntax (parameter IN) and puts the result into an 8 single integers array which defines the destination address. The destination address can be provided either by the OUT parameter of the ADDM function or directly by an array of 8 INT objects. However, it is strongly recommended to use the ADDM function to address a communication EF.

The destination address' structure is as follows:

Field	Size	Value
Type	Byte	Reserved.
ClientID	Byte	Reserved.
Rack	Byte	Rack slot number.
Slot	Byte	Module slot number.
Channel	Byte	Channel number.
ProtId	Byte	Reserved. 0 for Modbus.
AddrLen	Byte	This byte can have the following values: ● value 0 if module or channel server is addressed (neither UnitId nor AddrExt is used) ● value 1 if device number is specified ● value >1 if AddrExt is also used
UnitId	Byte	Device number, Modbus device for example.
AddrExt	4 Integers	Reserved. It is used to encode additional address information for TCP/IP addresses.

Chapter 5
ADDMX: Address Conversion

Description

Function Description

The ADDMX function is used to convert a character string respecting the Unity Pro device address syntax into an array of integers in order to manage (e)X80 serial link modules in an Ethernet I/O drop.

The ADDMX function can be used directly by the following communication functions with a local Modicon M340 CPU or a remote M340 module in a Quantum system:

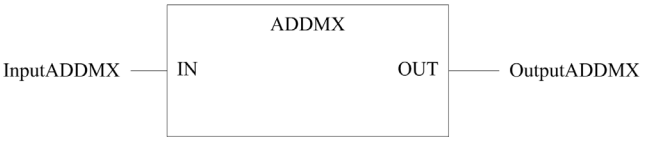
- READ_STS_QX
- WRITE_CMD_QX
- EXCH_QX
- READ_REG_QX
- WRITE_REG_QX
- PRINT_CHAR_QX

The ADDMX function can be used directly by the following communication functions in an M580 system:

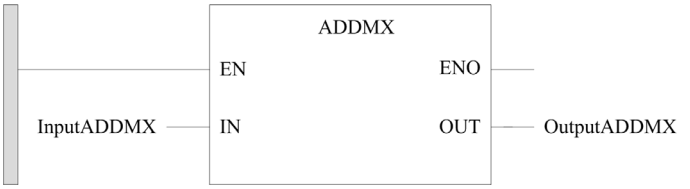
- READ_STS_MX
- WRITE_CMD_MX
- READ_PARAM_MX
- RESTORE_PARAM_MX
- SAVE_PARAM_MX
- WRITE_PARAM_MX
- INPUT_CHAR
- PRINT_CHAR
- READ_VAR
- WRITE_VAR
- INPUT_BYTE
- DATA_EXCH
- SEND_EMAIL

The additional parameters EN and ENO can be configured.

FBD Representation



LD Representation



IL Representation

```
ADDMX (
      IN := InputADDMX
      )
ST OutputADDMX
```

ST Representation

```
OutputADDMX := ADDMX (IN := InputADDMX);
```

Parameter Description

The following table describes the input parameter:

Parameter	Type	Comment
IN	Character string	String complying with Unity Pro device address syntax that contains the path from the CPU module to the EIO drop module and the path inside the EIO drop module to reach the targeted module. Each path is separated by \\ characters inside the string.

To address a BMX CRA 312 •0 or BME CRA 312 •0 module directly with its IP address, the `IN` parameter takes one of the 2 following forms:

- 'r0.s0.ch0.{ip.ip.ip.ip}'
- 'r0.s0.ch0.{ip.ip.ip.ip}TCP.MBS'

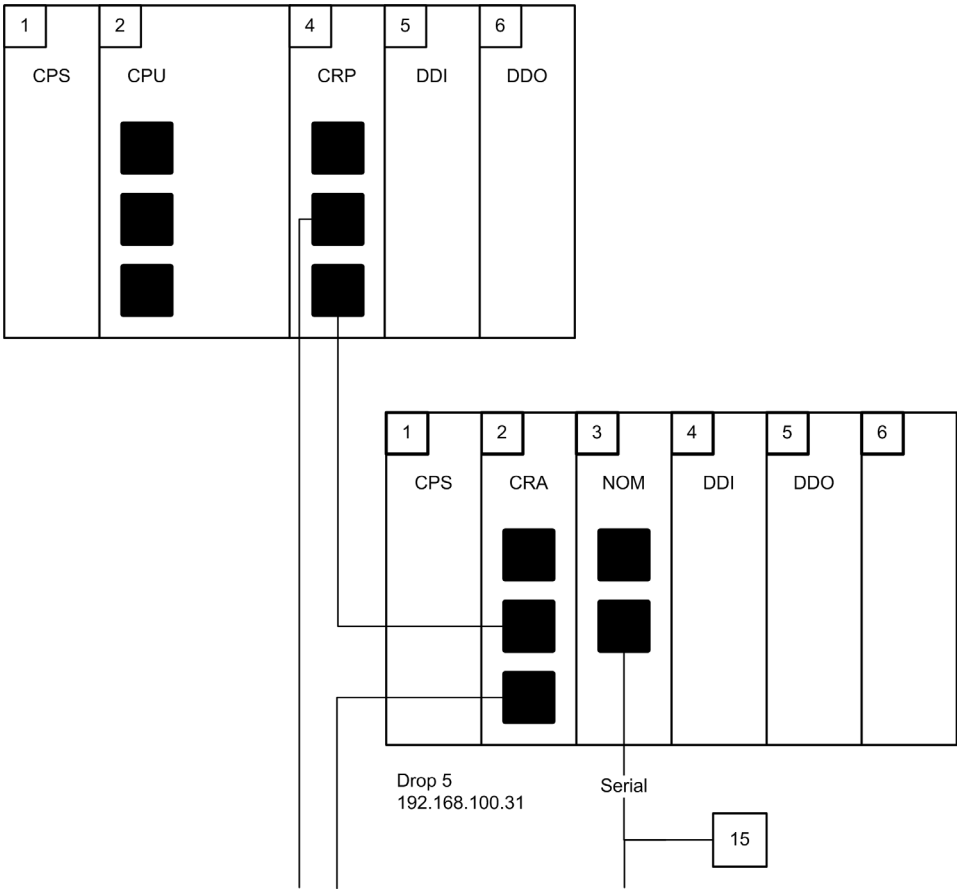
To address a Modbus server or slave using Modbus protocol *(see Communication Services and Architectures, Reference Manual)*, the `IN` parameter takes one of the 2 following forms:

- 'r0.s0.ch0.{ip.ip.ip.ip}TCP.MBS\\r1.s1.ch1.d'
- 'r0.s0.ch0.{ip.ip.ip.ip}TCP.MBS\\r1.s1.ch1.d.MBS'

Detailed description of Unity Pro device address syntax string content:

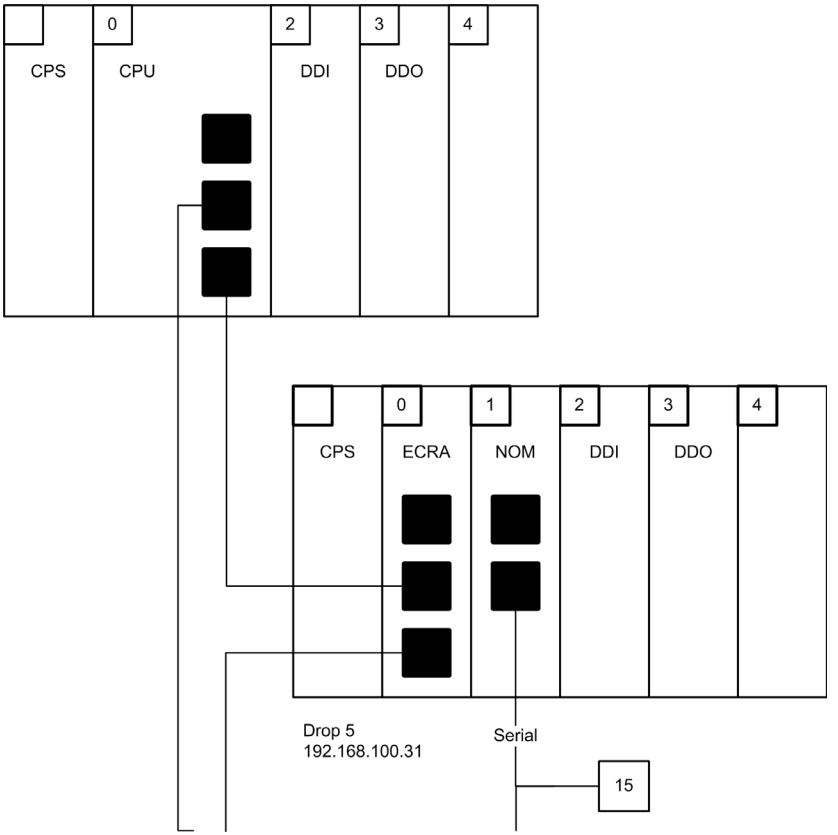
Field	Value	
r0.s0.ch0	Topological address of the Quantum CRP or Quantum NOE module or M580 Ethernet port that will send the TCP/Modbus request (rack, slot, and channel number). **NOTE:** In a Quantum rack: r0 = 1 and ch0 = 1. **NOTE:** For a M580: to configure a communication through the CPU Ethernet port, set channel number to 3 (0.0.3).	
{ip.ip.ip.ip}	BMX CRA 312 00 or BMX CRA 312 10 module with ip.ip.ip.ip address (protocol TCP port 502, default IP protocol, and port).	
TCP.MBS	Optional, used to address a Modbus server	
\\	Separator	
r1.s1.ch1.d	Modbus server module rack, slot, channel, and Modbus slave number Depending on the addressing mode, the parameters can be partly used as described:	
	r1.s1	Module server addressing mode: module server topological address in a rack with a BMX CRA 312 10 module. **NOTE:** Modicon M340 first rack number = 0 (r0 = 0).
	r1.s1.ch1	Channel server addressing mode: channel server address in the rack. **NOTE:** Modicon M340 first channel number = 0 (ch1 = 0).
	r1.s1.ch1.d	Modbus slave addressing mode: address Modbus slave n° d, using channel ch1 Modbus Master from a NOM module located in a rack with a BMX CRA 312 10 module.
MBS	Optional. To address a Modbus server if the NOM module is not a Modbus master.	

Examples of Quantum drop:



Device to Address	ADDMX Syntax
Module server of drop #5 CRA	ADDMX('1.4.1{192.168.100.31}SYS
Module server of BMX NOM in drop #5	ADDMX('1.4.1{192.168.100.31}\\0.3')
Modbus device #15 on serial link of BMX NOM in drop #5	ADDMX('1.4.1{192.168.100.31}\\0.3.1.15')

Examples of M580 drop:



Device to Address	ADDMX Syntax
Module server of drop #5 CRA	ADDMX('0.0.3{192.168.100.31}SYS
Module server of BMX NOM in drop #5	ADDMX('0.0.3{192.168.100.31}\\0.1')
Modbus device #15 on serial link of BMX NOM in drop #5	ADDMX('0.0.3{192.168.100.31}\\0.1.1.15')

The following table describes the output parameter:

Parameter	Type	Comment
OUT	ANY_ARRAY_INT	Array containing at least 2 ADDM_TYPE arrays. Each ADDM_TYPE array represents the address of a device (more details provided hereunder). This parameter can be used as an input parameter for several communication functions.

The ADDMX block analyses the addressing string syntax (IN parameter) and puts the result into an array of array which defines the destination address. The destination address can be provided either by the OUT parameter of the ADDMX function or directly by an array of array (n x 8 INT objects). However, it is strongly recommended to use the ADDMX function to address a communication EF.

Detailed description of ADDM_TYPE array:

Field	Size	Value
Type	1 byte	reserved (2)
ClientID	1 byte	0 if not specified
Rack	1 byte	module rack number
Slot	1 byte	module slot number
Channel	1 byte	channel number inside the module
ProtId	1 byte	This byte can have the following values: ● 0: Modbus protocol ● 2: user protocol ● 3: SMTP
AddrLen	1 byte	This byte can have the following values: ● 0: No optional field ● 1: Modbus address content ● >1: Ethernet address content (AddrExt is used)
UnitId	1 byte	Optional: Device address or UNIT-ID
AddrExt	Array[7]	Optional: Ethernet destination address extension

⚠ WARNING

UNEXPECTED BEHAVIOR OF APPLICATION

The use of reserved parameters is the responsibility of users and can lead to malfunctions.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF APPLICATION

Do not use wrong address parameters. For instance, do not set an address parameter that does not correspond to the targeted equipment.

Failure to follow these instructions can result in injury or equipment damage.

Chapter 6
ADDR: Address Conversion

Description

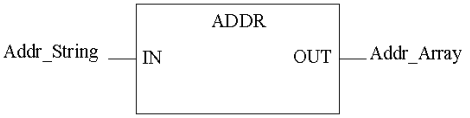
Function Description

The ADDR function is used to convert a character string into an address that can be used directly by communication functions.

The additional parameters EN and ENO can be configured.

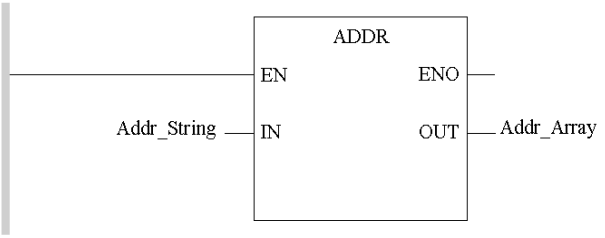
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Addr_String
ADDR
ST Addr_Array
```

ST Representation

Representation:

```
Addr_Array := ADDR(Addr_String);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
IN	`STRING`	Character string-type variable representing the device address on a bus or network. **Examples**: '{5.6}SYS', '{1.2}0.4.5.2'

The following table describes the output parameters:

Parameter	Type	Comment
`OUT`	ADDR_TYPE or ARRAY [0..5] OF INT	Array of 6 integers representing the `Addr_String` address. `Addr_Array` may be used directly as the first input parameter of the communication EFs.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF APPLICATION

Do not use wrong address parameters. For instance, do not set an address parameter that does not correspond to the targeted equipment.

Failure to follow these instructions can result in injury or equipment damage.

Chapter 7
CANCEL: Stopping an Exchange in Progress

Subject of this Chapter

This chapter describes the `CANCEL` function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	86
Example of How to Cancel an Exchange	88

Description

Function Description

The CANCEL function is used to interrupt an asynchronous communication function in progress. The exchange number assigned to each communication can be used to identify the function to be stopped.

The time between the CANCEL function request and the CANCEL action depends on the number of communication functions in progress. The use of a CANCEL function means that for all channels and EFs:

- The assigned asynchronous communication functions in progress are cancelled.
- The dedicated frames buffered before using the CANCEL function may be sent.

NOTE: In character mode, the assigned INPUT_CHAR, INPUT_BYTE and/or OUT_IN_CHAR elementary functions in progress are cancelled and the associated asynchronous serial link channel is unlocked if it is waiting for the ending criteria.

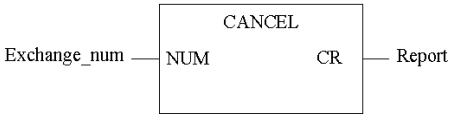
NOTE: The execution of this function is synchronous with the execution of the PLC program (the communication function is stopped in the PLC cycle during which the CANCEL function was executed).

The additional parameters EN and ENO can be configured.

NOTE: Cancelling an exchange is effective at the application level, but has no effect at the physical communication port level. After a communication function has been initiated, the exchange is managed at the communication port level.
Cancelling a communication function at the application level means the Activity bit is reset, the Report is written the value 2, the Exchange number is released. If a reply is finally received at the port level for the exchange request, it can no longer be taken into account at the application level. In the same manner, the Time Out event of communication function doesn't stop retries of a communicating module. This can happen when the network is slow and/or of low quality, causing retransmissions. In such a situation, consistency between the applicative Time Out at the EF level and protocol Time Outs configured at the module level, should be ensured.

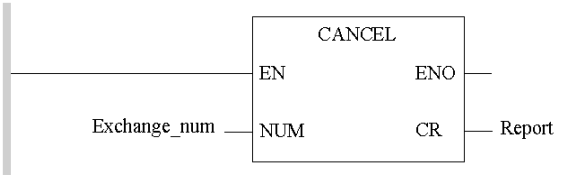
FBD Representation

Representation:



Ladder Representation

Representation:



IL Representation

Representation:

```
LD Exchange_Num
CANCEL
ST Report
```

ST Representation

Representation:

```
CANCEL(Exchange_Num, Report);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
NUM	INT	This parameter specifies the number of the exchange whose execution is to be stopped.

The following table describes the output parameters:

Parameter	Type	Comment
CR	INT	This operation report is assigned one of the following two values: ● 16#00: Correct operation. Communication is interrupted, the activity bit of the interrupted function is set to 0 and its report is assigned the value 2. ● 16#0C: Incorrect exchange number.

Example of How to Cancel an Exchange

At a Glance

This example covers canceling an exchange carried out with the OUT_IN_CHAR function.

When the exchange is launched, a unique number is assigned to it. This number remains valid until the end of the exchange.

The CANCEL function uses this number to interrupt the exchange to which it is assigned.

Programming the OUT_IN_CHAR Function

Programming in ST:

```
IF RE(%I0.3.8) AND NOT %MW170.0 THEN
   (* initialization of the data to send *)
   %MW173 := 10;
   (* communication function *)
   OUT_IN_CHAR(ADDR('{20.5}0.0.0.SYS'),1,Str_Out,Str_In,%MW170:4);
END_IF;
```

Programming the CANCEL Function

Programming in ST:

```
%MW180 := SHRZ_INT(%MW170,8);
IF RE(%I0.3.9) THEN
   CANCEL(%MW180,%MW185);
END_IF;
```

The CANCEL function comprises two parameters:

- on input: the exchange number to be cancelled
- on output: the report

It is necessary to initialize the first parameter with the number of the exchange to be cancelled. This number is located in the most significant byte of the first word of the management table. For us, this is the most significant byte of %MW170. Before sending the CANCEL function, it is necessary to create an 8 bit shift in order to recover the eight most significant bits of %MW170.

Request parameters:

Parameters	Description
%MW180	The least significant bit contains the number of the exchange to be cancelled. **NOTE**: The most significant byte must be zero.
%MW185	Function report.

Chapter 8
CREAD_REG: Continuous Register Reading

Introduction

This chapter describes the CREAD_REG block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	90
Derived Data Types	93
Function Mode	95
Parameter Description	96

Description

Function Description

This function block reads a register area continuously. It reads data from an addressed node via Modbus Plus, TCP/IP Ethernet, or SY/MAX Ethernet.

EN and ENO can be configured as additional parameters.

CREAD_REG, CWRITE_REG, READ_REG, WRITE_REG, and MBP_MSTR function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

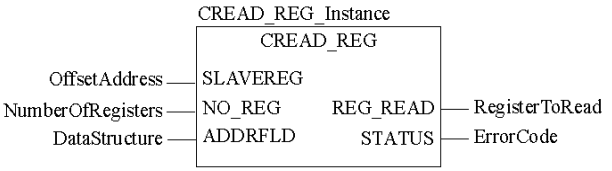
More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

NOTE: When programming a CREAD_REG function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the *Modbus Plus Network Planning and Installation Guide (Reference 31003525)*. If TCP/IP or SY/MAX Ethernet routing is implemented *(see Quantum with Unity Pro, TCP/IP Configuration, User Manual)*, standard Ethernet IP router products must be used.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

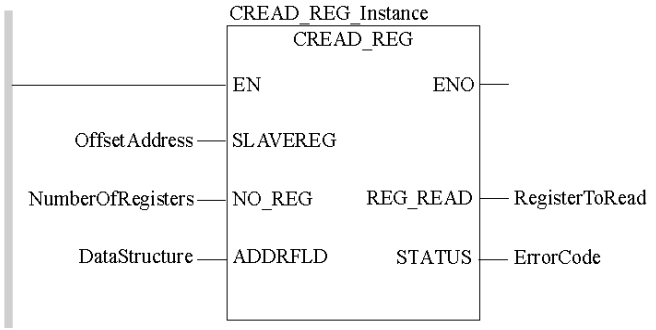
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL CREAD_REG_Instance (SLAVEREG:=OffsetAddress,    NO_REG:=NumberOfReg
isters, ADDRFLD:=DataStructure,    REG_READ=>RegisterToRead, STATUS=>Er
rorCode)
```

Representation in ST

Representation:

```
CREAD_REG_Instance (SLAVEREG:=OffsetAddress,    NO_REG:=NumberOfRegiste
rs, ADDRFLD:=DataStructure,    REG_READ=>RegisterToRead, STATUS=>ErrorC
ode) ;
```

Parameter Description

Description of input parameters:

Parameter	Data Type	Description
SLAVEREG	DINT	Offset address of the first %MW register in the slave to be read from.
NO_REG	INT	Number of addresses to be read from slave.
ADDRFLD	WordArr5	Data structure describing the Modbus Plus address, TCI/IP address, or SY/MAX IP address.

Description of output parameters:

Parameter	Data Type	Description
REG_READ	ANY	Data to be read For the file to be read a data structure must be declared as a located variable.
STATUS	WORD	If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

Runtime Error

Table of error codes for the Communication Library *(see page 496)*

Derived Data Types

Derived Data Type WordArr5 in Modbus Plus

Element descriptions:

Element	Data Type	Description
WordArr5[1]	WORD	Least significant byte: Routing register 1 is used for address specification (routing path addresses one of five) of the destination node during network transfer. The last byte in the routing path that is not zero is the destination node. Most significant byte: Source node address. ● slot position of the module when using the Modbus Plus Port on the NOM module ● set to 0 when using the Modbus Plus Port on the CPU, (regardless of the CPU slot)
WordArr5[2]	WORD	Routing register 2
WordArr5[3]	WORD	Routing register 3
WordArr5[4]	WORD	Routing register 4
WordArr5[5]	WORD	Routing register 5

Element Description for WordArr5 with TCP/IP Ethernet

Element description for WordArr5 with TCP/IP Ethernet

Element	Data Type	Description
WordArr5[1]	WORD	Least significant byte: MBP on Ethernet Transporter (MET) mapping index Most significant byte: Slot of the NOE module
WordArr5[2]	WORD	Byte 4 (MSB) of the 32-bit destination IP address
WordArr5[3]	WORD	Byte 3 of the 32-bit destination IP address
WordArr5[4]	WORD	Byte 2 of the 32-bit destination IP address
WordArr5[5]	WORD	Byte 1 (LSB) of the 32-bit destination IP address

Element Description for WordArr5 with SY/MAX Ethernet

Element description for WordArr5 with SY/MAX Ethernet

Element	Data Type	Description
WordArr5[1]	WORD	Least significant byte: MBP on Ethernet Transporter (MET) mapping index Most significant byte: Slot of the NOE module
WordArr5[2]	WORD	Destination drop number (or set to FF hex)
WordArr5[3]	WORD	Terminator (set to FF hex)
WordArr5[4]	WORD	No significance
WordArr5[5]	WORD	No significance

Function Mode

Function Mode of the CREAD_REG Block

Although a large number of CREAD_REG function blocks can be programmed; only four read operations may be active at the same time. In this case it is irrelevant whether they are the result of this function block or others (e.g. MBP_MSTR, MSTR, READ_REG). All function blocks use one data transaction path and require multiple cycles to complete a job.

NOTE: A TCP/IP communication between a Quantum PLC (NOE 211 00) and a Momentum PLC (all TCP/IP CPUs and all TCP/IP I/O modules) is only possible, when only **one** read or write job is carried out in every cycle. If several jobs are sent per PLC cycle, the communication stops without generating an error message in the status register of the function block.

All routing information is contained in the WordArr5 data structure of the ADDRFLD input. The type function block assigned to this input is defined by the network used.

Please use:

- Modbus Plus for function block ModbusP_ADDR
- TCP/IP Ethernet for function block TCP_IP_ADDR
- SY/MAX Ethernet for function block SYMAX_IP_ADDR

NOTE: The WordArr5 data structure can also be used with constants.

NOTE: This function block puts a heavy load on the network; therefore the network load must be carefully monitored. If the network load is too high, the program logic should be reorganized in order to work with the READ_REG function block, a variation of this function block that does not operate in a continuous mode, but under command control.

Parameter Description

SLAVEREG

Start of the area in the addressed slave from which the source data is read. The source area always resides within the %MW register area.

NOTE: For slaves for a **non-**Unity Pro PLC:
The source area always resides within the 4x register area. `SLAVEREG` expects the source reference as offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).

The parameter can be entered as an address, located variable, unlocated variable or literal.

NO_REG

Number of addresses to be read from the addressed slave (1 ... 100).

The parameter can be entered as an address, located variable or unlocated variable.

REG_READ

An `ARRAY` that is the same size as the requested transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

STATUS

If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

The parameter can be entered as an address, located variable or unlocated variable.

Chapter 9
CWRITE_REG: Continuous Register Writing

Introduction

This chapter describes the `CWRITE_REG` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	98
Derived Data Types	101
Function Mode	103
Parameter Description	104

Description

Function Description

This function block writes continuously to the register area. It transfers data from the PLC via Modbus Plus, TCP/IP Ethernet or SY/MAX Ethernet to an addressed slave.

EN and ENO can be configured as additional parameters.

CREAD_REG, CWRITE_REG, READ_REG, WRITE_REG, and MBP_MSTR function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

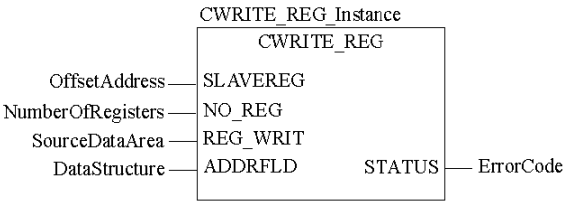
More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

NOTE: You must be familiar with the routing procedures of the network when programming a CWRITE_REG function. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the *Modbus Plus Network Planning and Installation Guide (Reference 31003525)*. If TCP/IP or SY/MAX Ethernet routing is implemented *(see Quantum with Unity Pro, TCP/IP Configuration, User Manual)*, standard Ethernet IP router products must be used.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

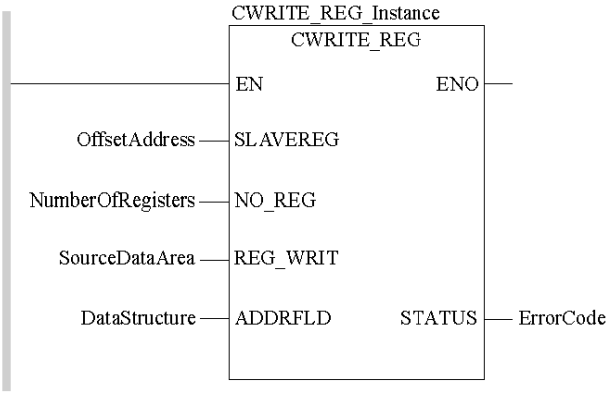
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL CWRITE_REG_Instance (SLAVEREG:=OffsetAddress,    NO_REG:=NumberOfRe
gisters, REG_WRIT:=SourceDataArea,    ADDRFLD:=DataStructure, STATUS=>E
rrorCode)
```

Representation in ST

Representation:

```
CWRITE_REG_Instance ( SLAVEREG:=OffsetAddress,    NO_REG:=NumberOfRegis
ters, REG_WRIT:=SourceDataArea,    ADDRFLD:=DataStructure, STATUS=>Erro
rCode) ;
```

Parameter Description

Description of input parameters:

Parameter	Data Type	Description
SLAVEREG	DINT	Offset address of the first %MW register in the slave to be written to.
NO_REG	INT	Number of addresses to be written from slave.
REG_WRIT	ANY	Source data (A data structure must be declared as a located variable for the source file.)
ADDRFLD	WordArr5	Data structure transferring the Modbus Plus-address, TCI/IP address, or SY/MAX-IP address.

Description of output parameters:

Parameter	Data Type	Description
STATUS	WORD	If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

Runtime Error

Table of error codes for the Communication Library *(see page 496)*

Derived Data Types

Element Description for WordArr5 in Modbus Plus

Element description for WordArr5 in Modbus Plus:

Element	Data Type	Description
WordArr5[1]	WORD	Least significant byte: Routing register 1 is used for address specification (routing path addresses one of five) of the destination node during network transfer. The last byte in the routing path that is not zero is the destination node. Most significant byte: Source node address: ● slot position of the module when using the Modbus Plus Port on the NOM module ● set to 0 when using the Modbus Plus Port on the CPU, (regardless of the CPU slot)
WordArr5[2]	WORD	Routing register 2
WordArr5[3]	WORD	Routing register 3
WordArr5[4]	WORD	Routing register 4
WordArr5[5]	WORD	Routing register 5

Element Description for WordArr5 with TCP/IP Ethernet

Element description for WordArr5 with TCP/IP Ethernet

Element	Data Type	Description
WordArr5[1]	WORD	Least significant byte: MBP on Ethernet Transporter (MET) mapping index Most significant byte: Slots of the NOE module
WordArr5[2]	WORD	Byte 4 (MSB) of the 32-bit destination IP address
WordArr5[3]	WORD	Byte 3 of the 32-bit destination IP address
WordArr5[4]	WORD	Byte 2 of the 32-bit destination IP address
WordArr5[5]	WORD	Byte 1 (LSB) of the 32-bit destination IP address

Element Description for WordArr5 with SY/MAX Ethernet

Element description for WordArr5 with SY/MAX Ethernet

Element	Data Type	Description
WordArr5[1]	WORD	Least significant byte: MBP on Ethernet Transporter (MET) mapping index Most significant byte: Slot of the NOE module
WordArr5[2]	WORD	Destination drop number (or set to FF hex)
WordArr5[3]	WORD	Terminator (set to FF hex)
WordArr5[4]	WORD	No significance
WordArr5[5]	WORD	No significance

Function Mode

Function Mode of the CWRITE_REG Block

Although a large number of CWRITE_REG function blocks can be programmed, only four write operations may be active at the same time. In this case it is irrelevant whether they are the result of this function block or others (e.g. MBP_MSTR, MSTR, WRITE_REG). All function blocks use one data transaction path and require multiple cycles to complete a job.

If several CWRITE_REG function blocks are used within an application, they must at least differ in the values of their NO_REG or REG_WRIT parameters.

NOTE: A TCP/IP communication between a Quantum PLC (NOE 211 00) and a Momentum PLC (all TCP/IP CPUs and all TCP/IP I/O modules) is only possible, when only **one** read or write job is carried out in every cycle. If several jobs are sent per PLC cycle, the communication stops without generating an error message in the status register of the function block.

All routing information is contained in the WordArr5 data structure of the ADDRFLD input. The type function block assigned to this input is defined by the network used.

Please use:

- Modbus Plus for function block ModbusP_ADDR
- TCP/IP Ethernet for function block TCP_IP_ADDR
- SY/MAX Ethernet for function block SYMAX_IP_ADDR

NOTE: The WordArr5 data structure can also be used with constants.

NOTE: This function block puts a heavy load on the network; therefore the network load must be carefully monitored. If the network load is too high, the program logic should be reorganized, in order to work with the WRITE_REG function block, a variation of this function block that does not operate in a continuous mode, but under command control.

Parameter Description

SLAVEREG

Start of the area in the addressed slave to which the source data is written. The source area always resides within the %MW address area.

NOTE: For slaves for a **non-**Unity Pro PLC:
The destination area always resides within the 4x register area. `SLAVEREG` expects the target address as an offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).
The parameter can be entered as an address, located variable, unlocated variable or literal.

NO_REG

Number of registers to be written to slave processor (1 ... 100). The parameter can be entered as an address, located variable, unlocated variable or literal.

STATUS

If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

The parameter can be entered as an address, located variable or unlocated variable.

REG_WRIT

An `ARRAY` that is the same size as the requested transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

Chapter 10
DATA_EXCH: Exchanging Data between Applications

Subject of this Chapter

This chapter describes the DATA_EXCH function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	106
Assisted Entry Screen	111
Example of How to Use a Fipway Network	113
Example of How to Enable/Disable HTTP or FTP/TFTP Services	115

Description

Function Description

The DATA_EXCH function is used to perform data transfers from one device to another with M340, M580 and Premium CPU:

- data transmission
- data reception
- data transmission then reception

NOTE:

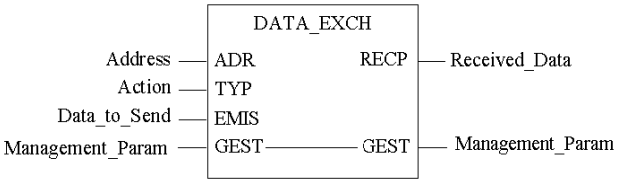
The type of request that can be sent is dependent on the type of CPU:

- On Modicon M340 and M580 CPU, this function can be used to send Modbus requests to another device. Modbus Application Protocol, is described in the **http://www.modbus.org** web site.
- On Premium CPU, this function can be used to send UNI-TE requests to another device. For this, it is necessary to make sure the data sent constitutes the entire frame required by the protocol being used.

The additional parameters EN and ENO can be configured.

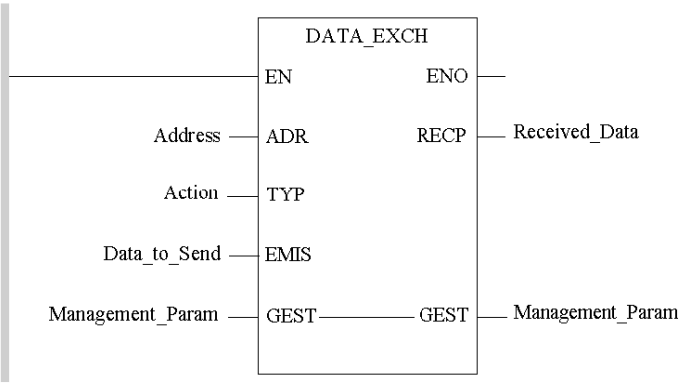
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
DATA_EXCH Action, Data_to_Send, Management_Param,
Received_Data
```

ST Representation

Representation:

```
DATA_EXCH(Address, Action, Data_to_Send, Management_Param,
Received_Data);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0.. 5] OF INT for Premium ARRAY [0..7] OF INT for Modicon M340 and M580	Address of the destination entity of the exchange. If the Action parameter is send/receive type, broadcast addresses are prohibited. For Premium CPU: ● Use the ADDR *(see page 83)* function. For Modicon M340 CPU: ● Use the ADDM *(see page 71)* function. ● The destination entity of the exchange is an array of 8 words: %MWx:8. %MWx:8: Initialized by the ADDM conversion block. **%MW0:8:=ADDM('0.3.0.0')** For Modicon M580 CPU: ● Address of the destination entity of the exchange operation, result of ADDMX *(see page 77)* function.
TYP	INT	Type of action to perform. For Premium CPU, the possible values are: ● 1: transmission followed by await reception (this action is not possible in slave Uni-Telway) ● 2: transmission only ● 3: receive mode For Modicon M340 and M580 CPU, the possible value is: ● 1: transmission followed by await reception
EMIS	ARRAY [n..m] OF INT	Integer table to be sent to the destination device of the request. **NOTE:** Verify that a minimum length of 1 element exists, even if there is no data to send (code 3 for action, for example). **NOTE:** Verify that the length of the data to be sent (in bytes) is assigned to the fourth word of the management table before launching the function, in order for this to be correctly executed.

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0..3] OF INT	Exchange management table consisting of the following words: ● Rank 1 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: exchange number ❍ Least significant byte: activity bit (rank 0) and cancel bit (rank 1) **NOTE:** The cancel bit is only available for Modicon M340 and M580 PLCs. This is another way to cancel the communication function than using the CANCEL *(see page 85)* EF. ● Rank 2 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: operation report ❍ Least significant byte: communication report ● Rank 3 word: a word managed by the user which defines the maximum response time using a time base of 100 ms. ● Rank 4 word: a word managed by the system which defines the length of the exchange. For detailed information, refer to section *Structure of the Management Parameters (see page 40)*.

The following table describes the output parameters:

Parameter	Type	Comment
RECP	ARRAY [n..m] OF INT	Integer table containing the data received during receive mode or a send/receive action. The size of the data received (in bytes) is written automatically by the system in the fourth word of the management table *(see page 46)* **NOTE:** Even if the action is a transmission only, verify that a RECP table exists with a minimum size of 1.

Modicon M340 Services

On Modicon M340 PLCs, the user can encode a private and free protocol in order to send any Modbus request. It offers the possibility on Modbus media to send or receive a free byte string.

The DATA_EXCH EF is useable over all communication ports except for the CANopen port. The limitations for the SendBuffer length are given by the characteristics of the destination port:

The following table describes the length for all communication ports:

Port	Length
Ethernet (NOE or Embedded port)	1 Kb
Modbus	256 bytes

The length of the message to be sent or received is indicated in the fourth word of the management table *(see page 46)*.

The ADDM conversion block is used to indicate the destination of the request.

See the ADDM *(see page 71)* function.

NOTE: Specfify TCP.MBS keyword when the DATA_EXCH EF is used for Modbus commands over Ethernet.

All other Ethernet entities are not supported. The DATA_EXCH EF does not manage directly a connection over TCP.

For open user protocol over Modbus, the following syntax is accepted:

- **rack.module** - Module server
- **rack.module.channel.device.MBS** - Modbus protocol
- **rack.module.channel.device** - User protocol

Modicon M340 Service Example

Objective: Write to a single register %MW100, Length := 5

(* REQUEST WRITE SINGLE REGISTER %MW100 Length := 5 *)

(* Data_to_send = Modbus request encoding *)

(* Byte 1 = Register Address Hi = 0 ; Byte 0 = Function code = 06 *)

Data_to_Send[0] := 6;

(* Byte 3 = Register Value Hi ; Byte 2 = Register Address Lo = 100 *)

Data_to_Send[1] := (RegisterValue & 16#FF00) + 100;

(* Byte 5 = unused; Byte 4 = Register Value Lo)

Data_to_Send[2] := RegisterValue & 16#FF;

IF ((Management_Param[ACTIVITY] & 1) = 0) THEN

Management_Param[LENGTH] := 5; (* LENGTH RQ WRITE *)

DATA_EXCH (ADDM('0.0.0.1'), 1, Data_To_Send, Management_Param, Received_Data);

END_IF;

NOTE: The MODBUS bus is BIG-ENDIAN and P-UNIT words are LITTLE-ENDIAN. For some queries, perform a conversion.
The ROL instruction can be used:
Value_read := ROL(Received_Data[1], 8); (* CONVERT BIG/LITTLE ENDIAN *)

Assisted Entry Screen

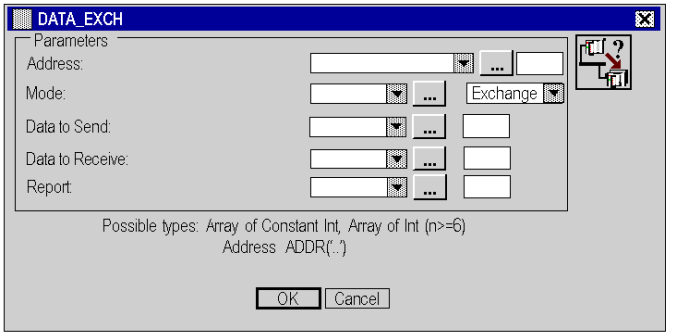
At a Glance

For this communication function, you can call up the assisted entry screen. Please note that the assisted entry screen is not available for Modicon M340.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an assisted entry screen of the function:



Address

For Premium PLCs, the types of object possible are:

- `ADDR(STRING)`
- `ARRAY [0..5] OF INT`

NOTE: If you enter a value directly in the field, the assisted address entry button turns gray.

Mode

Possible choices:

- 1 :Exchange
- 2 : Transmission
- 3 : Reception

NOTE: If you use the entry field instead of the menu, you can enter a variable of the `INT` type, either located or not.

Data to Send

The variable for transmission is an integer array. This array can be located or not.

Data to be Received

The reception zone is an array of integers, that can be located or not. The size of this array depends on the number of objects to be received.

Report

The report is an array of 4 integers, that can be located or not.

NOTE: Do not use several identical memory areas for report tables; otherwise, the variable read function may not work.

Example of How to Use a Fipway Network

At a Glance

Let us suppose we want two PLC stations to communicate on a Fipway network. Station 1 must send the data (20 bytes located in %MW70:10) to station 2, which receives it in a table located in %MW80:10. Each PLC has a management table starting with the address %MW90.

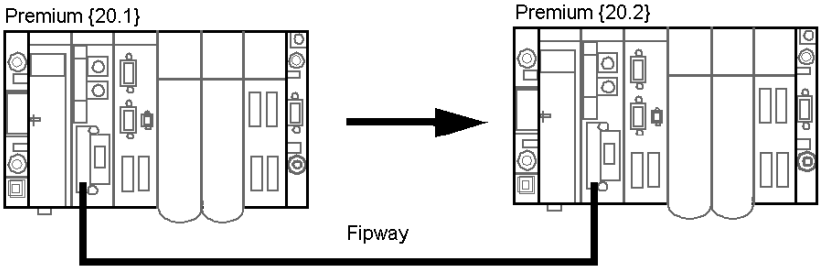
For station 1, the word %MW93 must contain the length of the data to be sent, or 20 (10 words to be sent).

For station 2, the word %MW93 is set to 0 before the exchange, in order to see the number of characters received at the end of the exchange.

The DATA_EXCH function requires both PLCs to be programmed, one to send, the other to receive.

Illustration

The two stations are connected via a Fipway network:



Transmission Programmed in Station 1

Programming in ST:

```
IF RE(%I0.3.2) AND NOT %MW90.0 THEN
   (* initialization of the data to send *)
   %MW93:= 20;
   (* communication function *)
   DATA_EXCH(ADDR('{20.2}APP'),2,%MW70:10,%MW90:4,%MW80:1);
END_IF;
```

Request parameters:

Parameters	Description
ADDR('{20.2}APP')	• 20 : network • 2 : station • APP: application of station 2
2	Type of communication: transmission
%MW70:10	Data to send
%MW90:4	Management table
%MW80:1	Reception zone for responses. Here, there is no data to receive, but a word must nevertheless be reserved (compulsory).

Reception Programmed in Station 2

```
IF RE(%I0.3.4) AND NOT %MW90.0 THEN
   (* initialization of the data to receive *)
   %MW93:= 0;
   (* communication function *)
   DATA_EXCH(ADDR('{20.1}APP'),3,%MW70:1,%MW90:4,%MW80:10);
END_IF;
```

Request parameters:

Parameters	Description
ADDR('{20.1}APP')	• 20 : network • 1 : station • APP: application of station 1
3	Type of communication: reception
%MW70:1	Data to send. Here there is none, but at least one word must be reserved.
%MW90:4	Management table
%MW80:10	Reception zone: 10 words are provided from station 1.

Example of How to Enable/Disable HTTP or FTP/TFTP Services

Description

An `Enable Disable HTTP or FTP/TFTP Services` operation changes the enabled state of the module's HTTP or FTP/TFTP services.

Network Implementation

An `Enable Disable HTTP or FTP/TFTP Services` operation can be performed on TCP/IP Ethernet networks.

Implementing the DATA_EXCH Block

A `DATA_EXCH` *(see page 106)* block can be used to change the state of the FTP/TFTP service or the HTTP service while the application is running. The `DATA_EXCH` block cannot change the state of the HTTP or FTP/TFTP services if the service was disabled using one of the configuration tools.

NOTE: It is although recommended to use the ETH_PORT_CTRL *(see page 119)* EFB instead of `DATA_EXCH` function to manage security on M340 and M580.

NOTE: Ethernet services of the embedded port of M580 can not be modified using the DATA_EXCH block. To do this, use the ETH_PORT_CTRL *(see page 119)* block.

Configuring the Address Variable

The Address variable identifies the local address of the module for which FTP/TFTP and HTTP will be enabled or disabled

Variable	Type	Description
Address	ARRAY [0..5] of INT for Premium ARRAY [0..7] of INT for M340	Address of the destination entity of the exchange. If the Action parameter is send/receive type, broadcast addresses are prohibited. For Premium: ● Use the **ADDR** function. ● Example: ADDR('0.4.0.SYS') For M340: ● Use the **ADDM** function. ● The destination entity of the exchange is an array of 8 words. ● Example: ADDM('0.4.0') **NOTE:** In each example, the address is: ● rack = 0 ● slot = 4 ● channel = 0

Configuring the Action Variable

The Action variable identifies the function type for the DATA_EXCH function block:

Variable	Type	Description	Value (hex)
Action	INT	1 = Transaction followed by await reception	16#0001

Configuring the Data_to_Send Variable

The Data_to_Send variable is an ARRAY [0..3] of INT and includes the following:

Variable	Description	Value (hex)
Data_to_Send[0]	• High byte = 16#00 • Low byte = request code (16#5A)	16#005A
Data_to_Send[1]	• High byte = 16#01 • Low byte = subrequest code (16#6E)	16#016E
Data_to_Send[2]	• Low byte = Ethernet service command (16#xx) on M340/M580. 16#00 on Premium • High byte = Ethernet service command (16#xx) on Premium. 16#00 on M340/M580 The values of the Ethernet service command can be: • 16#00 - No change • 16#01 - Enable FTP only • 16#02 - Disable FTP only • 16#10 - Enable HTTP only • 16#20 - Disable HTTP only • 16#11 - Enable FTP; Enable HTTP • 16#12 - Enable HTTP; Disable FTP • 16#21 - Disable HTTP; Enable FTP • 16#22 - Disable FTP; Disable HTTP	16#00xx on M340/M580 16#xx00 on Premium
Data_to_Send[3]	• High byte = Reserved (16#00) • Low byte = Reserved (16#00)	16#0000

Configuring the Management_Param Variable

The Management_Param variable is an ARRAY [0..3] of INT and includes the following:

Variable	Description	Value (hex)
Management_Param[0]	• High byte = Exchange number, filled by the system • Low byte = Activity bit, filled by the system	-
Management_Param[1]	• High byte = Operation report *(see page 44)*, filled by the system • Low byte = Communication report *(see page 43)*, filled by the system	-
Management_Param[2]	The time out value can be set to any value. Because the time-out base is 100 ms, in this example the time-out is set to 10 seconds.	16#0064
Management_Param[3]	• High byte = Length (16#00) • Low byte = Length (16#08) **NOTE:** The Length needs to be set to a value of 8 bytes.	16#0008

Viewing the Response

Use a Unity Pro Animation Table to display the response. The response is contained in the Received_Data variable, which is an ARRAY [0..3] of INT:

Variable	Value (hex)
Example of a successful response:	
Received_Data[0]	16#005A (M340 Modbus) 16#008A (Premium UNITE)
Received_Data[1]	16#xxFE (Success)
Received_Data[2]	16#00yy **NOTE:** The word created with the 16#yyxx has got the same meaning as ETH_SCE_STATUS *(see page 120)* output of ETH_PORT_CTRL *(see page 119)* EFB.
Received_Data[3]	16#0000
1. The detected error code appears in the format FD xx xx, where xx xx can be the following: • **50 86**: Busy: When the request occurs with 2 seconds of the previous request, regardless of the previous request success or detected error. • **40 01**: Same State: When the request is called to change the sate of both HTTP and FTP/TFTP to their currently existing state. • **20 04**: Invalid Data: When the data in the request is not in the form specified by the request type. • **50 69**: Disabled: When FTP/TFTP or HTTP is already disabled by Unity Pro and the request attempts to change the state of the disabled service. **NOTE:** In case of multiple detected errors, the order of priority in the display is Disabled, Invalid Data, Same State, Busy.	

Variable	Value (hex)
Example of an unsuccessful response:	
Received_Data[0]	16#005A (M340 Modbus) 16#008A (Premium UNITE)
Received_Data[1]	16#xxFD (Detected error)[1]
Received_Data[2]	16#00xx[1]
Received_Data[3]	16#0000

1. The detected error code appears in the format FD xx xx, where xx xx can be the following:
- **50 86**: Busy: When the request occurs with 2 seconds of the previous request, regardless of the previous request success or detected error.
- **40 01**: Same State: When the request is called to change the sate of both HTTP and FTP/TFTP to their currently existing state.
- **20 04**: Invalid Data: When the data in the request is not in the form specified by the request type.
- **50 69**: Disabled: When FTP/TFTP or HTTP is already disabled by Unity Pro and the request attempts to change the state of the disabled service.

NOTE: In case of multiple detected errors, the order of priority in the display is Disabled, Invalid Data, Same State, Busy.

Chapter 11
ETH_PORT_CTRL: Activating or Deactivating a Protocol

ETH_PORT_CTRL: Executing a Security Command in an Application

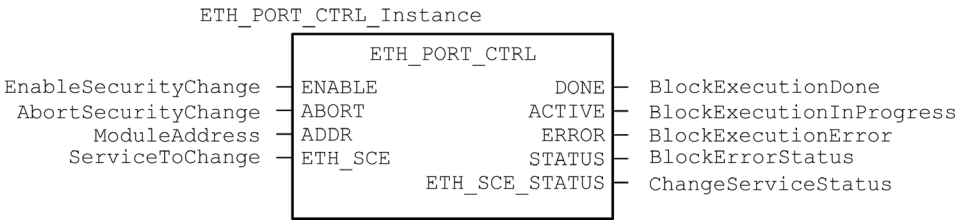
Function Description

Use the ETH_PORT_CTRL function block to control the FTP TFTP, HTTP, and DHCP / BOOTP protocols when they are enabled in the Unity Pro **Security** screen *(see Modicon M580, BMENOC0301/0311 Ethernet Communications Module, Installation and Configuration Guide)*. (By default, these protocols are disabled.) For cyber security reasons (to help protect data against requests to modify in the monitoring mode), map the inputs on variables and on unlocated variables in which the HMI property is disabled (the variable is not in the data dictionary).

The additional parameters EN and ENO may also be configured.

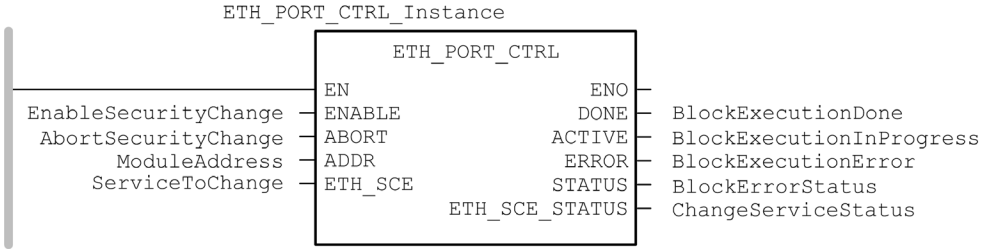
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

```
CAL ETH_PORT_CTRL_Instance (ENABLE := EnableSecurityChange,  ABORT :=
AbortSecurityChange, ADDR := ModuleAddress, ETH_SCE := ServiceToChange,
DONE => BlockExecutionDone,  ACTIVE => BlockExecutionInProgress,  ERROR
=> BlockExecutionError,  STATUS => BlockErrorStatus,  ETH_SCE_STATUS =>
ChangeServiceStatus)
```

ST Representation

```
ETH_PORT_CTRL_Instance (ENABLE := EnableSecurityChange,  ABORT :=
AbortSecurityChange, ADDR := ModuleAddress, ETH_SCE := ServiceToChange,
DONE => BlockExecutionDone,  ACTIVE => BlockExecutionInProgress,  ERROR
=> BlockExecutionError,  STATUS => BlockErrorStatus,  ETH_SCE_STATUS =>
ChangeServiceStatus);
```

Description of Parameters

This table describes the input parameters:

Parameter	Type	Comment
ENABLE	BOOL	Set to 1 to enable the operation.
ABORT	BOOL	Set to 1 to abort the currently active operation.
ADDR	ANY_ARRAY_INT	This array contains the address of the entity for which you want to change the security state, which is the result of the ADDMX *(see page 77)* or ADDMX or ADDM function *(see page 71)*. For example: ● ADDM('0.0.10') for a M580 CPU ● ADDM('0.3.0') for a BMENOC0301/11 plugged in slot 3 of main rack
ETH_SCE	WORD	For each protocol, use these binary values to control the protocol: ● 00: The protocol is unchanged. ● 01: Enable the protocol. ● 10: Disable the protocol. ● 11: reserved **NOTE:** A value of 11 reports a detected error in ETH_SCE_STATUS. These bits are used for the different protocols: ● 0, 1: FTP ● 2, 3: TFTP (Only available for Modicon M580) ● 4, 5: HTTP ● 6, 7: DHCP / BOOTP ● 8...15: reserved (value = 0)
(1) To address a module in the local rack, enter 0.0.10 (CPU main server address).		

This table describes the output parameters:

Parameter	Type	Comment
DONE	BOOL	Operation completed indication. Set to 1 when the execution of the operation is completed successfully.
ACTIVE	BOOL	Operation in progress indication. Set to 1 when the execution of the operation is in progress.
ERROR	BOOL	Set to 1 if an error is detected by the function block.
STATUS	WORD	Code providing the detected error identification *(see page 498)*.
ETH_SCE_STATUS	WORD	For each protocol, these values contain the response to any attempt to enable or disable the FTP, TFTP, HTTP, or DHCP / BOOTP protocols: ● 0: command executed ● 1: command not executed Reasons for not executing the command can be: ● The communication service has been disabled by the configuration. ● The communication service is already in the state requested by the command (**Enabled** or **Disabled**). ● The communication service (x) is not supported by the module or is a non-existing service. These bits are used for the different protocols: ● 0: FTP ● 1: TFTP ● 2: HTTP ● 3: DHCP / BOOTP ● 4 ... 15: reserved (value = 0)

Execution Type

Synchronous:

When used on the following M580 CPU modules, the ETH_PORT_CTRL function block is executed ***synchronously***. As a result, the DONE output turns **ON** as soon as the ENABLE input is set to **ON**. In this case, the ACTIVE output remains **OFF**.

- BMEP581020
- BMEP582020
- BMEP582040
- BMEP583020
- BMEP583040
- BMEP584020
- BMEP584040
- BMEP585040
- BMEP586040

- BMEH582040*
- BMEH584040*
- BMEH586040*

  * In BMEH58•040 Hot Standby CPUs, verify that the ETH_PORT_CTRL function block is executed equally on both primary and standby CPUs.

Asynchronous:

When used on the following modules, the ETH_PORT_CTRL function block is executed ***asynchronously*** and may take several cycles until the DONE output turns **ON**. Therefore, the ACTIVE output is set to **ON** until the completion of the ETH_PORT_CTRL function block.

- **M340 modules**:
  - BMXNOC0401
  - BMXNOE0100
  - BMXNOE0110
- **M580 modules**:
  - BMENOC0301/11

How to Use the ETH_PORT_CTRL EFB

Use the `ETH_PORT_CTRL` EFB:

Step	Action
1	Set the bits of the services you want to activate in `ETH_SCE`.
2	Set `ENABLE` input to activate the EFB.
3	`ENABLE` input should be an OR between a pulse command and the `ACTIVE` output of the EFB.
4	Check `STATUS` output value: • `STATUS`<>0: There is a communication issue. • `STATUS` = 0: Check `ETH_SCE_STATUS`. The services for which the bits are set haven't been modified as they should be.

Chapter 12
EthPort_Control_MX: Activating or Deactivating a Protocol

EthPort_Control_MX: Deactivating or Activating a Protocol

Function Description

The EthPort_Control_MX function is used in a **Unity Pro V8.0** version, for a Modicon M580 CPU embedded Ethernet, to deactivate or activate the following protocols:

- FTP
- TFTP
- HTTP

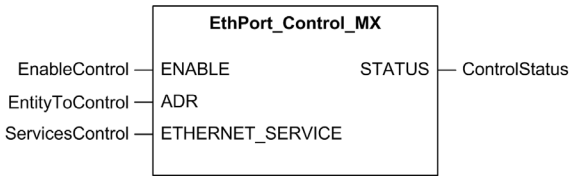
Enable the protocol in the configuration before the EthPort_Control_MX function is used to deactivate or re-activate the protocol.

If a protocol is not enabled in the configuration, the EthPort_Control_MX function does not apply any change and an error is reported.

The additional parameters EN and ENO can be configured.

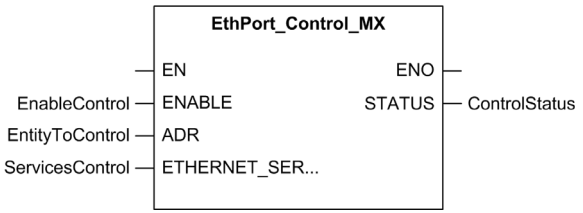
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
EthPort_Control_MX (ENABLE:=EnableControl, ADR:=EntityToControl,
ETHERNET_SERVICE:=ServicesControl)
ST ControlStatus
```

ST Representation

Representation:

```
ControlStatus:=EthPort_Control_MX (ENABLE:=EnableControl,
ADR:=EntityToControl, ETHERNET_SERVICE:=ServicesControl);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Description
ENABLE	BOOL	Enable or disable the EF.
ADR	ANY_ARRAY_INT	Array containing the address of the destination entity of the write operation, result of ADDM *(see page 71)* or ADDMX *(see page 77)* function. **NOTE:** EthPort_Control_MX is only used for Modicon M580 CPUs with a **Unity Pro V8.0** version, use ADDM('0.0.0') to enable/disable the CPU protocols services.
ETHERNET_SERVICE	WORD	Word structure: ● bit 0-1: FTP ● bit 2-3: TFTP ● bit 4-5: HTTP ● bit 6-15: reserved Bits combination meaning for each protocol: ● 00: no change in service state ● 01: enable the protocol ● 10: disable the protocol ● 11: reserved

The following table describes the output parameter:

Parameter	Type	Description
STATUS	INT	Status returned by the system: ● 3: Bad address format ● -1: Invalid Parameters ● 0: Success

Chapter 13
EXCH_QX: Exchanging Data between Applications on EIO Bus

Description

Function Description

The EXCH_QX function is used to perform data transfers (Modbus) to and from Modbus slaves connected to a Modicon X80 serial communication module (BMX NOM) located in an EIO drop.

NOTE: Modbus Application Protocol, is described in the http://www.modbus.org web site.

The additional parameters EN and ENO can be configured.

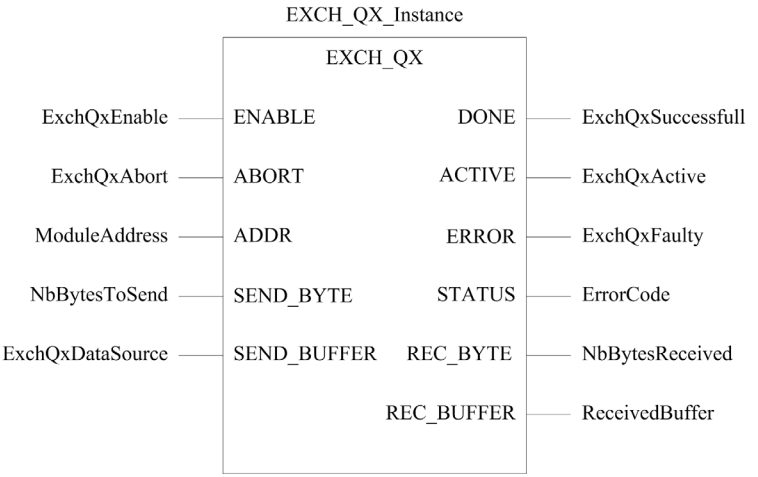
The communication function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

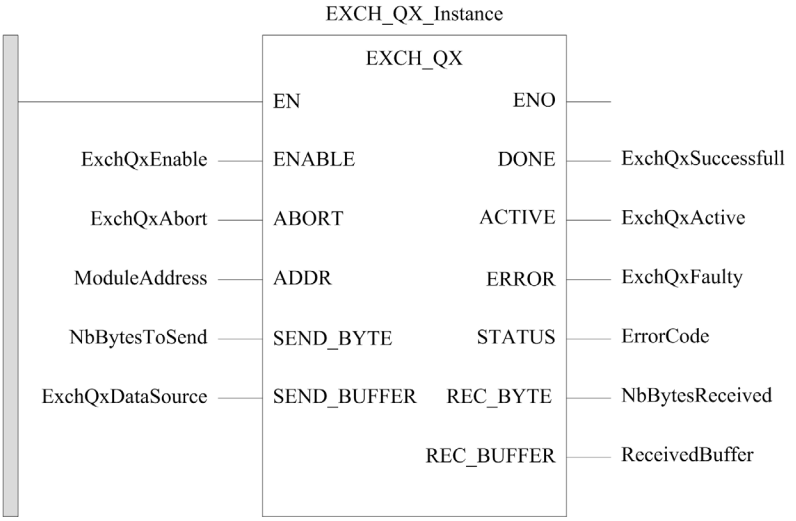
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
CAL EXCH_QX_Instance (ENABLE:=ExchQxEnable, ABORT:=ExchQxAbort,
ADDR:=ModuleAddress, SEND_BYTE:=NbBytesToSend, SEND_BUFFER:=ExchQxData-
Source, DONE=>ExchQxSuccessfull, ACTIVE=>ExchQxActive,
ERROR=>ExchQxFaulty, STATUS=>ErrorCode, REC_BYTE=>NbBytesReceived,
REC_BUFFER=>ReceivedBuffer)
```

ST Representation

Representation:

```
EXCH_QX_Instance (ENABLE:=ExchQxEnable, ABORT:=ExchQxAbort,
ADDR:=ModuleAddress, SEND_BYTE:=NbBytesToSend, SEND_BUFFER:=ExchQxData-
Source, DONE=>ExchQxSuccessfull, ACTIVE=>ExchQxActive,
ERROR=>ExchQxFaulty, STATUS=>ErrorCode, REC_BYTE=>NbBytesReceived,
REC_BUFFER=>ReceivedBuffer)
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ENABLE	BOOL	Set to 1 to perform data transfers.
ABORT	BOOL	Set to 1 to abort the current operation.
ADDR	ANY_ARRAY_INT	Array containing the Modbus slave address, result of ADDMX *(see page 77)* function.
SEND_BYTE	WORD	Number of bytes to be sent.
SEND_BUFFER	ANY_ARRAY_BYTE	Source data field. A data structure must be declared as a located variable for the source file.

The following table describes the output parameters:

Parameter	Type	Comment
DONE	BOOL	Set to 1 when the execution of the function is completed successfully.
ACTIVE	BOOL	Set to 1 when the execution of the function is in progress.
ERROR	BOOL	Set to 1 if an error is detected by the function block.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.
REC_BYTE	WORD	Number of bytes to be received from the Modbus slave.
REC_BUFFER	ANY_ARRAY_BYTE	Received data (For the file to be read a data structure must be declared as a located variable.)

ST Example

```
(* Example:
  we want to read the %MW1000 on slave number 5 (a M340 PLC for instance)
  wired to channel 0 of the NOM in slot 2 of which IP address is
192.168.10.17,
  the Quantum CRP is in slot 5 for instance.
First of all you should declare in the data editor the following
variables:
data_to_send as an ARRAY [0..4] OF BYTE (or more if you want to send more
bytes)
bytes_to_send as a WORD
slave_addres as a STRING[40]
received_data as an ARRAY [0..3] OF BYTE (or more if you need)
enable_sending as a BOOL

 and you should create an instance of EXCH_QX EFB named for instance
EXCH_QX_6 *)
(* initialize the data to send *)
data_to_send[0]:=16#03; (* Modbus function code read Holding register *)
data_to_send[1]:=16#03; (* MSB of the first register address *)
data_to_send[2]:=16#E8; (* LSB of the first register address, 16#03E8 =
1000*)
data_to_send[3]:=16#00; (* MSB of the quantity of registers to read *)
data_to_send[4]:=16#01; (* LSB of the quantity of registers to read *)

(* initialize the number of byte to send *)
bytes_to_send:=5;

(* initialize the address of the slave *)
slave_address:='1.5.1{192.168.10.17}\\0.2.0.5';
(* CRP slot 5, Drop address 192.168.10.17, NOM in slot 2 of main drop
rack. *)
```

```
EXCH_QX_6 (ENABLE := enable_sending,
  ADDR := addmx(slave_address),
  SEND_BYTE := bytes_to_send,
  SEND_BUFFER := data_to_send,
  REC_BUFFER => received_data);
(* to start the EFB EXCH_QX_6 set enable_sending, then
   received_data[0] will contain the function code sent (3)
   received_data[1] will contain the number of bytes received (2)
  received_data[2] and received_data[3] will contain the value of %MW1000
from the slave number 5*)
```

Chapter 14
INPUT_BYTE: Receiving Character Strings

Description

Function Description

INPUT_BYTE is used to send an array of bytes read request to a character mode communication module. The message received is stored in an array of bytes.

NOTE: In general, INPUT_BYTE provides same feature like INPUT_CHAR but it allows to pass a table off bytes instead of a string as output parameter. Because of this, it is possible to read a 0 (NULL) byte value in a byte stream from a serial port.

For Premium PLCs, this function is used to receive up to 4 Kbytes (120 bytes for the terminal port).

For Modicon M340 and M580 CPU, this function is used to receive up to 1024 bytes.

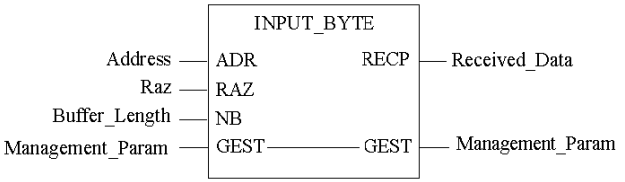
2 exclusive possibilities are available:

- Read a number of bytes: No condition should be configured.
- Read a message: A stop condition should be configured in the configuration screen.

The additional parameters EN and ENO can be configured.

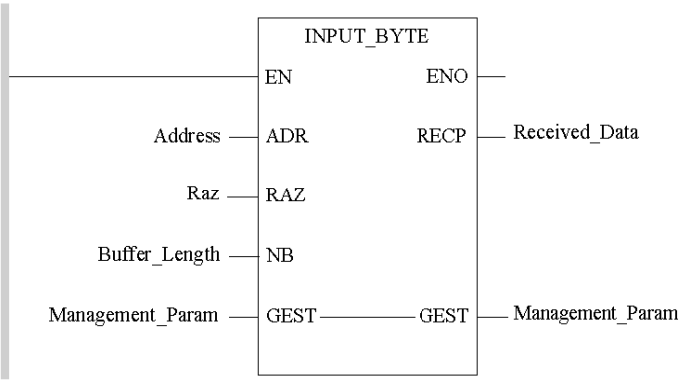
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
INPUT_BYTE Raz, Buffer_Length, Management_Param, Received_Data
```

ST Representation

Representation:

```
INPUT_BYTE(Address, Raz, Buffer_Length, Management_Param, Received_Data
);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0..5] OF INT for Premium ARRAY [0..7] OF INT for Modicon M340 and M580	For Premium PLC: ● Address of the message's receiving character mode channel is given by the ADDR *(see page 83)* function. ● Only those addresses ending in SYS are possible (for example, 0.0.0.SYS processor terminal port). For Modicon M340 PLC: ● Address of the message's receiving character mode channel is given by the ADDM *(see page 71)* function. ● The syntax of the address is of ADDM-type. Node is an optional field that may be SYS or empty (e.g. ADDM('0.0.0.SYS') equals ADDM('0.0.0'). For Modicon M580 CPU: ● Address of the destination entity of the input operation, result of ADDMX *(see page 77)* function.
RAZ	INT	Reset. This parameter is used to reset the coupler's receive memory. ● value = 0: no memory reset ● value = 1: memory reset **NOTE**: For terminal port communications of Premium PLCs, the value must be 1. **NOTE**: On Modicon M340 and M580 CPU, the INPUT_BYTE EF can be programmed with or without this parameter.
NB	INT	Length of the buffer or number of bytes to be received. ● Value = 0: Message read as soon as it is available on the channel. Here, a stop condition must be specified in the configuration screen. ● Value greater than 0: Specifies the number of bytes to be read. **NOTE**: For terminal port communications of Premium PLCs, 0 is the only value authorized. Here, the default end of message character is a carriage return (CR).

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0..3] OF INT	Exchange management table consisting of the following words: ● Rank 1 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: exchange number ❍ Least significant byte: activity bit (rank 0) and cancel bit (rank 1) **NOTE:** The cancel bit is only available for Modicon M340 and M580 PLCs. ● Rank 2 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: operation report ❍ Least significant byte: communication report ● Rank 3 word: a word managed by the user which defines the maximum response time using a time base of 100 ms. ● Rank 4 word: a word managed by the system which defines the length of the exchange. For detailed information, refer to section *Structure of the Management Parameters* *(see page 40)*.

The following table describes the output parameters:

Parameter	Type	Comment
RECP	ARRAY OF BYTE	Bytes received. This is then stored in a BYTE array. **NOTE:** The size of the data received (in bytes) is written automatically by the system in the fourth word of the management table.

Programming rules

When several INPUT_BYTE are launched simultaneously, the RAZ parameter must be set to 0 (module's receive memory not reset).

A request may be made to reset the module's memory for the next message in order to avoid receiving old data.

When the RAZ parameter is set to 1, the INPUT_BYTE function must be launched before sending the data.

On Premium PLCs, it takes several PLC cycles to receive a byte string of more than 240 bytes (the string is fragmented). It is therefore important to make sure that the management data is not modified during the processing of the function. The system receives the string consistently over several fragments.

On Modicon M340 and M580 CPU, it takes only one CPU cycle to receive a byte string of a maximum length of 1024 bytes. It is important to make sure that the management data is not modified during the processing of the function.

The Modicon M340 PLC's serial port is full duplex, so a PRINT_CHAR function can be sent even when an INPUT_BYTE function has been sent and is still pending.

Chapter 15
INPUT_CHAR: Receiving Character Strings

Subject of this Chapter

This chapter describes the `INPUT_CHAR` function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	138
Assisted Entry Screen	143
Example of Reading Character Strings via Fipway Network	145
Example of Reading Character Strings via Serial Link of Modicon M340 Processors	147

Description

Function Description

The INPUT_CHAR function is used to send a character string read request to a character mode communication module (await reception of character strings). The message received is stored in a character string.

For Premium PLCs, this function is used to receive up to 4 Kbytes (120 bytes for the terminal port).

For Modicon M340 and M580 CPU, this function is used to receive up to 1,024 bytes.

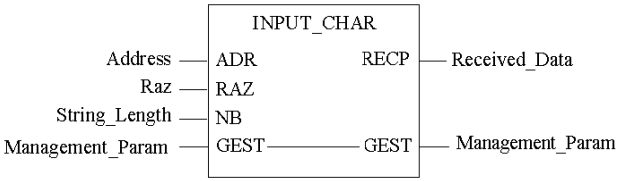
2 exclusive possibilities are available:

- Read a number of characters: No condition should be configured.
- Read a message: A stop condition should be configured in the configuration screen.

The additional parameters EN and ENO can be configured.

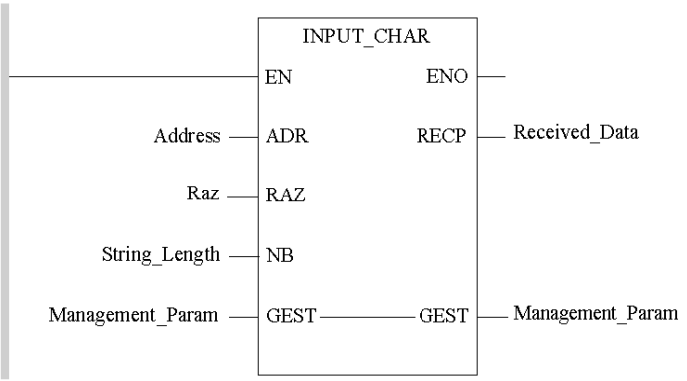
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
INPUT_CHAR Raz, String_Length, Management_Param, Received_Data
```

ST Representation

Representation:

```
INPUT_CHAR(Address, Raz, String_Length, Management_Param, Received_Data
);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0..5] OF INT for Premium ARRAY [0..7] OF INT for Modicon M340 and M580	For the Premium PLC: ● Address of the message's receiving character mode channel is given by the ADDR *(see page 83)* function. ● Only those addresses ending in SYS are possible (example: 0.0.0.SYS processor terminal port). For the Modicon M340 PLC: ● Address of the message's receiving character mode channel is given by the ADDM *(see page 71)* function. ● The syntax of the address is of ADDM ('r.m.c.node')-type. Node is an optional field that may be SYS or empty (e.g. ADDM('0.0.0.SYS') equals ADDM('0.0.0'). For the Modicon M580 CPU: ● Address of the destination entity of the input operation, result of an ADDMX *(see page 77)* function.
RAZ	INT	Reset. This parameter is used to reset the coupler's receive memory. ● value = 0: no memory reset ● value = 1: memory reset **NOTE**: For terminal port communications of Premium PLCs, the value must be 1. **NOTE**: On Modicon M340 and M580 CPU, the INPUT_CHAR EF can be programmed with or without this parameter.

Parameter	Type	Comment
NB	INT	Length of the character string or number of characters to be received. ● Value = 0: Message read as soon as it is available on the channel. Here, a stop condition must be specified in the configuration screen. ● Value greater than 0: Specifies the number of characters to be read. **NOTE**: For terminal port communications of Premium PLCs, 0 is the only value authorized. Here, the default end of message character is a carriage return (CR).

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0..3] OF INT	Exchange management table consisting of the following words: ● Rank 1 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: exchange number ❍ Least significant byte: activity bit (rank 0), cancel bit (rank 1) and immediate acknowledge bit (rank 2) **NOTE:** The cancel bit is only available for Modicon M340 and M580 PLCs. ● Rank 2 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: operation report ❍ Least significant byte: communication report ● Rank 3 word: a word managed by the user which defines the maximum response time using a time base of 100 ms. ● Rank 4 word: a word managed by the system which defines the length of the exchange. For detailed information, refer to section *Structure of the Management Parameters* *(see page 40)*.

The following table describes the output parameters:

Parameter	Type	Comment
RCP	STRING	String received. This is then stored in a character string. **NOTE:** The size of the data received (in bytes) is written automatically by the system in the fourth word of the management table.

Programming Rules

When several INPUT_CHAR are launched simultaneously, the RAZ parameter must be set to 0 (module's receive memory not reset).

A request may be made to reset the module's memory for the next message in order to avoid receiving old data.

When the RAZ parameter is set to 1, the INPUT_CHAR function must be launched before sending the data.

On Premium PLCs, it takes several PLC cycles to receive a character string of more than 240 bytes (the string is fragmented). It is therefore important to make sure that the management data is not modified during the processing of the function. The system receives the string consistently over several fragments.

On Modicon M340 and M580 CPU, it takes only one CPU cycle to receive a character string of a maximum length of 1,024 bytes. It is important to make sure that the management data is not modified during the processing of the function.

The Modicon M340 PLC's serial port is full duplex, so a PRINT_CHAR function can be sent even when an INPUT_CHAR function has been sent and is still pending.

It is possible to launch the INPUT_CHAR function before ending the characters to the PLC.

If the ending characters are used, if in the buffer there are many ending characters and the buffer hasn't been reset, each INPUT_CHAR function receives the beginning string of the buffer until it reaches the first ending character and then the buffer is removed from the read characters.

It works in the same way for reading a number of characters.

If ending characters are configured it could be possible to use the number of characters function.

NOTE: RTS/CTS management: In Character mode communication, when using EF INPUT_CHAR, the characters received on serial link are stored in a ring buffer. The size of this buffer is 1024. When the buffer is full the other characters are lost. To prevent this loss of characters it is possible to select the RTS/CTS management. In this case when the ring buffer is almost full, the CPU resets its RTS signal to stop the sending of the characters.

⚠ WARNING

LOSS OF DATA

When using RTS/CTS management the maximum number of characters of a string that are allowed to be sent by the remote equipment is 1000 (including ending character). If a string of more than 1000 characters is sent (for example 1013), the RTS will allow to receive only the first 1000 characters, while the INPUT_CHAR function will wait for the ending character indefinitely (if no timeout has been defined with the function).

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: If you want to send string of more than 1000 characters (1013 for example) you have to send the string twice, one sending of the first 1000 characters and the other for the remaining 13 for example.

Assisted Entry Screen

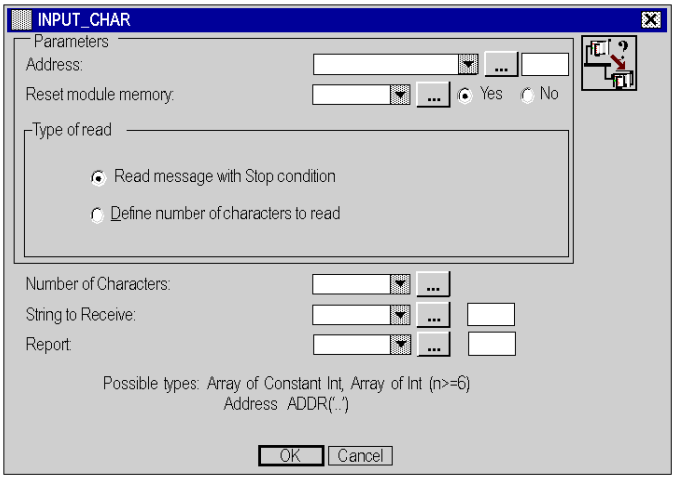
At a Glance

For this communication function, you can call up the assisted entry screen. Please note that the assisted entry screen is not available for the Modicon M340.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an example of the assisted entry screen of the function:



Address

For Premium PLCs, the possible type of object is:

- `ADDR(STRING)`
- `ARRAY [0..5] OF INT`

NOTE: If you enter a value directly in the field, the assisted address entry button turns gray.

Reset Module Memory

Possible INT type choice :

- `Yes`
- `No`

NOTE: Selecting the Yes/No buttons immediately displays the value 1 or 0.

Type of Read

The two radio buttons select the mode of operation and this choice must be made according to the configuration of the channel used.

Number of Characters

You can enter an integer, in the form of a variable or an immediate value.

String to be Received

The reception zone is a variable of the`STRING` type. The size of this variable is a function of the number of characters to be received. This variable must be declared before being used in this screen.

Report

The report is an array of 4 integers.

NOTE: Do not use several identical memory areas for report tables; otherwise, the variable read function may not work.

Example of Reading Character Strings via Fipway Network

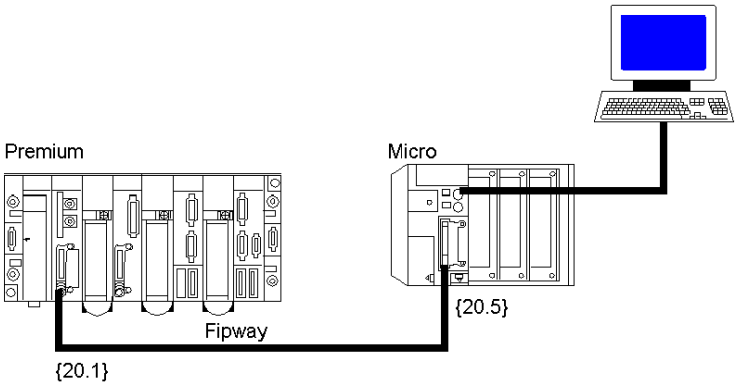
At a Glance

From a station with an address of 1 on Fipway network 20, we would like to read a character string sent by a video terminal (screen and keyboard) connected to the TER port of the PLC with a network address of 20, station 5.

This character string is stored in the `Str_1` variable, the management table of the communication function being `%MW110:4`.

Illustration

The two stations are connected via a Fipway network.



Programming

Programming in ST:

```
IF RE(%I0.3.6) AND NOT %MW110.0 THEN
   INPUT_CHAR(ADDR('{20.5}0.0.SYS'), 1, 0, %MW110:4, Str_1);
END_IF;
```

Request parameters:

Parameters	Description
ADDR('{20.5}0.0.0.SYS')	• {20.5} : network 20, station 5 • 0 : rack • 0 : module • 0 : channel 0 • SYS: system address (terminal port)
1	Reset
0	Read the entire character string
%MW110:4	Management table
Str_1	Variable of type STRING that is to receive the message

Example of Reading Character Strings via Serial Link of Modicon M340 Processors

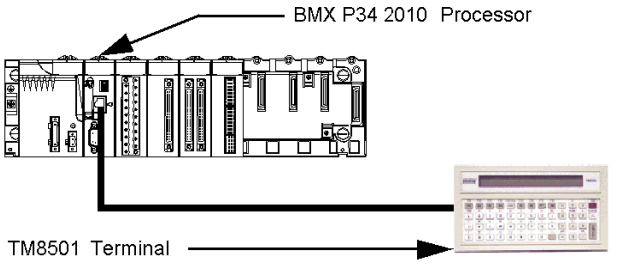
At a Glance

We would like to read a character string sent by a data entry/display terminal connected to a Modicon M340 processor's serial port.

This character string is stored in the `Str` variable, the management table of the communication function being `gestion`.

Illustration

A Modicon M340 PLC is linked to a data entry/display terminal TM8501:



Programming

Programming in ST:

```
IF (%M15) THEN
   INPUT_CHAR(ADDM('0.0.0'), 1, 0, gestion, Str);
END_IF;
```

Request parameters:

Parameters	Description
ADDM('0.0.0')	• 0 : rack • 0 : module • 0 : channel 0 • SYS: system address (optional on Modicon M340 PLCs)
1	Reset
0	Read the entire character string
gestion	Management table
Str	Variable of type STRING that is to receive the message

Chapter 16
INPUT_CHAR_QX: Receiving Character Strings on EIO Bus

Description

Function Description

The INPUT_CHAR_QX function is used to receive a character string from a serial device connected to a Modicon X80 serial communication module (BMX NOM) located in an EIO drop.

The message received is stored in a character string.

The additional parameters EN and ENO can be configured.

NOTE: The INPUT_CHAR_QX function works with a Modicon M340 BMX NOM 0200 module with a firmware version SV: 1.4 or later. With an earlier firmware version, an error (STATUS = 16#3301) is raised.

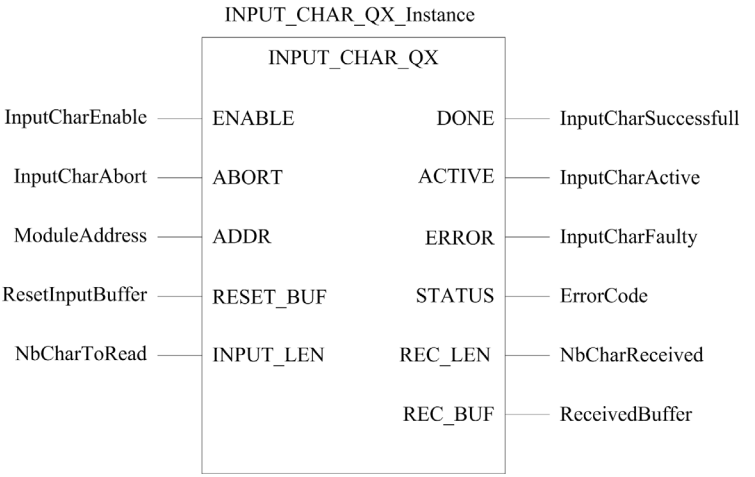
The communication function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

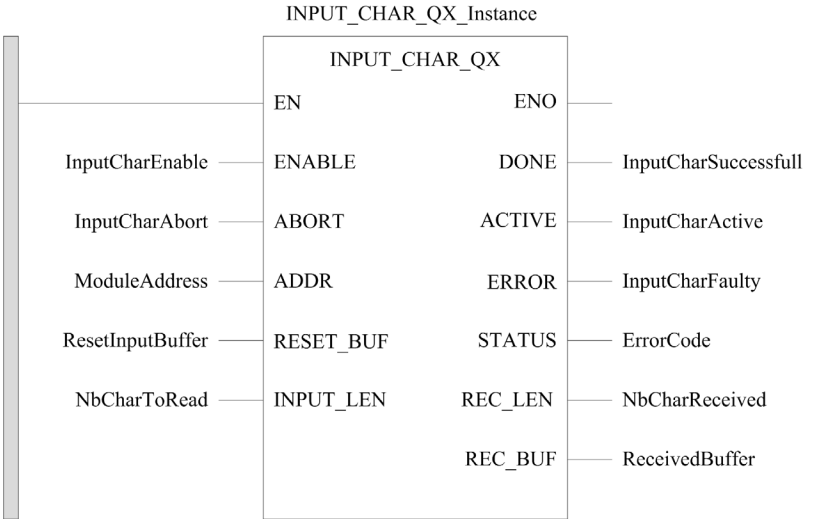
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
CAL INPUT_CHAR_QX_Instance (ENABLE:=InputCharEnable,
ABORT:=InputCharAbort, ADDR:=ModuleAddress, RESET_BUF:=ResetInput-
Buffer, INPUT_LEN:=NbCharToRead, DONE=>InputCharSuccessfull,
ACTIVE=>InputCharActive, ERROR=>InputCharFaulty, STATUS=>ErrorCode,
REC_LEN=>NbCharReceived, REC_BUF=>ReceivedBuffer)
```

ST Representation

Representation:

```
INPUT_CHAR_QX_Instance (ENABLE:=InputCharEnable, ABORT:=InputCharAbort,
ADDR:=ModuleAddress, RESET_BUF:=ResetInputBuffer,
INPUT_LEN:=NbCharToRead, DONE=>InputCharSuccessfull, ACTIVE=>InputChar-
Active, ERROR=>InputCharFaulty, STATUS=>ErrorCode,
REC_LEN=>NbCharReceived, REC_BUF=>ReceivedBuffer)
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ENABLE	BOOL	Set to 1 to receive a character string.
ABORT	BOOL	Set to 1 to abort the current operation.
ADDR	ANY_ARRAY_INT	Array containing the target Modicon M340 module address, result of ADDMX *(see page 77)* function.
RESET_BUF	BOOL	This parameter is used to reset the receiving memory: ● 0: no memory reset ● 1: memory reset
INPUT_LEN	INT	Number of characters to be received: ● 0: message read as soon as it is available on the channel. A STOP condition must be specified in the configuration screen. ● >0: number of characters to be read

The following table describes the output parameters:

Parameter	Type	Comment
DONE	BOOL	Set to 1 when the execution of the function is completed successfully.
ACTIVE	BOOL	Set to 1 when the execution of the function is in progress.
ERROR	BOOL	Set to 1 if an error is detected by the function block.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.

Parameter	Type	Comment
REC_LEN	INT	Number of characters received.
REC_BUF	ANY	Characters received.

Programming Rules

When several INPUT_CHAR_QX are launched simultaneously, the RESET_BUF parameter must be set to 0 (module's receive memory not reset).

A reset request (RESET_BUF=1) may be made to reset the module's memory for the next message in order to avoid receiving old data. When the RESET_BUF parameter is set to 1, the INPUT_CHAR_QX function must be launched before sending the data otherwise the characters already received are flushed.

It is possible to launch the INPUT_CHAR_QX function before the ending characters to the PLC.

If the ending characters are used, if in the buffer there are many ending characters and the buffer hasn't been reset, each INPUT_CHAR_QX function receives the beginning string of the buffer until it reaches the first ending character and then the buffer is removed from the read characters.

It works in the same way for reading a number of characters.

If ending characters are configured it could be possible to use the number of characters function.

NOTE: RTS/CTS management: In Character mode communication, when using EF INPUT_CHAR_QX, the characters received on serial link are stored in a ring buffer. The size of this buffer is 1024. When the buffer is full the other characters are lost. To prevent this loss of characters it is possible to select the RTS/CTS management. In this case when the ring buffer is almost full, the CPU resets its RTS signal to stop the sending of the characters.

⚠ WARNING

LOSS OF DATA

When using RTS/CTS management the maximum number of characters of a string that are allowed to be sent by the remote equipment is 1000 (including ending character). If a string of more than 1000 characters is sent (for example 1013), the RTS will allow to receive only the first 1000 characters, while the INPUT_CHAR_QX function will wait for the ending character indefinitely.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: If you want to send a string of more than 1000 characters (1013 for example) you have to send 2 strings. First send 1000 characters and then the remaining 13 characters for example.

Chapter 17
MBP_MSTR: Modbus Plus Master

Introduction

This chapter describes the `MBP_MSTR` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Block Description	154
Operational Function Codes	157
Network Control Block Structures	158
Read Data	161
Write Data	163
Get Local Statistics	165
Clear Local Statistics	166
Write Global Data	167
Read Global Data	168
Get Remote Statistics	169
Clear Remote Statistics	171
Peer Cop Health	172
Reset Optional Module	173
Read CTE	174
Write CTE	176
Send Email	178
Send Modbus Request	180
Close Connection Request	185
Read/Write Data	186
Enable / Disable HTTP or FTP/TFTP Services	187
Peer Cop Communications Health Status	189
Modbus Plus Network Statistics	191
TCP/IP Ethernet Network Statistics	196

Block Description

Function Description

You can select one of 14 available network communication operations *(see page 157)* using the MBP_MSTR function block.

The MBP_MSTR block is supported on various hardware platforms *(see page 29)*.

EN and ENO can be configured as additional parameters.

NOTE: Verify that the programmer is familiar with the routing procedures of your network when programming an MBP_MSTR function block. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the *Modbus Plus Network Planning and Installation Guide*. If TCP/IP or SY/MAX Ethernet routing is implemented *(see Quantum with Unity Pro, TCP/IP Configuration, User Manual)*, use standard EtherNet IP router products.

CREAD_REG, CWRITE_REG, READ_REG, WRITE_REG, and MBP_MSTR function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

NOTE: In FBD and LD sections, this function block can be used on the program level and with derived function blocks (DFBs). When using DFBs, verify that the parameters CONTROL and DATABUF are directly connected to the I/O pins of the DFB.

NOTE: A TCP/IP communication between a Quantum PLC and a Momentum PLC is only possible when one read or write job is carried out in every cycle. If several jobs are sent per PLC cycle, the communication stops without generating a detected error message in the status register of the function block.

NOTE: In Quantum or Premium Hot Standby, to help prevent the former standby CPU from executing communication functions (now in RUN offline state), add a condition on the status bits to disable the function if the CPU is offline.
Example:

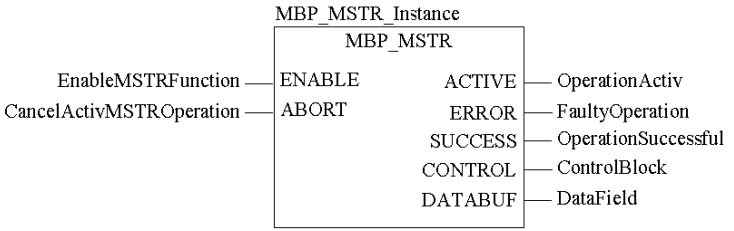
- You can send MBP_MSTR.Enable:=(HSBY_NOEPLCMSTR_ON) AND (%SW61.1) AND NOT (%SW61.0)
  or
- You can create a boolean variable, primary_state:=(%SW61.1) AND NOT (%SW61.0), and insert it for executing the section

NOTE: In M580 Hot Standby, to help prevent the former standby CPU from executing communication functions (now in RUN offline state), add a condition on the status bits to disable the function if the CPU is offline.
Example:

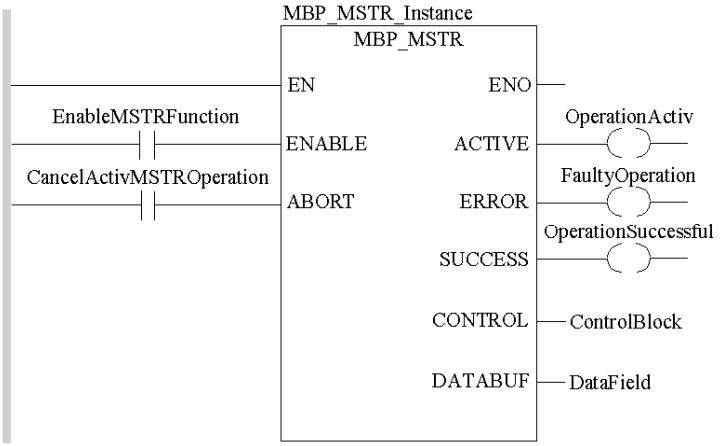
- You can set `MBP_MSTR`.Enable:=(HSBY_NOEPLCMSTR_ON AND ECPU_HSBY_1.LOCAL_HSBY_STS.RUN_PRIMARY)
  or
- Use the boolean variable ECPU_HSBY_1.LOCAL_HSBY_STS.RUN_PRIMARY as condition for executing the section

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

Representation in FBD



Representation in LD



Input Parameters

Parameter	Data Type	Description
ENABLE	BOOL	When ON, the operation specified in the first element of the CONTROL register is enabled.
ABORT	BOOL	When ON, the currently active operation *(see page 157)* is aborted.

Output Parameters

Parameter	Data Type	Description
ACTIVE	BOOL	ON when the operation is active.
ERROR	BOOL	ON when the operation is aborted without success.
SUCCESS	BOOL	ON when the operation concludes successfully.
CONTROL	WORD	This field contains the control block. The first element CONTROL[1] contains the number of the operation code of the operation to be performed *(see page 157)*. The content of the sequence register is determined by the operation. Verify that the data field is declared as a located variable. The structure of the control block differs according to the network used *(see page 158)*.
DATABUF	WORD	For operations providing data (for example a write operation) the data field is the data source. For operations receiving data (for example a read operation) the data field is the data destination. With Ethernet CTE read and write operations, the data field holds the contents of the Ethernet configuration extension table. Verify that DATABUF is defined as an array of at least 10 elements in this case. Verify that the data field is declared as a located variable.

Runtime Error

In the event of a detected error occurring during an MBP_MSTR operation, a hexadecimal error code is displayed in the CONTROL[2] register of the control block for one cycle.

NOTE: Function error codes *(see page 498)* are network-specific.

NOTE: For a list of all block error codes and values, refer to the tables of error codes for the communication library *(see page 496)*.

Operational Function Codes

Valid MBP_MSTR Function Codes

Using the MBP_MSTR block, network communication operations can be triggered via the network. As shown in the following table, each operation has a function code assigned to it. The availability of specific operations depends on both the type of network and the type of module you are using.

Function Code	Operation	Modbus Plus	TCP/IP Ethernet	SY/MAX Ethernet	CIP Ethernet
1	Write data	X	X	X	-
2	Read data	X	X	X	-
3	Get local statistics	X	X	-	-
4	Clear local statistics	X	X	-	-
5	Write global data, peer cop *(see page 167)*	X	-	-	-
6	Read global data, peer cop *(see page 168)*	X	-	-	-
7	Get remote statistics	X	X	-	-
8	Clear remote statistics *(see page 171)*	X	X	-	-
9	Peer cop health status *(see page 172)*	X	-	-	-
10	Reset optional module	-	X	X	-
11	Read CTE (Config extension)	-	X	X	-
12	Write CTE (Config extension)	-	X	X	-
13	Send E-mail *(see page 178)*	-	X	-	-
14	CIP Explicit message *(see Quantum, 140 NOC 77100 EtherNet/IP Communication Module, User Manual)*	-	-	-	X
15	Send Modbus Request *(see page 180)*	-	X	-	-
16	Close Connection Request *(see page 185)*	-	X	-	-
17	Change Modbus Plus Address *(see Modicon Quantum with Unity, Ethernet Network Modules, User Manual)*	X	-	-	-
23	Read / write data *(see page 186)*	-	X	-	-
65520 (dec) FFF0 (hex)	Enable / disable HTTP or FTP/TFTP	-	X	-	-

where:

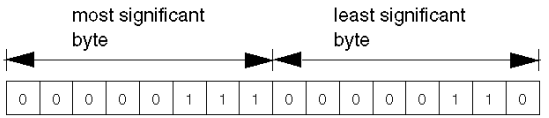
- X indicates *Yes*
- - indicates *No*

Network Control Block Structures

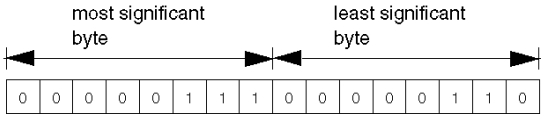
Summary

The structure of the MBP_MSTR control block varies according to the type of network you are using. Structures for Modbus Plus, TCP/IP Ethernet, and SyMax Ethernet are described below.

Control Block for Modbus Plus

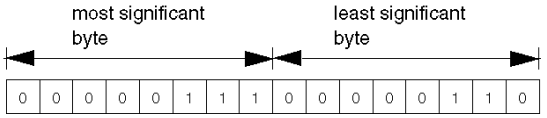
Register	Contents
CONTROL[1]	Indicates an operation that is valid for Modbus Plus
CONTROL[2]	Indicates the error status
CONTROL[3]	Indicates the length, i.e., the number of data units transferred (max. 100)
CONTROL[4]	Indicates MSTR operation-dependent information
CONTROL[5]	Routing register 1: used to specify a destination node during network transfer (routing path addresses one of five) Most significant byte: source node address, i.e., the slot for the Modbus Plus Network Options Module (NOM) When using the Modbus Plus Port on the CPU, this byte must be set to 0 (regardless of the CPU slot). Least significant byte: destination node address, i.e., a value that represents a direct or a bridge address. If there is no bridge, this value contains the destination node address. If there is a bridge, this value contains the address of the bridge. If the NOM is inserted in slot 7 on the module rack, the most significant byte of routing register 1 looks as follows (value 0x0706):  **Most significant byte** Slots 1 ... 16 **Least significant byte** Destination address (binary value between 1 and 64 (normal) or 65 to 255 (extended))
CONTROL[6]	Routing register 2, the destination node address (further bridge or Modbus Plus modules). If addressing in the previous routing register has finished, the value is set to 0.
CONTROL[7]	Routing register 3, similar to routing register 2
CONTROL[8]	Routing register 4, similar to routing register 2 (see Routing Register 2)
CONTROL[9]	Routing register 5, similar to routing register 2 (see Routing Register 2)

Control Block for TCP/IP Ethernet

Register	Contents
CONTROL[1]	Indicates an operation that is valid for TCP/IP
CONTROL[2]	Indicates the error status
CONTROL[3]	Indicates the length, i.e., the number of data units transferred (max. 100)
CONTROL[4]	Indicates `MSTR` operation-dependent information
CONTROL[5]	Routing register: used to specify a destination node during network transfer Most significant byte: source node address, i.e., the NOE slot for the NOE module When using an integrated Ethernet on the CPU, this byte must be set to 254 (FE hex) regardless of the CPU slot. Least significant byte: destination node address, i.e, a value that represents a direct or bridge address. If there is no bridge the value in the least significant byte is set to 0. If there is a bridge, this value contains the MBP for the Ethernet mapping index (MET). If the NOE is inserted in slot 7 on the module rack and the Ethernet mapping index (MET) is 6, the routing register looks as follows (value 0x0706):  **Most significant byte** Slots 1 ... 16 **Least significant byte** MBP on Ethernet Transporter (MET) mapping index
CONTROL[6]	Byte 4, MSB of the 32-bit destination IP address
CONTROL[7]	Byte 3 of the 32-bit destination IP address
CONTROL[8]	Byte 2 of the 32-bit destination IP address
CONTROL[9]	Byte 1, LSB of the 32-bit destination IP address
CONTROL[10]	Indicates `MSTR` operation-dependent information
CONTROL[11]	Indicates `MSTR` operation-dependent information

NOTE: CONTROL[10] and CONTROL[11] are used when configuring the MBP_MSTR block for a Read/Write Data operation (function code 23).

Control Block for SY/MAX Ethernet

Register	Contents
CONTROL[1]	Indicates an operation that is valid for SY/MAX
CONTROL[2]	Indicates the error status
CONTROL[3]	Indicates the length, i.e., the number of registers transferred (max. 100)
CONTROL[4]	Indicates `MSTR` operation-dependent information
CONTROL[5]	Routing register: used to specify a destination node during network transfer Most significant byte: source node address, ie.e, the slot for the NOE module Least significant byte: destination node address, i.e, a value that represents a direct or bridge address. If there is no bridge the value in the least significant byte is set to 0. If there is a bridge, this value contains the MBP for the Ethernet mapping index (MET). If NOM is inserted in slot 7 on the module rack and the Ethernet mapping index (MET) is 6, the routing register looks as follows (value 0x0706):  **Most significant byte** Slots 1 ... 16 **Least significant byte** MBP on Ethernet Transporter (MET) mapping index
CONTROL[6]	Destination drop number (or set to FF hex)
CONTROL[7]	Terminator (set to FF hex)

Read Data

Description

A read operation transfers data from a specified slave source device to a master destination device on the network. The operation uses a master transaction path, and may require several cycles to complete. To program an MBP_MSTR block to perform a write operation, use function code 2 *(see page 157)*.

NOTE: Do not attempt to program an MBP_MSTR to read to its own station address. This action causes the function block to generate an error in the CONTROL[2] register of the control block *(see page 158)*.
You can perform a read operation on a nonexistent slave register. The slave detects and logs the status, which can last for several cycles.

NOTE: For Quantum PLC communication with a Premium/Atrium PLC, confirm that the addressing is made with an offset of 1. In order to access an address object **n** of a Premium PLC, confirm that the communication function of the Quantum PLC uses the **n+1** address. IEC addressing starts at 0, and Modbus addressing starts at 1.

Network Implementation

The read operation can be performed on Modbus Plus, TCP/IP Ethernet, and SY/MAX Ethernet networks.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	2 = read data
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of registers to be read from the slave.
CONTROL[4]	Determines the %MW starting register in the slave from which the data is read, e.g., 1 = %MW1, 49 = %MW49.
CONTROL[5] ... CONTROL[9]	Routing register 1 is used to specify the address (routing path address 1 of 5) of the node during a network transfer. The last byte in the routing path that is not 0 is the destination node.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	2 = read data
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of addresses to be read from the slave.
CONTROL[4]	Determines the %MW starting register in the slave from which the data is read, e.g., 1 = %MW1, 49 = %MW49.
CONTROL[5]	Routing register • most significant byte: network adapter module slot • least significant byte: MBP on Ethernet Transporter (MET) mapping index
CONTROL[6] ... CONTROL[9]	Each address contains 1 byte of the 32-bit IP address, where the MSB is in CONTROL[6] and the LSB is in CONTROL[9].

Control Block Usage for SY/MAX Ethernet

Register	Meaning
CONTROL[1]	2 = read data
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of addresses to be read from the slave.
CONTROL[4]	Determines the %MW starting register in the slave to which the data is written, e.g., 1 = %MW1, 49 = %MW49.
CONTROL[5]	Routing register: • most significant byte: network adapter module slot • least significant byte: destination drop number
CONTROL[6] ... CONTROL[9]	terminator: FF hex

Write Data

Description

A write operation transfers data from a master source device to a specified slave destination device on the network. The operation uses a master transaction path, and may require several cycles to complete. To program an MBP_MSTR block to perform a write operation, use function code 1 *(see page 157)*.

NOTE: Do not attempt to program an MBP_MSTR to write to its own drop address. This action causes the function block to generate an error in the CONTROL[2] register of the control block *(see page 158)*.
You can perform a write operation to a nonexistent slave register. The slave detects and logs the status, which can last for several cycles.

NOTE: For Quantum PLC communication with a Premium/Atrium PLC, confirm that the addressing is made with an offset of 1. In order to access an address object **n** of a Premium PLC, confirm that the communication function of the Quantum PLC uses the **n+1** address. IEC addressing starts at 0, and Modbus addressing starts at 1.

Network Implementation

The write operation can be performed on Modbus Plus, TCP/IP Ethernet, and SY/MAX Ethernet networks.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	1 = write data
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of addresses sent to the slave.
CONTROL[4]	Determines the %MW starting register in the slave to which the data is written, e.g. 1 = %MW1, 49 = %MW49.
CONTROL[5] ... CONTROL[9]	Routing register 1 is used to specify the address (routing path address 1 of 5) of the node during a network transfer. The last byte in the routing path that is not 0 is the destination node.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	1 = write data
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of addresses sent to the slave.
CONTROL[4]	Determines the %MW start address of the slave to which the data is written.
CONTROL[5]	Routing register: • most significant byte: network adapter module slot • least significant byte: MBP on Ethernet transporter (MET) mapping index
CONTROL[6] ... CONTROL[9]	Each address contains 1 byte of the 32-bit IP address.

Control Block Usage for SY/MAX Ethernet

Register	Meaning
CONTROL[1]	1 = write data
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of addresses sent to the slave.
CONTROL[4]	Determines the %MW starting register in the slave to which the data is written, e.g., 1 = %MW1, 49 = %MW49.
CONTROL[5]	Routing register: • most significant byte: network adapter module slot • least significant byte: destination drop number
CONTROL[6] ... CONTROL[9]	terminator: FF hex

Get Local Statistics

Description

A `Get Local Statistics` operation reads the data from the local node in one cycle and does not require a master transaction path.To program an `MBP_MSTR` block to `Get Local Statistics`, use function code 3 *(see page 157)*.

Network Implementation

A `Get Local Statistics` operation can be performed on Modbus Plus and TCP/IP Ethernet networks *(see page 196)*.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	3 = get local statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of addresses to be read from local statistics (1 ... 54) **Note:** The size of databuf must be at least the size of this entry.
CONTROL[4]	First address from which the statistics table must be read (Reg1=0)
CONTROL[5]	Routing register 1 is used to specify the address (routing path address 1 of 5) of the node during a network transfer. The last byte in the routing path that is not 0 is the destination mode.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	3 = get local statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of addresses to be read from local statistics (1 ... 38) **Note:** The size of databuf must be the size of this entry.
CONTROL[4]	First address from which the statistics table must be read (Reg1=0)
CONTROL[5]	Routing register Most significant byte: Network adapter module slot
CONTROL[6] ... CONTROL[9]	Not used

Clear Local Statistics

Description

A clear local statistics operation clears the values from words 13 ... 22 in the statistics table of the local node. The operation is carried out in one cycle and does not require a master transaction path. To program an `MBP_MSTR` block to clear local statistics, use function code 4 *(see page 157)*.

Network Implementation

A clear local statistics operation can be performed on Modbus Plus and TCP/IP Ethernet networks *(see page 196)*.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	4 = clear local statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Reserved
CONTROL[4]	Reserved
CONTROL[5]	Routing register 1 is used to specify the address (routing path address 1 of 5) of the node during a network transfer. The last byte in the routing path that is not 0 is the destination mode.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	4 = clear local statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Reserved
CONTROL[4]	Reserved
CONTROL[5]	Routing register Most significant byte: network adapter module slot
CONTROL[6] ... CONTROL[9]	Reserved

Write Global Data

Description

A write global data operation transfers data to the communication processor of the current node; the data can be transmitted on the network as soon as the node receives the token and then read by all nodes connected to the local network *(see page 168)* .

A write global data operation is carried out in one cycle and does not require a master transaction path. To program an `MBP_MSTR` block to write global data, use function code 5 *(see page 157)*.

Network Implementation

A write global data operation can be performed only on Modbus Plus networks. The read and write global data operations comprise a Modbus Plus capability known as *Peer Cop*.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	5 = write global data
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of addresses to be written from state RAM into global data memory (comm processor) (1...32)
CONTROL[4]	Reserved
CONTROL[5]	If global data is sent via a NOM, enter the NOM module slot in the most significant byte of this register.

Read Global Data

Description

A read global data operation reads data from the communications processor of a node on the network that has written global data *(see page 167)*. A master transaction path is not required.

A read global data operation can take several cycles if the global data is not currently available with the nodes called. If global data is available, the operation is executed in one cycle. To program an `MBP_MSTR` block to write global data, use function code 6 *(see page 157)*.

Network Implementation

A read global data operation can be performed only on Modbus Plus networks. The read and write global data operations comprise a Modbus Plus capability known as *Peer Cop*.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	6 = read global data
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of addresses to be sent from global data memory (comm processor) (1...32)
CONTROL[4]	Displays the addresses available in the scanned node. (This register Is automatically updated.)
CONTROL[5]	The least significant byte contains the address of the node whose global data is to be read. It can be a value in the range 1 ... 64. If global data is received via a NOM, enter the NOM module slot in the most significant byte of this address.

Get Remote Statistics

Description

A get remote statistics operation can be used to read data from remote nodes on the network. With each query, the remote communications processor supplies a complete table of statistics even if the query does not refer to the entire table. It then copies only the words that you queried into identified $MW addresses.

An operation can take several cycles to complete; it does not require a master data transaction path. To program an `MBP_MSTR` block to get remote statistics, use function code 7 *(see page 157)*.

Network Implementation

A get remote statistics operation can be performed on Modbus Plus and TCP/IP Ethernet networks.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	7 = get remote statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of addresses to be read from the statistics data field (1 ... 54). **Note:** The size of `databuf` must be at least the size of this entry.
CONTROL[4]	First address from which the node statistics must be read. The number of available statistics registers cannot be exceeded.
CONTROL[5] ... CONTROL[9]	Routing address 1 ... 5 of the node. The last byte in the routing path that is not 0 is the destination node.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	7 = get remote statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of addresses to be read from the statistics data field (1 ... 38). **Note:** The size of `databuf` must be at least the size of this entry.
CONTROL[4]	First address from which the node statistics must be read. The number of available statistics registers cannot be exceeded.
CONTROL[5]	Routing register Most significant byte: network adapter module slot
CONTROL[6] ... CONTROL[9]	Each address contains 1 byte of the 32-bit IP address, where the value in CONTROL[6] is the MSB and the value in CONTROL[9] is the LSB.

Clear Remote Statistics

Description

A clear remote statistics operation clears remote-node values from words 13 ... 22 in the statistics table of the local node. It uses a master transaction path and may require several cycles to complete. To program an MBP_MSTR block to perform a clear remote statistics operation, use function code 8 *(see page 157)*.

Network Implementation

A clear remote statistics operation can be performed on Modbus Plus and TCP/IP Ethernet networks *(see page 196)*.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	8 = clear remote statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Reserved
CONTROL[4]	Reserved
CONTROL[5] ... CONTROL[9]	Routing register 1 is used to specify the address (routing path address 1 of 5) of the destination node during a network transfer. The last byte in the routing path that is not 0 is the destination mode.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	8 = clear remote statistics
CONTROL[2]	Indicates the error status
CONTROL[3]	Reserved
CONTROL[4]	Reserved
CONTROL[5]	Routing Register Most significant byte: network adapter module slot
CONTROL[6] ... CONTROL[9]	Each address contains one byte of the 32-bit IP address, where the MSB is in CONTROL[6] and the LSB is in CONTROL[9].

Peer Cop Health

Description

A peer cop health operation reads selected data from the peer cop communications health table and downloads the data to the specified %MW addresses in state RAM. To program an MBP_MSTR block to perform a clear remote statistics operation, use function code 9 *(see page 157)*.

NOTE: Peer cop health is operational only when a peer cop-based I/O scanner has been configured.

The peer cop communications health table is 12 words long; `MBP_MSTR` indexes these words with the numbers 0 ... 11.

Network Implementation

A peer cop health operation can be performed only on Modbus Plus networks.

Control Block Usage for Modbus Plus

Register	Meaning
CONTROL[1]	9 = peer cop health
CONTROL[2]	indicates the error status.
CONTROL[3]	Number of words wanted by the peer cop table (1...12)
CONTROL[4]	First word to be read from the peer cop table, where 0 = the first word and 11 = the last word)
CONTROL[5]	Routing address 1 If this is the second of two local nodes, set the value in the high byte to 1.

Reset Optional Module

Description

A reset optional module operation causes a Quantum NOE Ethernet communications module or the Ethernet port on a 140CPU65150/60 CPU module to enter a cycle that resets its working environment. To program an MBP_MSTR block to perform a reset option module operation, use function code 10 *(see page 157)*.

Network Implementation

A reset optional module operation can be performed on TCP/IP Ethernet *(see page 196)* and SY/MAX Ethernet networks.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	10 = reset optional module
CONTROL[2]	Indicates the error status
CONTROL[3]	No significance
CONTROL[4]	No significance
CONTROL[5]	Routing register *(see page 159)*
CONTROL[6] ... CONTROL[9]	No significance

Control Block Usage for SY/MAX Ethernet (CONTROL)

Register	Meaning
CONTROL[1]	10 = reset optional module
CONTROL[2]	Indicates the error status
CONTROL[3]	No significance
CONTROL[4]	No significance
CONTROL[5]	Routing register *(see page 160)*
CONTROL[6] ... CONTROL[9]	No significance

Read CTE

Description

A read CTE operation reads a specified number of bytes from the Ethernet configuration extension table in the specified buffer of PLC memory. These bytes start with a byte offset at the CTE table start. The CTE table contents are displayed in the `DATABUF` output parameter *(see page 156)*. To program an MBP_MSTR block to perform a clear remote statistics operation, use function code 11 *(see page 157)*.

Network Implementation

A read CTE operation can be performed on TCP/IP Ethernet and SY/MAX Ethernet networks.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	11 = read CTE
CONTROL[2]	Indicates the error status
CONTROL[3]	The length setting: a value from 12 to 37
CONTROL[4]	No significance
CONTROL[5]	Routing register Least significant byte = mapping index Either a value displayed in the byte of the register or is not used. or Most significant byte = network adapter module slot
CONTROL[6]... CONTROL[9]	No significance

Control Block Usage for SY/MAX Ethernet

Register	Meaning
CONTROL[1]	11 = read CTE
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of words transferred
CONTROL[4]	Byte offset in the PLC register structure, specifying from where the CTE bytes are read
CONTROL[5]	Routing register MSB: slot of the NOE module
CONTROL[6] ... CONTROL[9]	Terminator: FF hex

CTE Indicator Implementation (DATABUF)

The values in the CTE table are displayed in the DATABUF output when a CTE read operation is implemented. The registers display the following CTE data:

CTE indicator implementation (DATABUF):

Parameter	Register	Contents
Frame type	DATABUF[0]	1 = 802.3 2 = Ethernet
IP address	DATABUF[1]	First byte of the IP address
	DATABUF[2]	Second byte of the IP address
	DATABUF[3]	Third byte of the IP address
	DATABUF[4]	Fourth byte of the IP address
Lower netmask	DATABUF[5]	Most significant word
	DATABUF[6]	Least significant word
Gateway	DATABUF[7]	First byte of the gateway
	DATABUF[8]	Second byte of the gateway
	DATABUF[9]	Third byte of the gateway
	DATABUF[10]	Fourth byte of the gateway

Write CTE

Description

A write CTE operation writes the CTE configuration table from the specified data (DATABUF) to a specified Ethernet configuration extension table or to a specific slot. To program an MBP_MSTR block to perform a write CTE operation, use function code 12 *(see page 157)*.

Network Implementation

A write CTE operation can be performed on TCP/IP Ethernet *(see page 196)* and SY/MAX Ethernet networks.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	12 = write CTE
CONTROL[2]	Indicates the error status
CONTROL[3]	The length setting: a value from 12 to 37.
CONTROL[4]	No significance
CONTROL[5]	Routing register Least significant byte = mapping index Either a value displayed in the byte of the address or is not used. or Most significant byte = network adapter module slot
CONTROL[6] ... CONTROL[9]	No significance

Control Block Usage for SY/MAX Ethernet

Register	Meaning
CONTROL[1]	12 = Write CTE (Config extension table)
CONTROL[2]	Indicates the error status
CONTROL[3]	Number of words transferred
CONTROL[4]	Byte offset in the PLC address structure specifying where the CTE bytes are written
CONTROL[5]	Routing register Most significant byte = NOE module slot Least significant byte = Destination drop number

Register	Meaning
CONTROL[6]	Terminator: FF hex
CONTROL[7] ... CONTROL[9]	No significance

CTE Indicator Implementation (DATABUF)

The values in the Ethernet configuration extension table are displayed in the DATABUF output field when a write CTE operation is implemented. The registers are used to transfer the following CTE data:

CTE indicator implementation (DATABUF):

Parameter	Register	Contents
Frame type	DATABUF[0]	1 = 802.3 2 = Ethernet
IP address	DATABUF[1]	First byte of the IP address
	DATABUF[2]	Second byte of the IP address
	DATABUF[3]	Third byte of the IP address
	DATABUF[4]	Fourth byte of the IP address
Lower netmask	DATABUF[5]	Most significant word
	DATABUF[6]	Least significant word
Gateway	DATABUF[7]	First byte of the gateway
	DATABUF[8]	Second byte of the gateway
	DATABUF[9]	Third byte of the gateway
	DATABUF[10]	Fourth byte of the gateway

Send Email

Description

The electronic mail notification service allows controller-based projects to report alarms or events. The controller monitors the system and dynamically creates an electronic mail message, which alerts local or remote users.

A user-defined event or condition triggers the MSTR block to create a message. Each message uses one of three user-defined headers. Each message sent from the controller can contain text and variable information (with a maximum of 238 bytes).

The project selects the appropriate header. Each header contains:

- sender name
- list of recipients
- subject

To program an MBP_MSTR block to send email, use function code 13 *(see page 157)*.

Network Implementation

A send email operation can be performed on a TCP/IP Ethernet network.

Control Block Usage for TCP/IP Ethernet

Register	Meaning
CONTROL[1]	13 = send Email
CONTROL[2]	Indicates the email-specific error codes *(see page 510)*
CONTROL[3]	Number of words transferred
CONTROL[4]	Not used
CONTROL[5]	High byte: slot address of the NOE module or 0xFE for the 140 CPU 651 60
	Low byte: always 0
CONTROL[6] ... CONTROL[9]	Not used

DATABUF Parameter Description

Register	Contents
DATABUF 1	The mail header is the least significant byte with a value of 1, 2, or 3.
	The most significant byte contains the number (*n*) of characters in the subject, a value between 0 and 238.
DATABUF 2 ... DATABUF 119	The data (in ASCII format) that will be copied into the Email message. The first *n* characters are added to the configured Email subject. The remaining characters (2 * *N* -2 - *n*) are part of the message body, where *N* is the number of words transferred.

Send Modbus Request

At a Glance

Use MSTR operation 15 to send generic Modbus requests on the network.

NOTE: This operation is not available on Modbus Plus ports (embedded port on CPU or NOM modules) and the embedded Ethernet port on a CPU.

Block Operation

The MBP_MSTR block can send requests and receive responses up to 253 bytes long.

For the operation, refer to Block Operation *(see page 154)*:

- When the ENABLE input pin is turned ON, operation 15 begins.
- If the ABORT input pin is turned ON or if the ENABLE input pin is turned OFF, the operation ends.
- The ACTIVE output pin is ON during the operation.
- The ERROR output pin turns ON if the operation aborts without success.
- The SUCCESS output pin turns ON if the operation completes with success.
- The CONTROL and DATABUF output pins define the operation (refer to the Control Block *(see page 180)* and Data Buffer *(see page 181)*).
- EN and ENO can be configured as additional parameters.

Control Block

The format of the Control block is described in the following table:

Word	Description
CONTROL[1]	15 = Send Modbus Request
CONTROL[2]	Indicates detected error status
CONTROL[3]	DATABUF length (WORDs)
CONTROL[4]	Offset for the beginning of the response in the DATABUF (WORDs). **NOTE:** To avoid overwriting the request, the Response Offset value multiplied by 2 must be greater than the Request Length (CONTROL[10]).
CONTROL[5]	Routing register: High byte = Ethernet communication module slot Low byte = MBP on Ethernet transporter (MET) mapping index (also known as Unit ID)
CONTROL[6]	Byte 4 of the IP address (MSB)
CONTROL[7]	Byte 3 of the IP address
CONTROL[8	Byte 2 of the IP address
CONTROL[9]	Byte 1 of the IP address (LSB)

Word	Description
CONTROL[10]	Length of the DATABUF Request data (bytes)
CONTROL[11]	Length of the DATABUF Response received (bytes) **NOTE:** This is read only, it is set by option module after operation completion.

Data Buffer

The MODBUS protocol defines a simple protocol data unit (PDU) independent of the underlying communication layers.

The data buffer (DATABUF) consists of contiguous registers that include both the Modbus Request PDU and the Modbus Response PDU:

DATABUF *Data Buffer Length* is set in the CONTROL[3] word.	**Modbus Request** PDU: *Data Request Length* is set in the CONTROL[10] word.
	Modbus Response PDU: Data Response Start (*Response offset*) is set in the CONTROL[4] word. **NOTE:** To avoid overwriting the request, the Response Offset value multiplied by 2 must be greater than the Request Length (CONTROL[10]). *Data Response Length* is set in the CONTROL[11] word.

⚠ CAUTION
LOSS OF DATA Verify that the *Response Offset* is greater than the *Data Request Length*. **Failure to follow these instructions can result in injury or equipment damage.**

NOTE: Both the Request and Response must be structured in little endian order. Each word of 2 bytes of data in little endian format, where the least significant byte is stored in the smallest memory address.

Modbus Request

The **Modbus Request** PDU format is:

Byte offset	Field	Data type	Description
1	Function code	BYTE	Modbus function code
2	Request data	BYTE area	This field is function code dependent and contains information such as variable references, variable counts, data offsets, sub-function codes and so on.

Modbus Response

The **Modbus Response** PDU format is:

Byte offset	Field	Data type	Description
1	Function code	BYTE	Modbus function code
2	Response data	BYTE area	This field is function code dependent and contains information such as variable references, variable counts, data offsets, sub-function codes and so on.

Modbus Exception Response

The **Modbus Exception Response** PDU format is:

Byte offset	Field	Data type	Description
1	Function code	BYTE	Modbus function code + 80 (hex)
2	Exception code	BYTE	This field is defined in the *MODBUS Application Protocol Specification*.

Modbus Specification

The standard Modbus function codes are defined in the *MODBUS Application Protocol Specification*, V1.1b, which can be downloaded from www.modbus-ida.org.

Example of a Send Modbus Request

To read 4 contiguous Input registers (Modbus Function code 4) starting at register 100 (64 hex) in a remote device. A **Modbus Request** has to be sent on the network with the following **Modbus Response**.

Request Field Name	Value
Function code	04
Starting address, High	00
Starting address, Low	64
Number of registers, High	00
Number of registers, Low	04

Response Field Name	Value
Function code	04
Byte count	08
Register 100 value, High	00
Register 100 value, Low	01
Register 101 value, High	00
Register 101 value, Low	02
Register 102 value, High	00
Register 102 value, Low	03
Register 103 value, High	00
Register 103 value, Low	04

Using the following CONTROL word values:

- CONTROL[1] (MSTR operation) = 15
- CONTROL[2] (Error Code) = refer to CONTROL block description (read-only)
- CONTROL[3] (DATABUF length) = 11
- CONTROL[4] (Response Offset) = 5
- CONTROL[5] (Routing Register) = refer to CONTROL block description
- CONTROL[6] (IP1) = refer to CONTROL block description
- CONTROL[7] (IP1) = refer to CONTROL block description
- CONTROL[8] (IP1) = refer to CONTROL block description
- CONTROL[9] (IP1) = refer to CONTROL block description
- CONTROL[10] (Request Length) = 5
- CONTROL[11] (Response Length) = 10

the data encoding in the Data Buffer is as follows:

	Field	Value (hex)	Description
Request	DATABUF[1]	0400	Function code + Starting address, High
	DATABUF[2]	6400	Starting address, Low + Number of registers, High
	DATABUF[3]	0400	Number of registers, Low + NULL
	DATABUF[4]	0000	Null
	DATABUF[5]	0000	Null
Response	DATABUF[6]	0408	Function code + Byte count
	DATABUF[7]	0001	Register 100 value, High + Register 100 value, Low
	DATABUF[8]	0002	Register 101 value, High + Register 101 value, Low
	DATABUF[9]	0003	Register 102 value, High + Register 102 value, Low
	DATABUF[10]	0004	Register 103 value, High + Register 103 value, Low
	DATABUF[11]	0000	Null

Close Connection Request

At a Glance

Use MBP_MSTR operation 16 to close a Modbus TCP/IP connection on the network.

NOTE: This operation is not available on Modbus Plus ports (embedded port on CPU or NOM modules).

Block Operation

The MBP_MSTR block can send requests and receive responses up to 253 bytes long. For block operation, refer to the block description *(see page 154)*:

- When the ENABLE input pin is turned ON, operation 15 begins.
- If the ABORT input pin is turned ON or if the ENABLE input pin is turned OFF, the operation ends.
- The ACTIVE output pin is ON during the operation.
- The ERROR output pin turns ON if the operation aborts without success.
- The SUCCESS output pin turns ON if the operation completes with success.
- The CONTROL and DATABUF output pins *(see page 156)* define the operation.
- EN and ENO can be configured as additional parameters.

Control Block

The format of the CONTROL block is described below:

Register	Function	Description
CONTROL[1]	Operation	16 = close connection
CONTROL[2]	Error status	Indicates detected error status *(see page 498)* (read only)
CONTROL[3]	(not used)	–
CONTROL[4]	(not used)	–
CONTROL[5]	Routing Register	High byte = Ethernet communication module slot
		Low byte = MBP on Ethernet transporter (MET) mapping index (also known as Unit ID)
CONTROL[6]	IP Address	Byte 4 of the IP address (MSB)
CONTROL[7]		Byte 3 of the IP address
CONTROL[8]		Byte 2 of the IP address
CONTROL[9]		Byte 1 of the IP address (LSB)

Read/Write Data

Introduction

In a single transaction, the MSTR read and write operations can transfer data from a master source device to a specified slave destination device, then transfer data from this specified slave source to the master. It uses a master transaction path and may require several cycles to complete. To program an MBP_MSTR block to perform a combined read/write operation, use function code 23 *(see page 157)*.

The combined read/write operation can be used only with these models:

- Quantum: NOE 771 01 (3.0 or later) or NOE 771 11 (3.0 or later)
- Momentum Unity: M1E Unity CPU (1.0 or later)

Control Block Usage

Register	Content
CONTROL[1]	23 = read/write data.
CONTROL[2]	Indicates the error status.
CONTROL[3]	Number of registers to be sent to the slave.
CONTROL[4]	Specifies the %MW starting register in the slave to which the data will be written, e.g. 1 = %MW1, 49 = %MW49.
CONTROL[5]	Routing register: Most significant byte: network adapter module slot. Least significant byte: MBP on Ethernet Transporter (MET) mapping index.
CONTROL[6] ... CONTROL[9]	Each address contains 1 byte of the 32-bit IP address, where the MSB is in CONTROL[6] and the LSB is in CONTROL[9].
CONTROL[10]	Number of registers to be read from slave.
CONTROL[11]	Specifies the %MW starting register in the slave from which the data is read, e.g. 1 = %MW1, 49 = %MW49.

NOTE:

When configuring the MBP_MSTR block for a read/write data operation, note that

- The DATABUF output parameter is used to store, in the following sequence, both:
  1. the data to be written
  2. the data to be read
- The size of the DATABUF output parameter must equal the combined size of the data to be written and the data to be read; if the size is smaller, data will be overwritten and may be lost.
- Both the CONTROL and DATABUF parameters must be stored at located addresses, for example %MW addresses.

Enable / Disable HTTP or FTP/TFTP Services

Description

An `Enable Disable HTTP or FTP/TFTP Services` operation changes the enabled state of the module's HTTP or FTP/TFTP services.

Network Implementation

An `Enable Disable HTTP or FTP/TFTP Services` operation can be performed on TCP/IP Ethernet networks *(see page 196)*.

Control Block Usage for TCP/IP Ethernet

When HTTP or FTP/TFTP has been enabled using Unity Pro configuration tools *(see Modicon Quantum with Unity, Ethernet Network Modules, User Manual)*, an MSTR block can be used to change the enabled state of the service while the application is running. The MSTR block cannot change the state of the HTTP or FTP/TFTP services if the service was disabled using one of the configuration tools.

Register	Meaning
CONTROL[1]	65520 (dec) FFF0 (hex) = enable / disable HTTP or FTP/TFTP
CONTROL[2]	Indicates the error status. Possible return codes include: 0x000 (Success): MSTR block with operational code 65520 was called and the enabled state of HTTP or FTP/TFTP was changed. 0x5068 (Busy): MSTR block with operational code 65520 was called within 2 seconds of the previous call (regardless of return code from previous call). 0x4001 (Same state): MSTR block with operational code 65520 was called to change the enabled state of HTTP and FTP/TFTP to the states they were already in. 0x2004 (Invalid data): MSTR block with operational code 65520 was called and the data in the control block did not match the specifications. 0x5069 (Disabled): If the HTTP or FTP/TFTP service was already disabled via the Unity Pro interface when the MSTR block with operational code 65520 was called to change the state of the disabled service.
CONTROL[3]	Set this register to 1
CONTROL[4]	
CONTROL[5]	Module slot number and destination ID High byte = Module slot number communication module slot Low byte = Destination ID
CONTROL[6]	Request mode Bit 0 (LSB) = 1: Enable FTP/TFTP Bit 0 (LSB) = 0: Disable FTP/TFTP Bit 1 = 1: Enable HTTP Bit 1 = 0: Disable HTTP

Register	Meaning
CONTROL[7]	Set this register to 0
CONTROL[8]	
CONTROL[9]	

HTTP, FTP, and TFTP service state changes made by MSTR with operation code 65520 (dec) are overridden by the configured value when the module is power-cycled or reset and when a new application is downloaded to the module.

Here are some examples:

State Configured By Unity Pro	Action attempted using MSTR with operation code 65520) (dec)	Result
Disabled	Any	MSTR returns error code 0x5069 (service was already disabled by configuration)
Enabled	Disable	MSTR returns code 0x000 (success). ● Another MSTR block action enables the service --OR-- ● The module is reset or power-cycled --OR-- ● A new application is downloaded with the service disabled by configuration
	Enable	MSTR returns error code 0x4001 (same state). No change made.

Peer Cop Communications Health Status

Peer Cop Communications Health Status

The table containing Peer Cop status information fills 12 contiguous registers indexed with the numbers 0 ... 11 in an `MBP_MSTR` operation. Each individual bit in the table words is used to present one aspect of communications health for a specific node on the Modbus Plus network. To program an MBP_MSTR block to get Peer Cop health status, use function code 9 *(see page 157)*.

Network Implementation

A Peer Cop communications health status operation can be performed only on Modbus Plus networks.

Relation Bit Network Node

The bits of the words 0 to 3 represent the health at the global communications input of nodes 1 to 64. The bits of words 4 ... 7 represent the health of the output of a specific node.

The bits in words 8 to 11 represent the health of the input of a specific node.

Status type	Word index	Relation bit network node
Global receive	0	16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
	1	32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17
	2	48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33
	3	64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49
Send direct	4	16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
	5	32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17
	6	48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33
	7	64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49

Status type	Word index	Relation bit network node
Receive direct	8	16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
	9	32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17
	10	48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33
	11	64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49

Health Bit Status

The status of the Peer Cop health bit indicates the current communications status of its assigned node. A health bit is set when the associated node accepts input for its Peer Cop data block or when it receives a signal that another node has accepted specific output data from its Peer Cop output data block. A health bit is deleted if the associated data block does not accept any communication within the configured Peer Cop health timeout period.

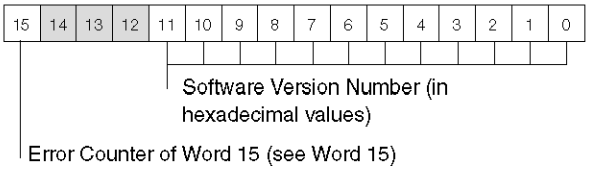
All health bits are deleted when interface command **Put Peer Cop** is executed during PLC startup. The table values become valid when the token is completely bypassed, after the **Put Peer Cop** command has been carried out. The health bit of a specific node is always 0 when the assigned Peer Cop entry is 0.

Modbus Plus Network Statistics

Modbus Plus Network Statistics

The following table shows the statistics available on Modbus Plus. You can obtain this data by running the corresponding `MBP_MSTR` operation (Modbus function code 8).

NOTE: If you edit the clear local statistics *(see page 166)* or clear remote statistics *(see page 171)* operation, only words 13 to 22 in the statistics table are cleared.

Word	Bits	Description
00	Node type ID	
	0	Unknown node type
	1	PLC node
	2	Modbus bridge node
	3	Host computer node
	4	Bridge Plus node
	5	Peer I/O node
	6 ... 15	Reserved
01	0 ... 11	Software version number as hexadecimal value (to read this, isolate bits 12-15 from the word)
	12 ... 14	Reserved
	15	Defines error counters from word 15. The most significant bit defines the use of error counters in word 15. The lower valued half of the most significant byte together with the least significant byte contain the software  version.
02		Network address of this station

Word	Bits	Description
03	MAC status variable:	
	0	Startup status
	1	Offline status indicator signals
	2	Duplicated offline status
	3	Idle status
	4	Token utilization status
	5	Work response status
	6	Token transfer status
	7	Response request status
	8	Status check of transfer
	9	Token request status
	10	Response request status
04	Peer status (LED code); indicates status of this device relative to the network:	
	0	Monitor connect operation
	32	Normal connect operation
	64	Never receives token
	96	Single station
	128	Duplicate station
05	Token transfer counter; increments each time this station receives the token	
06		Token cycle time in ms
07	LOW	Bit representation data master fail during token ownership
	HIGH	Bit representation (bitmap) program master fail during token ownership
08	LOW	Bitmap activity token ownership of the data master
	HIGH	Bitmap activity token ownership of the program master
09	LOW	Bitmap activity token ownership of the data slave
	HIGH	Bitmap activity token ownership of the program slave
10	LOW	
	HIGH	Bitmap transfer request command data slave/slave poll
11	LOW	Bitmap response transfer request program master/master poll
	HIGH	Bitmap transfer request command program slave/slave poll
12	LOW	Bitmap connect status of the program master
	HIGH	Bitmap automatic log-off of program slave
13	LOW	Pretransfer delay error counter
	HIGH	Receive buffer DMA overrun error counter

Word	Bits	Description
14	LOW	Receive counter repeat command
	HIGH	Error counter data block size
15	If bit 15 of word 1 = 0, word 15 has the following significance:	
	LOW	Error counter receiver collision abort
	HIGH	Error counter receiver alignment
	If bit 15 of word 1 = 1, word 15 has the following significance:	
	LOW	Data block error on cable A
	HIGH	Data block error on cable B
16	LOW	Error counter CRC receiver
	HIGH	Error counter wrong packet length
17	LOW	Error counter wrong link address
	HIGH	Error counter DMA underflow transfer buffer storage
18	LOW	Error counter wrong internal packet length
	HIGH	Error counter wrong MAC function code
19	LOW	Communication retry counter
	HIGH	Error counter communication failed
20	LOW	Counter package receipt successful
	HIGH	Error counter no response receipt
21	LOW	Error counter unexpected response receipt
	HIGH	Error counter unexpected path
22	LOW	Error counter unexpected response
	HIGH	Error counter skipped transaction
23	LOW	Bitmap active station table, nodes 1 through 8
	HIGH	Bitmap active station table, nodes 9 through 16
24	LOW	Bitmap active station table, nodes 17 through 24
	HIGH	Bitmap active station table, nodes 25 through 32
25	LOW	Bitmap active station table, nodes 33 through 40
	HIGH	Bitmap active station table, nodes 41 through 48
26	LOW	Bitmap active station table, nodes 49 through 56
	HIGH	Bitmap active station table, nodes 57 through 64
27	LOW	Bitmap token station table, nodes 1 through 8
	HIGH	Bitmap token station table, nodes 9 through 16
28	LOW	Bitmap token station table, nodes 17 through 24
	HIGH	Bitmap token station table, nodes 25 through 32

Word	Bits	Description
29	LOW	Bitmap token station table, nodes 33 through 40
	HIGH	Bitmap token station table, nodes 41 through 48
30	LOW	Bitmap token station table, nodes 49 through 56
	HIGH	Bitmap token station table, nodes 57 through 64
31	LOW	Bitmap table regarding existence of global data, nodes 1 through 8
	HIGH	Bitmap table regarding existence of global data, nodes 9 through 16
32	LOW	Bitmap table regarding existence of global data, nodes 17 through 24
	HIGH	Bitmap table regarding existence of global data, nodes 25 through 32
33	LOW	Bitmap table regarding existence of global data, nodes 33 through 40
	HIGH	Bitmap table regarding existence of global data, nodes 41 through 48
34	LOW	Bitmap table regarding existence of global data, nodes 49 through 56
	HIGH	Bitmap table regarding existence of global data, nodes 57 through 64
35	LOW	Bitmap receive buffer used, buffers 1 through 8
	HIGH	Bitmap receive buffer used, buffers 9 through 16
36	LOW	Bitmap receive buffer used, buffers 17 through 24
	HIGH	Bitmap receive buffer used, buffers 25 through 32
37	LOW	Bitmap receive buffer used, buffers 33 through 40
	HIGH	Counter of activated processed commands for station administration
38	LOW	Counter activation command output path 1 of the data master
	HIGH	Counter activation command output path 2 of the data master
39	LOW	Counter activation command output path 3 of the data master
	HIGH	Counter activation command output path 4 of the data master
40	LOW	Counter activation command output path 5 of the data master
	HIGH	Counter activation command output path 6 of the data master
41	LOW	Counter activation command output path 7 of the data master
	HIGH	Counter activation command output path 8 of the data master
42	LOW	Counter command processing input path 41 of the data slave
	HIGH	Counter command processing input path 42 of the data slave
43	LOW	Counter command processing input path 43 of the data slave
	HIGH	Counter command processing input path 44 of the data slave
44	LOW	Counter command processing input path 45 of the data slave
	HIGH	Counter command processing input path 46 of the data slave
45	LOW	Counter command processing input path 47 of the data slave
	HIGH	Counter command processing input path 48 of the data slave

Word	Bits	Description
46	LOW	Counter command activation output path 81 of the program master
	HIGH	Counter command activation output path 82 of the program master
47	LOW	Counter command activation output path 83 of the program master
	HIGH	Counter command activation output path 84 of the program master
48	LOW	Counter command activation output path 85 of the program master
	HIGH	Counter command activation output path 86 of the program master
49	LOW	Counter command activation output path 87 of the program master
	HIGH	Counter command activation output path 88 of the program master
50	LOW	Counter command processing input path C1 of the program slave
	HIGH	Counter command processing input path C2 of the program slave
51	LOW	Counter command processing input path C3 of the program slave
	HIGH	Counter command processing input path C4 of the program slave
52	LOW	Counter command processing input path C5 of the program slave
	HIGH	Counter command processing input path C6 of the program slave
53	LOW	Counter command processing input path C7 of the program slave
	HIGH	Counter command processing input path C8 of the program slave

TCP/IP Ethernet Network Statistics

TCP/IP Ethernet Network Statistics

A TCP/IP Ethernet module replies to local and remote statistics commands from the `MBP_MSTR` block with the the contents of the `databuf` array (see the information in the table below):

Word	Meaning		
00 to 02	MAC address e.g. MAC address 00 00 54 00 12 34 is displayed as follows:	**Word** 00 01 02	**Contents** 00 00 54 00 12 34
03	Board status (refer to the following table)		
04 and 05	Number of receiver interrupts		
06 and 07	Number of transfer interrupts		
08 and 09	Transfer timeout error count		
10 and 11	Collision detection error count		
12 and 13	Omitted packets		
14 and 15	Memory error count		
16 and 17	Number of restarts performed by the driver		
18 and 19	Receive framing error count		
20 and 21	Overflow error count receiver		
22 and 23	Receive CRC error counter		
24 and 25	Receive buffer error counter		
26 and 27	Transfer buffer error counter		
28 and 29	Transfer bin underflow counter		
30 and 31	Late collision counter		
32 and 33	Lost carrier counter		
34 and 35	Number of retries		
36 and 37	IP address e.g. the IP address 198.202.137.113 (or C6 CA 89 71) is represented as follows:	**Word** 36 37	**Contents** 89 71 C6 CA

Board Status Word Bit Definition

NOTE: It is best to view the board status word in binary format.

The following table describes the bit definitions of the board status word:

- 140 NOE 771 *x*1, versions 2.0, 3.0, 3.1, 3.3 and 3.6 or higher
- 140 NOE 771 *x*0, versions 3.0, 3.3 and 3.4 or higher

Bit #	Definition
15	0 = Link LED off 1 = Link LED on
14	0 = Appl LED off 1 = Appl LED on
13	0 = twisted pair 1 = fiber
12	0 = 10 Mbit 1 = 100 Mbit
11 ... 8	(Reserved)
7 ... 4	Module type (see table, below)
3	(Reserved)
2	0 = half duplex 1 = full duplex
1	0 = not configured 1 = configured
0	0 = not running 1 = running

NOTE: Bits are numbered from right to left, starting with bit 0 (low bit). For example, **PLC running =** 0000 0000 0000 0001 and **LED connection =** 1000 0000 0000 0000.

The following table describes the word bit definitions for board status for the:

- 140 NOE 771 *x*1, version 3.5
- 140 NOE 771 *x*0, versions 1.02 and 2.0
- 140 CPU 651 *x*0

Bit #	Definition
15 ... 12	Module type (see table below)
11	(Reserved)
10	0 = half duplex 1 = full duplex
9	0 = not configured 1 = configured
8	0 = PLC not running 1 = PLC/NOE running
7	0 = Link LED off 1 = Link LED on
6	0 = Appl LED off 1 = Appl LED on
5	0 = twisted pair 1 = fiber
4	0 = 10 Mbit 1 = 100 Mbit
3 ... 0	(Reserved)

NOTE: Bits are counted from right to left, starting with bit 0 (low bit). For example, **PLC running =** 0x0100, **Application LED =** 0x0040, and **LED Connection =** 0x0080.

Board Status Word Bit Definition by Module Type

The following table describes the values of the module types:

Value of Bits 7...4 or 15...12 Note: See the previous tables for the bit range that applies to your module's software version.	Module Type
0	NOE 2x1
1	ENT
2	M1E
3	NOE 771 00
4	ETY
5	CIP
6	(reserved)
7	140 CPU 651 x0
8	(reserved)
9	(reserved)
10	NOE 771 10
11	NOE 771 01
12	NOE 771 11
13 ... 15	(reserved)

Chapter 18
ModbusP_ADDR: Modbus Plus Address

Introduction

This chapter describes the `ModbusP_ADDR` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	200
Detailed Description	203

Description

Function Description

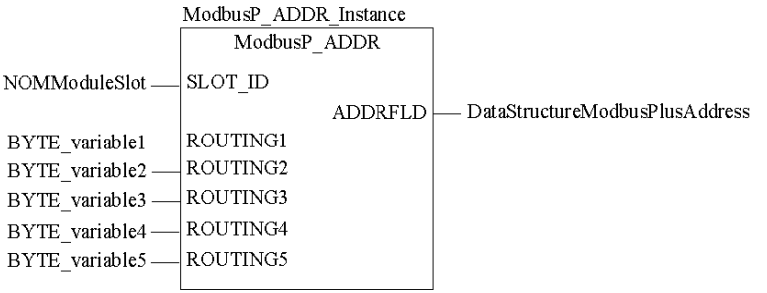
This function block allows the entry of the Modbus Plus address for function blocks READ_REG, CREAD_REG, WRITE_REG and CWRITE_REG. The address is transferred as a data structure.

EN and ENO can be configured as additional parameters.

NOTE: You must be familiar with your network when programming the ModbusP_ADDR function block. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the *Modbus Plus Network Planning and Installation Guide (Reference 31003525)*.

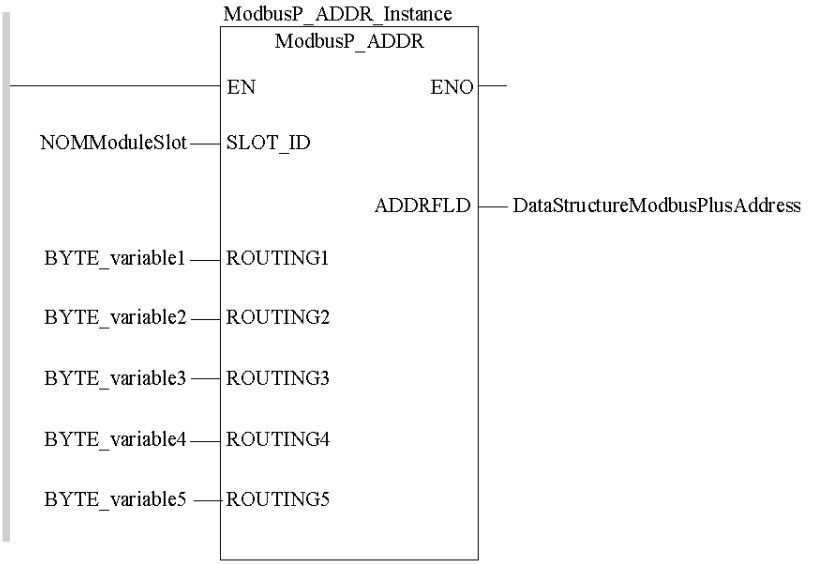
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL ModbusP_ADDR_Instance (SLOT_ID:=NOMModuleSlot,    ROUTING1:=BYTE_va
riable1, ROUTING2:=BYTE_variable2,    ROUTING3:=BYTE_variable3, ROUTING
4:=BYTE_variable4,    ROUTING5:=BYTE_variable5,    ADDRFLD=>DataStructu
reModbusPlusAddress)
```

Representation in ST

Representation:

```
ModbusP_ADDR_Instance (SLOT_ID:=NOMModuleSlot,    ROUTING1:=BYTE_variab
le1, ROUTING2:=BYTE_variable2,    ROUTING3:=BYTE_variable3, ROUTING4:=B
YTE_variable4,    ROUTING5:=BYTE_variable5,    ADDRFLD=>DataStructureMo
dbusPlusAddress) ;
```

Parameter Description

Description of input parameters:

Parameter	Data Type	Description
Slot_ID	BYTE	Slot ID NOM module slot
ROUTING1	BYTE	Routing 1 is used for address specification (routing path addresses one of five) of the destination node during network transfer. The last byte in the routing path that is not zero is the destination node.
ROUTING2	BYTE	Routing 2
ROUTING3	BYTE	Routing 3
ROUTING4	BYTE	Routing 4
ROUTING5	BYTE	Routing 5

Description of output parameters:

Parameter	Data Type	Description
ADDRFLD	WordArr5	Data structure used to transfer the Modbus Plus address

Detailed Description

Derived Data Types

Element description for `WordArr5`:

Element	Data Type	Description
`WordArr5[1]`	`WORD`	Routing tab 1 Least significant byte: used for address specification (routing path addresses one of five) of a destination node during network transfer. Most significant byte: Slot of the network adapter module (NOM), if available.
`WordArr5[2]`	`WORD`	Routing tab 2
`WordArr5[3]`	`WORD`	Routing tab 3
`WordArr5[4]`	`WORD`	Routing tab 4
`WordArr5[5]`	`WORD`	Routing tab 5

Slot_ID

If a Modbus Plus network option module (NOM) in the rack of a Quantum controller is addressed as the destination node, the value at the `Slot_ID` input represents the physical NOM slot, i.e. if the NOM is plugged in at Slot 7 of the rack, the value appears as follows:

0	0	0	0	0	1	1	1

Routing x

The `Routing x` input is used for address specification (routing path addresses one of five) of the destination node during network transfer. The last byte in the routing path that is not zero is the destination node.

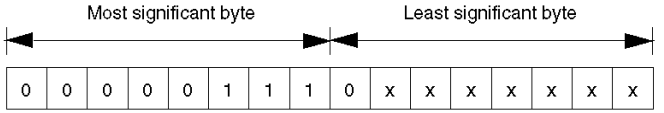
0	x	x	x	x	x	x	x

Destination address (binary value between 1 and 64 (normal) or 65 to 249 (extended))

Routing Tab 1

If a Modbus Plus Network Options Module (NOM) is addressed as the destination node in a Quantum control module rack, the most significant byte represents the physical slot of the NOM. If the destination node is a CPU, the most significant byte (regardless of the CPU slot) is set to "0".

If NOM is inserted in slot 7 on the module rack, the most significant byte of routing tab 1 looks as follows:



Most significant byte Slots 1 ... 16
Least significant byte Destination address (binary value between 1 and 64 (normal) or 65 to 255 (extended))

Chapter 19
OUT_IN_CHAR: Sending/Receiving Character Strings

Subject of this Chapter

This chapter describes the `OUT_IN_CHAR` communication function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	206
Assisted Entry Screen	209
Example of How to Send/Receive a Character String	211

Description

Function Description

The OUT_IN_CHAR function is used to send a string of a maximum of 210 bytes (120 for the terminal port), followed by a message receive request (send only or receive only is also possible).

The character string may be contained in a static variable *(see Unity Pro, Program Languages and Structure, Reference Manual)* or stipulated in the form of an immediate value (series of bytes between apostrophes, e.g.: 'Message to be sent').

These character strings may contain special characters, and must start with the character $ followed by the hexadecimal value of the character to be sent, for example $0D.

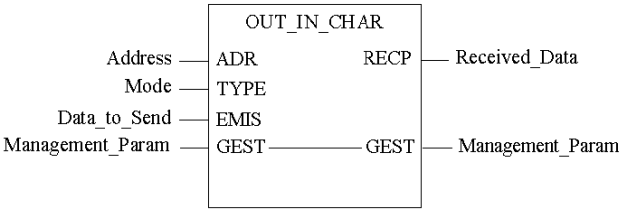
Certain special characters *(see Unity Pro, Program Languages and Structure, Reference Manual)* may be used, such as:

$R = CR (carriage return), $L = LF (line feed), $N = CR+LF.

The additional parameters EN and ENO can be configured.

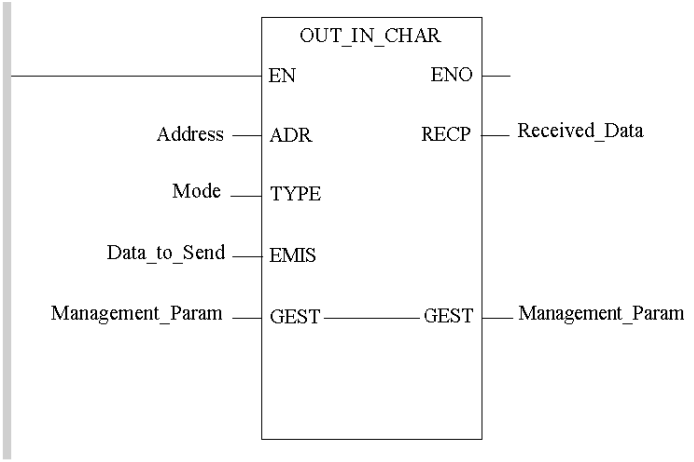
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
OUT_IN_CHAR Mode, Data_to_Send, Management_Param, Received_Data
```

ST Representation

Representation:

```
OUT_IN_CHAR(Address, Mode, Data_to_Send, Management_Param, Received_Dat
a);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
Address	ARRAY [0...5] OF INT	Address of the destination entity of the exchange. Only those addresses ending in SYS are possible (e.g.: {Network.Station}SYS).
Mode	INT	Operating mode: ● 1: transmission followed by receive mode ● 2: send a message ● 3: message in receive mode
Data_to_Send	STRING	Character string to send. It is essential for this character string to exist when the function is called, even if there is no data to send (for example in receive only mode).

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0...3] OF INT	Exchange management table *(see page 40)*

The following table describes the output parameters:

Parameter	Type	Comment
Received_Data	STRING	Character string received. It is essential for this character string to exist when the function is called, even if there is no data to receive (for example in send only mode).

NOTE: At the time of transmission or of a transmission followed by reception, the user is advised to initialize the fourth word of the management table (length) before executing the function. Also, the length of the character string received is stored in this same word at the end of this exchange. However, prior to this, during configuration, it is necessary to define a stop condition *(see Premium and Atrium Using Unity Pro, Asynchronous Serial Link, User Manual)*.
To send character strings containing end of string characters (ZERO). You must:

- Use localized STRINGS.
- Initialize the last word of the exchange management table with the number of characters to send. If you initialize this word with the value 0, the string that is sent will stop at the first ZERO character it encounters. If you initialize it will a value, the length of the character string sent will be equal to this value.

Assisted Entry Screen

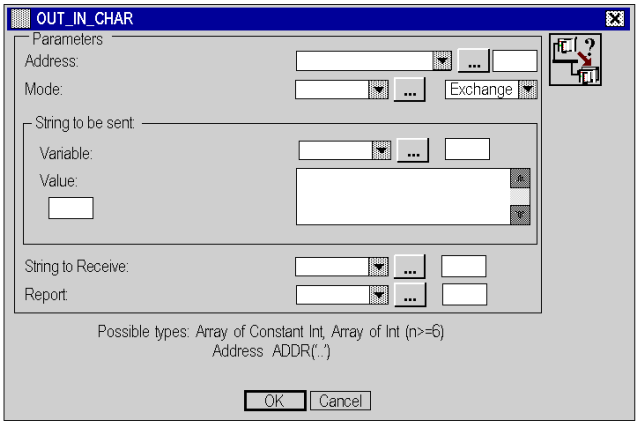
At a Glance

For this communication function, you can call up the assisted entry screen. Please note that the assisted entry screen is not available for Modicon M340.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an example of the assisted entry screen of the function:



Address

For Premium PLCs, the possible type of object is:

- `ADDR(STRING)`
- `ARRAY [0..5] OF INT`

NOTE: If you enter a value directly in the field, the assisted address entry button turns gray.

Mode

The selection from the drop-down list:

- Exchange
- Transmission
- Reception

displays directly the immediate value 1, 2 or 3.

NOTE: Simply choose from the solutions offered in the drop-down menu.

NOTE: If you use the entry field instead of the menu, you can enter a variable of the INT type, either located or not.

String to be Sent

The string to be send is a variable of the STRING type. This variable must be declared before being used.

If the choice of string is a variable (example `String_0`), the immediate value entry field disappears.

String to be Received

The reception zone is a variable of the`STRING` type. The size of this variable is a function of the number of characters to be received. This variable must be declared before being used in this screen.

Report

The report is an array of 4 integers.

NOTE: Do not use several identical memory areas for report tables; otherwise, the variable read function may not work.

Example of How to Send/Receive a Character String

At a Glance

Let us assume we want to send a character string `Str_1` from station 1 of network 20 to the terminal port of station 5, then receive a character string `Str_2` from the terminal port of station 5 of the same network.

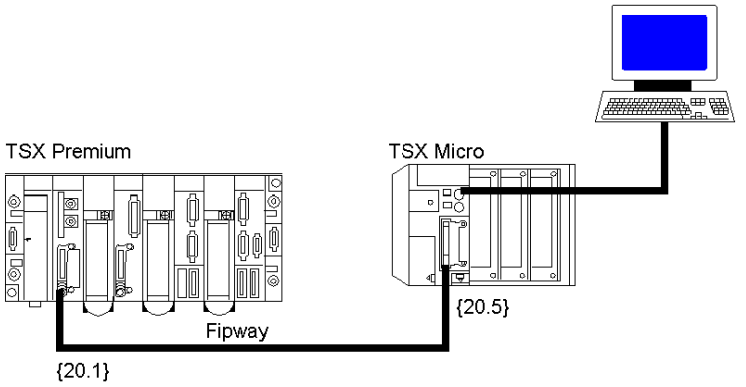
Here, a video terminal is connected to the terminal port of station 5 of network 20.

The string to be sent contains 10 characters.

The function's management table is made up of the table of words `%MW170:4`.

Illustration

The two stations are connected via a Fipway network.



Programming

Programming in ST:

```
IF RE(%I0.3.8) AND NOT %MW170.0 THEN
   (* initialization of the data to send *)
   %MW173 := 10;
   (* communication function *)
        OUT_IN_CHAR(ADDR('{20.5}0.0.0.SYS'),1,Str_1,%MW170:4,Str_2);
END_IF;
```

Request parameters:

Parameters	Description
ADDR('{20.5}0.0.0.SYS')	• {20.5}: network 20, station 5 • 0: rack • 0: module • 0: channel 0 • SYS: system address
1	Transmission then reception
Str_1	Variable of type `STRING` containing the message to send
%MW170:4	Management table
Str_2	Variable of type `STRING` that will contain the message received

NOTE: Before the function is launched, initialize the length parameter (in the example: %MW173) with the value corresponding to the number of characters (in bytes) to send to Str_1).
At the end of the exchange, %MW173 will contain the length of the data received (in bytes).

Chapter 20
OUT_IN_MBUS: Modbus Communication Function

Aim of this Chapter

This chapter describes the OUT_IN MBUS communication function.

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
20.1	General Presentation of the OUT_IN_MBUS Communication Block	214
20.2	Description of the OUT_IN_MBUS Communication Block	219
20.3	Installing the OUT_IN_MBUS Communication Block	230
20.4	Sample Use of a OUT_IN_MBUS Communication Block	238

Section 20.1
General Presentation of the OUT_IN_MBUS Communication Block

Aim of this Section

This section presents a summary description of the OUT_IN_MBUS communication block.

What Is in This Section?

This section contains the following topics:

Topic	Page
Function Description	215
Sample Use	216
Functionalities	218

Function Description

Introduction

The OUT_IN_MBUS function is used to emulate a Modbus master communication from a serial link configured in character mode.

Combined with the possibility of passing a Modbus slave configuration on the fly to a configuration in character mode via the function WRITE_CMD, this function allows the PLC to operate either as master or as Modbus slave on the same link.

NOTE: This function is only useful when the two modes Modbus master and slave are operating concurrently. If this is not the case, the EF standards READ_VAR and WRITE_VAR are recommended for managing the Modbus master function (Modbus slave is implicitly managed by the system). In this case see the Modbus documentation.

NOTE: Be careful that two masters (on the same bus) do not send requests simultaneously otherwise the requests are lost and each report will have a bad result which could be 16#0100 (request could not be processed) or 16#ODFF (slave is not present).

⚠ WARNING

UNEXPECTED APPLICATION BEHAVIOR - WRITING VARIABLES TO SLAVES

Before writing variables from a master to a slave ensure that the resulting application behavior remains acceptable.

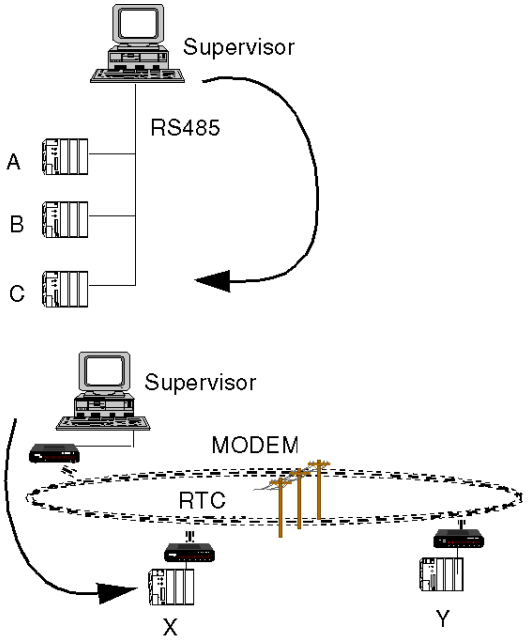
Failure to follow these instructions can result in death, serious injury, or equipment damage.

Sample Use

Nominal Mode

Most applications are comprised of a PC used for supervision applications. The supervisor is a Modbus master that communicates with different slaves. This operating mode is called the nominal mode.

The figure below illustrates the nominal mode (supervisor to slave):

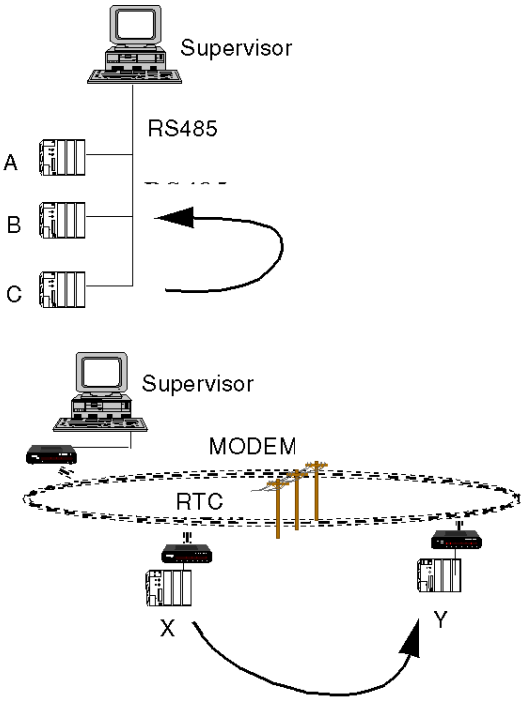


The Exception Mode

On request, a PLC should be able to toggle from Modbus slave mode to Modbus master mode in order to send requests to other PLCs or devices. This toggling is called the exception mode.

When the exception mode is terminated, the system should be able to return to the initial state (to nominal mode).

The figure below illustrates the exception mode (emulated master to slave):



Functionalities

At a Glance

The OUT_IN_MBUS function supports:

- The Modbus codes 1, 2, 3, 4, 5, 6, 15, 16 *(see page 223)*
- The ASCII and RTU modes *(see page 226)*
- The complete addressing *(see page 223)*

The OUT_IN_MBUS function is present as a DFB and it must be restarted at each PLC cycle as long as the activity bit is set to 1.

The function is available on Premium PLCs using a TSXSCP111 or TSXSCP114 communication card installed in the processor or a host module (TSXSCY21601 or TSXSCY11601).

Restrictions

On each of the ports, you must not simultaneously activate more than one OUT_IN_MBUS DFB. Similarly, you must not simultaneously use the PRINT_CHAR, INPUT_CHAR or OUT_IN_CHAR blocks.

The OUT_IN_MBUS function does not support:

- changing the parameters of the physical layers or link layers: baud rate, character format, change from RTU to ASCII or the inverse, RS232 signal management
- communication conflicts and errors that may result from the presence of two simultaneously active Modbus masters on the same link
- all operating modes that may result from a fault or a temporary connection with a communication card
- configuration and management of modems

Section 20.2
Description of the OUT_IN_MBUS Communication Block

Aim of this Section

This section presents a detailed description of the OUT_IN_MBUS communication block.

What Is in This Section?

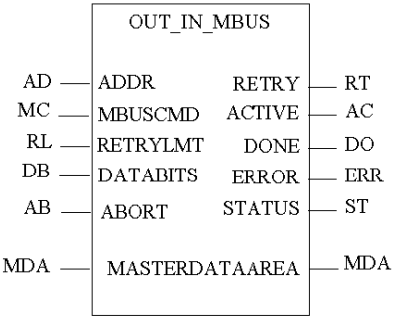
This section contains the following topics:

Topic	Page
Representations and Parameters	220
The MbusCmd Parameter	223
The RetryLmt Parameter	225
The DataBits Parameter	226
The RespTout Parameter	227
The MasterDataArea Parameter	228
The Status Parameter	229

Representations and Parameters

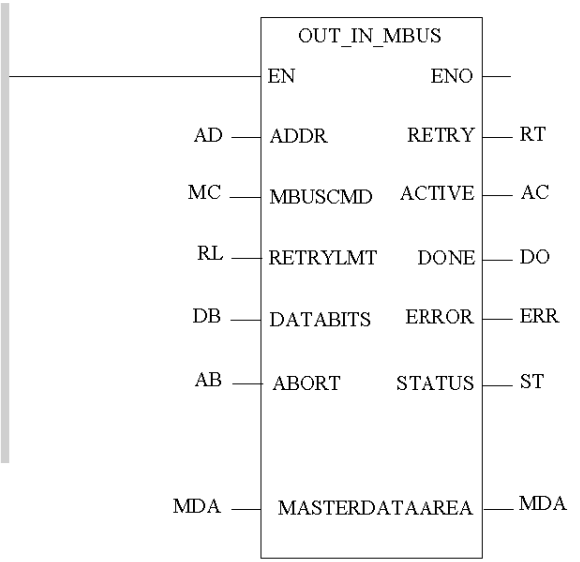
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
OUT_IN_MBUS AD, MC, RL, DB, AB, MDA, RT, AC, DO, ERR, ST
```

ST Representation

Representation:

```
OUT_IN_MBUS(AD, MC, RL, DB, AB, MDA, RT, AC, DO, ERR, ST);
```

Parameter Description

The following array describes the input parameters:

Parameter	Type	Comment
ADDR	ADDR_TYPE	Information on the address of the communication port of the slave device. This address must be in the following form: ADDR('r.m.c.SYS'). Abbreviations Used: r = rack, m = module slot, c = channel.
MBUSCMD	ARRAY [1.. 4] OF INT	Modbus array definition *(see page 223)*
RETRYLMT	INT	Number of attempts *(see page 225)* to send a message made by the block
DATABITS	BOOL	Modbus Messages *(see page 226)* to send in ASCII mode (DATABITS =0) or RTU mode (DATABITS=1).
RESPTOUT	INT	Block wait *(see page 227)* time.
ABORT	BOOL	DFB cancel bit.

The following array describes the input/output parameter:

Parameter	Type	Comment
MASTERDATAAREA	ARRAY [x.. y] OF INT	Data *(see page 228)* area of the master PLC

NOTE: In order to read one of the last 8 bits of the memory it is necessary to read all the last 8, otherwise the function sends back an error status report. Then it possible to extract the bit needed.

NOTE: The input/output parameter uses a dynamic array, you must in this case check in Unity Pro Allow dynamic arrays [ANY_ARRAY_XXX] under **tool → project option → language extension**.

The following array describes the output parameters (read only):

Parameter	Type	Comment
RETRY	INT	The value displayed indicates the current number of attempts made by the block.
ACTIVE	BOOL	The value 1 indicates that an operation is in progress.
DONE	BOOL	The value 1 indicates that the operation is completed.
ERROR	BOOL	The value 1 indicates that an error has occurred.
STATUS	INT	The value displays a status *(see page 229)* code generated by the block.

The MbusCmd Parameter

Definition of the MbusCmd Parameter

The parameter MbusCmd represents the Modbus command.

The parameter MbusCmd is comprised of an array of 4 registers as presented below:

Contents	Description	Description
MbusCmd[1]	Slave address	This word contains the address of the Modbus slave PLC. The range of allowed addresses is from 0 to 248. The address 0 is reserved for sending a Modbus message to several PLCs. This type of transmission is called broadcast mode. The broadcast mode supports only Modbus function codes writing master PLC data to slave PLCs. It does not support Modbus function codes reading data of slave PLCs. Address 248 is reserved for point-to-point communication when the slave address is not known. This address is not supported by all devices.
MbusCmd[2]	Modbus function code	The block OUT_IN_MBUS supports the following function codes: ● 01 = reading of several output bits (0x) ● 02 = reading of several input bits (1x) ● 03 = reading of several output registers (4x) ● 04 = reading of several input registers (3x) ● 05 = writing of a single output bit (0x) ● 06 = writing of a single output register (4x) ● 15 = writing of several output bits (0x) ● 16 = writing of several output registers (4x) **Note**: When the slave PLC is of Premium type, all bits become **%M**s and the registers become **%MW**s.
MbusCmd[3]	Data area of the slave PLC	For a read command, the data area of the slave PLC is the source of data. For a write command, the data area of the slave PLC is the destination of data. For example: ● To read the output bits 300 to 500 of a slave PLC, enter 300 in this field. ● To write the data of a master PLC in register 100 of type 4x of a slave PLC, enter 100 in this field. According to the type of Modbus command (read or write), the source and target data areas must match those in the array below *(see page 224)*.
MbusCmd[4]	Quantity	This register contains the quantity of data to write or to read in the slave PLC. For example, enter 100 to read 100 output registers in the slave PLC or enter 32 to write 32 output bits in the slave PLC. There is a size limit, that depends on the Modbus function code used and the transmission mode (RTU or ASCII). These MbusCmd[4] limit values are detailed in the array below *(see page 224)*. This size is not significant for function codes 5 and 6. **Note**: The memory area is limited depending on the device and the slave parameters.

MbusCmd[3]

The array below presents the slave PLC data area for **MbusCmd[3]**. This data area depends on the Modbus function code used and the type of slave PLC:

Function Code	Data Area for Standard Modbus Devices	Data Area of a Premium PLC
01 (reading of several output bits (0x))	0x (source)	%M (source)
02 (reading of several input bits (1x))	1x (source)	%M (source)
03 (reading of several output registers (4x))	4x (source)	%MW (source)
04 (reading of several input registers (3x)	3x (source)	%MW (source)
05 (writing of a single output bit (0x))	0x (destination)	%M (destination)
06 (writing of a single output register (4x))	4x (destination)	%MW (destination)
15 (writing of several output bits (0x))	0x (destination)	%M (destination)
16 (writing of several output registers (4x)).	4x (destination)	%MW (destination)

MbusCmd[4]

The array below presents the limit value of **MbusCmd[4]**. This value depends on the Modbus function code used and the transmission mode:

Function Code	RTU Mode (8 Bits)	ASCII Mode (7 Bits)
01 (reading of several output bits (0x))	1000	500
02 (reading of several input bits (1x))	1000	500
03 (reading of several output registers (4x))	100	50
04 (reading of several input registers (3x)	100	50
05 (writing of a single output bit (0x))	1	1
06 (writing of a single output register (4x))	1	1
15 (writing of several output bits (0x))	1000	500
16 (writing of several output registers (4x)).	100	50

The `RetryLmt` Parameter

Definition

This parameter corresponds to the number of attempts to send a message made by the `OUT_IN_MBUS` block prior to receiving a correct response from a slave device (PLC, modem, etc.).

When the response is not completely structured in the assigned time, the block generates an error and an error code.

The number of new attempts should be between 0 and 32767.

This field is used in conjunction with `RespTout`.

The `DataBits` Parameter

Definition

The Modbus messages may be sent in ASCII or RTU mode.

The ASCII mode uses 7 data bits whereas the RTU modes uses 8. To send a message in RTU characters, Databits should be set to 1.

The value must match the configuration of the communication card.

NOTE: To avoid any problem in the event of configuration changes, use the system constants of type `%KWr.m.c.1.8` to initialize the parameter.

The RespTout Parameter

Definition

This parameter corresponds to the wait time delay of the OUT_IN_MBUS block prior to receiving a correct response from a slave device (PLC, modem, etc.).

When the response is not completely structured in the assigned time, the block generates an error. This system allows no response after this delay.

The base time is 100 ms. The valid values are between 0 (infinite wait) and 32767.

The timeout begins after the last character of the message is sent.

The `MasterDataArea` Parameter

Definition

For a read command, the data area of the master PLC is the destination of the data sent by the slave.

For a write command, the data area of the master PLC is the source of data.

For Modbus command codes 1, 2, 5 and 15, coding of bits is performed as follows:

- Bits 1 to 16 are stored in the first item of the INT array set as argument, the first bit being in the least significant bit of the item.
- Bits 17 to 32 are stored in the second item of the array, bit 17 corresponding to the least significant bit of the item.
- Etc.

Thus, to exchange 1,000 bits, you must declare an array of 63 INT (1,000/16 + 8).

NOTE: A data area of integer type %MW may be used directly in the MasterDataArea parameter (for example: %MW100:50 designates an array of 50 integers starting at address 100).
A series of array items of type %M must first be converted and recopied in an array of integers (INT) according to the above-described coding.

The `Status` Parameter

Definition

This parameter displays a status code generated by the `OUT_IN_MBUS` block.

The array below shows the different status codes.

Status Code	Description of Status
1	Modbus exception - Incorrect function
2	Modbus exception - Incorrect data address
3	Modbus exception - Incorrect data value
4	Modbus exception - Slave node error
5	Modbus exception - Confirmation
6	Modbus exception - Slave node occupied
7	Modbus exception - Negative confirmation
8	Modbus exception - Memory parity error
104	Data length cannot be zero
108	Undefined error
113	LRC checksum of slave PLC not valid
114	CRC checksum of slave PLC not valid
115	Modbus function code not valid
116	Modbus response timeout message
124	Internal status not defined
125	Broadcast mode not allowed for this Modbus function
128	Unexpected response from Modbus slave
130	Modified command word activity in progress
131	Incorrect number of characters
200	Slave address out of range
201	Communication error with serial port
202	Invalid binary number
203	Too much data
204	Master data area too small
205	Timeout must be positive
206	Unknown Modbus exception
207	Action canceled by user
208	RetryLmt must be positive

Section 20.3
Installing the OUT_IN_MBUS Communication Block

Aim of this Section

This section describes installation of the OUT_IN_MBUS communication block.

What Is in This Section?

This section contains the following topics:

Topic	Page
Configuration of the Serial Link	231
Procedure for Programming	234
Using a Modem	236

Configuration of the Serial Link

Introduction

Use of the `OUT_IN_MBUS` block requires prior correct configuration of the serial link.

The following parameters are used:
- linked to transmission and configured from Unity Pro
- linked to the application and passed as function arguments

Reminder of Transmission Parameters

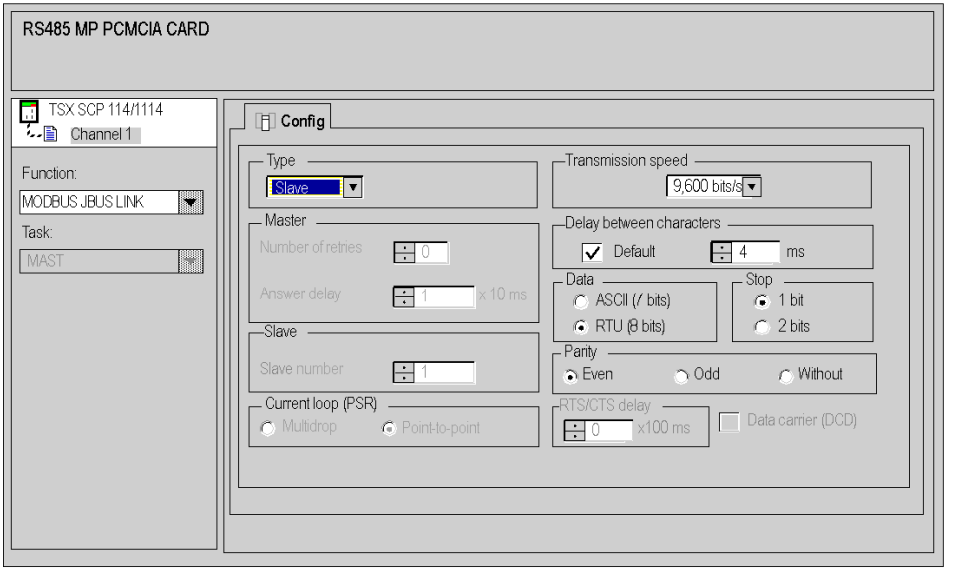
The serial link of the cards is configured from Unity Pro. These parameters are the following:
- Transmission speed
- Delay between characters
- Data
- Stop
- Parity

To allow exchange of data between all equipment connected on the bus, configuration of the serial link should be identical for all.

The configuration screen used to enter the parameters depends on the configuration selected in nominal mode.

Transmission Parameters of the Modbus Mode

The figure below shows a configuration screen when the nominal mode is in Modbus mode.

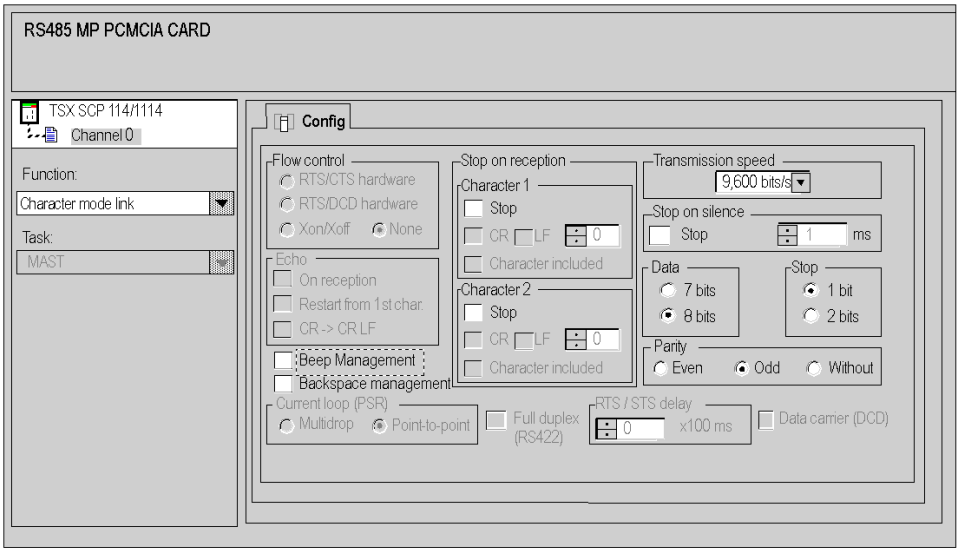


NOTE: The value of the Databit *(see page 226)* input parameter of the `OUT_IN_MBUS` block should match the value checked in the Data window of the configuration screen.

Use of `OUT_IN_MBUS` requires dynamic switching to character mode. In this mode, the conditions for stopping (on characters or silence) are disabled and not modifiable by the application. The `OUT_IN_CHAR` function is thus no longer usable in receive mode (stop on timeout), and only the `INPUT_CHAR` and `INPUT_BYTES` functions may be used for receiving by specifying the number of characters to receive.

Transmission Parameters of the Character Mode

The figure below shows a configuration screen when the nominal mode is in character mode.



NOTE: The value of the Databit *(see page 226)* input parameter of the `OUT_IN_MBUS` block should match the value checked in the Data window of the configuration screen.

When the card configuration is the character mode:

- The conditions for stopping (on characters or silence) configurable in this mode must be disabled for proper operation of `OUT_IN_MBUS`. The function `OUT_IN_CHAR` is thus no longer usable in receive mode (stop on timeout), and only the functions `INPUT_CHAR` and `INPUT_BYTES` may be used for receiving by specifying a number of characters to receive.
- The delay between characters cannot be configured. It is thus necessary to make sure that the value of this parameter in the remote Modbus devices is compatible with the character mode configuration.

Parameters for the Application

Two parameters for the application are sent as arguments of the `OUT_IN_MBUS` function.

These parameters are the following:

- The number of retries *(see page 225)*
- The answer delay *(see page 227)*

Procedure for Programming

Procedure to Follow

The array below shows the procedure to follow for programming the OUT_IN_MBUS block.

Step	Action	details
1	Preparing the communication port	• If the serial port is not configured to character mode, change the Modbus mode of the port to character mode by sending the WRITE_CMD *(see page 235)* command to the serial port. • For a modem transmission, send the HAYES command by using the PRINT_CHAR block or the OUT_IN_CHAR block to configure the modem *(see page 236)*. • For a modem transmission, use the HAYES command to send a dial message to the modem by using the PRINT_CHAR block or the OUT_IN_CHAR block. The dial message is used to send a telephone number to the modem *(see page 236)*.
2	Initialization of parameters	Initialize the DFB block input parameters. It is not useful to repeat this operation for each PLC cycle.
3	OUT_IN_MBUS block call	• OUT_IN_MBUS must be called at each PLC cycle until the activity bit is at zero. • As soon as the activity bit is at zero, force a bit in the block call condition to prevent a new call. • Check the error bit (in the event of an error, the status word will specify the cause of the error).
4	Resetting the communication port	• For a modem transmission, send the HAYES command to send a disconnection message to the modem *(see page 236)* by using the PRINT_CHAR block or the OUT_IN_CHAR block. • If the port was switched to character mode (in step 1), return to the starting mode of the serial port using the WRITE_CMD *(see page 235)* command.

Writing the Command Words to a Communication Port

The following steps should be executed to send a WRITE_CMD to a communication port:

Step	Action
1	Test to determine whether any command is pending. ● Before executing a WRITE_CMD, test whether an exchange is currently in progress using the %MWr.m.c.0 language object. To refresh this word, use the READ_STS block.
2	Assign the command word. ● You must next modify the value of the command language object in order to perform the required command. For a Modbus link, the object language is the internal word %MWr.m.c.15. For example, to switch from Modbus mode to character mode, %MWr.m.c.15 is set to 16#4000 (%MWr.m.c.15.14=1). **Note**: A single command bit must then be switched from 0 to 1 before transmitting the WRITE_CMD.
3	Send the command. ● Finally, a WRITE_CMD must be executed to acknowledge the command.

In the example *(see page 238)* that follows, we use the corresponding IODDT interface to communicate with the serial port channel.

Using a Modem

Description

It is necessary to know three commands to interface telephonic modems to PLCs. These commands are the following:

- Initialize modem
- Renumerate
- Disconnect modem

It is imperative to send an initialization message followed by a dial message to the modem before sending it an ASCII or Modbus message.

When the connection is successful between the two modems, you may send an unlimited number of ASCII or Modbus messages.

When all the messages have been sent, you must send the disconnection string to the modem.

Initializing the Modem

The initialization message is an ASCII message comprised of a maximum of 512 characters, although 50 characters are generally adequate for initializing a modem.

You can use any Hayes AT command as a component of the initialization string.

Example: A typical `Hayes` initialization message:

- `AT&F&K0&Q0&D0V1X0Q0 <CR><LF>`

NOTE: To simplify programming, you may initialize the modem via a terminal (for example: Windows hyperterminal) and not use the `OUT_IN_CHAR` function. Once the parameters have been loaded in the modem, they may be saved in non-volatile memory with an AT command, usually &W.

Dialing the Modem

The dial message is used to send the telephone number to the modem.

Only AT commands relating to dialing should be included in the message.

Example 1: Frequency dialing:

- `AT DT 6800326 <CR><LF>`

Example 2: Pulse dialing:

- `AT DP 6800326 <CR><LF>`

Example 3: Frequency dialing with tone waiting:

- `AT DT W,6800326 <CR><LF>`

NOTE: The timeout value should be large because the connection between two modems takes time (for example, set timeout to 30,000 ms). A Status code *(see page 229)* 116 is generated by the `OUT_IN_MBUS` block is the value is too short. Several attempts may be necessary before finding the optimum time.

Disconnecting the Modem

The disconnection message is used to disconnect the modem.

Example 1: Typical `Hayes` disconnection message:

- `+++AT H0 <CR><LF>`

NOTE: The timeout value should be large because the disconnection of a modem takes time (for example, set timeout to 30,000 ms). A Status code *(see page 229)* 116 is generated by the `OUT_IN_MBUS` block if the value is too short. Several attempts may be necessary before finding the optimum time.

Section 20.4
Sample Use of a OUT_IN_MBUS Communication Block

Aim of this Section

This section presents a sample use of the OUT_IN_MBUS communication block.

What Is in This Section?

This section contains the following topics:

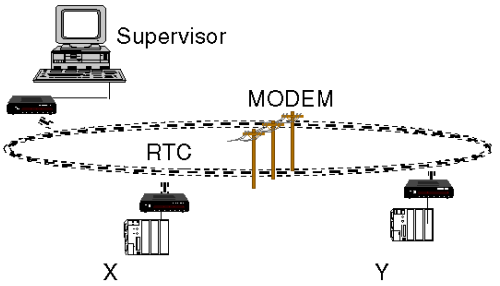
Topic	Page
Example Description	239
Programming Structure	240
Declaration of Variables	242
Programming	243

Example Description

At a Glance

The example selected is a Modbus communication application via modems.

The figure below illustrates the example:



The devices communicate with each other using modems. The supervisor is Modbus master whereas the X and Y PLCs are slaves.

The goal of the example is to write the data area values of PLC X to PLC Y.

PLC X must write a data area of 41 integers starting with the %MW100 address in PLC Y from %MW100.

To do this, PLC X must become Modbus master. The Modbus address of PLC X is 9, Y is 10.

To simplify programming, the modems have been initialized with the correct parameters via a programming terminal. These parameters are stored in non-volatile memory by the AT&W commands.

Programming Structure

Step Comments

The array below summarizes the programming steps in the example:

Step Number	Step Description
0	Initial state of function Wait for change to 1 of `Start_4` bit to go to step 5.
5	If no command is in progress on the serial port, a command is sent to change the serial port from Modbus mode to character mode. Go to step 10.
10	Read status of serial port. ● If there is an error on the serial port then ❍ `Error_4` is at -2. ❍ Go to step 65. ● If there is no error on the serial port, ❍ And character mode is active, then go to step 15. ❍ And no character mode active, then test the status of change to character mode on 1000 cycles. If, after 1000 cycles, the modem has not changed, then `Error_4` is at -1, and go to step 65.
15	Sending a dial command to the modem via the `PRINT_CHAR` block and waiting for the response via `INPUT_CHAR`. Go to step 20.
20	If the result of `PRINT_CHAR` is conclusive then go to step 25 otherwise go to step 65 with `Error_4` at -3.
25	If the result of `INPUT_CHAR` is conclusive then go to step 30 otherwise go to step 65 with `Error_4` at -4.
30	If the modem responds then go to step 35 otherwise go to step 65 with `Error_4` at -5.
35	Initialization of OUT_IN_MBUS block parameters. Go to step 40.
40	● Call of `OUT_IN_MBUS` block. ● If the bit `Active_4` is at 0, ❍ And if the bit `Flag_Error_4` is at 0, then go to step 45. ❍ And if the bit `Flag_Error_4` is at 1, then go to step 45 with `Error_4` at -6.
45	Sending a disconnection command to the modem using the `PRINT_CHAR` block. Go to step 50.
50	If the result of `PRINT_CHAR` is conclusive then go to step 55 otherwise go to step 65 with `Error_4` at 1.
55	If no command is in progress on the serial port, a command is sent to change the serial port from character mode to Modbus mode. Go to step 60.

Step Number	Step Description
60	Read status of serial port. ● If there is an error on the serial port then ❍ `Error_4` is at 3. ❍ Go to step 65. ● If there is no error on the serial port, ❍ And go to Modbus mode, then go to step 65. ❍ And no Modbus mode, then test the status of change to character mode on 1000 cycles. If, after 1000 cycles, the modem has not changed, then `Error_4` is at 2, and go to step 65.
65	Return to step 0 (initial function state) and bit `Start_4` at 0.

Declaration of Variables

At a Glance

The following array shows details on the variables used:

Variable	Type	Definition
Start_4	BOOL	Function start bit
Step_4	INT	Function step
Error_4	INT	Function error code
MngtPrint_4	ARRAY[0..4] of INT	Array of the communication parameters for the `INPUT_CHAR` block
MngtInput_4	ARRAY[0..4] of INT	Array of the communication parameters for the block `PRINT_CHAR`
ReqString_4	STRING	Character string to modem command
AnsString_4	STRING	Character string from modem answer
Out_In_Mbus_4	OUT_IN_MBUS	Instance of the `OUT_IN_MBUS` block
Adr_4	INT	Communication port of the slave PLC
MbusCmd_4[1]	INT	Modbus function code
MbusCmd_4[2]	INT	Quantity of data to read
MbusCmd_4[3]	INT	Modbus address of the slave PLC
MbusCmd_4[4]	INT	Start of the data area from the slave to read
RetryLmt_4	INT	Number of attempts
DataBits_4	INT	Transmission mode (1 for RTU and 0 for ASCII)
RespTout_4	INT	Timeout
Retry_4	INT	Number of attempts made by the block
Active_4	BOOL	The value 1 indicates that the operation was successful.
Done_4	BOOL	The value 1 indicates that the operation was successful.
Flag_Error_4	BOOL	The value 1 indicates that an error has occurred or that the current operation has terminated
Status_4	INT	Status code generated by the block

Programming

Programming in ST Language

The example is programmed in ST structured litteral language. The dedicated section is under the same master task (MAST).

```
(* Function to write %MW100 to %MW140 in slave Y *)
(* ---------------------------------------------- *)

CASE Step_4 OF


  0:
        (* Initialization *)
    IF (Start_4) THEN (* trigger flag *)
        Error_4 := 0;
        Step_4 := 5; (* next step *)
    END_IF;


  5:
        (* Send command to switch serial port from Modbus to character
mode *)

    READ_STS(Ioddt_Pcmcia_0_3_1); (* read serial port status *)
    IF (Ioddt_Pcmcia_0_3_1.EXCH_STS = 0) THEN (* no active command *)
        Ioddt_Pcmcia_0_3_1.CONTROL := 16#00; (* reset control word *)
        SET(Ioddt_Pcmcia_0_3_1.MB_TO_CHAR); (* set MB_TO_CHAR command b
it *)
        WRITE_CMD (Ioddt_Pcmcia_0_3_1); (* send command *)
        i := 0; (* initialize retry counter *)
        Step_4 := 10; (* next step *)
    END_IF;
```

```
 10:
        (* Test result of switch command *)
    READ_STS(Ioddt_Pcmcia_0_3_1); (* read serial port status *)
    IF (Ioddt_Pcmcia_0_3_1.EXCH_STS = 0) THEN (* command completed *)
        RESET(Ioddt_Pcmcia_0_3_1.MB_TO_CHAR); (* reset MB_TO_CHAR comma
nd bit *)
        IF (Ioddt_Pcmcia_0_3_1.EXCH_RPT = 0) THEN (* no error *)
            IF (AND(Ioddt_Pcmcia_0_3_1.PROTOCOL, 16#0F) = 03) THEN (* c
haracter mode OK *)
                Step_4 := 15; (* next step *)
            ELSE
                i := i + 1;
                IF (i > 1000) THEN
                    Error_4 := -1; (* error *)
                    Step_4 := 65; (* next step = end *)
                END_IF;
            END_IF;
        ELSE (* error in sending command to port *)
            Error_4 := -2; (* error *)
            Step_4 := 65; (* next step = end *)
        END_IF;
    END_IF;



 15:
        (* Send dial command to modem *)

    Adr_4 := ADDR('0.3.1.SYS'); (* communication port *)
    MngtPrint_4[3] := 50; (* timeout *)
    MngtPrint_4[4] := 16; (* number of bytes to send *)
    ReqString_4 := 'ATDT0102030405$N'; (* dial message *)
    PRINT_CHAR(Adr_4, ReqString_4, MngtPrint_4);
    MngtInput_4[3] := 300; (* timeout *)
    MngtInput_4[4] := 0; (* number of bytes to send *)
```

```
    INPUT_CHAR(Adr_4, 0, 12, MngtInput_4, AnsString_4); (* wait modem r
eply *)
    Step_4 := 20; (* next step *)



 20:
        (* Test PRINT_CHAR function result *)
    IF (NOT MngtPrint_4[1].1) THEN
        IF (MngtPrint_4[2] = 0) THEN
              Step_4 := 25; (* success : next step *)
        ELSE
              Error_4 := -3; (* error *)
           Step_4 := 65; (* next step = end *)
        END_IF;
    END_IF;



 25:
    (* Test INPUT_CHAR function result *)
    IF (NOT MngtInput_4[1].1) THEN
        IF (MngtInput_4[2] = 0) THEN
              Step_4 := 30; (* success : next step *)
        ELSE
               Error_4 := -4; (* error *)
            Step_4 := 65; (* next step = end *)
        END_IF;
    END_IF;



 30:
    (* Test Modem reply *)
    IF (AnsString_4 = 'CONNECT 9600') THEN
           Step_4 := 35; (* success : next step *)
    ELSE
```

```
            Error_4 := -5; (* error *)
        Step_4 := 65; (* next step = end *)
    END_IF;


 35:
    (* Initialize OUT_IN_MBUS parameters *)
    MbusCmd_4[1] := 10; (* slave PLC address *)
    MbusCmd_4[2] := 16#06; (* Modbus function 16#06 *)
    MbusCmd_4[3] := 100; (* slave PLC area = %MW100 *)
    MbusCmd_4[4] := 41; (* quantity of data *)
    RetryLmt_4 := 2; (* number of retry *)
    DataBits_4 := %KW0.3.1.1.8; (* 1 = 8 bits -> RTU mode, 0 = 7 bits -
> ASCII mode *)
    RespTout_4 := 300; (* timeout = 30s *)
    Flag_Error_4 := 0;
    Step_4 := 40; (* next step *)


 40:
    (* Call OUT_IN_MBUS *)
    Out_In_Mbus_4 (Adr_4, MbusCmd_4, RetryLmt_4, DataBits_4, RespTout_4
, Abort_4,
                   %MW100:41, Retry_4, Active_4, Done_4, Flag_Error_4,
Status_4);

    IF (NOT Active_4) THEN (* request completed *)
        IF (NOT Flag_Error_4) THEN (* no error *)
            Step_4 := 45; (* next step *)
        ELSE (* error *)
            Error_4 := -6; (* error *)
            Step_4 := 45; (* next step *)
        END_IF;
    END_IF;
```

```
45:
    (* Hangup modem *)
    MngtPrint_4[3] := 50; (* timeout *)
    MngtPrint_4[4] := 9; (* number of bytes to send *)
    ReqString_4 := '+++ATH0$N'; (* hangup message *)
    PRINT_CHAR(Adr_4, ReqString_4, MngtPrint_4);
    Step_4 := 50; (* next step *)


50:
    (* Test PRINT_CHAR function result *)
    IF (NOT MngtPrint_4[1].1) THEN
        IF (MngtPrint_4[2] = 0) THEN
              (* Success : next step *)
              Step_4 := 55;
        ELSE
              (* End on error *)
              Error_4 := 1;
           Step_4 := 65;
        END_IF;
    END_IF;


55:
    (* Send command to switch serial port from Modbus to character mode
*)
    READ_STS(Ioddt_Pcmcia_0_3_1); (* read serial port status *)
    IF (Ioddt_Pcmcia_0_3_1.EXCH_STS = 0) THEN (* no active command *)
        Ioddt_Pcmcia_0_3_1.CONTROL := 16#00; (* reset control word *)
        SET(Ioddt_Pcmcia_0_3_1.CHAR_TO_MB); (* set MB_TO_CHAR command b
it *)
        WRITE_CMD (Ioddt_Pcmcia_0_3_1); (* send command *)
```

```
        i := 0; (* initialize retry counter *)
        Step_4 := 60; (* next step *)
    END_IF;



 60:
    (* Test result of switch command *)
    READ_STS(Ioddt_Pcmcia_0_3_1); (* read serial port status *)
    IF (Ioddt_Pcmcia_0_3_1.EXCH_STS = 0) THEN (* command completed *)
        RESET(Ioddt_Pcmcia_0_3_1.CHAR_TO_MB); (* reset CHAR_TO_MB comma
nd bit *)
        IF (Ioddt_Pcmcia_0_3_1.EXCH_RPT = 0) THEN (* no error *)
            IF (AND(Ioddt_Pcmcia_0_3_1.PROTOCOL, 16#0F) = 07) THEN (* M
odbus mode OK *)
                Step_4 := 65; (* next step *)
            ELSE
                i := i + 1;
                IF (i > 1000) THEN
                    Error_4 := 2; (* error *)
                    Step_4 := 65; (* next step *)
                END_IF;
            END_IF;
        ELSE (* error in sending command to port *)
            Error_4 := 3; (* error *)
            Step_4 := 65; (* next step *)
        END_IF;
    END_IF;
```

```
 65:
    (* End *)
    Start_4 := 0; (* allow new demand *)
    Step_4 := 0; (* goto waiting state *)

END_CASE;
```

Chapter 21
PRINT_CHAR: Sending character strings

Subject of this Chapter

This chapter describes the `PRINT_CHAR` function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	252
Assisted entry screen	257
Example of Sending Character Strings via Fipway Network	258
Example of Sending Character Strings via Serial Link of Modicon M340 Processors	260

Description

Function Description

For Premium PLCs, the PRINT_CHAR function is used to send a character string of a maximum of 4 Kbytes, 120 bytes at the terminal port, to be transmitted on a character mode link.

For Modicon M340 and M580 CPU, the PRINT_CHAR function is used to send a character string of a maximum of 1,024 bytes.

The character string may be contained in a static variable *(see Unity Pro, Program Languages and Structure, Reference Manual)* or stipulated in the form of an immediate value (series of bytes between apostrophes, e.g.: 'Message to be sent').

These character strings may contain special characters, and must start with the character $ followed by the hexadecimal value of the character to be sent, for example $0D.

Certain special characters *(see Unity Pro, Program Languages and Structure, Reference Manual)* may be used, such as:

$R = CR (carriage return), $L = LF (line feed), $N = CR+LF.

The additional parameters EN and ENO can be configured.

⚠ WARNING

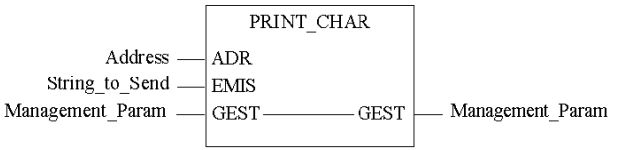
UNINTENDED EQUIPMENT OPERATION

Implement a request coordination between P-HSBY and Target in a 2-wires application to abort simultaneous PRINT_CHAR during a swap.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

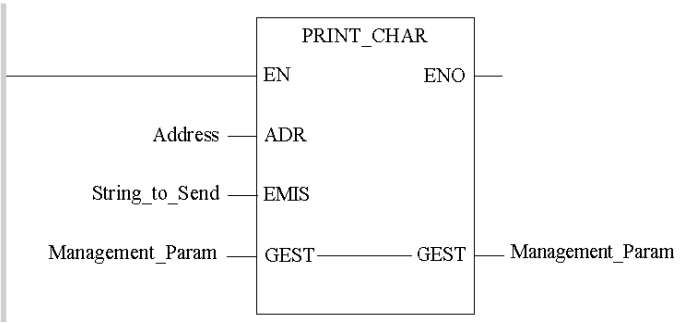
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
PRINT_CHAR String_to_Send, Management_Param
```

ST Representation

Representation:

```
PRINT_CHAR(Address, String_to_Send, Management_Param);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0..5] OF INT for Premium ARRAY [0..7] OF INT for Modicon M340 and M580	The following statements are true only for Premium PLC: ● Address of the message's receiving character mode channel is given by the ADDR *(see page 83)* function ● Only those addresses ending in SYS are possible (example: 0.0.0.SYS processor terminal port). The following statements are true only for Modicon M340 PLC: ● Address of the message's receiving character mode channel is given by the ADDM *(see page 71)* function ● The syntax of the address is of ADDM ('r.m.c.node')-type. Node is an optional field that may be SYS or empty (e.g. ADDM('0.0.0.SYS') equals ADDM('0.0.0'). The following statements are true only for Modicon M580 CPU: ● Address of the destination entity of the print operation, result of ADDMX *(see page 77)* function.
EMIS	STRING	Character string to send. This is either latched in a character string or provided as an immediate value. **Note**: this character string must exist, even if there is no data to be sent.

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0..3] OF INT	Exchange management table consisting of the following words: ● Rank 1 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: exchange number ❍ Least significant byte: activity bit (rank 0) and cancel bit (rank 1) **NOTE:** The cancel bit is only available for Modicon M340 and M580 PLCs. ● Rank 2 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: operation report ❍ Least significant byte: communication report ● Rank 3 word: a word managed by the user, which defines the maximum response time using a time base of 100 ms. ● Rank 4 word: a word managed by the user which defines the length of the exchange. ❍ If this parameter length is set to 0 then the system sends the string entirely. ❍ If this parameter length is greater than the length of the string then the error 16#0A (Insufficient send buffer size) is returned into the 2nd management word and no character is sent. **NOTE:** In case of PRINT_CHAR, the fourth management word (length to send) must not be written while the activity bit is 1 (i.e. once EF is running). Otherwise PRINT_CHAR could be locked with error 0xB. For detailed information, refer to section *Structure of the Management Parameters* *(see page 40)*.

Programming Rules

Special characters are preceded by the character $ in the string to be transmitted. The $ characters are not transmitted by the sender, and must therefore not be counted during the initialization of the length parameter.

The spaces between two characters are counted in a byte.

Therefore, for the example PRINTING IN PROGRESSLR, the length of the data to be sent is 22 bytes.

On Premium PLCs, it takes several PLC cycles to send a character string of more than 240 bytes (the string is fragmented). It is therefore important to make sure that the management data is not modified during the processing of the function. The system transmits the string consistently over several fragments, but does not prohibit the transmission of another character string between two fragments.

On Modicon M340 and M580 CPU, it takes one CPU cycle to send a character string of a maximum length of 1,024 bytes. It is important to make sure that the management data is not modified during the processing of the function.

The Modicon M340 PLC's serial port is full duplex, so a PRINT_CHAR function can be sent even when an INPUT_CHAR function has been sent and is still pending.

NOTE:
To send character strings containing end of string characters (ZERO). You must:

- use localized STRINGS,
- initialize the last word of the exchange management table with the number of characters to send. If you initialize this word with the value 0, the string that is sent will stop at the first ZERO character it encounters. If you initialize it with a value, the length of the character string sent will be equal to this value.

Assisted entry screen

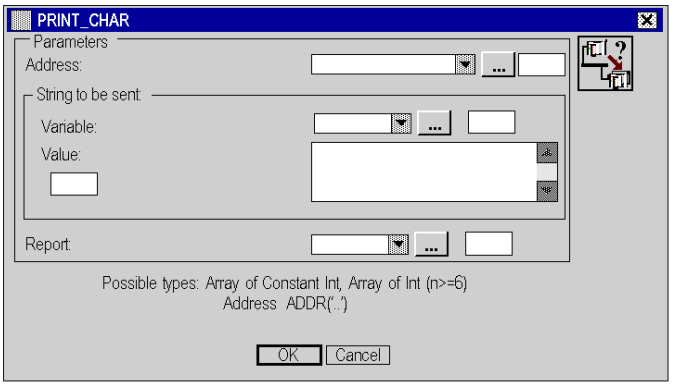
At a Glance

For this communication function, you can call up the assisted entry screen. Please note that the assisted entry screen is not available for the Modicon M340/M580.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an example of the assisted entry screen of the function:



Address

For Premium PLCs, the possible type of object is:

- `ADDR(STRING).`
- `ARRAY [0..5] OF INT.`

NOTE: if you enter a value directly in the field, the assisted address entry button turns gray.

String to be sent

The string to be sent is a variable of the STRING type or an immediate value. If a STRING type variable is selected, the value field disappears.

Report

The report is an array of 4 integers, that can be located or not.

NOTE: be careful not to use several identical memory areas for report tables, otherwise the variable read function may not work.

Example of Sending Character Strings via Fipway Network

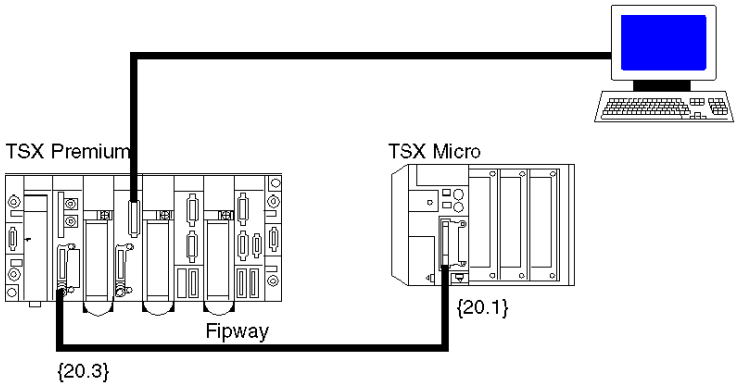
At a Glance

Let us assume we want to send a character string to a video terminal connected to the built-in link of the TSX SCY 21601 module of a PLC with a network address of 20, station 3. The TSX SCY 21601 module is located in slot 2 of the base rack.

The management table is located in `%MW110:4` and the string to be sent is located in the variable `Str_1`.

Illustration

The two stations are connected via a Fipway network.



Transmission

Programming in ST:

```
IF RE(%I0.3.6) AND NOT %MW110.0 THEN
   PRINT_CHAR(ADDR('{20.3}0.2.0.SYS'),Str_1,%MW110:4);
END_IF;
```

Request parameters:

Parameters	Description
`ADDR('{20.3}0.2.0.SYS')`	• {20.3}: network 20, station 3 • 0: rack • 2: module • 0: channel 0 • SYS: system address
Str_1	Character string to send, variable `Str_1` is of type `STRING`.)
%MW110:4	Management table

NOTE: Each time the function is launched, initialize the length parameter (in the example: %MW113) with the value corresponding to the number of characters (in bytes) to send to `Str_1`).

Example of Sending Character Strings via Serial Link of Modicon M340 Processors

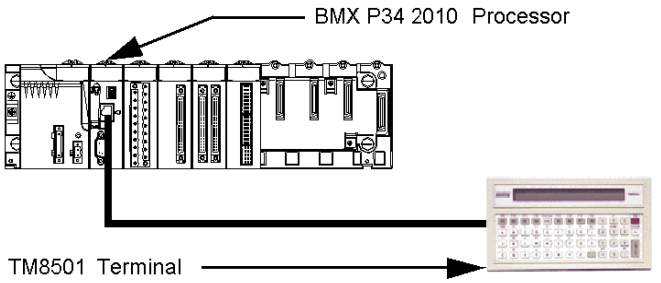
At a Glance

Let us assume we want to send a character string to a compact data entry/display terminal connected to a Modicon M340 processor's serial port.

The management table is located in `%MW110:4` and the string to be sent is located in the variable `caract`.

Illustration

A Modicon M340 PLC is linked to a data entry/display terminal TM8501:



Transmission

Programming in ST:

```
IF (%M16) THEN
   PRINT_CHAR(ADDM('0.0.0'),caract,gestion);
END_IF;
```

Request parameters:

Parameters	Description
`ADDM('0.0.0')`	• 0: rack • 0: module • 0: channel 0 • SYS: system address (optional on Modicon M340 PLCs)
caract	Character string to send, variable `caract`.
gestion	Management table

NOTE: Each time the function is launched, initialize the length parameter (For example: %MW113 if the management table is located from %MW110 to %MW113) with the value corresponding to the number of characters (in bytes) to send to caract).

Chapter 22
PRINT_CHAR_QX: Sending Character Strings on EIO Bus

Description

Function Description

The PRINT_CHAR_QX function is used to send a character string of 1000 bytes maximum to a serial device connected to a Modicon X80 serial communication module (BMXNOM0200) located in an EIO drop.

The character string may be contained in a static variable *(see Unity Pro, Program Languages and Structure, Reference Manual)* or stipulated in the form of an immediate value (series of bytes between apostrophes, for example: 'Message to be sent').

These character strings may contain special characters, and must start with the character $ followed by the hexadecimal value of the character to be sent, for example $0D.

Certain special characters *(see Unity Pro, Program Languages and Structure, Reference Manual)* may be used, such as:

$R = CR (carriage return), $L = LF (line feed), $N = CR+LF.

The additional parameters EN and ENO can be configured.

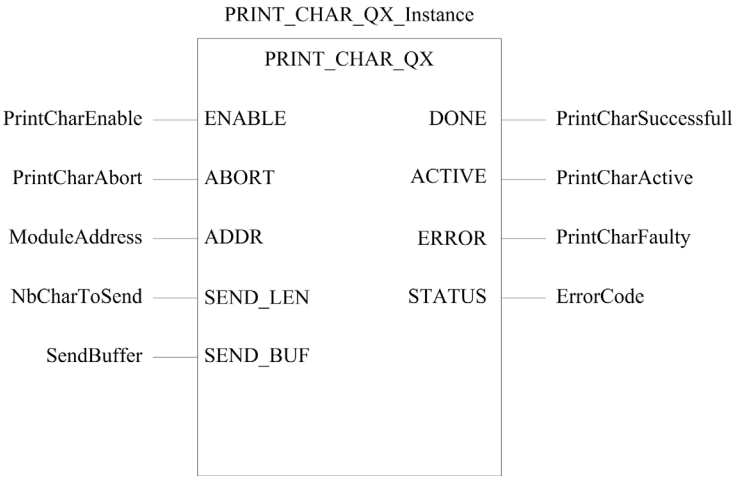
The communication function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

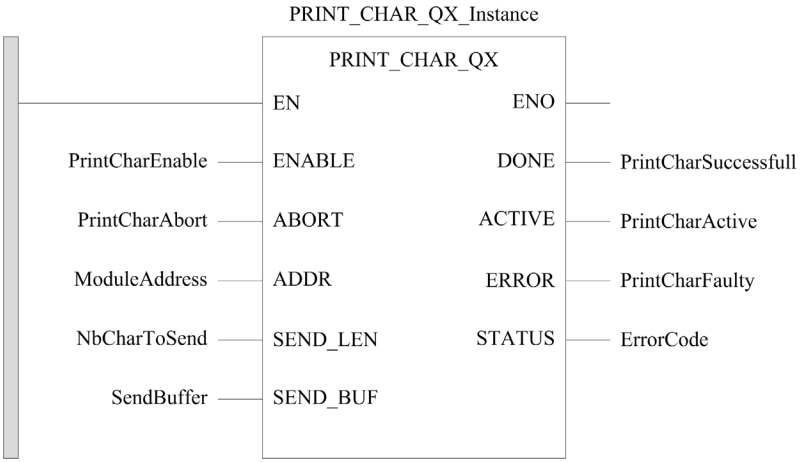
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
CAL PRINT_CHAR_QX_Instance (ENABLE:=PrintCharEnable,
ABORT:=PrintCharAbort, ADDR:=ModuleAddress, SEND_LEN:=NbCharToSend,
SEND_BUF:=SendBuffer, DONE=>PrintCharSuccessfull, ACTIVE=>PrintChar-
Active, ERROR=>PrintCharFaulty, STATUS=>ErrorCode)
```

ST Representation

Representation:

```
PRINT_CHAR_QX_Instance (ENABLE:=PrintCharEnable, ABORT:=PrintCharAbort,
ADDR:=ModuleAddress, SEND_LEN:=NbCharToSend, SEND_BUF:=SendBuffer,
DONE=>PrintCharSuccessfull, ACTIVE=>PrintCharActive, ERROR=>PrintChar-
Faulty, STATUS=>ErrorCode);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ENABLE	BOOL	Set to 1 to send a character string.
ABORT	BOOL	Set to 1 to abort the current operation.
ADDR	ANY_ARRAY_INT	Array containing the target Modicon X80 module address, result of ADDMX *(see page 77)* function.
SEND_LEN	INT	Number of characters to send.
SEND_BUF	ANY	Characters to send. **Note**: This character parameter must exist, even if there is no data to be sent.

Description of output parameters:

Parameter	Data type	Meaning
DONE	BOOL	Set to 1 when the execution of the function is completed successfully.
ACTIVE	BOOL	Set to 1 when the execution of the function is in progress.
ERROR	BOOL	Set to 1 if an error is detected by the function block.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.

Programming Rules

Special characters are preceded by the character `$` in the string to be transmitted. The `$` characters are not transmitted by the sender, and must therefore not be counted during the initialization of the length parameter.

The spaces between two characters are counted in a byte.

Therefore, for the example `PRINTING IN PROGRESS$L$R`, the length of the data to be sent is 22 bytes.

NOTE:
To send character strings containing end of string characters (ZERO), you must:

- use localized STRINGS
- initialize `SEND_LEN` with the number of characters to send.

Chapter 23
RCV_TLG: Receiving telegrams

Subject of this Chapter

This chapter describes the `RCV_TLG` function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	268
Example of how to receive a telegram	271

Description

Function Description

The RCV_TLG function is used to read telegram-type data from a remote application.

The data received must be a maximum of 16 bytes in length. Unlike the other communication functions, this function is processed immediately (synchronous): there are therefore no activity bit and timeout parameters.

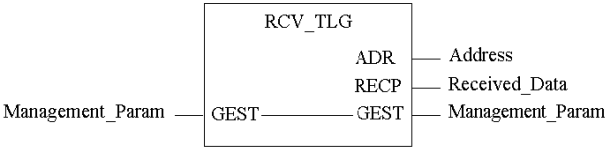
The integer table assigned to the management parameters therefore only uses two words instead of four (the exchange and timeout numbers being unnecessary).

NOTE: this function can be used on Fipway in the processor only, and for stations 0 to 15.

The additional parameters EN and ENO can be configured.

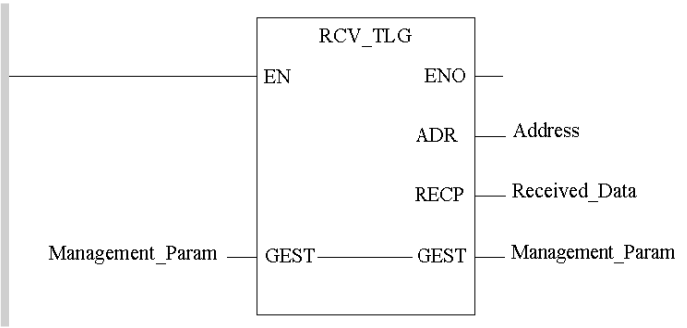
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
LD Management_Param
RCV_TLG Address, Received_Data
```

ST representation

Representation:

```
RCV_TLG(Management_Param, Address, Received_Data);
```

Parameter Description

The following table describes the input/output parameters:

Parameter	Type	Comment
`Management_Param`	ARRAY [0.. 1] OF `INT`	Table of words used to manage the exchange. This table is made up of 2 words: the report word and the word indicating the amount of data received (in number of bytes). The report comprises: • the operation report (most significant byte of the first word), • the communication report (least significant byte of the first word), The communication report takes one of the following values: • 16#00 : correct exchange, • 16#05 : incorrect management parameters, • 16#06 : incorrect specific parameters, • 16#09 : insufficient receive buffer size, • 16#0B: No system resources: the number of simultaneous communication EFs exceeds the maximum that can be managed by the processor, • 16#0D: no telegram received, • 16#10 : network module missing, • 16#0F: telegram service not configured.

The following table describes the output parameters:

Parameter	Type	Comment
Address	INT	Address of the entity transmitting the telegram at the end of the exchange. ● The least significant byte corresponds to the hexadecimal network number. ● The most significant byte corresponds to the hexadecimal station number.
Received_Data	ARRAY [n... m] OF INT	Receive buffer. Integer table containing the data received. This table must be a maximum of 8 integers (16 bytes) in length.

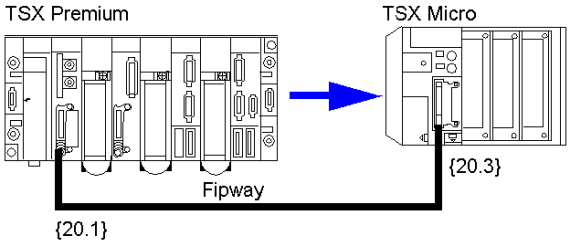
Example of how to receive a telegram

At a Glance

Let us assume we want to receive an 8-word (16 byte) telegram from a remote application on a Fipway network.

Illustration

The two stations are connected via a Fipway network.



Programming

Programming in ST:

```
IF RE(%I0.3.11) THEN
   RCV_TLG(%MW200:2, %MW300,%MW310:8);
END_IF;
```

Request parameters:

Parameters	Description
%MW200:2	Management table
%MW300	Contains the address of the sender at the end of the exchange.
%MW310:8	Contents of the telegram received

NOTE: When a `RCV_TLG` function is programmed in an event task, it cannot be used in the MAST or FAST task.
To execute this function synchronously, it is necessary to test the operation report immediately after the program line that activates the execution of this function.

Chapter 24
READ_ASYN: Reading data asynchronously

Description

Function Description

The READ_ASYN function is used to read 1 Kbyte of data via the asynchronous messaging channel of TSX ETY modules in TCP/IP mode.

The following data can be read-accessed:

- internal bits,
- internal words.

Asynchronous reading can only take place between two stations on the same Ethernet TCP/IP network segment.

The READ_ASYN function is sent at the end of the MAST task only if this is configured in periodic mode. It is possible to activate 8 functions simultaneously.

The operating principle is identical to that of the WRITE_ASYN *(see page 362)* function.

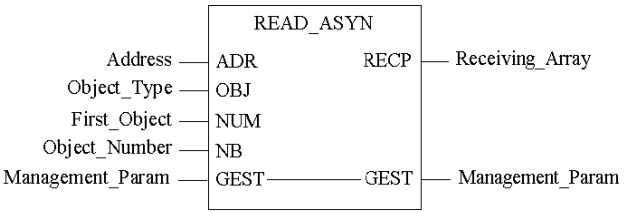
The size of the send and receive buffers is expressed in words. It is 512 words, or 1024 bytes.

NOTE: The asynchronous server function supports UNI-TE V1 or V2.0 protocols. The READ_ASYN function uses the UNI-TE V2.0 protocol.

The additional parameters EN and ENO can be configured.

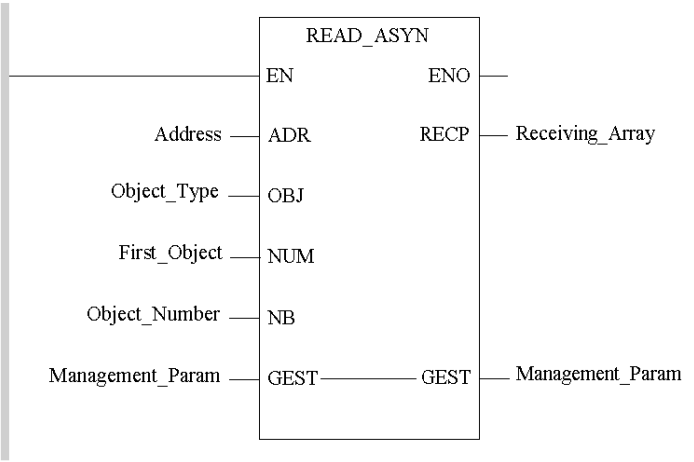
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
LD Address
READ_ASYN Object_Type, First_Object, Object_Number, Management_Param,
Receiving_Array
```

ST representation

Representation:

```
READ_ASYN(Address, Object_Type, First_Object, Object_Number,
Management_Param, Receiving_Array);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
Address	ARRAY [0.. 5] OF INT	Address of the destination entity of the exchange. Addresses are of the type: ADDR('{Network.Station}SYS.
Object_Type	STRING	Type of objects to read: ● '%M': internal bits, ● '%MW': internal words, ● '%S': system bits, ● '%SW': system words.
First_Object	DINT	Index of the first object to read.
Object_Number	INT	Number of objects to read.

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0.. 3] OF INT	Exchange management table *(see page 40)*

The following table describes the output parameters:

Parameter	Type	Comment
Receiving_Array	ARRAY [n... m] OF INT	Word table containing the value of the objects read.

Chapter 25
READ_GDATA: Reading Modbus Plus Global Data

Description

Function Description

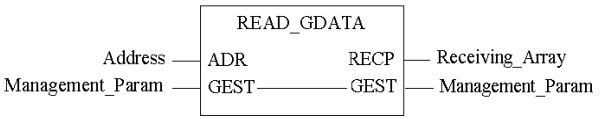
The READ_GDATA function is used to read shared data, or **Global Data**, on a **Modbus Plus** network.

Global Data is shared between a maximum of 64 stations on the same Modbus Plus network. Each station may write up to 32 integers, which can then be used by all stations on the network. Conversely, each station can read the 32 integers (maximum) of all the other network stations.

The additional parameters EN and ENO can be configured.

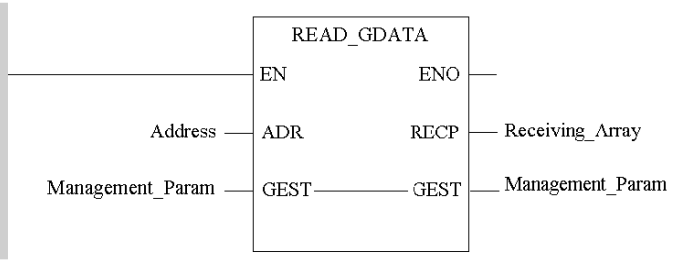
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
LD Address
READ_GDATA Management_Param, Receiving_Array
```

ST representation

Representation:

```
READ_GDATA(Address, Management_Param, Receiving_Array);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
Address	ARRAY [0.. 5] OF INT	Address of the destination entity of the exchange. This address is initialized, before the exchange, with the value of the node on which the station is connected whose Global Data is required. Example address: ADDR('0.0.1.10') corresponds to the station connected to node 10 of the network. The first three figures (0.0.1) correspond to the address of the channel of the Premium Modbus Plus PCMCIA card.

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0.. 3] OF INT	Exchange management table *(see page 40)*. It is not necessary to initialize the length parameter before launching the exchange. At the end of this exchange, this length parameter (the fourth word) contains the number of bytes that makes up the data produced by the station specified in the address.

The following table describes the output parameters:

Parameter	Type	Comment
Receiving_Array	ARRAY [n... m] OF INT	Word table containing the value of the objects read.

Chapter 26
READ_REG: Read Register

Introduction

This chapter describes the `READ_REG` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	280
Derived Data Types	283
Function mode	285
Parameter description	286

Description

Function description

With a rising edge at the REQ input, this function block reads a register area from an addressed slave via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet.

CREAD_REG, CWRITE_REG, READ_REG, WRITE_REG, and MBP_MSTR function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

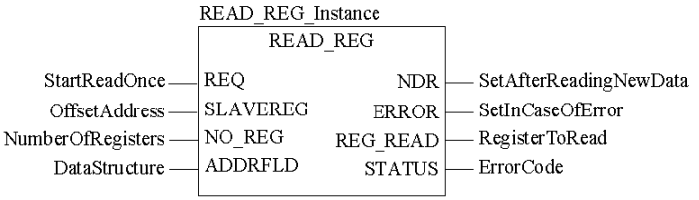
More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

NOTE: When programming a READ_REG function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the *Modbus Plus Network Planning and Installation Guide (Reference 31003525)*. If TCP/IP or SY/MAX Ethernet routing is implemented *(see Quantum with Unity Pro, TCP/IP Configuration, User Manual)*, standard Ethernet IP router products must be used.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

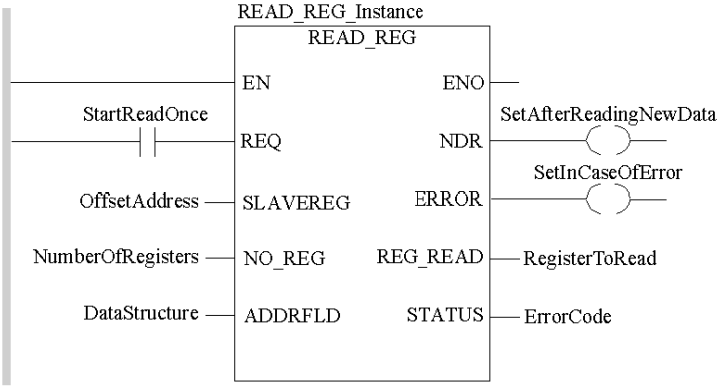
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL READ_REG_Instance (REQ:=StartReadOnce,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    ADDRFLD:=DataStructure, NDR=>SetAfterReadingNewData,
    ERROR=>SetInCaseOfError, REG_READ=>RegisterToRead,
    STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
READ_REG_Instance (REQ:=StartReadOnce,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    ADDRFLD:=DataStructure, NDR=>SetAfterReadingNewData,
    ERROR=>SetInCaseOfError, REG_READ=>RegisterToRead,
    STATUS=>ErrorCode) ;
```

Parameter description

Description of input parameters:

Parameter	Data Type	Meaning
REQ	BOOL,	With a rising edge at the REQ input, this function block reads a register area from an addressed slave via Modbus Plus, TCP/IP-Ethernet or SY/MAX-Ethernet.
SLAVEREG	DINT	Offset address of the first %MW register in the slave to be read from.
NO_REG	INT	Number of addresses to be read from slave
ADDRFLD	WordArr5	Data structure describing the Modbus Plus-address, TCP/IP address or SY/MAX-IP address.

Description of output parameters:

Parameter	Data Type	Meaning
NDR	BOOL	Set to 1 for one cycle after reading new data
ERROR	BOOL	Set to 1 for one cycle if an error occurs
STATUS	WORD,	If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.
REG_READ	ANY	Writing data (For the file to be read a data structure must be declared as a located variable.)

Runtime error

For a list of all block error codes and values, see the table of error codes *(see page 496)*.

Derived Data Types

Element description for `WordArr5` in Modbus Plus

Element description for `WordArr5` in Modbus Plus:

Element	Data type	Description
`WordArr5[1]`	`WORD`	Least significant byte: Routing register 1 is used for address specification (routing path addresses one of five) of the destination node during network transfer. The last byte in the routing path that is not zero is the destination node. Most significant byte: Source node address. ● slot position of the module when using the Modbus Plus Port on the NOM module. ● set to 0 when using the Modbus Plus Port on the CPU, (regardless of the CPU slot).
`WordArr5[2]`	`WORD`	Routing register 2
`WordArr5[3]`	`WORD`	Routing register 3
`WordArr5[4]`	`WORD`	Routing register 4
`WordArr5[5]`	`WORD`	Routing register 5

Element description for `WordArr5` with TCP/IP Ethernet

Element description for `WordArr5` with TCP/IP Ethernet

Element	Data type	Description
`WordArr5[1]`	`WORD`	Least significant byte: MBP on Ethernet Transporter (MET) mapping index Most significant byte: Slot of the NOE module (16#FE if Ethernet is integrated on the CPU)
`WordArr5[2]`	`WORD`	Byte 4 (MSB) of the 32-bit destination IP address
`WordArr5[3]`	`WORD`	Byte 3 of the 32-bit destination IP address
`WordArr5[4]`	`WORD`	Byte 2 of the 32-bit destination IP address
`WordArr5[5]`	`WORD`	Byte 1 (LSB) of the 32-bit destination IP address

Element description for WordArr5 with SY/MAX Ethernet

Element description for WordArr5 with SY/MAX Ethernet:

Element	Data type	Description
WordArr5[1]	WORD	Least significant byte: MBP on Ethernet Transporter (MET) mapping index Most significant byte: Slot of the NOE module
WordArr5[2]	WORD	Destination drop number (or set to FF hex)
WordArr5[3]	WORD	Terminator (set to FF hex)
WordArr5[4]	WORD	No significance
WordArr5[5]	WORD	No significance

Function mode

Function mode of READ_REG blocks

Although a large number of READ_REG function blocks can be programmed, only four read operations may be active at the same time. In such a case it is insignificant whether they are the result of this function block or others (e.g. MBP_MSTR, CREAD_REG). All function blocks use one data transaction path and require multiple cycles to complete a job.

NOTE: A TCP/IP communication between a Quantum PLC (NOE 211 00) and a Momentum PLC (all TCP/IP CPUs and all TCP/IP I/O modules) is only possible, when only **one** read or write job is carried out in every cycle. If several jobs are sent per PLC cycle, the communication stops without generating an error message in the status register of the function block.

All routing information is contained in the WordArr5 data structure of the ADDRFLD input. The type function block assigned to this input is defined by the network used.

Please use:

- Modbus Plus for function block ModbusP_ADDR
- TCP/IP Ethernet for function block TCP_IP_ADDR
- SY/MAX Ethernet for function block SYMAX_IP_ADDR

NOTE: The WordArr5 data structure can also be used with constants.

Parameter description

REQ

A rising edge triggers the read transaction.

The parameter can be entered as a direct address, located variable, unlocated variable or literal.

SLAVEREG

Start of the area in the addressed slave from which the source data is read. The source area always resides within the %MW register area.

NOTE: For slaves for a **non-**Unity Pro PLC:
The source area always resides within the 4x register area. `SLAVEREG` expects the source reference as offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).
The parameter can be entered as a direct address, located variable, unlocated variable or literal.

NO_REG

Number of addresses to be read from the addressed slave (1 ... 100).

The parameter can be entered as a direct address, located variable, unlocated variable or literal.

NDR

Transition to ON state for one program cycle signifies receipt of new data ready to be processed.

The parameter can be entered as a direct address, located variable or unlocated variable.

ERROR

Transition to ON state for one program cycle signifies the detection of a new error.

The parameter can be entered as a direct address, located variable or unlocated variable.

REG_READ

An `ARRAY` that is the same size as the requested transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

STATUS

If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

The parameter can be entered as an address, located variable or unlocated variable.

Chapter 27
READ_REG_QX: Read Register on EIO Bus

Description

Function description

The READ_REG_QX function is used to read registers in a Modbus slave connected to a Modicon X80 serial communication module located in an EIO drop.

When programming a READ_REG_QX function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the Modicon Modbus Plus Network Planning and Installation Guide *(see page 15)*.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

The additional parameters EN and ENO can be configured.

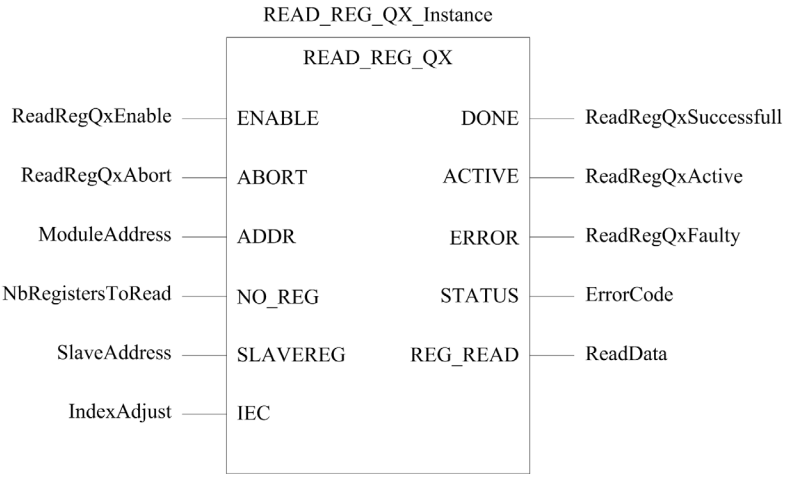
The communication function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

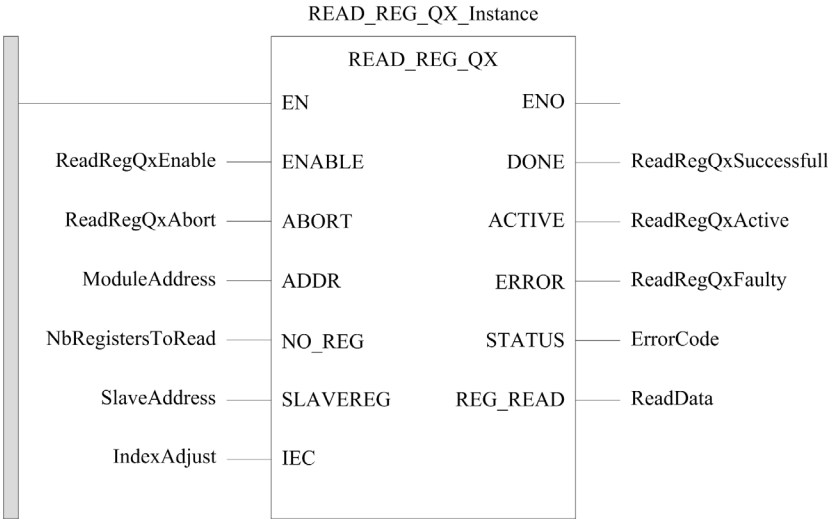
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL READ_REG_QX_Instance (ENABLE:=ReadRegQxEnable, ABORT:=ReadRegQ-
xAbort, ADDR:=ModuleAddress, NO_REG:=NbRegistersToRead,
SLAVEREG:=SlaveAddress, IEC:=IndexAdjust, DONE=>ReadRegQxSuccessfull,
ACTIVE=>ReadRegQxActive, ERROR=>ReadRegQxFaulty, STATUS=>ErrorCode,
REG_READ=>ReadData)
```

Representation in ST

Representation:

```
READ_REG_QX_Instance (ENABLE:=ReadRegQxEnable, ABORT:=ReadRegQxAbort,
ADDR:=ModuleAddress, NO_REG:=NbRegistersToRead, SLAVEREG:=SlaveAddress,
IEC:=IndexAdjust, DONE=>ReadRegQxSuccessfull, ACTIVE=>ReadRegQxActive,
ERROR=>ReadRegQxFaulty, STATUS=>ErrorCode, REG_READ=>ReadData)
```

Parameter description

Description of input parameters:

Parameter	Data type	Meaning
ENABLE	BOOL	Set to 1 to read registers.
ABORT	BOOL	Set to 1 to abort the current operation.
ADDR	ANY_ARRAY_INT	Array containing the Modbus slave address, result of ADDMX *(see page 77)* function.
NO_REG	INT	Number of registers to be read in the Modbus slave.
SLAVEREG	DINT	Offset address of the first %MW register to read in the Modbus slave.
IEC	INT	Index adjust applied to SLAVEREG parameter. IEC parameter can have 2 values: • 0: $\%MW_x$ • 1: $\%MW_{x+1}$ IEC default value is 0 for a Modicon M340 target.

Description of output parameters:

Parameter	Data type	Meaning
DONE	BOOL	Set to 1 when the execution of the function is completed successfully.
ACTIVE	BOOL	Set to 1 when the execution of the function is in progress.
ERROR	BOOL	Set to 1 if an error is detected by the function block.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.
REG_READ	ANY_ARRAY_WORD	Read data (A data structure must be declared as a located variable to receive the response from the Modbus slave.)

Chapter 28
READ_SDO: Reading Service Data Object

READ_SDO: Reading Service Data Object

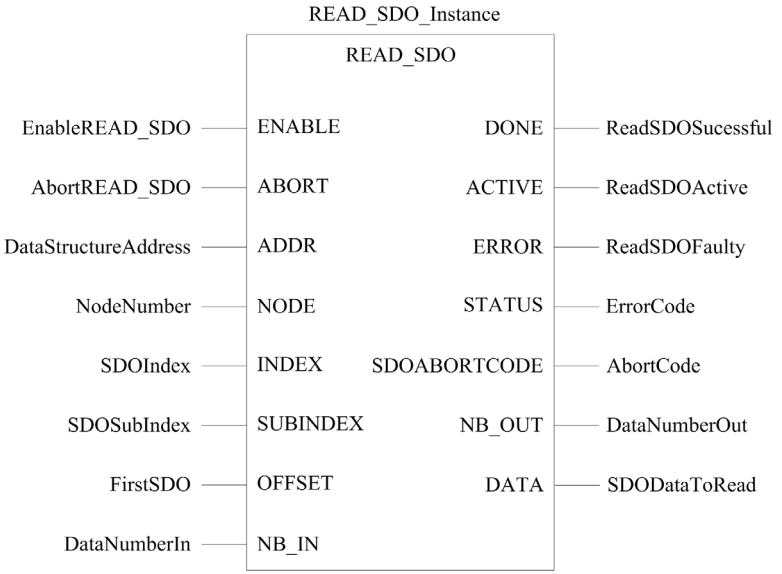
Function Description

The `READ_SDO` function block reads (explicit exchange) from the PLC application up to the device (SDO).

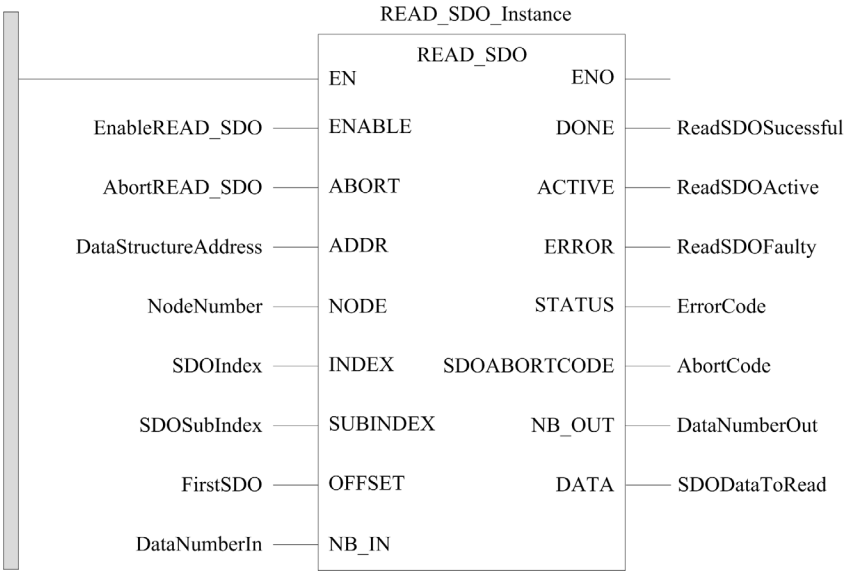
This function block provides access to the abort code when the SDO command is not successful (only if the fieldbus is in RUN mode, and only towards the configured devices).

FBD Representation

Representation:



LD Representation



IL Representation

Representation:

```
CAL READ_SDO_Instance (ENABLE := EnableREAD_SDO,
ABORT := AbortREAD_SDO, ADDR := DataStructureAddress,
NODE := NodeNumber, INDEX := SDOIndex, SUBINDEX := SDOSubIndex,
OFFSET := FirstSDO, NB_IN := DataNumberIn, DONE => ReadSDOSuccessful,
ACTIVE => ReadSDOActive, ERROR => ReadSDOFaulty, STATUS => ErrorCode,
SDOABORTCODE => AbortCode, NB_OUT => DataNumberOut,
DATA => SDODataToRead)
```

ST Representation

Representation:

```
READ_SDO_Instance (ENABLE := EnableREAD_SDO, ABORT := AbortREAD_SDO,
ADDR := DataStructureAddress, NODE := NodeNumber, INDEX := SDOIndex,
SUBINDEX := SDOSubIndex, OFFSET := FirstSDO, NB_IN := DataNumberIn,
DONE => ReadSDOSuccessful, ACTIVE => ReadSDOActive,
ERROR => ReadSDOFaulty, STATUS => ErrorCode, SDOABORTCODE => AbortCode,
NB_OUT => DataNumberOut, DATA => SDODataToRead)
```

Parameter Description

The following table describes the input parameters:

Input parameter	Data type	Description
ENABLE	BOOL	ON: the operation is enabled.
ABORT	BOOL	ON: the currently active operation is aborted.
ADDR	ANY_ARRAY_INT	Array containing the address of the destination entity of the read operation, result of ADDMX function.
NODE	BYTE	Byte used to select a particular NMT slave device on the CANopen network (16#01 to 16#7F).
INDEX	INT	Two bytes used to access a particular object in a CANopen SDO server device.
SUBINDEX	BYTE	Byte used to access a particular subobject in a CANopen SDO server device.
OFFSET	INT	Two bytes indicating the starting offset into the selected object. It can be non-zero when performing segmented SDO transfers. **NOTE:** Not used when addressing an EtherNet/IP module (address with CIP suffix).
NB_IN	INT	Two bytes providing a count of the desired number of data values to read (in bytes). **NOTE:** • If set to 0, the number of data to read is set to the size of the variable associated to the output parameter DATA. • When used with the BMECXM0100 module, this input parameter equal 0 whatever the value you set.

The following table describes the output parameters:

Output parameter	Data type	Description
DONE	BOOL	ON: the operation concludes successfully.
ACTIVE	BOOL	ON: the operation is active.
ERROR	BOOL	ON: the operation is aborted without success.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.
SDOABORTCODE	DWORD	SDO abort code *(see page 518)* when STATUS = 16#4007
NB_OUT	INT	Size of data (in BYTES) actually returned in the output parameter DATA.
DATA	ANY_ARRAY_BYTE	Read data.

Chapter 29
READ_VAR: Reading variables

Subject of this Chapter

This chapter describes the `READ_VAR` communication function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	296
Assisted entry screen	303
Example of use on a Uni-Telway bus	305
Example of Reading Bits	307
Example of use in a network	309
Example of Reading Words via Serial Link of Modicon M340 Processors	311
Example including execution check	313

Description

Function Description

The READ_VAR function is used to read the value of one or more language objects:

- internal bits,
- internal words.

Verify that the objects to be read are consecutive. They may be located in a remote CPU or in a device connected to a communication channel.

On Modicon M340 and M580 CPU, the READ_VAR function can read up to 2,000 consecutive bits in a remote device.

On Premium PLCs, the READ_VAR function can read up to 1,000 consecutive bits in a remote device, whatever the device and the protocol used may be (Uni-Telway or Modbus/Jbus).

NOTE: For Premium PLCs only, reading over 1000 bits requires the use of the SEND_REQ function.

⚠ WARNING

EXCHANGED DATA INCOMPATIBILITY

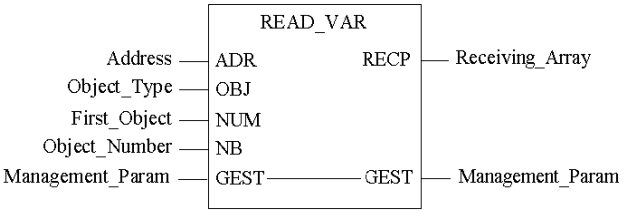
Data structure alignments are not the same for Premium/Quantum and M340/M580 so verify that the data exchanged are compatible. See DDT: Mapping rules *(see Unity Pro, Program Languages and Structure, Reference Manual)* page for alignment rules.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

The additional parameters EN and ENO can be configured.

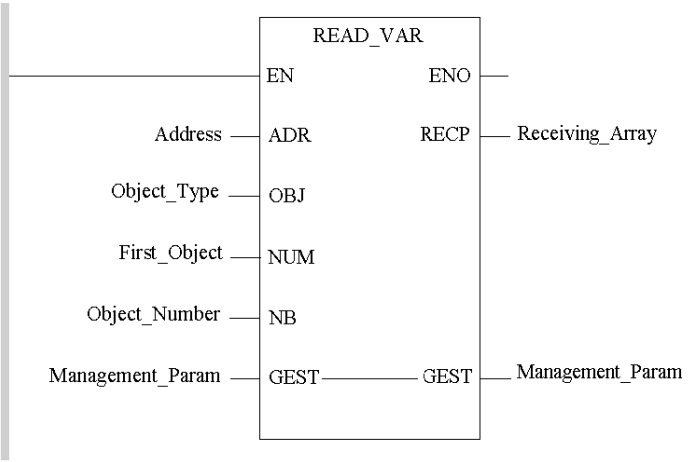
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
```

```
READ_VAR Object_Type, First_Object, Object_Number, Management_Param, Re
ceiving_Array
```

ST Representation

Representation:

```
READ_VAR(Address, Object_Type, First_Object, Object_Number, Management_
Param, Receiving_Array);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0..5] OF INT for Premium ARRAY [0..7] OF INT for Modicon M340 and M580	Specific statements by hardware platforms: ● Premium: ❍ Address of the message's receiving character mode channel is given by the ADDR *(see page 83)* function. ❍ Address of the destination entity of the exchange. The following addresses are prohibited: • {Network.Station}APP, • {Network.Station}APP.num. ● Modicon M340: ❍ Address of the message's receiving character mode channel is given by the ADDM *(see page 71)* function. ❍ The syntax of the address is of ADDM ('r.m.c.node')-type. ● Modicon M580: ❍ Address of the destination entity of the read operation, result of ADDMX *(see page 77)* function.
OBJ	STRING	Type of objects to read for Premium PLC: ● '%M': internal bits ● '%MW': internal words ● '%S': system bits ● '%SW': system words ● '%I': input bits ● '%IW': input words Type of objects to read for Modicon M340 and M580 CPU: ● '%M': internal bits ● '%MW': internal words ● '%I': input bits ● '%IW': input words
NUM	DINT	Index of the first object to read.
NB	INT	Number of objects to read.

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0..3] OF INT	Exchange management table consisting of the following words: ● Rank 1 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: exchange number ❍ Least significant byte: activity bit (rank 0) and cancel bit (rank 1) **NOTE:** The cancel bit is only available for Modicon M340 and M580 PLCs. ● Rank 2 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: operation report ❍ Least significant byte: communication report ● Rank 3 word: a word managed by the user which defines the maximum response time using a time base of 100 ms. ● Rank 4 word: a word managed by the system which defines the length of the exchange. For detailed information, refer to section *Structure of the Management Parameters (see page 40)*.

The following table describes the output parameters:

Parameter	Type	Comment
RECP	ARRAY [n..m] OF INT	Word table containing the value of the objects read. **NOTE:** The size of the data received (in bytes) is written automatically by the system in the fourth word of the management table.

Rules of Use

Verify that object types are entered consistently. Write entries either all in lower case or all in upper case; otherwise, the function sends back a report equal to 16#06 (incorrect specific parameters).

For Micro, Premium or Atrium PLCs receiving the `READ_VAR` function, read-only access to internal bits has the following particularities:

- The forcing value of the bits is returned in the response,
- Reading 1 bit therefore comprises 2 response bytes:
  - The first contains the value of the 8 bits from the one requested.
  - The second indicates whether these bits have been forced.
- In order to read one of the last 8 bits of the memory it is necessary to read all the last 8 bits, otherwise the function sends back a 16#01 operation report. This last particularity also applies to Modicon M340 and M580 PLCs.

NOTE: Verify that space for the forcing indication bytes are provided for in the receive table; otherwise, a 16#03 detected error code is sent back in the report.

NOTE: For Modicon M340/M580 PLCs, forced bits are not accessed by `READ_VAR` and `WRITE_VAR` functions, since Modbus protocol does not support it.

Simultaneous Transactions

The following table gives the capacities of each communication channel to simultaneously process transactions according to various configurations on Micro and Premium PLCs.

Configuration	Micro	TSX 57 10	TSX 57 20	TSX 57 23/30/40/45/55, PCX 57, PMX 57	TSX 57 46/56
Uni-Telway master terminal port	4	4	4	4	8
Uni-Telway master PCMCIA or SCY link	1	8	8	8	8
Uni-Telway client slave terminal port	4	1	1	1	8
Uni-Telway client slave PCMCIA or SCY link	1	1	1	1	1
Uni-Telway server slave terminal port	4	4	4	4	4
Uni-Telway server slave PCMCIA or SCY link	4	6	6	6	6
Modbus terminal port	4	-	-	-	-
Modbus PCMCIA or SCY link	4	8	8	8	8
Character mode terminal block	1	1	1	1	1
Character mode PCMCIA or SCY link	4	8	8	8	8
CANopen PCMCIA	-	10	10	10	10
Fipway PCMCIA or SCY link	4	8	8	8	8
Modbus Plus	4	4	4	4	4

Configuration	Micro	TSX 57 10	TSX 57 20	TSX 57 23/30/40/45/55, PCX 57, PMX 57	TSX 57 46/56
Ethernet	-	16	16	16	16
Embedded Ethernet	-	-	-	-	64

Capacities of each communication channel to simultaneously process transactions according to various configurations on Modicon M340 PLCs:

Configuration	BMX P34 1000	BMX P34 2000	BMX P34 2010/ 20102	BMX P34 2020	BMX P34 2030/ 20302
Embedded CANopen	-	-	16	-	16
Embedded Ethernet	-	-	-	16	16
Modbus master serial port	8	16	16	16	-

Maximum number of requests as client per scan on *standalone* M580 CPUs:

Configuration	CPU Reference (BME P58 ...)					
	10•0	20•0	30•0	40•0	5040	6040
Ethernet	16	32	48	80	80	96

Maximum number of requests as client per scan on *Hot Standby* M580 CPUs:

Configuration	CPU Reference (BME H58 ...)		
	2040	4040	6040
Ethernet	32	80	96

Maximum number of requests served per scan on *standalone* M580 CPUs:

Configuration	CPU Reference (BME P58 ...)					
	10•0	20•0	30•0	40•0	5040	6040
CPU from all origin (1.)	16	24	32	40	48	64
Embedded Ethernet	8	12	16	24 (2.)	32	32
USB	4	4	4	4	4	4

1. `%SW90` system word allows to adjust the number of requests served per cycle. The minimum number of requests served per cycle is set to 2 in order to reduce the CPU jitter.
2. Maximum number of requests is 16 for M580 CPU BME P58 40•0 with firmware version ≤ 1.20.

Maximum number of requests served per scan on *Hot Standby* M580 CPUs:

Configuration	CPU Reference (BME H58 ...)		
	2040	4040	6040
CPU from all origin (1.)	24	40	64
Embedded Ethernet	12	24	32
USB	4	4	4
1. %SW90 system word allows to adjust the number of requests served per cycle. The minimum number of requests served per cycle is set to 2 in order to reduce the CPU jitter.			

NOTE: Verify that the maximum number of simultaneous requests given in all tables takes into account the maximum number of simultaneous requests that each CPU can manage per cycle, as Client and as Server *(see Modicon M580, Hardware, Reference Manual)*.

Assisted entry screen

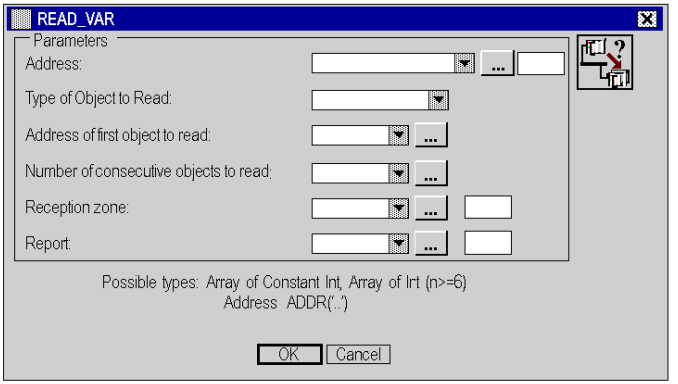
At a Glance

For this communication function, you can call up the assisted entry screen. Please note that the assisted entry screen is not available for Modicon M340/M580.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an example of the assisted entry screen of the function:



Address

For Premium PLCs, the possible type of object is:

- `ADDR(STRING).`
- `ARRAY [0..5] OF INT.`

NOTE: if you enter a value directly in the field, the assisted address entry button turns gray.

Type of object to read.

For Premium PLCs, possible choices are:

- `'%M'` for reading internal bits.
- `'%MW'` for reading internal words.
- `'%S'` for reading system bits.
- `'%SW'` for reading system words.
- `'%I'` for reading input bits.
- `'%IW'` for reading input words.

NOTE: simply choose from the solutions offered in the drop-down menu.

Address of first object to read

The possible objects are of the DINT type:

- variables,
- constants,
- immediate value.

NOTE: if you enter a constant, an entry field appears to enter the constant. If you enter a variable, this can be located or not.

Number of consecutive objects to read

The possible objects are of the INT type:

- variables,
- constants,
- immediate value.

NOTE: if you enter a constant, an entry field appears to enter the constant. If you enter a variable, this can be located or not.

Reception zone

The reception zone is an integer array. The size of this array depends on the number of objects to read. This integer array can be located or not.

Report

The report is an array of 4 integers.

NOTE: be careful not to use several identical memory areas for report tables, otherwise the variable read function may not work.

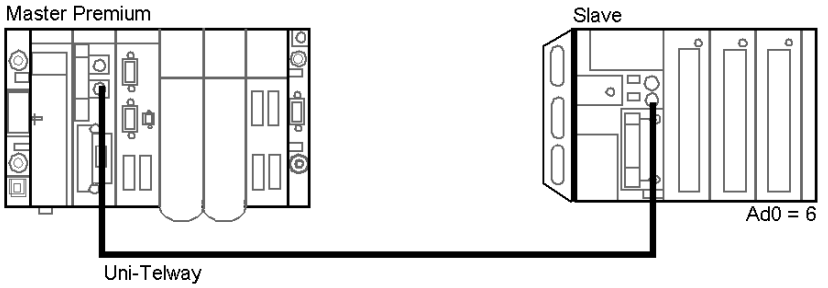
Example of use on a Uni-Telway bus

At a Glance

The master PLC must read internal words %MW100 to %MW109 of address station 6 on a Uni-Telway bus. The values of the words that are read are sorted on the basis of the internal word %MW10, and management parameters are stored on the basis of %MW40.

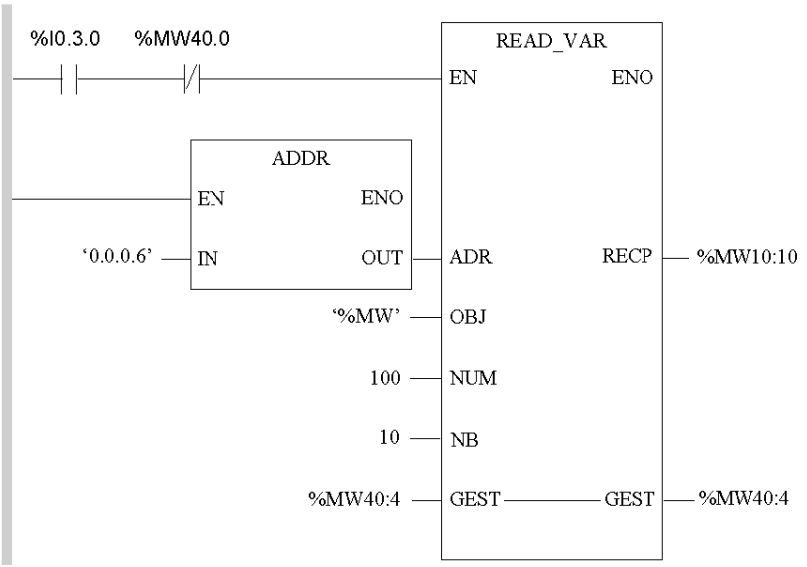
Illustration

Both stations are linked by a Uni-Telway bus.



Programming

Programming in LD:



Request parameters:

Parameters	Description
'0.0.0.6'	• 0 : rack • 0 : module • 0 : channel 0 • 6 : destination address
'%MW'	Type of object (internal word)
100	Address of first object
10	Number of consecutive objects
%MW40:4	Report
%MW10:10	Contents of the response

Example of Reading Bits

At a Glance

On Modicon M340 PLCs, forced bits are not accessed by READ_VAR and WRITE_VAR functions as Modbus protocol does not support it. Therefore the following statements about forced bits cannot be applied to Modicon M340 PLCs.

The following examples illustrate the READ_VAR communication function for reading bits. In this case the receive table consecutively contains the value of the bits as well as the forcing indication.

Reading 32 internal bits

In ST, the syntax of the internal bit reading function is as follows:

```
READ_VAR (ADDR('{20.1}0.5.1.3'),'%M', 0, 32, %MW100:4, %MW50:4);
```

The receive table must include 8 bytes (4 words), 4 bytes for the value and 4 bytes for the forcing indication.

	Word	Byte 3	Byte 2	Byte 1	Byte 0
Value	%MW100	0000	0000	**1100**	**1100**
	%MW101	1111	1111	0000	1111
Forcing	%MW102	0000	0000	**0101**	**0101**
	%MW103	0000	0000	0000	1111

Forcing can be performed for each %MW102 or %MW103 bit set to 1; the forcing value being that of the corresponding bit read.

Example:

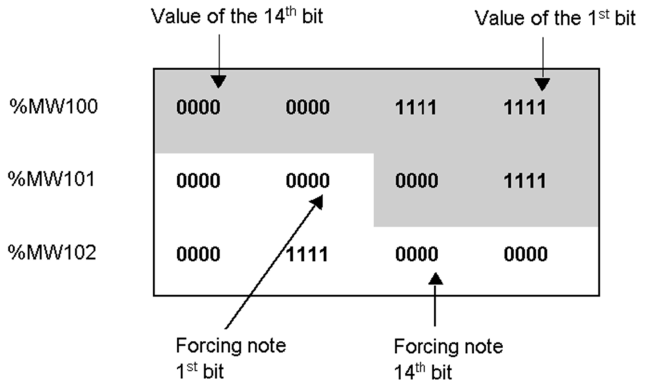
Value of the first 4 bits (byte 0 of word %MW100)	Forcing of the first 4 bits (byte 0 of word %MW102)	Description
0	1	The bit is forced to 0
0	0	The bit is not forced
1	1	The bit is forced to 1
1	0	The bit is not forced

Reading 18 internal bits

In ST, the syntax of the internal bit reading function is as follows:

```
READ_VAR (ADDR('{20.1}0.5.1.3'),'%M', 0, 18, %MW100:3, %MW50:4);
```

The receive table must contain 3 words (or 6 bytes). In fact, to obtain the value of the 18 bits, 3 bytes plus a further 3 bytes are needed in order to contain the forcing value of the 18 bits.



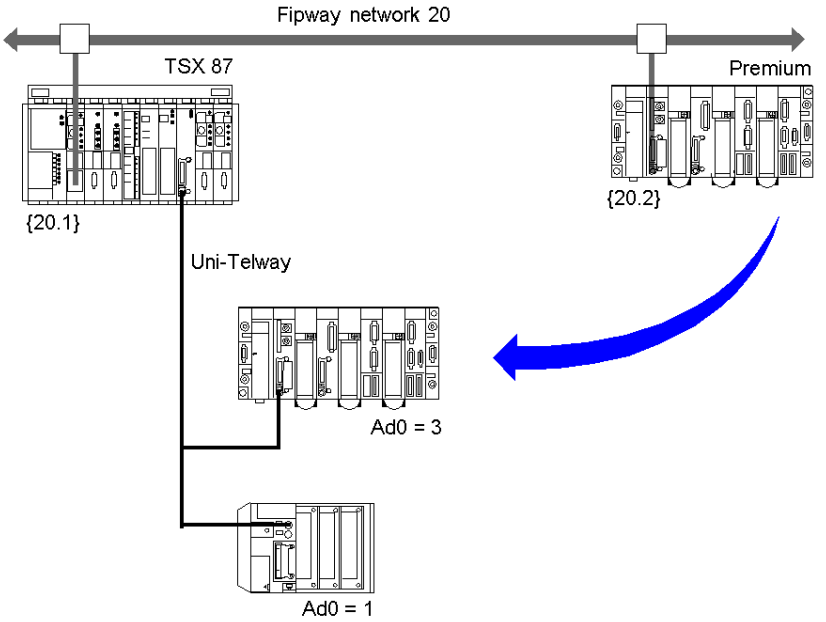
Example of use in a network

At a Glance

Station 2 of network 20 must read a 5-word table %MW0 to %MW4 of the Uni-Telway slave, network address 20, station 1, communication module TSX SCM 2116 in slot 5, channel 1 in the communication module, server address Ad0 = 3.

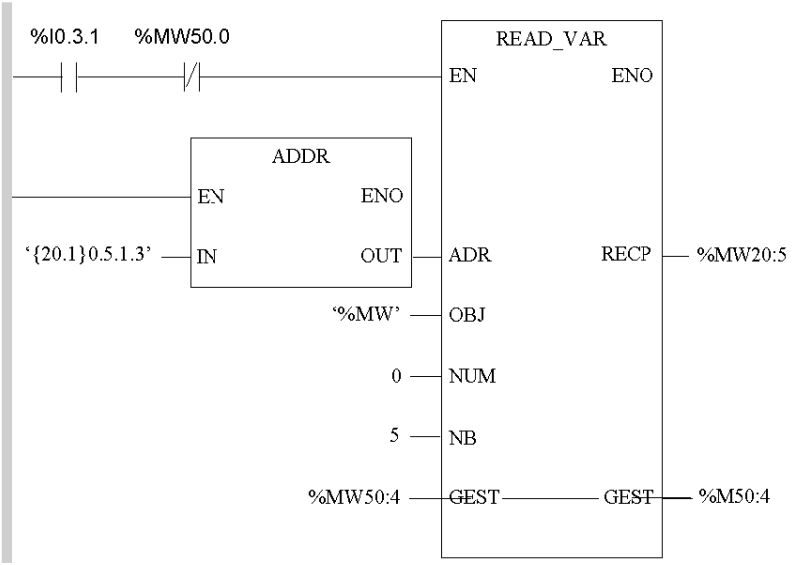
Illustration

The two stations are connected via a Fipway network.



Programming

Programming in LD:



Request parameters:

Parameters	Description
ADDR('{20.1}0.5.1.3')	• {20.1} : network 20, station 1 • 0 : rack • 5 : module • 1 : channel 1 • 3 : read destination slave address
'%MW'	Type of object (internal word)
0	address of first object
5	Number of consecutive objects
%MW50:4	Management table
%MW20:5	Contents of the response

Example of Reading Words via Serial Link of Modicon M340 Processors

At a Glance

That example deals with 2 Modicon M340 processors which communicate via Modbus serial link. The Modbus master PLC must read internal words `%MW100` to `%MW109` of the Modbus slave PLC.

Description of the Example

The values of the words that are read are sorted on the basis of the internal word `%MW10`.

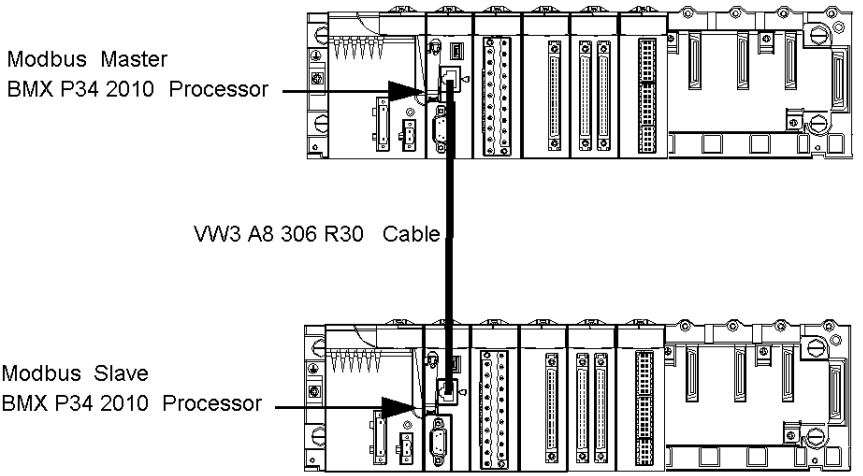
The following example uses unlocated variables, and covers the reception of data in an unlocated 10-word table named `Tab_recp` (declared as an `ARRAY [0..9] OF INT`). The management parameters are located in a table of 4 integers called `Management_Parameter` (declared as `ARRAY [0..3] OF INT`).

In that example, the Modbus slave's number is 7, so the input ADDM parameter is '0.0.0.7':

- 0: rack number of the processor equal to 0
- 0: slot number of the processor within the rack, equal to 0 as the slot number of a Modicon M340 processor is always 0
- 0: channel number, equal to 0 as the serial link of a Modicon M340 processor is always channel 0
- 7: configured slave number is 7.

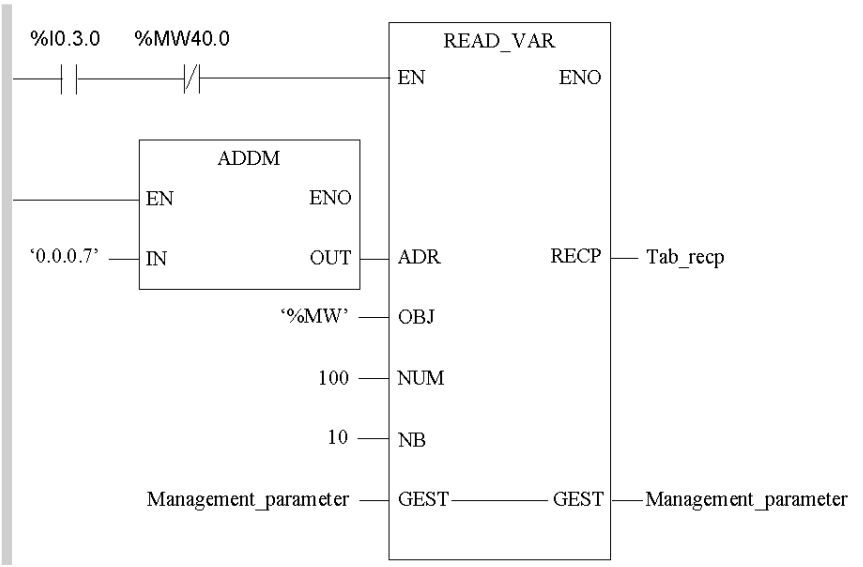
Illustration

The two Modicon 340 processors are connected via a Modbus link:



Programming

Programming in LD:



The request parameters are as follows:

Parameters	Description
ADDM('0.0.0.7')	• 0: rack number of the slave processor • 0: slot number of the slave processor • 0: channel number (serial port number) • 7: configured slave number
'%MW'	Type of object (internal word)
100	Address of the first object to read
10	Number of consecutive objects to read
Tab_recp	Contents of the response
Management_Parameter	Management table

Example including execution check

At a Glance

The following example illustrates the READ_VAR function with a management parameter check.

Programming the function

Programming in ST:

```
IF NOT %M21 AND %I0.1.2 THEN
   %MW210:4 := 0;
   %MW212 := 50;
   READ_VAR(ADDR('0.3.1.7'),'%MW',20,1,%MW210:4,%MW1701:1);
   SET %M21;
END_IF;
```

- the input bit %I0.1.2 controls the function,
- the internet bit %M21 is used to test the activity of the function,
- %MW210:4 := 0; initializes the management table to 0,
- MW212 := 50; initializes the timeout value to 5 seconds.

NOTE: READ_VAR(ADDM('0.3.1.7'),'%MW',20,1,%MW210:4,%MW1701:1); syntax must be used for Modicon M340 PLCs as ADDR function cannot be used by a Modicon M340 PLC.

Programming the exchange check

Programming in ST:

```
IF %M21 AND NOT %M210.0 THEN
   INC %MW214;
   IF %MW211 = 0 THEN
      INC %MW215;
   ELSE
      SET %Q0.2.2;
      INC %MW216;
      %MW217 := %MW211;
   END_IF;
END_IF;
```

- `%MW214` counts the number of exchanges,
- `%MW215` counts the number of correct exchanges,
- `%MW216` counts the number of exchanges generating errors,
- `%MW217` stores the error message,
- `%Q0.2.2` indicates an exchange failure.

Chapter 30
SEND_EMAIL: Sending Email

Send Email

Function Description

The SEND_EMAIL function is used for sending an email with a **Modicon M340**.

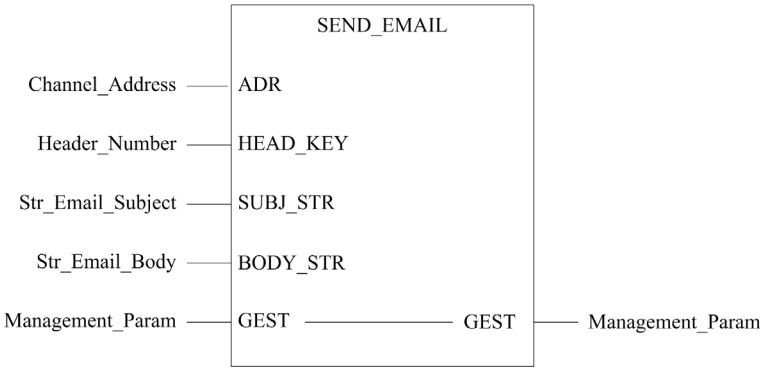
The SEND_EMAIL service is limited to the embedded Ethernet port and the BMX NOC 0401 modules. It is not supported through a BMX NOE module.

NOTE: For the SEND_EMAIL service to function properly, the IP address of the SMTP server and usable email destinations must first be configured in the Unity Pro software.

The additional parameters EN and ENO can be configured.

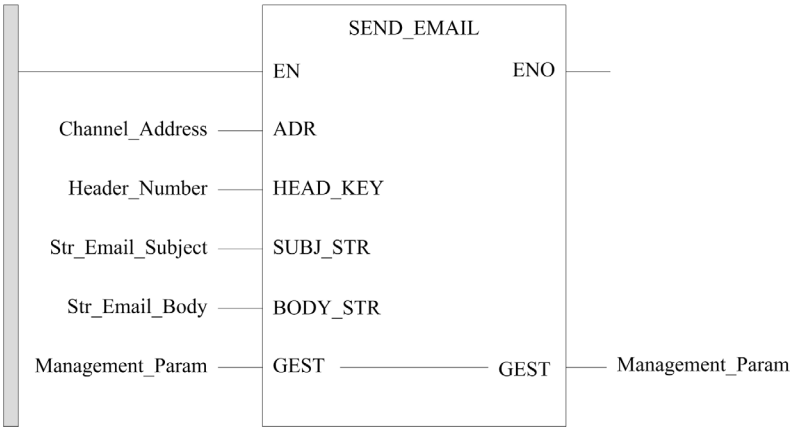
FBD representation

Representation:



LD representation

Representation:



IL representation

```
CAL SEND_EMAIL (ADR := Channel_Address, HEAD_KEY := Header_Number,
SUBJ_STR := Str_Email_Subject, BODY_STR := Str_Email_Body, GEST :=
Management_Param);
```

ST representation

```
SEND_EMAIL (ADR := Channel_Address, HEAD_KEY := Header_Number, SUBJ_STR
:= Str_Email_Subject, BODY_STR := Str_Email_Body, GEST :=
Management_Param);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Description
`Channel_Address`	`ARRAY [0..7] OF INT`	Address or result of the `ADDM`. It uses `ADDM` EF to build this field.ADDM Function *(see page 71)* To send an email through a CPU Ethernet port, use `ADDM('0.0.3')`. To send an email through a BMX NOC module configured at rack 0, slot 4, channel 0, use `ADDM('0.4.0')`.
`Header_Number`	`INT`	Corresponds to the emails addresses handled with Unity Pro software (only 1, 2, 3 are accepted).

Parameter	Type	Description
Str_Email_Subject	STRING	Represents the dynamic part of the subject that is appended to the static subject string.
Str_Email_Body	STRING	Represents the body of the email.

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0..3] OF INT	Exchange management table *(see page 40)*. 4-word table used to control parameters of the execution. The length parameter (4th word of the management table) is an output parameter. This word is written by the system with the total length of the email (header + body). The maximum size of header + body is 1024 characters.

Send Email Rules

After the launch of a SEND_EMAIL EF, the activity bit is set until the mail is sent. However, there is no confirmation that the mail arrived at the destination. If a timeout is programmed (3rd management word <> 0), the mail is canceled if it has not been sent during the programmed time.In this case, the 2nd management word receives a negative response **Exchange stop on timeout** (0x01).

Run-time characteristics : the **Modicon M340** system is able to manage four simultaneous send requests, coming from four EFs. A 5th EF trying to send an email will get the information **No processor system resources**(0x0B) until one of the resources is free.

Send Email Service Example

```
IF (default_id = 0) and FE(default_state) THEN
   (* PUMP IS OK *)
  SEND_EMAIL(ADDM('0.0.3'), 1, 'Pump n°3 is OK', ' ' , Management_Param);
ELSEIF RE(default_state) THEN
   (* PUMP IS FAULTY *)
   str_default := INT_TO_STRING(default_id);
   str_Email_Body := CONCAT_STR(' Default = ', str_default);
   SEND_EMAIL(ADDM('0.0.3'), 1, 'Pump n°3 is faulty', str_Email_Body,
Management_Param);
END_IF;
```

Chapter 31
SEND_REQ: Sending requests

Subject of this Chapter

This chapter describes the `SEND_REQ` function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	320
List of UNI-TE Requests	324
Assisted entry screen	330
Example of how to send a UNI-TE request	332
Changing IP Parameters with SEND_REQ (Example)	334
Using the SEND_REQ function	335

Description

Function Description

The SEND_REQ function is used to code and send all UNI-TE and Modbus/Jbus requests, as well as receive the associated responses.

Coding details for UNI-TE requests are provided in the reference manual: Communication Reference TSX DR NET.

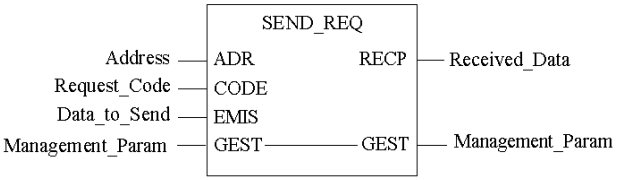
Details on coding the Modbus/Jbus requests are provided in the TSX DG MDB manual.

Modbus requests common to all Schneider devices are presented in Architecture and Communication Services *(see Communication Services and Architectures, Reference Manual)*.

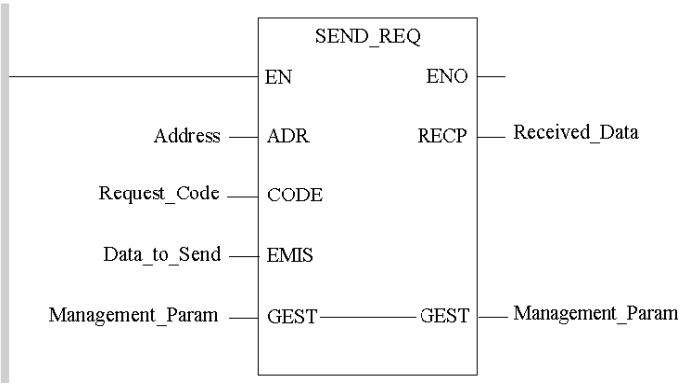
NOTE: this function could not be used to send Modbus/Jbus requests through a Premium ETY module.

The additional parameters EN and ENO can be configured.

FBD Representation



LD Representation



IL Representation

```
LD Address
SEND_REQ Request_Code, Data_to_Send, Management_Param, Received_Data
```

ST Representation

```
SEND_REQ(Address, Request_Code, Data_to_Send, Management_Param,
Received_Data);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
Address	ARRAY [0.. 5] OF INT	Address of the destination entity of the exchange. The type of address depends on the request sent. It is therefore possible, for example, to broadcast the RUN request (ALL, 0 for the TSX SCY 11601 module) whereas it is impossible to send an identification request to several devices simultaneously.
Request_Code	INT	Request to be sent to the destination device, also called **server**. Requests may be UNI-TE requests *(see page 324)* or Modbus requests.
Data_to_Send	ARRAY [n... m] OF INT	Integer table to be sent to the destination device of the request. This table depends on the request sent. It must have a minimum length of 1 element, even if the request used does not require data to be sent (for example an identification request). **Note**: It is imperative that the length of the data to be sent (in bytes) be assigned to the fourth word of the management table before launching the function, in order for this to be correctly executed.

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0.. 3] OF INT	Exchange management table *(see page 40)*

The following table describes the output parameters:

Parameter	Type	Comment
Received_Data	ARRAY [n... m] OF INT	Integer table containing the data returned by the **server** device which was the destination of the request. Although certain requests do not require a response (for example a Run request), it is nevertheless necessary to reserve a minimum table of 1 integer each time the SEND_REQ function is used. **Note**: The size of the data received is written automatically by the system in the fourth word of the management table *(see page 46)*. **Note**: In certain cases (reading of word tables for example) it is necessary to resequence the objects received by using the ROR1_ARB *(see Unity Pro, Obsolete, Block Library)* function (shift of a byte in a table).

Simultaneous Transactions

The following table gives the capacities of each communication channel to simultaneously process transactions according to various configurations on Micro and Premium PLCs. (SEND_REQ is not available for Modicon M340/M580 and for Modicon Quantum PLCs.)

Configuration	Micro	TSX 57 10	TSX 57 20	TSX 57 23/30/40/45/55, PCX 57, PMX 57	TSX 57 46/56
Uni-Telway master terminal port	4	4	4	4	8
Uni-Telway master PCMCIA or SCY link	1	8	8	8	8
Uni-Telway client slave terminal port	4	1	1	1	8
Uni-Telway client slave PCMCIA or SCY link	1	1	1	1	1
Uni-Telway server slave terminal port	4	4	4	4	4
Uni-Telway server slave PCMCIA or SCY link	4	6	6	6	6
Modbus terminal port	4	-	-	-	-
Modbus PCMCIA or SCY link	4	8	8	8	8
Character mode terminal block	1	1	1	1	1
Character mode PCMCIA or SCY link	4	8	8	8	8
CANopen PCMCIA	-	10	10	10	10
Fipway PCMCIA or SCY link	4	8	8	8	8
Modbus Plus	4	4	4	4	4

Configuration	Micro	TSX 57 10	TSX 57 20	TSX 57 23/30/40/45/55, PCX 57, PMX 57	TSX 57 46/56
Ethernet	-	16	16	16	16
Embedded Ethernet	-	-	-	-	64

List of UNI-TE Requests

At a Glance

The UNI-TE protocol is used to:

- identify and diagnose all the devices that have a UNI-TE server
- provide a set of services giving read/write access to type data
- download data from device to device
- protect a server against concurrent connections during a critical period

These different services can be obtained using the `SEND_REQ` function, with coding of the UNI-TE request to be sent.

NOTE: To find out details and the list of requests recognized by each device, refer to the TSX DR NET reference manual.

The following tables provide a non-exhaustive list of the requests recognized by Premium devices.

General Usage Requests

These requests are used to identify and diagnose all the types of devices that have a UNI-TE server.

Request Name	Request Code	Report Code	Comment
`IDENTIFICATION`	16#0F	16#3F	Provides the following information: ● product range ● specific application type ● product type ● catalogue reference
`READ_CPU`	16#4F	16#7F	Performs system diagnostics on any device.
`PROTOCOL_VERSION`	16#30	16#60	Used to adapt the version of the protocol between two communicating entities.
`MIRROR`	16#FA	16#FB	Tests the correct routing of data between two communicating devices.

Dynamic Modification of IP Parameters

Change IP Parameters

Request Name	Request Code	Comment
REQUEST CODE	16#37	request code function
CHANGE IP PARAMETERS	16#13	changes IP parameter subfunction

To ensure that the ETY module is ready for operation, allow the PLC to run (in run mode) for 15 seconds after the last stop mode before issuing the SEND_REQ function. After the user issues the CHANGE IP PARAMETERS command and the ETY module accepts the new parameters, the ETY module resets and begins operations with the new parameters.

NOTE: FDR clients that use the ETY module for a server must be rebooted after the ETY's IP address change is completed. Otherwise, these FDR clients can not update their parameter files on the FDR server (the ETY module).

NOTE: Constant words contain the original configuration parameters, not those that were updated after the IP address change.

NOTE: You can see the new configuration (IP parameters, subnet mask, and gateway address) in the Unity Pro ETY debug screen *(see Premium and Atrium Using Unity Pro, Ethernet Network Modules, User Manual)*. You can also see the newly assigned IP address in the Ethernet Module Statistics web page *(see Premium and Atrium Using Unity Pro, Ethernet Network Modules, User Manual)*; however, you must be aware that the IP Address, Subnetwork mask, and Gateway data on this page reflect the old configuration.

Parameter Usage for IP Address Change

This table is based on a sample IP address of 139.158.10.7, subnet mask 255.255.248.0, and gateway address of 139.158.8.1.

Parameter	Type	Value	Comment
ADDRESS	array [0...5] of INT	ADDR (‘rack.slot.channel.SYS’)	Example: ADDR (0.x.0.SYS’) x = slot in which ETY module is installed
REQUEST_CODE	INT	16#37	

Parameter	Type	Value	Comment
Data_to_Send	array [0...8] of INT	byte 1: subfunction (13h)	high byte
		byte 2: subfunction (96h)	low byte
		byte 3: 0	Value is ignored.
		byte 4: 0	Value is ignored.
		byte 5: IP address 2 (158)	1-239 (high byte)
		byte 6: IP address 1 (139)	0-255 (low byte)
		byte 7: IP address 4 (7)	0-255 (high byte)
		byte 8: IP address 3 (10)	0-255 (low byte)
		byte 9: subnet mask 2 (255)	255
		byte 10: subnet mask 1 (255)	0-255
		byte 11: subnet mask 4 (0)	0-255
		byte 12: subnet mask 3 (248)	0-255
		byte 13: gateway 2 (158)	1-239 (Gateway address must be on the same subnet as the IP address.)
		byte 14:gateway 1 (139)	0-255
		byte 15:gateway 4 (1)	0-255
		byte 16:gateway 3 (8)	0-255
Manage_Param	*word number*	*high byte*	*low byte*
	1	activity report	00
	2	operation report (see note)	communication report (see note)
	3	timeout (ms)	
	4	length: 18 (INT)	
	NOTE: The following table gives details for IP address codes (successful and error).		

Refer to the example for Changing IP Parameters with SEND_REQ *(see page 334)*.

Change IP Address Codes

Operation Report	Communication Report	Meaning
successful code		
FE (hex)	00 (hex)	(SEND_REQ) successfully changed IP address
error codes		
01 (hex)	FF (hex)	invalid request code value (e.g., not 16#37)
00 (hex)	03 (hex)	SEND_REQ to ETY IP address instead of across backplane
00 (hex)	07 (hex)	bad address mapping to the ETY

Operation Report	Communication Report	Meaning
16 (hex)	FF (hex)	invalid IP address
17 (hex)	FF (hex)	invalid subnet
18 (hex)	FF (hex)	invalid gateway address
19 (hex)	FF (hex)	invalid network address
1A (hex)	FF (hex)	ETY's IP address was already set to SEND_REQ)
FD (hex)	00 (hex)	(SEND_REQ) did not successfully changed IP address

Parameter Usage for Reset Module Command

A Reset Module operation causes Premium ETY and ETY PORT communication modules to enter a cycle to reset its working environment. To program a SEND_REQ function block to perform a Reset Module command, use function code 37, subfunction code 10.

Parameter	Type	Value	Comment
ADDRESS	array [0...5] of INT	ADDR ('rack.slot.channel.SYS')	Example: ADDR (0.x.0.SYS') x = slot in which ETY module is installed
REQUEST_CODE	INT	16#37	
Data_to_send	array [0...1] of INT	byte 1: subfunction (10h)	high byte
		byte 2: subfunction (96h)	low byte
		bytes 3, 4: 0	reserved
Manage_Param	*word number*	*high byte*	*low byte*
	1	activity report	00
	2	operation report (see note)	communication report (see note)
	3	timeout (ms)	
	4	length: 4 (INT) (in this example)	
	NOTE: The following table gives details for Reset Module codes (successful and error).		

Reset Module Command Codes

Operation Report	Communication Report	Meaning
successful code		
FE (hex)	00 (hex)	SEND_REQ successfully reset module
error codes		
01 (hex)	FF (hex)	invalid request code value (e.g., not 16#37)
00 (hex)	07 (hex)	bad address mapping to the ETY
FD (hex)	00 (hex)	SEND_REQ did not reset module

Accessing Objects

These requests provide a set of services that give read/write access to internal bit and word-type data, system bit and word-type data, floating point data, constant data and SFC data.

Request Name	Request Code	Report Code	Comment
READ_OBJECT	16#36	16#66	Used to read one or more consecutive objects of the same type.
WRITE_OBJECT	16#37	16#FE	Used to write one or more consecutive objects of the same type.
READ_INTERNAL_BIT	16#00	16#30	Used to read the value of an internal bit.
WRITE_INTERNAL_BIT	16#10	16#FE	Used to write the value of an internal bit.
READ_INTERNAL_WORD	16#04	16#34	Used to read the value of an internal word.
WRITE_INTERNAL_WORD	16#14	16#FE	Used to write the value of an internal word.

Management of Operating Modes

These requests provide a set of services that can be used to manage a processor's operating modes.

Request Name	Request Code	Report Code	Comment
RUN	16#24	16#FE	Used to launch the execution of a processor's tasks.
STOP	16#25	16#FE	Used to stop the execution of a processor's tasks.
INIT	16#33	16#63	Used to trigger a hot or cold re-start.

Reservation Management

These requests provide a reservation mechanism that is used to protect a server against concurrent connections during a critical period.

Request Name	Request Code	Report Code	Comment
RESERVE	16#1D	16#FE	Enables a client to generally or partially reserve a server's functions.
RELEASE	16#1E	16#FE	Enables a client to free up the reserved server.
I_AM_ALIVE	16#2D	16#FE	Used to maintain the reservation.

NOTE: to reserve the PLC, write %SW110 to 1 and use the `SEND_REQ` function with Request Code 16#1D.

```
If NOT Management_Table[0].0 then (*reservation 1D*)
   Management_Table[2]:=4; (* time out *)
   SEND_REQ (ADR :=ADDR('sys'),
       CODE := 16#001D,
       EMIS := Emis_table,
       GEST := Management_Table,
       RECP => Receive_buffer);
END_IF;
```

With variables created in de the Data Editor as:

- `Management_Table: ARRAY [0..3] OF INT`
- `Emis_table: ARRAY [0..0] OF INT`
- `Receive_buffer: ARRAY [0..0] OF INT`

NOTE: create the variables without any value inside, only the timeout value has to be set in `Management_Table[2]`.

Assisted entry screen

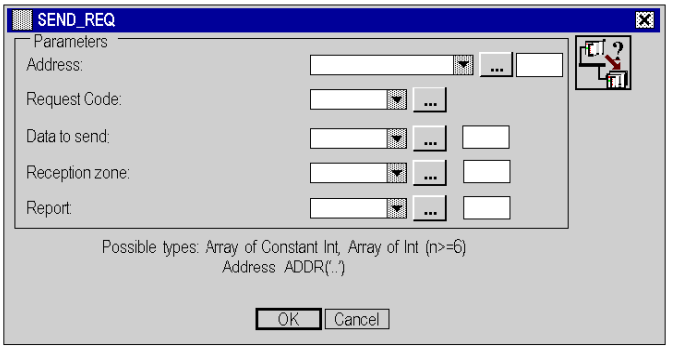
At a Glance

For this communication function, you can call up the assisted entry screen.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an example of the assisted entry screen of the function:



Address

For Premium PLCs, the possible type of object is:

- `ADDR(STRING).`
- `ARRAY [0..5] OF INT.`

NOTE: if you enter a value directly in the field, the assisted address entry button turns gray.

Request Code

The possible objects are of the INT type:

- variables,
- constants,
- immediate value.

NOTE: if you enter a constant, an entry field appears to enter the constant. If you enter a variable, it can be located or not.

Data to Send

These data to be sent are stored in the form of an integer array. This array can be located or not.

Reception Zone

The reception zone is an array of integers. This array can be located or not, and its size depends on the request code used.

Report

The report is an array of 4 integers.

NOTE: be careful not to use several identical memory areas for report tables, otherwise the variable read function may not work.

Example of how to send a UNI-TE request

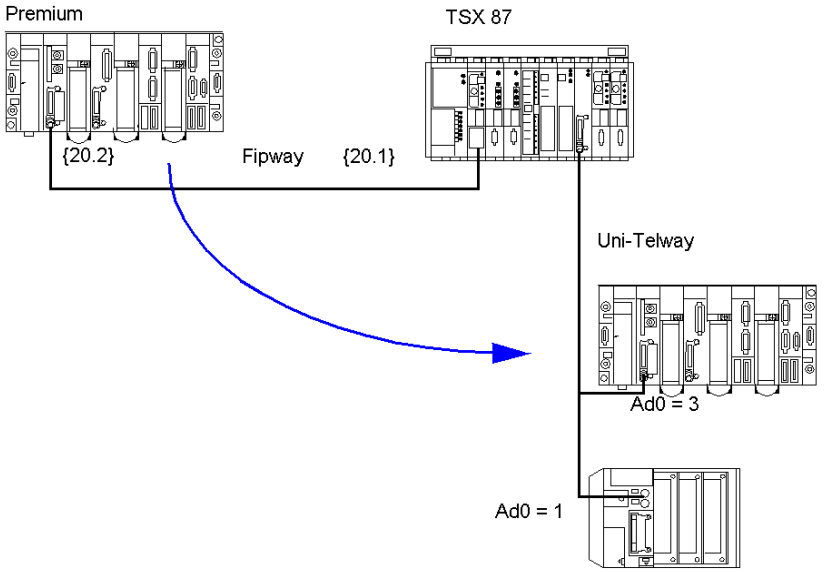
At a Glance

Station 2 of network 20 must send an identification request to the device with address Ad0=3 on the Uni-Telway bus of station 1 of the same network. The identification request has the decimal code 15 (or 16#0F).

The management table is located at `%MW10:4`.

Illustration

The two stations are connected via a Fipway network:



Programming

Programming in ST:

```
IF RE(%I0.3.2) AND NOT %MW10.0 THEN
   SEND_REQ(ADDR('{20.1}0.5.1.3'),15,%MW0:1, %MW10:4,%MW100:24);
END_IF;
```

Request parameters:

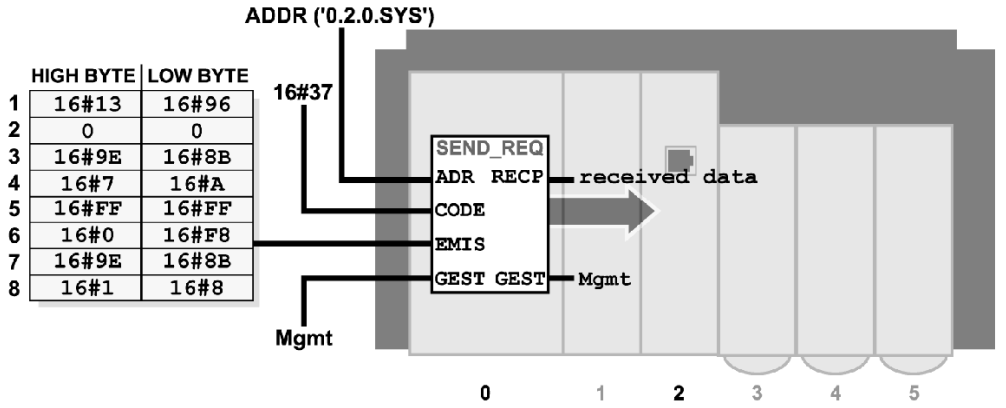
Parameters	Description
ADDR('{20.1}0.5.1.3')	• {20.1}: network 20, station 1 • 0: rack • 5: module • 1: channel 1 • 3: destination address
15	Request 15 (or 16#0F if the coding is hexadecimal)
%MW0:1	Data sent (for example: no data to send)
%MW10:4	Management table
%MW100:24	Contents of the response (reception of 24 words)

NOTE: Each time the function is launched, initialize the length parameter (in the example: `%MW13 = 0`).

Changing IP Parameters with SEND_REQ (Example)

Illustration

The graphic shows you how to set IP parameters for the ETY module in slot 2 with the SEND_REQ block:



Note:

- ADR: reflects the ETY's position in slot 2.
- CODE: reflects the value of the REQUEST_CODE.
- EMIS: contains the IP parameters in Data_to_Send:
  - address (139.158.10.7)
  - subnet mask (255.255.248.0)
  - gateway (139.158.8.1)
- GEST: reflects Management_Param (management parameters).
  You have to assign a time to the third word of Management_Param. The fourth word should have the INT value 18.
- RECP: This parameter requires a minimum INT value of 1, even when no response message is returned, as in the case of an IP change request..

Using the SEND_REQ function

At a Glance

The SEND_REQ function is used to code and send all UNI-TE and Modbus/Jbus requests, as well as receive the associated responses.

In certain cases (reading of word tables for example) it is necessary to resequence the objects received by using the ROR1_ARB function (shift of a byte in a table). .

Example

Objects to read:

16#0201

16#0403

16#0605

16#0807

16#0A09

Reception table after execution of a SEND_REQ (read object):

%MW100=16#01**07**

%MW101=16#0302

%MW102=16#0504

%MW103=16#0706

%MW104=16#0908

%MW105=16#000A

Reception table after a ROR1_ARB(%MW100:6):

%MW100=16#0201

%MW101=16#0403

%MW102=16#0605

%MW103=16#0807

%MW104=16#0A09

%MW105=16#**07**00

Chapter 32
SEND_TLG: Sending telegrams

Subject of this Chapter

This chapter describes the `SEND_TLG` function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	338
Example of how to send a telegram	341

Description

Function Description

The SEND_TLG function is used to send telegram-type data to a remote application.

The data to be sent must be a maximum of 16 bytes in length. Unlike the other communication functions, this function is processed immediately (synchronous): there are therefore no activity bit and timeout parameters.

The integer table assigned to the management parameters therefore only uses two words instead of four (the exchange and timeout numbers being unnecessary).

⚠ WARNING

INOPERABLE FUNCTION - INVALID CONFIGURATION

Use the SEND_TLG function on a Fipway network only after installing a TSX FPP 20 card:

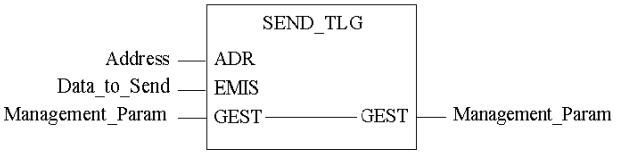
- on channel 1 of the processor
- for stations 0 to 15

Failure to follow these instructions can result in death, serious injury, or equipment damage.

The additional parameters EN and ENO can be configured.

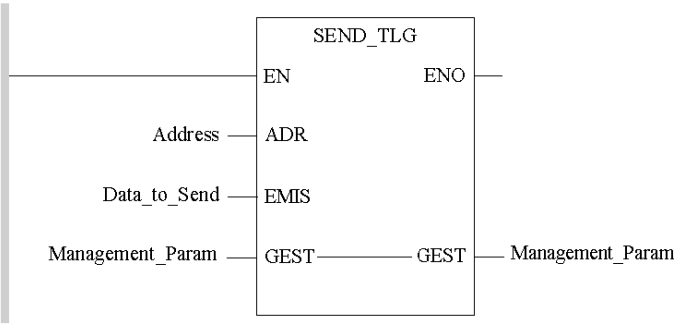
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
SEND_TLG Data_to_Send, Management_Param
```

ST Representation

Representation:

```
SEND_TLG(Address, Data_to_Send, Management_Param);
```

Description of the Parameters

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0.. 5] OF INT	Address of the destination entity of the exchange. Only the {Network.Station}APP or {Network.Station}APP.num addresses are authorized.
EMIS	ARRAY [n.. m] OF INT	Integer table to be sent to the destination device of the request. This must be a maximum of 8 integers (16 bytes) in length. **Note**: it is essential that the number of bytes to be sent is placed in the second word of the management table before executing the exchange.

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0 ... 1] OF INT	Table of words used to manage the exchange. This table is made up of 2 words: the report word and the length word for the data to be sent. The report comprises: ● the operation report (most significant byte of the first word), ● the communication report (least significant byte of the first word), The operation report takes one of the following values: ● 16#00: correct exchange, ● 16#03: incorrect address format, ● 16#04: incorrect destination address, ● 16#05: incorrect management parameters (for example length), ● 16#06: incorrect specific parameters, ● 16#07: module faulty, ● 16#0A: insufficient send buffer size, ● 16#0B: No system resources: the number of simultaneous communication EFs exceeds the maximum that can be managed by the processor, ● 16#0F: Telegram Service not configured.

Example of how to send a telegram

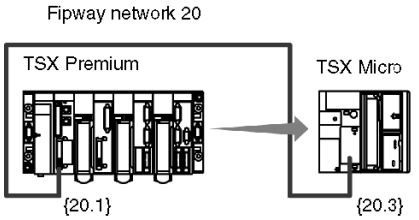
At a Glance

Let us suppose we want to send an 8-word telegram from station 1 to the remote station 3 on Fipway network 20.

We will use table %MW190:8 to contain these words to be sent, and table %MW200:2 will contain the management table of the exchange.

Illustration

The two stations are connected via a Fipway network.



Programming

Programming in ST:

```
IF RE(%I0.3.10) THEN
   SEND_TLG(ADDR('{20.3}APP'), %MW190:8, %MW200:2);
END_IF;
```

Request parameters:

Parameters	Description
ADDR('{20.3}APP')	• {20.2}: network 20, station 3 • APP: application
%MW190:8	Contents of the telegram to be sent
%MW200:2	Management table

NOTE: The word %MW 201 must be initialized to 16 (8 words) before the request is sent.
To execute this function synchronously, it is necessary to test the operation report immediately after the program line that activates the execution of this function.

Chapter 33
SYMAX_IP_ADDR: SY/MAX IP Address

Introduction

This chapter describes the `SYMAX_IP_ADDR` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	344
Detailed Description	346

Description

Function Description

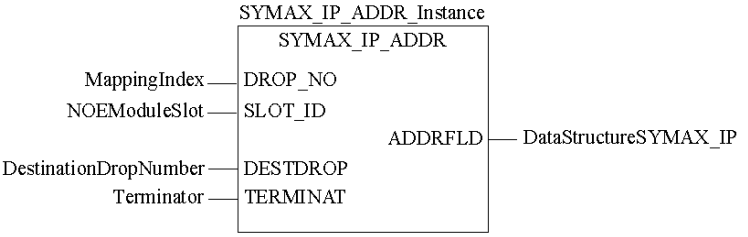
This function block allows the entry of the SY/MAX IP address for function blocks `REAG_REG`, `CREAD_REG`, `WRITE_REG` and `CWRITE_REG`. The address is transferred as a data structure.

`EN` and `ENO` can be configured as additional parameters.

NOTE: You must be familiar with your network when programming the `SYMAX_IP_ADDR` function block.

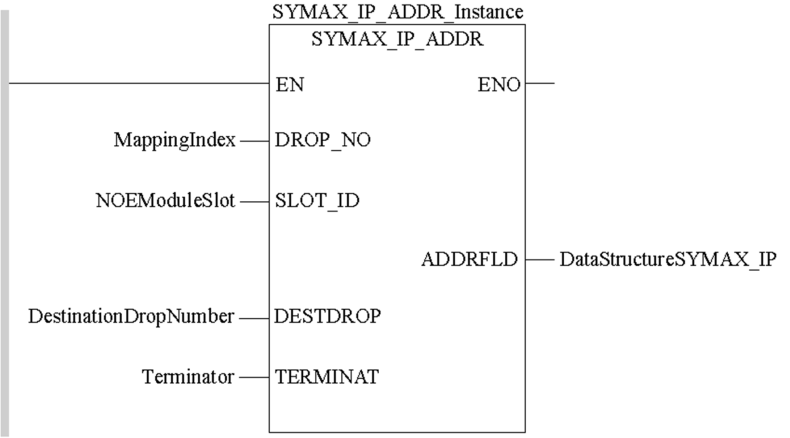
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL SYMAX_IP_ADDR_Instance (DROP_NO:=MappingIndex,
    SLOT_ID:=NOEModuleSlot, DESTDROP:=DestinationDropNumber,
    TERMINAT:=Terminator, ADDRFLD=>DataStructureSYMAX_IP)
```

Representation in ST

Representation:

```
SYMAX_IP_ADDR_Instance (DROP_NO:=MappingIndex,
    SLOT_ID:=NOEModuleSlot, DESTDROP:=DestinationDropNumber,
    TERMINAT:=Terminator, ADDRFLD=>DataStructureSYMAX_IP) ;
```

Parameter description

Description of input parameters:

Parameters	Data type	Description
DROP_NR	BYTE	MBP on Ethernet Transporter (MET) mapping index
Slot_ID	BYTE	Slot of the NOE module
DESTDROP	WORD	Destination drop number (or set to FF hex)
TERMINAT	WORD	Terminator (set to FF hex)

Description of output parameters:

Parameters	Data type	Description
ADDRFLD	WordArr5	Data structure used to transfer the SY/MAX IP address

Detailed Description

Element description of WordArr5

Element description for WordArr5:

Element	Data type	Description
WordArr5[1]	WORD	Most significant byte: Slots of the NOE module Least significant byte: MBP on Ethernet Transporter (MET) mapping index
WordArr5[2]	WORD	Destination drop number (or set to FF hex)
WordArr5[3]	WORD	Terminator (set to FF hex)
WordArr5[4]	WORD	No significance
WordArr5[5]	WORD	No significance

DROP_NR

The MBP to Ethernet Transporter (MET) mapping index is given at the DROP_NR input, i.e. if MET is 6, the value appears as follows:

0	0	0	0	0	1	1	0

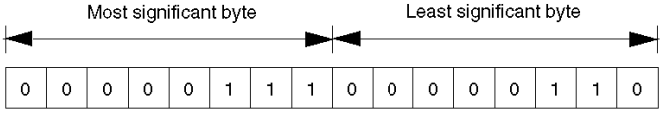
Slot_ID

If a NOE in the rack of a Quantum controller is addressed as destination node, the value at the SLOT_ID input represents the physical NOE slot, i.e. if the NOE is plugged in at Slot 7 of the rack, the value appears as follows:

0	0	0	0	0	1	1	1

ADDRFLD

If a NOE in the rack of a Quantum controller is addressed as a destination node, the value in the most significant byte represents the physical slot of the NOE and the least significant byte represents the MBP on Ethernet Transporter (MET) mapping index, i.e. if the NOE is inserted in slot 7 of the rack and the MET mapping index is 6, the first element of the data structure looks as follows:



Most significant byte Slots 1 to ... 16
Least significant byte MBP on Ethernet Transporter (MET) mapping index

Chapter 34
TCP_IP_ADDR: TCP/IP Address

Introduction

This chapter describes the `TCP_IP_ADDR` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	350
Detailed Description	353

Description

Function description

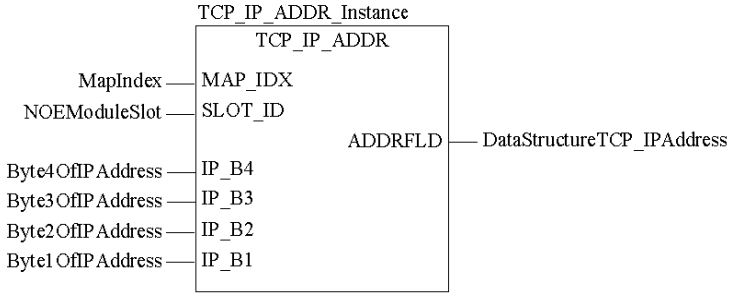
This function block allows the entry of the TCP/ IP address for function blocks READ_REG, CREAD_REG, WRITE_REG and CWRITE_REG. The address is transferred as a data structure.

EN and ENO can be configured as additional parameters.

NOTE: You must be familiar with your network when programming the TCP_IP_ADDR function block. A complete description of the TCP/IP routing is provided in the *Quantum with Unity Pro, TCP/IP Configuration, User Manual*.

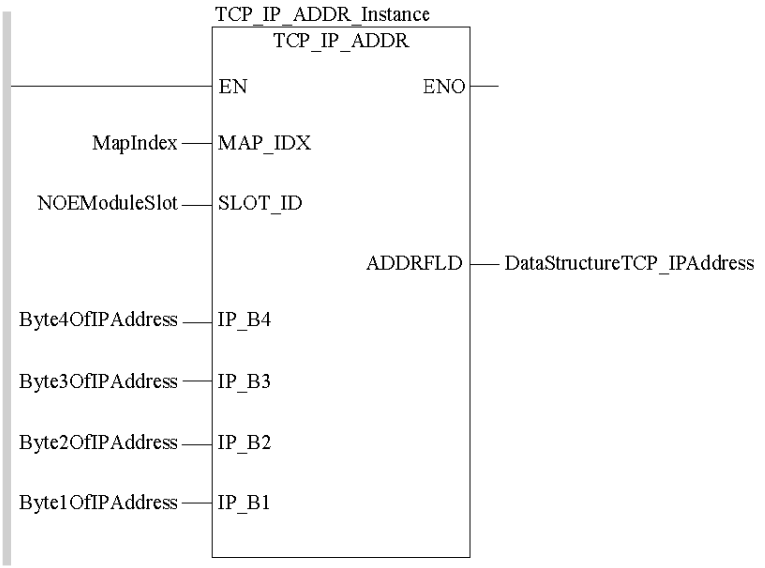
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL TCP_IP_ADDR_Instance (MAP_IDX:=MapIndex,
    SLOT_ID:=NOEModuleSlot, IP_B4:=Byte4OfIPAddress,
    IP_B3:=Byte3OfIPAddress, IP_B2:=Byte2OfIPAddress,
    IP_B1:=Byte1OfIPAddress,
    ADDRFLD=>DataStructureTCP_IPAddress)
```

Representation in ST

Representation:

```
TCP_IP_ADDR_Instance (MAP_IDX:=MapIndex,
    SLOT_ID:=NOEModuleSlot, IP_B4:=Byte4OfIPAddress,
    IP_B3:=Byte3OfIPAddress, IP_B2:=Byte2OfIPAddress,
    IP_B1:=Byte1OfIPAddress,
    ADDRFLD=>DataStructureTCP_IPAddress) ;
```

Parameter description

Description of input parameters:

Parameters	Data type	Description
MAP_IDX	BYTE	Map index MBP on Ethernet Transporter (MET) mapping index
Slot_ID	BYTE	Slot ID Slot of the NOE module
IP_B4	BYTE	Byte 4 (MSB) of the 32-bit destination IP address
IP_B3	BYTE	Byte 3 of the 32-bit destination IP address
IP_B2	BYTE	Byte 2 of the 32-bit destination IP address
IP_B1	BYTE	Byte 1 (LSB) of the 32-bit destination IP address

NOTE: For the Slot_ID parameter: when using an integrated Ethernet CPU module such as the 140 CPU 651 •0, the slot ID must be 254 (FE hex) regardless of the CPU slot.

Description of output parameters:

Parameters	Data type	Description
ADDRFLD	WordArr5	Data structure used to transfer the TCP/IP address

Detailed Description

Element description of WordArr5

Element description for WordArr5:

Element	Data type	Description
WordArr5[1]	WORD	Most significant byte: Slot of the NOE module Least significant byte: MBP on Ethernet Transporter (MET) mapping index
WordArr5[2]	WORD	Byte 4 of the 32-bit destination IP address
WordArr5[3]	WORD	Byte 3 of the 32-bit destination IP address
WordArr5[4]	WORD	Byte 2 of the 32-bit destination IP address
WordArr5[5]	WORD	Byte 1 of the 32-bit destination IP address

MAP_IDX

The MBP to Ethernet Transporter (MET) mapping index is given at the Map_Idx+ input, i.e. if MET is 6, the value appears as follows:

0	0	0	0	0	1	1	0

Slot_ID

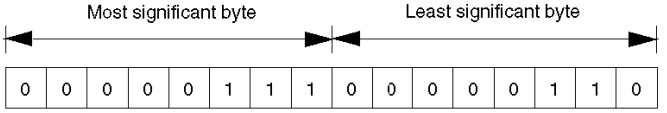
If a NOE in the rack of a Quantum controller is addressed as destination node, the value at the SLOT_ID input represents the physical NOE slot. Example : if the NOE is plugged in at Slot 7 of the rack, the value appears as follows:

0	0	0	0	0	1	1	1

NOTE: When using an integrated Ethernet CPU module such as the 140 CPU 651 x0, the slot ID must be 254 (FE hex) regardless of the CPU slot.

ADDRFLD

If a NOE in the rack of a Quantum controller is addressed as a destination node, the value in the most significant byte represents the physical slot of the NOE and the least significant byte represents the MBP on Ethernet Transporter (MET) mapping index, i.e. if the NOE is inserted in slot 7 of the rack and the MET mapping index is 6, the first element of the data structure looks as follows:



Most significant byte Slots 1 to ... 16
Least significant byte MBP on Ethernet Transporter (MET) mapping index

Chapter 35
UNITE_SERVER: Immediate server

Subject of this Chapter

This chapter describes the `UNITE_SERVER` communication function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	356
Example of immediate server	359

Description

Function Description

The UNITE_SERVER function is used to process UNI-TE requests immediately from the application program.

This function can be activated in the MAST task or the FAST task.

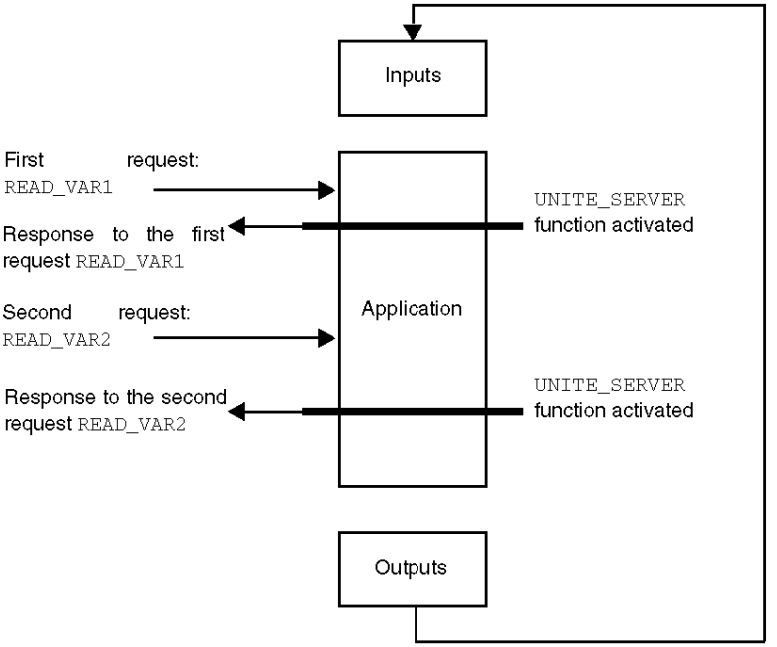
NOTE: At any given time, only one UNITE_SERVER function can be activated by the application.

NOTE: The UNITE_SERVER function is used to process requests from a Modbus link (TSX SCP 114 PCMCIA card in a TSX SCY 21601 module configured as a Modbus slave with immediate server *(see Premium and Atrium Using Unity Pro, Asynchronous Serial Link, User Manual)*).

The additional parameters EN and ENO can be configured.

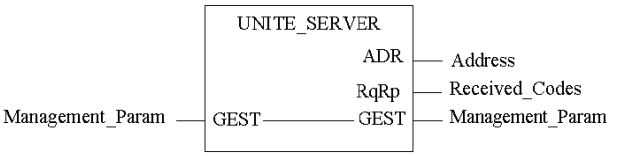
Exchange principle

The following diagram illustrates the exchanges made during the use of the UNITE_SERVER communication function.



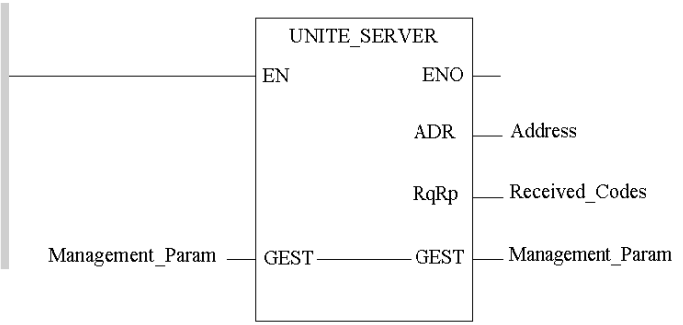
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
LD Management_Param
UNITE_SERVER Address, Received_Codes
```

ST representation

Representation:

```
UNITE_SERVER(Management_Param, Address, Received_Codes);
```

Parameter Description

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0.. 1] OF INT	Table of words used to manage the exchange. This table is made up of 2 words: the first word contains the exchange number and the activity bit, the second word contains the report. The report comprises: ● the operation report (most significant byte), ● the communication report (least significant byte). The communication report takes one of the following values: ● 16#00: correct exchange, ● 16#01: stop on timeout, the response was not able to be sent in under 2 seconds, ● 16#02: stop on user request (STOP, %S0, INIT, hot or cold re-start), ● 16#03: unknown address format, ● 16#05: incorrect management parameters, ● 16#07: problem in sending to the destination, ● 16#11: no request received, ● 16#12: UNITE_SERVER function already called by another task, ● 16#FF: message refused. **Note**: When a message is refused (code 16#FF), the operation report can then take the value 16#14 (server stopped).

The following table describes the output parameters:

Parameter	Type	Comment
Address	ARRAY [0.. 2] OF INT	Address of the destination entity of the exchange. This address corresponds to the channel to which the sender of the request is linked.
Received_Codes	INT	At the end of the exchange, this word contains: ● the request code received (least significant), ● the response code sent back (most significant).

Example of immediate server

At a Glance

This example covers implementation of a `UNITE_SERVER` function as an immediate server for a `READ_VAR` communication function. The Modbus link concerned is connected to the PCMCIA card of a TSX SCY 21601 module located in slot 2 of the base rack.

Programming

Programming in ST:

```
IF NOT %MW100:X0 THEN
   UNITE_SERVER(%MW100:2, %MW110:3, %MW10);
END_IF;
```

Request parameters:

Parameters	Description
%MW100:2	Management table
%MW110:3	Example: the immediate server is SCP114 configured as slave 49 in a SCY21601 (slot 4, rack 0). ● Word 1: 16#FE00 corresponds to rack, ● Word 2: 16#0405 corresponds to slot, ● Word 3: 16#0095 correspond to channel. See X-WAY addressing modes *(see Communication Drivers, Installation Manual)* for details.
%MW10	Response: ● Least significant byte: 16#03, UNITE read n words function received code. ● Most significant byte: 16#03, UNITE read n words function response code.

Chapter 36
WRITE_ASYN: Writing data asynchronously

Description

Function Description

The WRITE_ASYN function is used to write 1 Kbyte of data via the asynchronous messaging channel of TSX ETY modules in TCP/IP mode.

The following data can be write-accessed:

- internal bits,
- internal words.

Asynchronous writing can only take place between two stations on the same Ethernet TCP/IP network segment.

The WRITE_ASYN function is sent at the end of the MAST task only if this is configured in periodic mode. It is possible to activate 8 functions simultaneously.

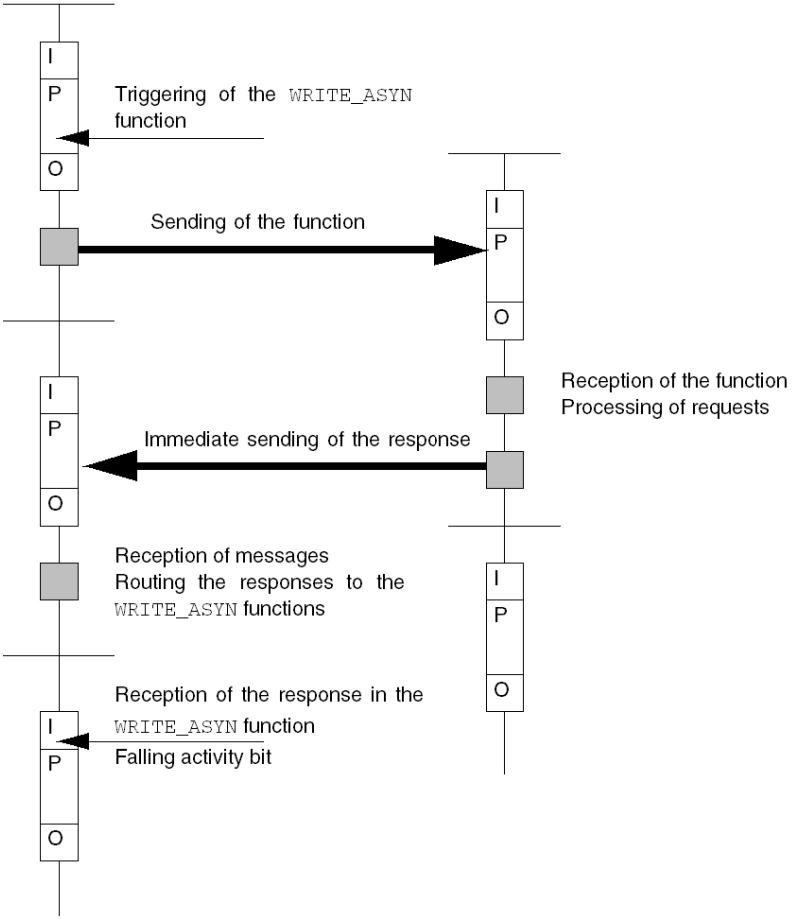
The size of the send and receive buffers is expressed in words. It is 512 words, or 1024 bytes.

NOTE: The asynchronous server function supports UNI-TE V1 or V2.0 protocols. The WRITE_ASYN function uses the UNI-TE V2.0 protocol.

The additional parameters EN and ENO can be configured.

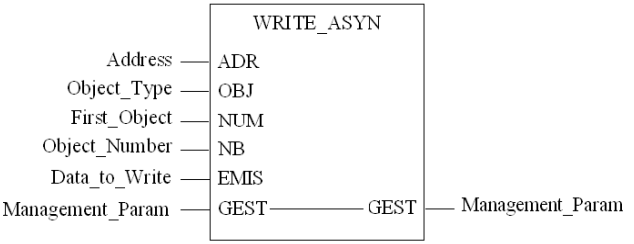
Exchange principle

The following diagram illustrates the exchanges between two stations for a WRITE_ASYN function:



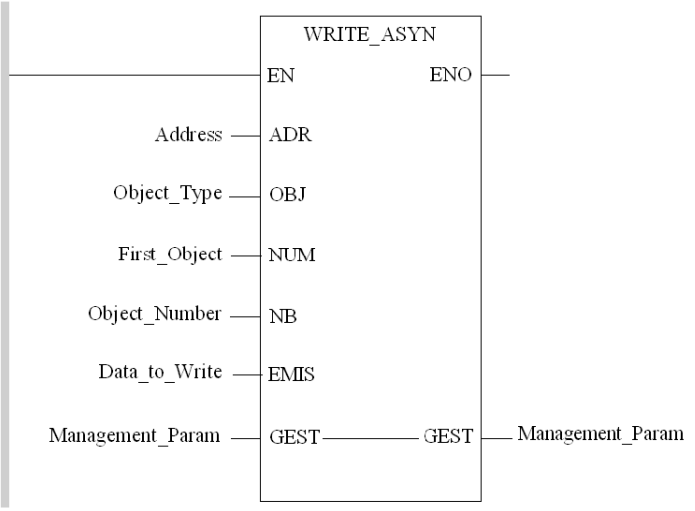
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
LD Address
WRITE_ASYN Object_Type, First_Object, Object_Number, Data_to_Write,
Management_Param
```

ST representation

Representation:

```
WRITE_ASYN(Address, Object_Type, First_Object, Object_Number,
Data_to_Write, Management_Param);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
Address	ARRAY [0.. 5] OF INT	Address of the destination entity of the exchange. Addresses are of the type: ADDR('{Network.Station}SYS.
Object_Type	STRING	Type of objects to write: ● '%M': internal bits, ● '%MW': internal words, ● '%S': system bits, ● '%SW': system words.
First_Object	DINT	Index of the first object to be written in the destination device.
Object_Number	INT	Number of objects to write.
Data_to_Write	ARRAY [n... m] OF INT	Word table containing the value of the objects to be written.

The following table describes the input/output parameters:

Parameter	Type	Comment
Management_Param	ARRAY [0.. 3] OF INT	Exchange management table *(see page 40)*. The operation report takes one of the following values: ● 16#00 : correct exchange, ● 16#01 : stop on timeout, the response was not able to be sent in under 2 seconds, ● 16#02 : stop on user request (STOP, S0, INIT, hot or cold re-start), ● 16#03 : incorrect address format, ● 16#05 : incorrect management parameters, ● 16#07 : destination missing, ● 16#09 : insufficient receive buffer size, ● 16#10 : insufficient send buffer size, ● 16#11 : no system resources (8 functions already active), ● 16#19 : incorrect exchange number, ● 16#FF: message refused. **Note**: do not forget to program a Timeout value to stop an exchange in progress when the response is not returned to the sender.

Chapter 37
WRITE_GDATA: Writing the Global Data Modbus Plus

Description

Function Description

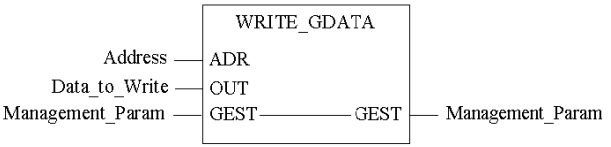
The WRITE_GDATA function is used to write shared data, or **Global Data**, on a **Modbus Plus** network.

Global Data is shared between a maximum of 64 stations on the same Modbus Plus network. Each station may write up to 32 integers, which can then be used by all stations on the network. Conversely, each station can read the 32 integers (maximum) of all the other network stations.

The additional parameters EN and ENO can be configured.

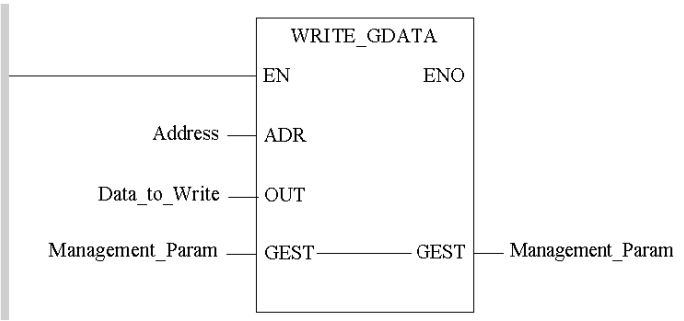
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
LD Address
WRITE_GDATA Data_to_Write, Management_Param
```

ST representation

Representation:

```
WRITE_GDATA(Address, Data_to_Write, Management_Param);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
`Address`	ARRAY [0.. 5] OF `INT`	Address of the PCMCIA card that connects the Premium to the Modbus Plus network. The value of this address is: `ADDR('0.0.1.SYS').` **Note**: data is written to the PCMCIA card, which then shares it.
`Data_to_Write`	ARRAY [n... m] OF `INT`	Word table containing the value of the objects to be written. This table must always be 32 integers of 16 bits in size, the maximum size of Global Data on a Modbus Plus network.

The following table describes the input/output parameters:

Parameter	Type	Comment
`Management_Param`	ARRAY [0.. 3] OF `INT`	Exchange management table *(see page 40)*. It is not necessary to initialize the length parameter before launching the exchange.

Chapter 38
WRITE_REG: Write Register

Introduction

This chapter describes the `WRITE_REG` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	368
Derived Data Types	371
Function mode	373
Parameter description	374

Description

Function description

On a rising edge at the REQ input, this function block writes a register area from a register area of a device connected via Modbus Plus, TCP/IP Ethernet or SY/MAX-Ethernet, with the content of a register area of the PLC.

EN and ENO can be configured as additional parameters.

CREAD_REG, CWRITE_REG, READ_REG, WRITE_REG, and MBP_MSTR function blocks use one data transaction path and require multiple cycles to complete an operation. The number of transaction paths available by module and by MAST cycle depends on the communication port used:

- Modbus Plus embedded port or NOM modules support up to 4 blocks at the same time.
- TCP/IP Ethernet embedded port support up to 4 blocks at the same time.
- TCP/IP Ethernet NOE, NOC, and 140 CRP 312 00 modules support up to 16 blocks at the same time.

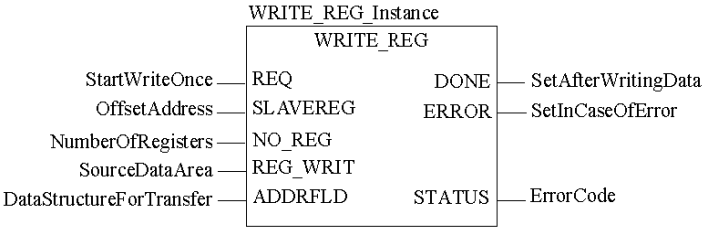
More communication function blocks may be programmed on the same communication port. However, a communication block exceeding the maximum number on that port is not serviced until one of the transaction paths is available. Then, the next block on the same port becomes active and begins using an available path.

NOTE: When programming a WRITE_REG function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the communication architectures manual *(see Communication Services and Architectures, Reference Manual)* and the *Modbus Plus Network Planning and Installation Guide (Reference 31003525)*. If TCP/IP or SY/MAX Ethernet routing is implemented *(see Quantum with Unity Pro, TCP/IP Configuration, User Manual)*, standard Ethernet IP router products must be used.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

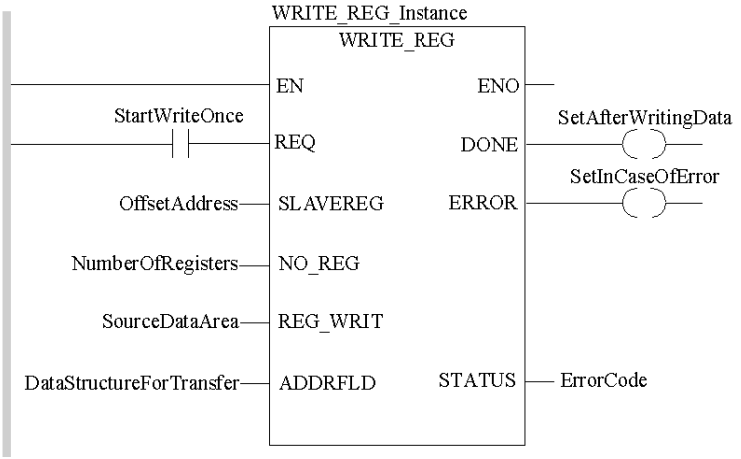
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL WRITE_REG_Instance (REQ:=StartWriteOnce,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    REG_WRIT:=SourceDataArea,
    ADDRFLD:=DataStructureForTransfer,
    DONE=>SetAfterWritingData, ERROR=>SetInCaseOfError,
    STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
WRITE_REG_Instance (REQ:=StartWriteOnce,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    REG_WRIT:=SourceDataArea,
    ADDRFLD:=DataStructureForTransfer,
    DONE=>SetAfterWritingData, ERROR=>SetInCaseOfError,
    STATUS=>ErrorCode) ;
```

Parameter description

Description of input parameters:

Parameter	Data type	Meaning
REQ	BOOL	On a rising edge at the REQ input, this function block writes a register area of a device connected via Modbus Plus, TCP/IP Ethernet or SY/MAX-Ethernet, with the content of a register area of the PLC.
SLAVEREG	DINT	Offset address of the first %MW register in the slave to be written to.
NO_REG	INT	Number of addresses to be written from slave
REG_WRIT	ANY	Source data field (A data structure must be declared as a located variable for the source file.)
ADDRFLD	WordArr5	Data structure transferring the Modbus Plus-address, TCP/IP address or SY/MAX-IP address.

Description of output parameters:

Parameter	Data type	Meaning
DONE	BOOL	Set to 1 for one cycle after writing data.
ERROR	BOOL	Set to 1 for one cycle if an error occurs.
STATUS	WORD	If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

Runtime error

For a list of all block error codes and values, see the table of error codes *(see page 496)*.

Derived Data Types

Element description for WordArr5 in Modbus Plus

Element description for WordArr5 in Modbus Plus:

Element	Data type	Description
WordArr5[1]	WORD	Least significant byte: Routing register 1 is used for address specification (routing path addresses one of five) of the destination node during network transfer. The last byte in the routing path that is not zero is the destination node. Most significant byte: Source node address. ● slot position of the module when using the Modbus Plus Port on the NOM module. ● set to 0 when using the Modbus Plus Port on the CPU, (regardless of the CPU slot).
WordArr5[2]	WORD	Routing register 2
WordArr5[3]	WORD	Routing register 3
WordArr5[4]	WORD	Routing register 4
WordArr5[5]	WORD	Routing register 5

Element description for WordArr5 with TCP/IP Ethernet

Element description for WordArr5 with TCP/IP Ethernet

Element	Data type	Description
WordArr5[1]	WORD	Most significant byte: Slot of the NOE module Least significant byte: MBP on Ethernet Transporter (MET) mapping index
WordArr5[2]	WORD	Byte 4 (MSB) of the 32-bit destination IP address
WordArr5[3]	WORD	Byte 3 of the 32-bit destination IP address
WordArr5[4]	WORD	Byte 2 of the 32-bit destination IP address
WordArr5[5]	WORD	Byte 1 (LSB) of the 32-bit destination IP address

Element description for WordArr5 with SY/MAX Ethernet

Element description for WordArr5 with SY/MAX Ethernet:

Element	Data type	Description
WordArr5[1]	WORD	Most significant byte: Slot of the NOE module Least significant byte: MBP on Ethernet Transporter (MET) mapping index
WordArr5[2]	WORD	Destination drop number (or set to FF hex)
WordArr5[3]	WORD	Terminator (set to FF hex)
WordArr5[4]	WORD	No significance
WordArr5[5]	WORD	No significance

Function mode

Function mode of the WRITE_REG block

Although a large number of WRITE_REG function blocks can be programmed, only four write operations may be active at the same time. In such a case it is insignificant whether they are the result of this function block or others (e.g. MBP_MSTR, CWRITE_REG). All function blocks use one data transaction path and require multiple cycles to complete a job.

If several WRITE_REG function blocks are used within an application, they must at least differ in the values of their NO_REG or REG_WRIT parameters.

NOTE: A TCP/IP communication between a Quantum PLC (NOE 211 00) and a Momentum PLC (all TCP/IP CPUs and all TCP/IP I/O modules) is only possible, when only **one** read or write job is carried out in every cycle. If several jobs are sent per PLC cycle, the communication stops without generating an error message in the status register of the function block.

The status signals DONE and ERROR report the function block state to the user program.

All routing information is contained in the WordArr5 data structure of the ADDRFLD input. The type function block assigned to this input is defined by the network used.

Please use:

- Modbus Plus for function block ModbusP_ADDR *(see page 199)*
- TCP/IP Ethernet for function block TCP_IP_ADDR *(see page 349)*
- SY/MAX Ethernet for function block SYMAX_IP_ADDR *(see page 343)*

NOTE: The WordArr5 data structure can also be used with constants.

Parameter description

REQ

A rising edge triggers the write transaction.

The parameter can be entered as an address, located variable, unlocated variable or literal.

SLAVEREG

Start of the area in the addressed slave to which the source data is written. The source area always resides within the %MW address area.

NOTE: For slaves for a **non-**Unity Pro PLC:
The destination area always resides within the 4x register area. `SLAVEREG` expects the target address as an offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).
The parameter can be entered as an address, located variable, unlocated variable or literal.

NO_REG

Number of addresses to be written to slave processor (1 ... 100).

The parameter can be entered as an address, located variable, unlocated variable or literal.

REG_WRIT

An `ARRAY` that is the same size as the planned transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

DONE

Transition to ON state for one program scan signifies that the data has been transferred.

The parameter can be entered as an address, located variable or unlocated variable.

ERROR

Transition to ON state for one program cycle signifies the detection of a new error.

The parameter can be entered as an address, located variable or unlocated variable.

STATUS

If an error occurs while the function is being executed, an error code *(see page 498)* remains at this output for one cycle.

The parameter can be entered as an address, located variable or unlocated variable.

Chapter 39
WRITE_REG_QX: Write Register on EIO Bus

Description

Function description

The `WRITE_REG_QX` function is used to write registers in a Modbus slave connected to a Modicon M340 Modbus communication module (BMX NOM) located in an EIO drop.

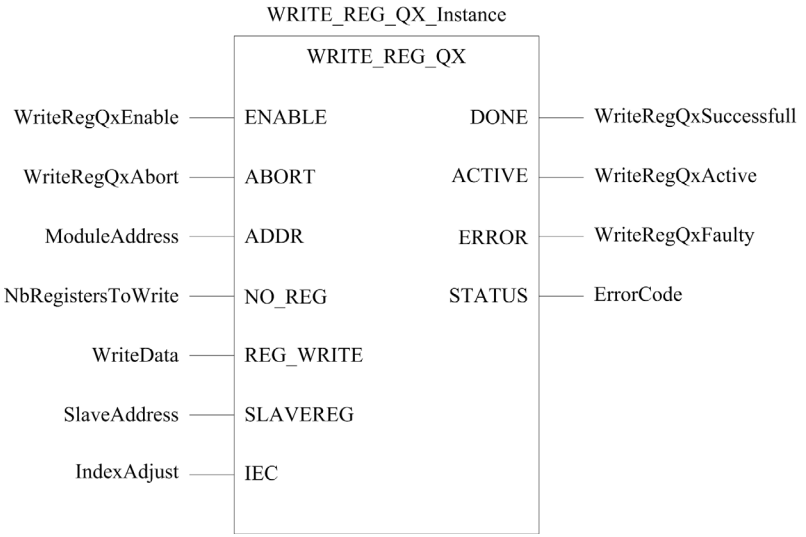
When programming a `WRITE_REG_QX` function, you must be familiar with the routing procedures used by your network.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

The additional parameters `EN` and `ENO` can be configured.

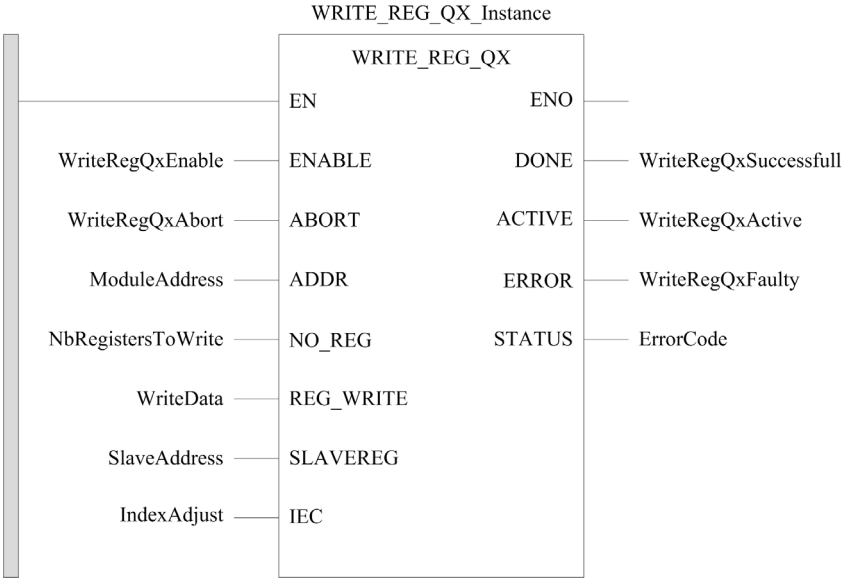
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL WRITE_REG_QX_Instance (ENABLE:=WriteRegQxEnable, ABORT:=WriteRegQ-
xAbort, ADDR:=ModuleAddress, NO_REG:=NbRegistersToWrite,
REG_WRITE:=WriteData, SLAVEREG:=SlaveAddress, IEC:=IndexAdjust,
DONE=>WriteRegQxSuccessfull, ACTIVE=>WriteRegQxActive,
ERROR=>WriteRegQxFaulty, STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
WRITE_REG_QX_Instance (ENABLE:=WriteRegQxEnable, ABORT:=WriteRegQ-
xAbort, ADDR:=ModuleAddress, NO_REG:=NbRegistersToWrite,
REG_WRITE:=WriteData, SLAVEREG:=SlaveAddress, IEC:=IndexAdjust,
DONE=>WriteRegQxSuccessfull, ACTIVE=>WriteRegQxActive,
ERROR=>WriteRegQxFaulty, STATUS=>ErrorCode)
```

Parameter description

Description of input parameters:

Parameter	Data type	Meaning
ENABLE	BOOL	Set to 1 to write registers.
ABORT	BOOL	Set to 1 to abort the current operation.
ADDR	ANY_ARRAY_INT	Array containing the Modbus slave address, result of ADDMX *(see page 77)* function.
NO_REG	INT	Number of registers to be written to the Modbus slave.
REG_WRITE	ANY_ARRAY_WORD	Source data field (A data structure must be declared as a located variable for the source file.)
SLAVEREG	DINT	Offset address of the first %MW register to be written in the Modbus salve.
IEC	INT	Index adjust applied to SLAVEREG parameter. IEC parameter can have 2 values: • 0: $\%MW_x$ • 1: $\%MW_{x+1}$ IEC default value is 0 for a Modicon M340 target.

Description of output parameters:

Parameter	Data type	Meaning
DONE	BOOL	Set to 1 when the execution of the function is completed successfully.
ACTIVE	BOOL	Set to 1 when the execution of the function is in progress.
ERROR	BOOL	Set to 1 if an error is detected by the function block.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.

Chapter 40
WRITE_SDO: Writing Service Data Object

WRITE_SDO: Writing Service Data Object

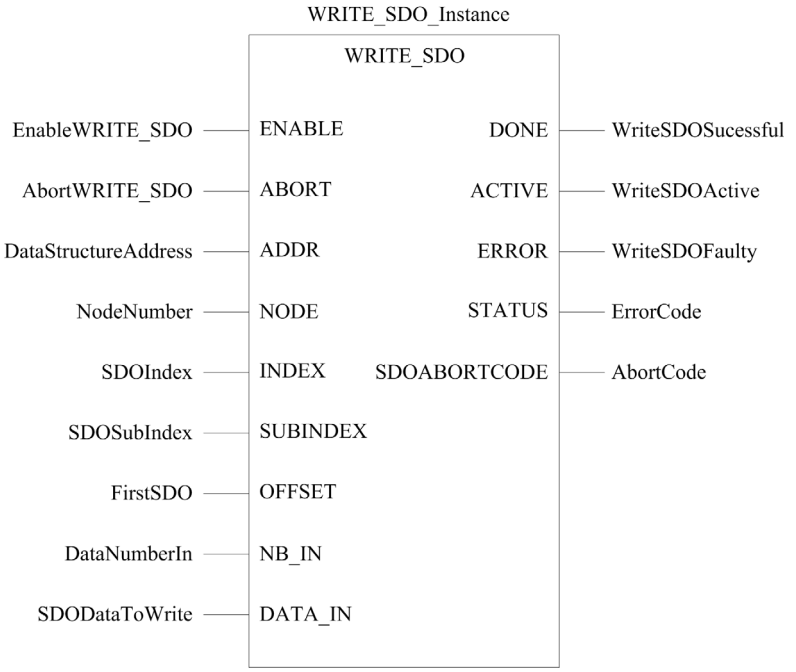
Function Description

The `WRITE_SDO` function block writes (explicit exchanges) from the PLC application up to the device (SDO).

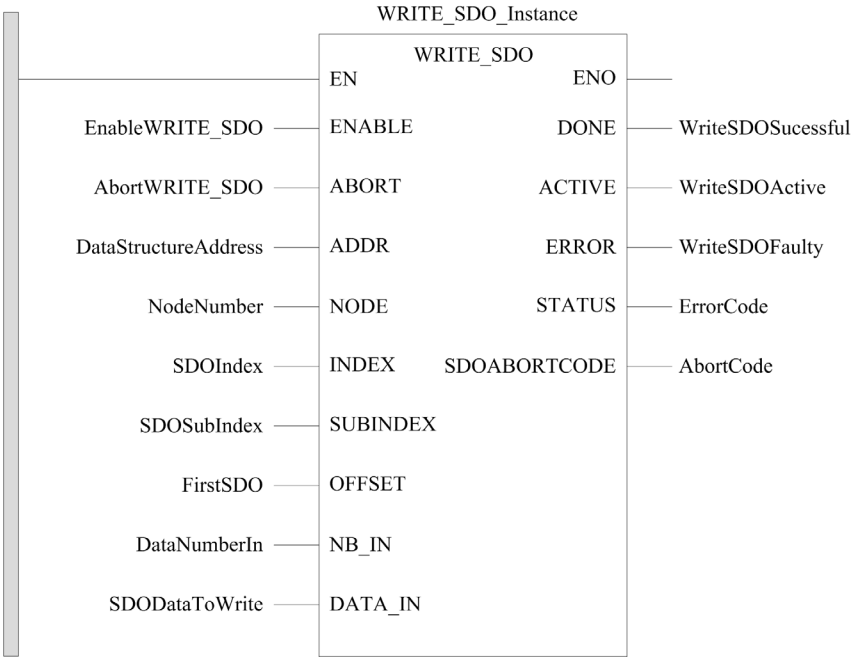
This function block provides access to the abort code when the SDO command is not successful (only if the fieldbus is in RUN mode, and only towards the configured devices).

FBD Representation

Representation:



LD Representation



IL Representation

Representation:

```
CAL WRITE_SDO_Instance (ENABLE := EnableWRITE_SDO,
ABORT := AbortWRITE_SDO, ADDR := DataStructureAddress,
NODE := NodeNumber, INDEX := SDOIndex, SUBINDEX := SDOSubIndex,
OFFSET := FirstSDO, NB_IN := DataNumberIn, DATA_IN := SDODataToWrite,
DONE => WriteSDOSuccessful, ACTIVE => WriteSDOActive,
ERROR => WriteSDOFaulty, STATUS => ErrorCode,
SDOABORTCODE => AbortCode)
```

ST Representation

Representation:

```
WRITE_SDO_Instance (ENABLE := EnableWRITE_SDO, ABORT := AbortWRITE_SDO,
ADDR := DataStructureAddress, NODE := NodeNumber, INDEX := SDOIndex,
SUBINDEX := SDOSubIndex, OFFSET := FirstSDO, NB_IN := DataNumberIn,
DATA_IN := SDODataToWrite, DONE => WriteSDOSuccessful,
ACTIVE => WriteSDOActive, ERROR => WriteSDOFaulty, STATUS => ErrorCode,
SDOABORTCODE => AbortCode)
```

Parameter Description

The following table describes the input parameters:

Input parameter	Data type	Description
ENABLE	BOOL	ON: the operation is enabled.
ABORT	BOOL	ON: the currently active operation is aborted.
ADDR	ANY_ARRAY_INT	Array containing the address of the destination entity of the read operation, result of ADDMX function.
NODE	BYTE	Byte used to select a particular NMT slave device on the CANopen network (16#01 to 16#7F).
INDEX	INT	Two bytes used to access a particular object in a CANopen SDO server device.
SUBINDEX	BYTE	Byte used to access a particular subobject in a CANopen SDO server device.
OFFSET	INT	Two bytes indicating the starting offset into the selected object. It can be non-zero when performing segmented SDO transfers. **NOTE:** Not used when addressing an EtherNet/IP module (address with CIP suffix).
NB_IN	INT	Two bytes providing a count of the desired number of data values to be written (in bytes).
DATA_IN	ANY_ARRAY_BYTE	Data to write.

The following table describes the output parameters:

Output parameter	Data type	Description
DONE	BOOL	ON: the operation concludes successfully.
ACTIVE	BOOL	ON: the operation is active.
ERROR	BOOL	ON: the operation is aborted without success.
STATUS	WORD	Provides the error code *(see page 498)* if an error is detected by the function block.
SDOABORTCODE	DWORD	SDO abort code *(see page 518)* when STATUS = 16#4007.

Chapter 41
WRITE_VAR: Writing variables

Subject of this Chapter

This chapter describes the WRITE_VAR communication function.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	384
Assisted entry screen	390
Example of how to write words on a network	392
Example of Writing Words via Serial Link of Modicon M340 PLCs	394
Example including execution check	396

Description

Function Description

The WRITE_VAR function is used to write one or more language objects of the same type:

- internal bits
- internal words

Verify that he objects to be written are consecutive. They may be located in a remote CPU or in a device connected to a communication channel.

⚠ WARNING

EXCHANGED DATA INCOMPATIBILITY

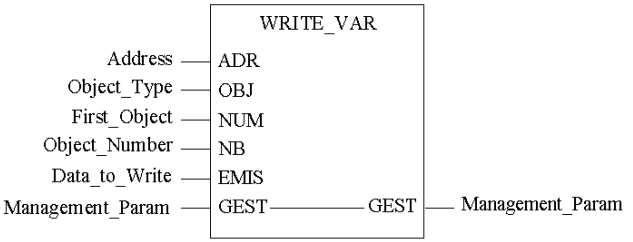
Data structure alignments are not the same for Premium/Quantum and M340/M580 so verify that the data exchanged are compatible. See DDT: Mapping rules *(see Unity Pro, Program Languages and Structure, Reference Manual)* for alignment rules.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

The additional parameters EN and ENO can be configured.

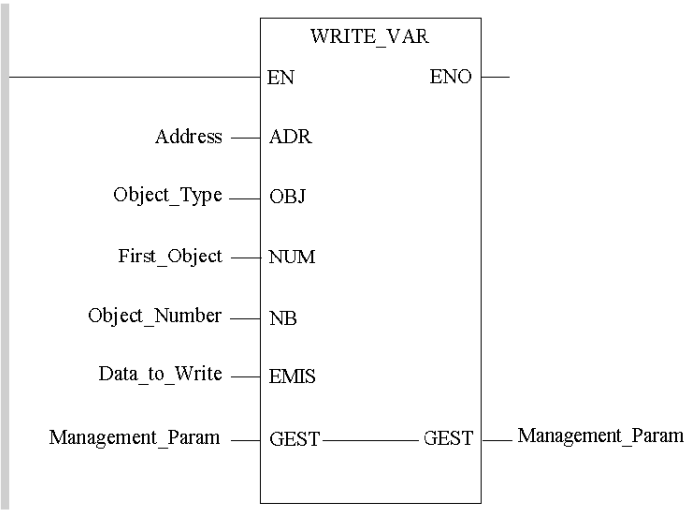
FBD Representation

Representation:



LD Representation

Representation:



IL Representation

Representation:

```
LD Address
WRITE_VAR Object_Type, First_Object, Object_Number, Data_to_Write,
Management_Param
```

ST Representation

Representation:

```
WRITE_VAR(Address, Object_Type, First_Object, Object_Number,
Data_to_Write, Management_Param);
```

Parameter Description

The following table describes the input parameters:

Parameter	Type	Comment
ADR	ARRAY [0..5] OF INT for Premium ARRAY [0..7] OF INT for Modicon M340 and M580	Specific statements by hardware platforms: ● Premium: ❍ Address of the message's receiving character mode channel is given by the ADDR *(see page 83)* function. ❍ Address of the destination entity of the exchange. The following addresses are prohibited: • {Network.Station}APP, • {Network.Station}APP.num. • broadcast addresses (ALL, 0 for TSX SCY 11601 module). ● Modicon M340: ❍ Address of the message's receiving character mode channel is given by the ADDM *(see page 71)* function. ❍ The syntax of the address is of ADDM ('r.m.c.node')-type. ● Modicon M580: ❍ Address of the destination entity of the write operation, result of ADDMX *(see page 77)* function.
TYP	STRING	Type of objects to write for Premium PLCs: ● '%M': internal bits, ● '%MW': internal words, ● '%S': system bits, ● '%SW': system words. Type of objects to write for Modicon M340 and M580 PLCs: ● '%M': internal bits, ● '%MW': internal words.
NUM	DINT	Index of the first object to be written in the destination device.
NB	INT	Number of objects to write.
EMIS	ARRAY [n..m] OF INT	Word table containing the value of the objects to be written.

The following table describes the input/output parameters:

Parameter	Type	Comment
GEST	ARRAY [0...3] OF INT	Exchange management table consisting of the following words: ● Rank 1 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: exchange number ❍ Least significant byte: activity bit (rank 0) and cancel bit (rank 1) **NOTE:** The cancel bit is only available for Modicon M340 and M580 PLCs. ● Rank 2 word: a word managed by the system and consisting of two bytes: ❍ Most significant byte: operation report ❍ Least significant byte: communication report ● Rank 3 word: a word managed by the user which defines the maximum response time using a time base of 100 ms. ● Rank 4 word: a word managed by the system which defines the length of the exchange. For detailed information, refer to section *Structure of the Management Parameters* *(see page 40)*.

NOTE: Verify that the length parameter is not initialized before launching the function.

NOTE: Premium CPU embedded Ethernet of TSX P574xxx, TSX P575xxx and TSX P576xxx used as client, can only write 100 words instead of 123 words for other Ethernet modules and Premium CPUs with ETY PORT.

Simultaneous Transactions

The following table gives the capacities of each communication channel to simultaneously process transactions according to various configurations on Micro and Premium PLCs.

Configuration	Micro	TSX 57 10	TSX 57 20	TSX 57 23/30/40/45/55, PCX 57, PMX 57	TSX 57 46/56
Uni-Telway master terminal port	4	4	4	4	8
Uni-Telway master PCMCIA or SCY link	1	8	8	8	8
Uni-Telway client slave terminal port	4	1	1	1	8
Uni-Telway client slave PCMCIA or SCY link	1	1	1	1	1
Uni-Telway server slave terminal port	4	4	4	4	4

Configuration	Micro	TSX 57 10	TSX 57 20	TSX 57 23/30/40/45/55, PCX 57, PMX 57	TSX 57 46/56
Uni-Telway server slave PCMCIA or SCY link	4	6	6	6	6
Modbus terminal port	4	-	-	-	-
Modbus PCMCIA or SCY link	4	8	8	8	8
Character mode terminal block	1	1	1	1	1
Character mode PCMCIA or SCY link	4	8	8	8	8
CANopen PCMCIA	-	10	10	10	10
Fipway PCMCIA or SCY link	4	8	8	8	8
Modbus Plus	4	4	4	4	4
Ethernet	-	16	16	16	16
Embedded Ethernet	-	-	-	-	64

The following table gives the capacities of each communication channel to simultaneously process transactions according to various configurations on Modicon M340 PLCs.

Configuration	BMX P34 1000	BMX P34 2000	BMX P34 2010/ 20102	BMX P34 2020	BMX P34 2030/ 20302
Embedded CANopen	-	-	16	-	16
Embedded Ethernet	-	-	-	16	16
Modbus master serial port	8	16	16	16	-

Maximum number of requests as client per scan on Modicon M580 CPUs:

Configuration	CPU Reference (BME P58 ...)					
	10•0	20•0	30•0	40•0	5040	6040
Ethernet	16	32	48	80	80	96

Maximum number of requests as client per scan on *Hot Standby* Modicon M580 CPUs:

Configuration	CPU Reference (BME H58 ...)		
	2040	4040	6040
Ethernet	32	80	96

Maximum number of requests served per scan on Modicon M580 CPUs:

Configuration	CPU Reference (BME P58 ...)					
	10•0	20•0	30•0	40•0	5040	6040
CPU from all origin (1.)	16	24	32	40	48	64
Embedded Ethernet	8	12	16	24 (2.)	32	32
USB	4	4	4	4	4	4
1. %SW90 system word allows to adjust the number of requests served per cycle. The minimum number of requests served per cycle is set to 2 in order to reduce the CPU jitter. 2. Maximum number of requests is 16 for M580 CPU BME P58 40•0 with firmware version ≤ 1.20.						

Maximum number of requests served per scan on *Hot Standby* Modicon M580 CPUs:

Configuration	CPU Reference (BME H58 ...)		
	2040	4040	6040
CPU from all origin (1.)	24	40	64
Embedded Ethernet	12	24	32
USB	4	4	4
1. %SW90 system word allows to adjust the number of requests served per cycle. The minimum number of requests served per cycle is set to 2 in order to reduce the CPU jitter.			

NOTE: Verify that the maximum number of simultaneous requests given in all tables takes into account the maximum number of simultaneous requests that each CPU can manage per cycle, as Client and as Server *(see Modicon M580, Hardware, Reference Manual)*.

Assisted entry screen

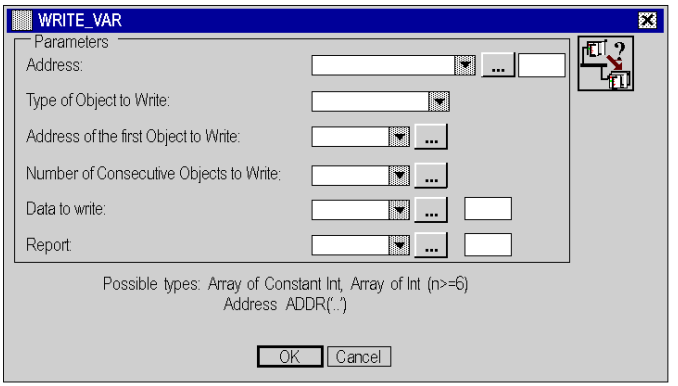
At a Glance

For this communication function, you can call up the assisted entry screen. Please note that the assisted entry screen is not available for Modicon M340/M580.

NOTE: The variable symbols are accepted in the different fields of the screen.

Illustration

The following screen shows an example of the assisted entry screen of the function:



Address

For Premium PLCs, the types of object possible are:

- `ADDR(STRING)`,
- `ARRAY [0..5] OF INT`.

NOTE: if you enter a value directly in the field, the assisted address entry button turns gray.

Type of object to write.

For Premium PLCs, the possible choices are:

- `'%M'` for writing internal bits.
- `'%MW'` for writing internal words.
- `'%S'` for writing system bits.
- `'%SW'` for writing system words.

NOTE: simply choose from the solutions offered in the drop-down menu.

Address of the first object to write

The possible objects of the DINT type are:

- variables,
- constants,
- immediate value.

NOTE: if you enter a constant, an entry field appears to enter the constant. If you enter a variable, it can be localised or not. On the other hand the objects to be written are mandatory located variables.

Number of consecutive objects to write

The possible objects of the INT type are:

- variables,
- constants,
- immediate value.

NOTE: if you enter a constant, an entry field appears to enter the constant. If you enter a variable, it can be localised or not.

Data to write

The data to write zone is an integer array. The size of this array depends on the number of objects to write. This integer array can be located or not.

Report

The report is an array of 4 integers.

NOTE: be careful not to use several identical memory areas for report tables, otherwise the variable read function may not work.

Example of how to write words on a network

At a Glance

Until now, the examples have been written using direct addressing (use of %MWi), but it is also possible to create these same examples using unlocated variables.

The following example uses unlocated variables, and covers the writing of a 50-word table named `Tab_1` (declared as an ARRAY [0..49] OF INT) in the Uni-Telway slave, with the address:

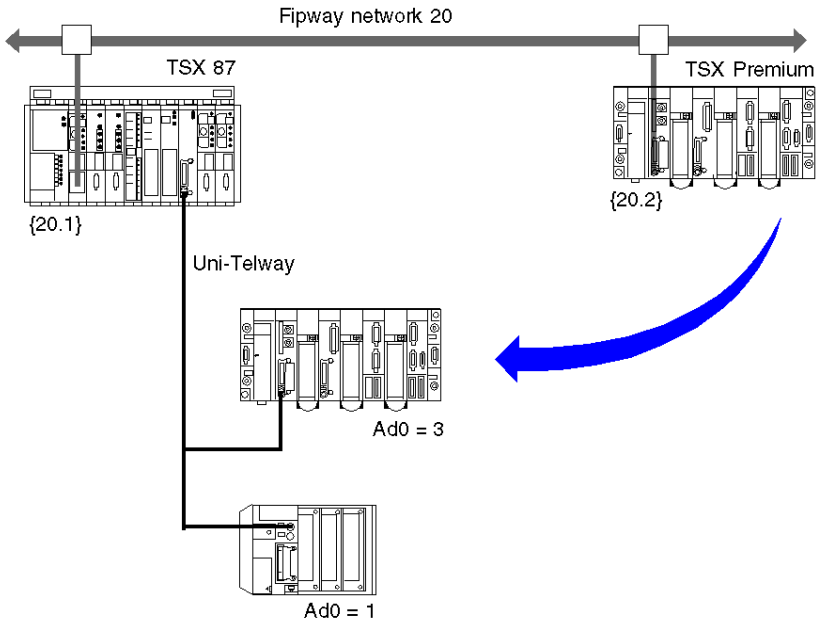
- Network 20
- Station 1
- SCM2116 module in a TSX 87 in slot 5 of the base rack
- Channel 1
- Server address of slave Ad0 = 3.

The values to write are in the variable `Tab_1` of the sender.

The management parameters are located in a table of 4 integers called `Management_Parameter` (declared as ARRAY [0..3] OF INT).

Illustration

The two stations are connected via a Fipway network.



Programming

Programming in ST:

```
IF RE(%I0.3.1) AND NOT Management_Parameter[0].0 THEN

WRITE_VAR(ADDR('{20.1}0.5.1.3'),'%MW',0,50,Tab_1,Management_Parameter);

END_IF;
```

Request parameters:

Parameters	Description
ADDR('{20.1}5.1.3')	• {20.1}: network 20, station 1 • 0: rack • 5: module • 1: channel 1 • 3: destination address
'%MW'	Type of object (internal word)
0	address of the first object to write
50	Number of consecutive objects to write
Tab_1	Data to write
Management_Parameter	Management table

Example of Writing Words via Serial Link of Modicon M340 PLCs

At a Glance

That example deals with 2 Modicon M340 processors which communicate via Modbus serial link.

Description of the Example

The following example uses unlocated variables, and covers the writing of an unlocated 50-word table named `Tab_1` (declared as an `ARRAY [0..49] OF INT`) in the Modbus slave. The management parameters are located in a table of 4 integers called `Management_Parameter` (declared as `ARRAY [0..3] OF INT`).

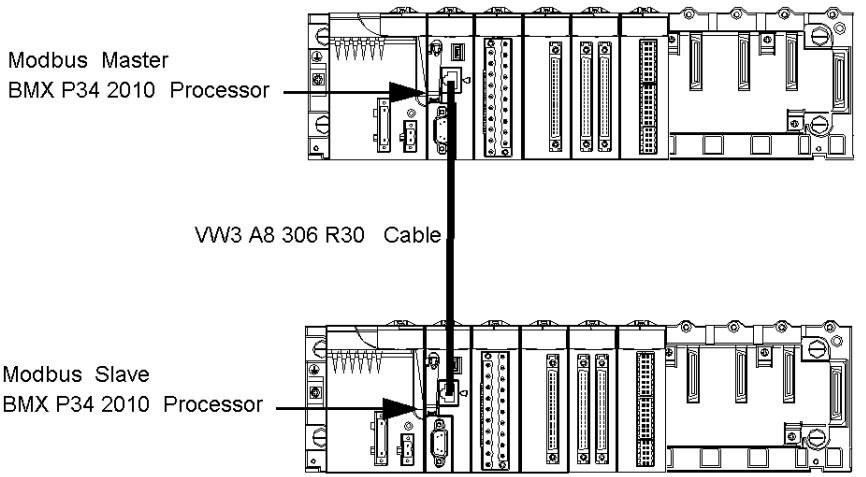
In that example, the Modbus slave's number is 7, so the input `ADDM` parameter is '0.0.0.7':

- 0: rack number of the processor equal to 0
- 0: slot number of the processor within the rack, equal to 0 as the slot number of a Modicon M340 processor is always 0
- 0: channel number, equal to 0 as the serial link of a Modicon M340 processor is always channel 0
- 7: configured slave number is 7.

The values to write are in the variable `Tab_1` of the sender.

Illustration

The two Modicon 340 processors are connected via a Modbus link:



Programming

Programming in ST:

```
IF RE(%I0.3.1) AND NOT Management_Parameter[0].0 THEN
   WRITE_VAR(ADDM('0.0.0.7'),'%MW',0,50,Tab_1,Management_Parameter);

END_IF;
```

The request parameters are as follows:

Parameters	Description
ADDM('0.0.0.7')	• 0: rack number of the slave processor • 0: slot number of the slave processor • 0: channel number (serial port number) • 7: configured slave number
'%MW'	Type of object (internal word)
0	address of the first object to write
50	Number of consecutive objects to write
Tab_1	Data to write
Management_Parameter	Management table

Example including execution check

At a Glance

The following example illustrates the `WRITE_VAR` function with a management parameter check.

Programming the function

Programming in ST:

```
IF NOT %M20 AND %I0.1.2 THEN
   %MW200:4:= 0;
   INC %MW1700
   %MW202:= 50;
  WRITE_VAR(ADDR('0.3.1.7'),'%MW',20,1,%MW1700:1,%MW200:4);
   SET %M20;
END_IF;
```

- the input bit %I0.1.2 controls the function,
- the internet bit %M20 is used to test the activity of the function,
- `%MW200:4:= 0;` sets the management table to 0,
- `INC %MW1700;` increments the word %MW1700,
- `MW202:= 50;` initializes the timeout value to 5 seconds.

NOTE: `WRITE_VAR(ADDM('0.3.1.7'),'%MW',20,1,%MW1700:4,%MW200:4);` syntax must be used for Modicon M340 PLCs as `ADDR` function cannot be used by a Modicon M340 PLC.

Programming the exchange check

Programming in ST:

```
IF %M20 AND NOT %M200.0 THEN
   INC %MW204;
   IF %MW201 = 0 THEN
      INC %MW205;
   ELSE
      SET %Q0.2.2;
      INC %MW206;
      %MW207 := %MW201;
   END_IF;
END_IF;
```

- %MW204 counts the number of exchanges,
- %MW205 counts the number of correct exchanges,
- %MW206 counts the number of exchanges generating errors,
- %MW207 stores the error message,
- %Q0.2.2 indicates an exchange failure.

Chapter 42
XMIT: Transmit

Introduction

This chapter describes the XMIT block. It differs from the XXMIT block only in the interface. Therefore for the functional descriptions, see the latter block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Brief Description	400
Representation	401
Parameter Description	404
Description of Data Structure XMIT_SET	406
Description of Data Structure XMIT_CFG	407
Application Example	413

Brief Description

Function Description

The XMIT (Transmit) function block sends Modbus messages from a master PLC to multiple slave PLCs or sends ASCII character strings from the PLC's Modbus slave port #1 or port #2 to ASCII printers and terminals.

XMIT sends these messages through telephone dialup modems, radio modems, or direct connection.

XMIT performs general ASCII input functions in the communication mode including simple ASCII and terminated ASCII. You may import ASCII or binary data into your PLC and convert it into various ASCII or binary data to send to DCE devices (Data Communication Equipment) based upon the needs of your application.

NOTE: XMIT block differs from the XXMIT block only in the interface. Therefore for the functional descriptions, see XXMIT *(see page 419)*.

EN and ENO can be configured as additional parameters.

Diagnostics

The block has built-in diagnostics that check that no other XMIT block is active in the PLC on the same port. In the block a control table allows you to control the communication link between the PLC and the DCE devices attached to Modbus port #1 or port #2 of the PLC. The block does NOT activate the port LED when it is transmitting data.

XMIT with Multiple Modbus Masters

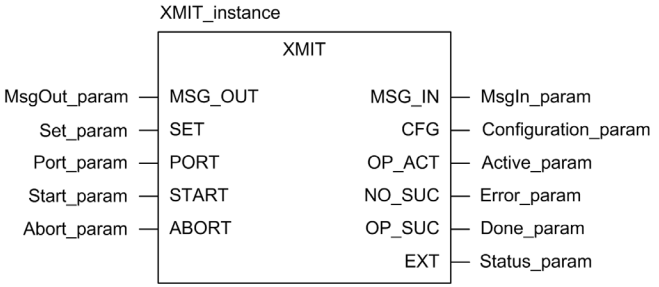
As a master/slave protocol, Modbus is designed to have only one master polling multiple slaves. Therefore, when using the XMIT block in a network with multiple masters, contention resolution and collision avoidance is your responsibility and may easily be addressed through user logic programming.

Restrictions

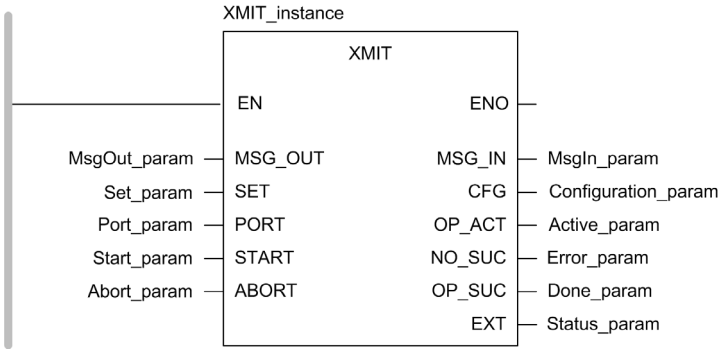
The block controls Modbus port #1 and #2 of the Momentum CPUs. It can be used with the stripped exec only. It works just as its LL984 counterpart, but without ASCII string conversion, copy and compare functions and without the Port Status functions.

Representation

FBD Representation



LD Representation



IL representation

```
CAL XMIT_instance(START:=Start_param, ABORT:=Abort_param,
    MSG_OUT:=MsgOut_param, SET:=Set_param, PORT:=Port_param,
    MSG_IN=>MsgIn_param, CFG=>Configuration_param,
    OP_ACT=>Active_param, NO_SUC=>Error_param,
    EXT=>Status_param)
```

ST representation

```
XMIT_instance (START:=Start_param, ABORT:=Abort_param,
    MSG_OUT:=MsgOut_param, SET:=Set_param, PORT:=Port_param,
    MSG_IN=>MsgIn_param, CFG=>Configuration_param,
    OP_ACT=>Active_param, NO_SUC=>Error_param,
    EXT=>Status_param);
```

Parameter Description

Description of the block parameter

Parameters	Data type	Significance
SET	XMIT_SET	Data structure for the XMIT configuration
MSG_OUT (1)	ANY	Buffer for outgoing messages (in 4x range)
PORT	BYTE	Port (0 or 1)
START	BOOL	1: Starts XMIT operation
ABORT	BOOL	1: Aborts current XMIT operation
MSG_IN (1)	ANY	Buffer for incoming messages (in 4x range)
CFG	XMIT_SET	Data structure with all components of the XMIT configuration, including the automatically set and not used variables. Only for display and must be in 4x range
OP_ACT	BOOL	1: XMIT operation in progress
NO_SUC	BOOL	1: Operation terminated UNsuccessfully
OP_SUC	BOOL	1: Operation terminated successfully
EXT (2)	INT	Displays an error code generated by XMIT
(1) No check for mapping in quad register. It may be an unlocated data.		
(2) Used for another purpose with hot standby Momentum		

XMIT_SET Data Structure

Element	Data type	Corresponding component in LL984 XMIT
BaudRate	WORD	4x+3 register (data rate)
DataBits	BYTE	4x+4 register (data bits)
Parity	BYTE	4x+5 register (parity)
StopBits	BYTE	4x+6 register (stop bits)
CommandWord	WORD	4x+8 register (command word)
MessageLen	WORD	4x+10 register (message length); in event of a terminated ASCII receipt, this component is set automatically.
RespTimeOut	WORD	4x+11 register (response time-out (ms))
RetryLimit	WORD	4x+12 register (retry limit)
XmStartDelay	WORD	4x+13 register (start of transmission delay (ms))
XmEndDelay	WORD	4x+14 register (end of transmission delay (ms)).

XMIT_CFG Data Structure

Element	Data type	Corresponding component in LL984 XMIT
FaultStatus	WORD	4x+1 register (fault status)
UserAvail_1	WORD	4x+2 register (available to user)
BaudRate	WORD	4x+3 register (data rate)
DataBits	WORD	4x+4 register (data bits)
Parity	WORD	4x+5 register (parity)
StopBits	WORD	4x+6 register (stop bits)
UserAvail_2	WORD	4x+7 register (available to user)
CommandWord	WORD	4x+8 register (command word)
MessagePtr	WORD	4x+9 register (message pointer)
MessageLen	WORD	4x+10 register (message length)
RespTimeOut	WORD	4x+11 register (response time-out (ms))
RetryLimit	WORD	4x+12 register (retry limit)
XmStartDelay	WORD	4x+13 register (start of transmission delay (ms))
XmEndDelay	WORD	4x+14 register (end of transmission delay (ms))
CurrentRetry	WORD	4x+15 register (current retry).

Parameter Description

MSG_OUT

MSG_OUT contains the message data to be transferred, for example, ASCII characters for an ASCII transfer, definition of termination characters for terminated ASCII input or Modbus templates for Modbus master messages.

The data type that must be assigned to the parameter has to be a data type WORD array. This array has to be assigned to a 4x register range. The field length must equal the length of the MSG_IN field. If the field is assigned to the range for Unlocated variables, a runtime error message will be generated.

SET

SET contains the configuration of the XMIT function block in form of the XMIT_SET data structure. This parameter may be assigned to an Unlocated variable.The data structure components have the same function as the components of the LL984 XMIT configuration. There is only one difference, the variables are set automatically by the system and the unused variables are not shown in this data structure. This means, a complete configuration requires that all components in this data structure have to be defined.

PORT

PORT specifies the communications interface. The only authorized values are 0 and 1.

NOTE: When using XMIT, L9_XMIT, or XXMIT EF blocks on the two serial ports of the Momentum 171 CBU 78090 CPU, create an EF block instance for each port. **Do not** dynamically change the port number. Set the port number as an initial value for each instance because the block stores one single port configuration.

START

A 1-signal at START initiates the XMIT operation. The 1-signal must be applied until the operation has finished or until an error has occurred.

ABORT

A 1-signal terminates the current XMIT operation and writes the abort code 121 to the FaultStatus component of the XMIT_CFG data structure at the CFG output.

MSG_IN

MSG_IN contains the incoming message data, for example, terminated ASCII input or responses of a Modbus master command which was previously sent by the XMIT function block.The data type that must be assigned to the parameter has to be a data type WORD array. This array has to be assigned to a 4x register range. The field length must equal the length of the MSG_OUT field. If the field is assigned to the range for Unlocated variables, a runtime error message will be generated.

CFG

CFG contains an XMIT function block copy of the configuration defined on SET which has the form of data structure XMIT_CFG, it includes the automatically set and not used variables.The data structure components have the same function as the components of the LL984 XMIT configuration.This data structure has to be assigned to a 4x register range. If the data structure is assigned to the range for Unlocated variables, a runtime error message will be generated. CFG is used to verify the actually applied configuration.

OP_ACT

A 1-signal indicates that an XMIT operation is in progress.

NO_SUC

A 1-signal indicates that an error has occurred or that the current XMIT operation is terminated.

OP_SUC

A 1-signal indicates that the XMIT operation has been completed successfully.

EXT

Displays an error code (except in hot standby)

Description of Data Structure XMIT_SET

XMIT_SET

This data structure contains the particular configuration for the XMIT operation. This variable may be stored in the unlocated memory. The elements of this structure have the same meaning as the corresponding elements of the XMIT_CFG *(see page 407)* structure. XMIT_SET is used to configure the XMIT block. The values of this data structure are transfered to XMIT_CFG.

NOTE: XMIT_SET does not contain the MessagePtr element. This is automatically set to the adress of the MSG_IN array and placed into XMIT_CFG.

Description of Data Structure XMIT_CFG

At a Glance

This data structure contains the actual configuration data the XMIT block uses. Do not write directly to this array, as the content is automatically generated or copied from XMIT_SET. The following is a detailed description of each of the (16) XMIT communication control table registers.

XMIT_CFG.Revision Read Only

Displays the current revision number of XMIT block. This number is automatically loaded by the function block and over writes any other number entered into this register.

XMIT_CFG.FaultStatus Read Only

This field displays a fault code generated by the XMIT block. A complete list is shown in the table below.

Fault Code	Fault Description
1	Modbus exception - Illegal function
2	Modbus exception - Illegal data address
3	Modbus exception - Illegal data value
4	Modbus exception - Slave device failure
5	Modbus exception - Acknowledge
6	Modbus exception - Slave device busy
7	Modbus exception -Negative acknowledge
8	Modbus exception -Memory parity error
9 ... 99	Reserved
100	Slave PLC data area cannot equal zero
101	Master PLC data area cannot equal zero
102	Coil (0x) not configured
103	Holding register (4x) not configured
104	Data length cannot equal zero
105	Pointer to message table cannot equal zero
106	Pointer to message table is outside the range of configured holding registers (4x)
107	Transmit message time-out (This error is generated when the UART cannot complete a transmission in 10 seconds or less. This error bypasses the retry counter and will activate the error output on the first error).
108	Undefined error
109	Modem returned ERROR

Fault Code	Fault Description
110	Modem returned NO CARRIER
111	Modem returned NO DIALTONE
112	Modem returned BUSY
113	Invalid LRC checksum from the slave PLC
114	Invalid CRC checksum from the slave PLC
115	Invalid Modbus function code
116	Modbus response message time-out
117	Modem reply time-out
118	XMIT could not gain access to PLC communications port #1 or port #2
119	XMIT could not enable PLC port receiver
120	XMIT could not set PLC UART
121	User issued an abort command
122	not used
123	not used
124	Undefined internal state
125	Broadcast mode not allowed with this Modbus function code
126	DCE did not assert CTS
127	Illegal configuration (data rate, data bits, parity, or stop bits)
128	Unexpected response received from Modbus slave
129	Illegal command word setting
130	Command word changed while active
131	Invalid character count
132	Invalid register block
133	ASCII input FIFO overflow error
134	Invalid number of start characters or termination characters

XMIT_CFG.UserAvail_1

The XMIT block does not use this register. However, it may be used in the user logic as a pointer.

XMIT_CFG.DataBits

XMIT supports the following data bits: 7 and 8. To configure a data bit size, enter its decimal number into this element. Modbus messages may be sent in ASCII mode (7 or 8 data bits) or RTU mode (8 data bits). When an invalid data bit is entered, the block displays an illegal configuration error (error code 127) in the XMIT_CFG.FaultStatus element. For more details on Modbus message formats refer to *Modicon Modbus Protocol Reference Guide* (PI MBUS 300).

XMIT_CFG.Parity

XMIT supports the following parity: none, odd and even. Enter a decimal of either: 0 = no parity, 1 = odd parity, or 2 = even parity. When an invalid parity is entered, the block displays an illegal configuration error (error code 127) in the XMIT_CFG.FaultStatus element.

XMIT_CFG.StopBits

XMIT supports one or two stop bits. Enter a decimal of either: 1 = one stop bit, or 2 = two stop bits. When an invalid stop bit is entered, the block displays an illegal configuration error (error code 127) in the XMIT_CFG.FaultStatus element.

XMIT_CFG.UserAvail_2

The XMIT block does not use this element. However, it may be used in the user logic as a pointer.

XMIT_CFG.CommandWord

The XMIT interprets each bit of the command word as a function to perform. If bit 7 and 8 are on simultaneously or if any two or more of bits 13, 14, 15 or 16 are on simultaneously or if bit 7 is not on when bits 13, 14, 15, or 16 are on error 129 will be generated. Other restrictions apply. For more details refer to XXMIT Communication Functions *(see page 436)* . The individual bit definitions are shown in the table below.

Bit	Definition
Bit 1 (msb)	Reserved
Bit 2 Enable RTS/CTS modem control	Set to 1 when a DCE connected to the PLC requires hardware handshaking using RTS/CTS control. This bit may be used in conjunction with values contained in XMIT_CFG.XmStartDelay and XMIT_CFG.XmEndDelay. Start of transmission delay keeps RTS asserted for (X mS) before XMIT sends message out of PLC port. Likewise, end of transmission delay keeps RTS asserted for (X mS) after XMIT has finished sending a message out of the PLC port. Once the end of transmission delay expires XMIT de-assert RTS.
Bit 3 Enable RS485 mode	Set to 1 when the selected port should operate in RS485 mode. Otherwise it defaults to 0, which is RS232 mode.
Bit 4	Reserved
Bit 5 Terminated ASCII input	Set to 1 to remove and discard all characters from FIFO until the starting string is matched, then these starting characters and subsequent characters are written into the MSG_IN array until the terminator sequence is matched. The terminator string is also written into the MSG_IN array. Refer to Terminated ASCII Input Function *(see page 438)* for more details.

Bit	Definition
Bit 6 Simple ASCII input	Set to 1 to remove the ASCII characters from FIFO for writing into the MSG_IN array. The Message pointer (XMIT_CFG.MessagePtr) is automatically set to the register address specified for the MSG_IN array. Refer to Simple ASCII Input Function *(see page 440)* for more details.
Bit 7 Enable ASCII string messaging	Set to 1 when you want to send ASCII messages out of the PLC. XMIT sends ASCII strings up to 1024 characters in length. You program the ASCII message into the MSG_OUT array. Two characters allowed per register. Only use Bit 7 OR Bit 8, do not try to use both. Refer to ASCII String Messaging *(see page 440)* for more details.
Bit 8 Enable Modbus messaging	Set to 1 when you want to send Modbus messages out of the PLC. Modbus messages may be in either RTU or ASCII formats. When data bits=8, XMIT uses Modbus RTU format. When data bits=7, XMIT uses Modbus ASCII format. Only use Bit 7 OR Bit 8, do not try to use both.
Bit 9 Enable ASCII receive FIFO	Set to 1 to allow the XMIT block to take control over the selected port (1 or 2) from the PLC. The block begins to receive ASCII characters into an empty 512 byte circular FIFO. Refer to ASCII Receive FIFO *(see page 450)* for more details.
Bit 10 Enable back space	Set to 1 to allow special handling of ASCII back space character (BS, 8Hex). When using either Simple ASCII Input (Bit 6) or Terminated ASCII Input (Bit 5) each back space character is removed from FIFO and may or may NOT be stored into the MSG_IN array. Refer to Enable Back space *(see page 450)* for more details.
Bit 11 Enable RTS/CTS flow control	Set to 1 to allow full duplex hardware flow control using the RTS and CTS handshaking signals for ASCII massaging. The RTS/CTS operates in both the input and output modes. Refer to Enable RTS/CTS Flow Control *(see page 451)* for more details.
Bit 12 Enable Xon/Xoff flow control	Set to 1 to allow full duplex software flow control using the ASCII Xon character (DC1, 11 Hex) and the ASCII Xoff character (DC3, 13 Hex). The Xon/Xoff operates in both the input and output modes. Refer to Enable Xon/Xoff Flow Control *(see page 452)* for more details.

Bit	Definition
Bit 13 Pulse dial modem	Set to 1 when using a Hayes compatible dial-up modem and you wish to pulse dial a telephone number. You program the phone number into the MSG_IN array. The length of the message must be in XMIT_SET.MessageLen. Pulse dialed numbers are sent to the modem automatically preceded by ATDP and with carriage return <CR> and line feed <LF> appended. Since the dial message is an ASCII string, bit 7 must be ON prior to sending the number to be dialed.
Bit 14 hangup modem	Set to 1 when using a Hayes compatible dial-up modem and you want to hangup the modem. You must use ladder logic to turn this bit ON. Since the hangup message is an ASCII string, bit 7 must be ON prior to sending the message. Hang up messages are sent to the modem automatically preceded by +++AT and with carriage return <CR> and line feed <LF> appended. XMIT looks for a correct disconnect response from the modem before it turns ON the OP_SUC output signal, noting a successful completion.
Bit 15 Tone dial modem	Set to 1 when using a Hayes compatible dial-up modem and you wish to tone dial a telephone number. The the dial message must be placed in MSG_OUT array and the length of the message in XMIT_SET.MessageLen. Tone dial numbers are sent to the modem automatically preceded by ATDT and with carriage return <CR> and line feed <LF> appended. Since the dial message is an ASCII string, bit 7 must be ON prior to sending the number to be dialed.
Bit 16 Initialize modem	Set to 1 when using a Hayes compatible dial-up modem and you want to initialize the modem. You program the initialization message into the MSG_OUT array and the length of the message XMIT_SET.MessageLen. All messages are sent to the modem automatically preceded by AT and with a carriage return <CR> and line feed <LF> appended. Since the initialization message is an ASCII string, bit 7 must be ON prior to sending the message.

XMIT_CFG.MessagePtr

This pointer is automatically handled by the XMIT block. It points to the beginning of the message table which is either the MSG_IN *(see page 404)* array or the MSG_OUT *(see page 404)* array depending on the chosen XMIT function. Each array element (4x register) holds up to two ASCII characters. Each ASCII string may be up to 1024 characters in length. For example, when you want to send 10 ASCII messages out of the PLC, you must transfer the10 ASCII characters strings into the MSG_OUT array one after another after each successful operation of XMIT.

XMIT_CFG.MessageLen

You enter the length of the current message. When XMIT is sending Modbus messages for function codes 01, 02, 03, 04, 05, 06, 08, 15 and 16, the length of the message is automatically set to five. When XMIT is receving Terminated ASCII input the length of the message must be set to five or an error results. When XMIT is sending Modbus messages for function codes twenty and twenty-one, the length of the message is automatically set to six. When XMIT is sending ASCII messages, the length may be 1 ... 1024 ASCII characters per message.

XMIT_CFG.RespTimeOut

You enter the time value in milliseconds (mS) to determine how long XMIT waits for a valid response message from a slave device (PLC, modem, etc.). In addition, the time applies to ASCII transmissions and flow control operations. When the response message is not completely formed within this specified time, XMIT issues a fault. The valid range is 0 ... 65535 mS. The timeout is initiated after the last character in the message is sent.

XMIT_CFG.RetryLimit

You enter the quantity of retries to determine how many times XMIT sends a message to get a valid response from a slave device (PLC, modem, etc.). When the response message is not completely formed within this specified time, XMIT issues a fault and a fault code. The valid range is 0 ... 65535 # of retries. This field is used in conjunction with response time-out (4x + 11).

XMIT_CFG.StartDelay

You enter the time value in milliseconds (mS) when RTS/CTS control is enabled, to determine how long XMIT waits after CTS is received before it transmits a message out of the PLC port #1. Also, you may use this register even when RTS/CTS is NOT in control. In this situation, the entered time value determines how long XMIT waits before it sends a message out of the PLC port #1. You may use this as a pre message delay timer. The valid range is 0 ... 65535 mS.

XMIT_CFG.XmEndDelay

You enter the time value in milliseconds (mS) when RTS/CTS control is enabled, to determine how long XMIT keeps RTS asserted once the message is sent out of the PLC port #1. After the time expires, XMIT deassert RTS. Also, you may use this register even when RTS/CTS is NOT in control. In this situation, the entered time value determines how long XMIT waits after it sends a message out of the PLC port #1. You may use this as a post message delay timer. The valid range is 0 ... 65535 mS.

XMIT_CFG.XmCurrentRetry

The value displayed here indicates the current number of retry attempts made by the XMIT block. This register is read only.

Application Example

Description

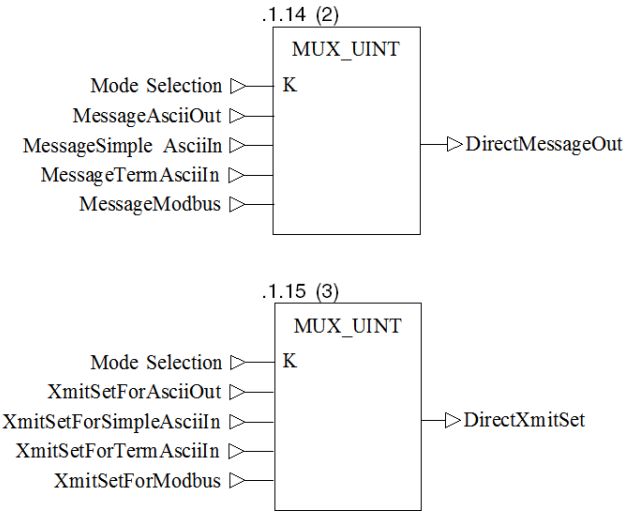
The following program is a short demo application which allows to easily switch between the four main functions of the XMIT block:

- ASCII Message Out (0)
- Simple ASCII In (1)
- Terminated ASCII In (2)
- Modbus Master (3)

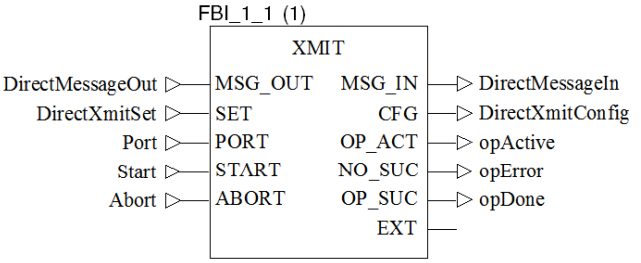
To select the function enter the appropriate number into the ModeSelection variable. A rising signal on X_Trigger activates the XMIT block by setting the Start variable 1. Start remains 1 until the function has been performed or an error occured

IEC Section

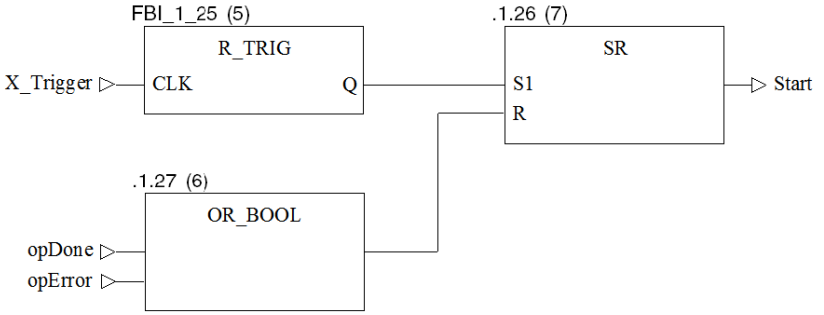
Selection of configuration data



XMIT function block



Keeps Start "on" until command finished or an error occurs



Variable Declarations

The following tables show the variables used in this example

Variable table

Variable Name	Data Type	Address	Description
Start	BOOL		Must remain ON until the XMIT has been finished
Abort	BOOL		Aborts XMIT operation
opActive	BOOL		Indicates XMIT Status

Variable Name	Data Type	Address	Description
opError	BOOL		Indicates XMIT Status
opDone	BOOL		Indicates XMIT Status
Port	BYTE		Select port 1 or 2
DirectMessageOut	WordArr256	400257	Actual configuration data or data to be transmitted
DirectXmitSet	XMIT_SET	400513	Actual configuration data
DirectMessageIn	WordArr256	400001	Data received by ASCII In functions
DirectXmitConfig	XMIT_CFG	400523	Actual configuration data
X_Trigger	BOOL		Triggers the XMIT function
ModeSelection	UINT		Select the requested function 0..3
Message AsciiOut	WordArr256		Message to be transmitted by ASCII Out
MessageSimpleAsciiIn	WordArr256		(no content needed)
MessageTermAsciiIn	WordArr256		Configuration data for Terminated ASCII In
MessageModbus	WordArr256		Configuration data for Modbus
XmitSetForAsciiOut	XMIT_SET		Configuration data for ASCII Out
XmitSetForSimpleAsciiIn	XMIT_SET		Configuration data for Simple ASCII In
XmitSetForTermAsciiIn	XMIT_SET		Configuration data for Terminated ASCII In
XmitSetForModbus	XMIT_SET		Configuration data for Modbus

Înitial Values

The following tables show the initial values for the different arrays used:

Content of XmitSetForAaciiOut Data Structure

Element Name	Data Type	Address	Comment
BaudRate	WORD	9600	
DataBits	BYTE	8	
Parity	BYTE	2	
StopBits	BYTE	1	
CommandWord	WORD	512	Bit 7 set
MessageLen	WORD	16	Transmits the first 16 Characters from MessageAsciiOut array
RespTimeOut	WORD	100	
RetryLimit	WORD	100	

Element Name	Data Type	Address	Comment
XmStartDelay	WORD	100	
XmEndDelay	WORD	100	

Content of XmitSetForSimpleAsciiIn Data Structure

Element Name	Data Type	Address	Comment
BaudRate	WORD	9600	
DataBits	BYTE	8	
Parity	BYTE	2	
StopBits	BYTE	1	
CommandWord	WORD	1152	Bits 6 and 9 set
MessageLen	WORD	16	opDone is set to 1 after receiving 16 characters
RespTimeOut	WORD	100	
RetryLimit	WORD	100	
XmStartDelay	WORD	100	
XmEndDelay	WORD	100	

Content of XmitSetForTermAsciiIn Data Structure

Element Name	Data Type	Address	Comment
BaudRate	WORD	9600	
DataBits	BYTE	8	
Parity	BYTE	2	
StopBits	BYTE	1	
CommandWord	WORD	2176	Bits 5 and 9 set
MessageLen	WORD	5	The DirectXmitSet.MessageLen element will be automatically set to 5 independent of this entry
RespTimeOut	WORD	100	
RetryLimit	WORD	100	
XmStartDelay	WORD	100	
XmEndDelay	WORD	100	

Content of XmitSetForModbus Data Structure

Element Name	Data Type	Address	Comment
BaudRate	WORD	9600	
DataBits	BYTE	8	RTU Mode
Parity	BYTE	2	
StopBits	BYTE	1	
CommandWord	WORD	256	Bit 8 set
MessageLen	WORD	5	For Function Codes 01...06, 15 and 16
RespTimeOut	WORD	1000	
RetryLimit	WORD	10	
XmStartDelay	WORD	100	
XmEndDelay	WORD	100	

Content of MessageAsciiOut Data Structure

Element Name	Data Type	Address	Comment
MessageAsciiOut[2]	WORD	17220	'CD' to be transmitted
MessageAsciiOut[3]	WORD	17734	'EF'
MessageAsciiOut[4]	WORD	18248	'GH'
MessageAsciiOut[5]	WORD	18762	'IJ'
MessageAsciiOut[6]	WORD	19276	'KL'
MessageAsciiOut[7]	WORD	19790	'MN'
MessageAsciiOut[8]	WORD	20304	'OP'
MessageAsciiOut[...]	WORD	...	

Content of MessageTermAsciiIn Data Structure

Element Name	Data Type	Address	Comment
MessageAsciiOut[1]	WORD	258	0x0102 1 starting and 2 termination characters
MessageAsciiOut[2]	WORD	9216	0x2400 First starting character '$'
MessageAsciiOut[3]	WORD	3338	0x0D0A Termination characters [CR][LF]

Content of MessageModbus Data Structure

Element Name	Data Type	Address	Comment
MessageAsciiOut[1]	WORD	3	Read multiple holding registers (4x)
MessageAsciiOut[2]	WORD	32	Read 32 registers
MessageAsciiOut[3]	WORD	10	Slave PLC modbus address
MessageAsciiOut[4]	WORD	101	Start with register 40101
MessageAsciiOut[5]	WORD	701	Data destination is register 40701

Chapter 43
XXMIT: Transmit

Introduction

This chapter describes the XXMIT block.

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
43.1	Introduction to XXMIT	420
43.2	XXMIT:Functions	421
43.3	XXMIT: Programming Rules	464
43.4	Technical References for XXMIT	466
43.5	Cabling Information	477

Section 43.1
Introduction to XXMIT

XXMIT Functionality

Function Overview

The XXMIT function block enables the use of the PLCs serial port for communication under the control of the application program.

The following communication types are supported:
- Modbus as Master
- Simple ASCII Input/Output
- ASCII Input with one or two termination characters
- Modem Communication

Function Description

The XXMIT function block sends Modbus messages from a master PLC to multiple slave PLCs or sends ASCII character strings from the PLCs Modbus slave port number 1 to ASCII printers and terminals. XXMIT sends these messages over telephone dialup modems, radio modems,

⚠ CAUTION

INVALID DATA

In the case of ASCII messages, there is no control (similar to a CRC or checksum) that guarantees the validity of the received data.

To prevent reception of invalid data due to electrical perturbations, it is advised to use Modbus Messages that include a Cyclical Redundancy Checking (CRC).

Failure to follow these instructions can result in injury or equipment damage.

Section 43.2
XXMIT:Functions

Overview

This section describes XXMIT.

What Is in This Section?

This section contains the following topics:

Topic	Page
Brief Description	422
Representation	423
Detailed Parameter Description	426
XXMIT Communication Functions	436
XXMIT ASCII Functions	437
XXMIT Modem Functions	442
XXMIT Modbus Functions	444
FIFO and Flow Control	450
Application Examples	453

Brief Description

Function Description

The XXMIT function block sends Modbus messages from a master PLC to multiple slave PLCs or sends ASCII character strings from the PLC's Modbus slave port number 1 to ASCII printers and terminals. XXMIT sends these messages over telephone dialup modems, radio modems, or simply direct connections.

NOTE: The XXMIT function block can only be used in MAST task. There is an immediate fault code (127) if it is started in the FAST / AUX or EVENT tasks (there is no control performed by Unity Pro at build time).

NOTE: EN and ENO should NOT be used with the XXMIT, otherwise the output parameters may freeze.

NOTE: Modbus communication through radio modems with Quantum PLC is only possible with the XXMIT function block configured as Modbus Master.

NOTE: Be careful that two masters (on the same bus) do not send requests simultaneously otherwise the requests are lost and each report will have a bad result which could be 16#0100 (request could not be processed) or 16#ODFF (slave is not present).

Software and Hardware Required

Software

XXMIT requires the following software:

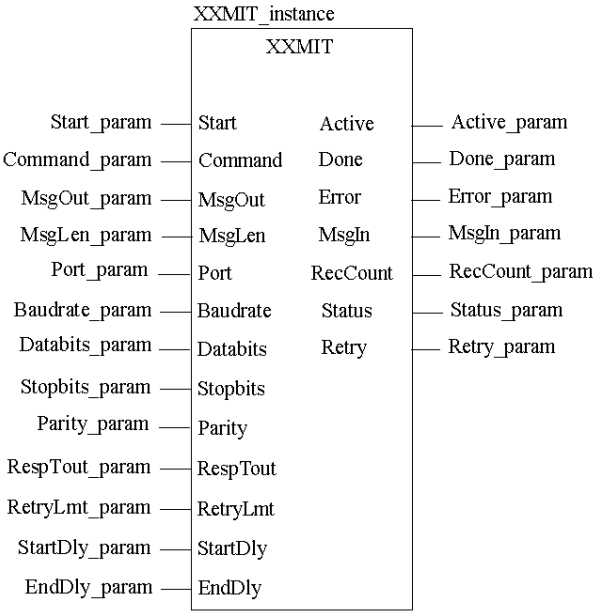
PLC	Operating System	Unity Pro
Quantum	2.3 or later	2.3 or later
Momentum	1.0 or later	8.0 or later

Refer to Block availability on various hardware platforms *(see page 29)* for a list of supported hardware on the XXMIT function block.

Representation

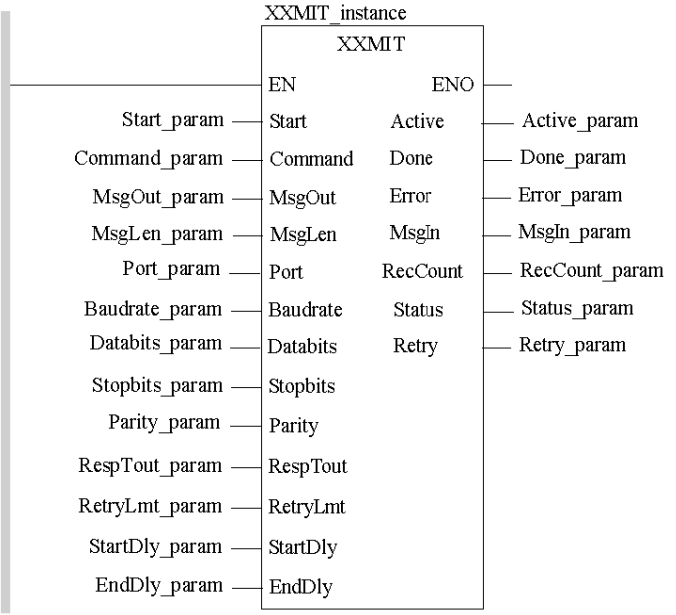
FBD representation

Representation:



LD representation

Representation:



IL representation

Representation:

```
CAL XXMIT_instance(Start:=Start_param,
    Command:=Command_param, MsgOut:=MsgOut_param,
    MsgLen:=Msglen_param, Port:=Port_param,
    Baudrate:=Baudrate_param, Databits:=Databits_param,
    Stopbits:=Stopbits_param, Parity:=Parity_param,
    RespTout:=RespTout_param, RetryLmt:=RetryLmt_param,
    StartDly:=StartDly_param, EndDly:=EndDly_param,
    Error=>Error_param, MsgIn=>MsgIn_param,
    RecCount=>RecCount_param, Status=>Status_param,
    Retry=>Retry_param)
```

ST representation

Representation:

```
XXMIT_instance (Start:=Start_param, Command:=Command_param,
    MsgOut:=MsgOut_param, MsgLen:=Msglen_param,
    Port:=Port_param, Baudrate:=Baudrate_param,
    Databits:=Databits_param, Stopbits:=Stopbits_param,
    Parity:=Parity_param, RespTout:=RespTout_param,
    RetryLmt:=RetryLmt_param, StartDly:=StartDly_param,
    EndDly:=EndDly_param, Error=>Error_param,
    MsgIn=>MsgIn_param, RecCount=>RecCount_param,
    Status=>Status_param, Retry=>Retry_param);
```

Parameter Description

Description of the block parameter

Parameters	Data type	Significance
Start	BOOL	Value of 1 starts XXMIT operation *(see page 426)*
Command	WORD	Specifies the command to be performed *(see page 426)*
MsgOut	ANY	Message to be sent *(see page 428)*
MsgLen	INT	Message lenght of output message *(see page 429)*
Port	BYTE	Selection of communication interface *(see page 429)*
Baudrate	INT	Baudrate *(see page 429)*
Databits	BYTE	Databits *(see page 429)*
Stopbits	BYTE	Stopbits *(see page 429)*
Parity	BYTE	Parity *(see page 430)*
RespTout	INT	Time to wait for a valid response *(see page 430)*
RetryLmt	INT	Number of retries until receiving a valid response *(see page 430)*
StartDly	INT	Waiting time before message transmit *(see page 430)*
EndDly	INT	Waiting time after message transmit *(see page 431)*
Active	BOOL	Value of 1 indicates that an XXMIT operation is in progress *(see page 431)*
Done	BOOL	Value of 1 indicates that the XXMIT operation has been completed successfully *(see page 431)*
Error	BOOL	Value of 1 indicates that an error occured or that the current XXMIT operation is terminated *(see page 431)*
MsgIn	ANY	Incoming message *(see page 431)*
RecCount	INT	Display the number of received characters *(see page 432)*
Status	INT	Display a fault code generated by XXMIT *(see page 433)*
Retry	INT	Indicates the current number of retry attempts made by XXMIT *(see page 435)*

Detailed Parameter Description

Start

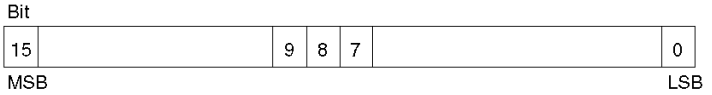
A value of 1 on the Start signal triggers the XXMIT operation. The value of 1 must be applied until the operation has finished or until an error has occurred.

NOTE: The XXMIT input parameters must be initialized before setting the START input. They must not be changed during the function block activity.

Command

The XXMIT interprets each bit of the command word as a function to perform. If bit 9 and 8 are on simultaneously or if any two or more of bits 3, 2, 1 or 0 are on simultaneously or if bit 9 is not on when bits 3, 2, 1, or 0 are on error 129 will be generated. For more details refer to *XXMIT Communication Functions, page 436*. The individual bit definitions are shown in the following table.

Command Word Layout



XXMIT Command Word Bit Definitions

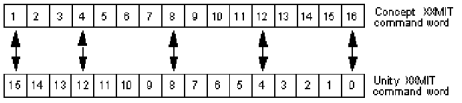
Bit	Definition
Bit 15 (msb)	With Unity Momentum processors, once XMIT *(see page 399)*, XXMIT *(see page 419)* or L9_XMIT *(see Unity Pro, UnityLL984, Block Library)* block becomes active, the serial port will not revert back to a Slave mode. This command bit has been created to set the Momentum serial port to its initial configuration. Set to 1 will set the port configuration as it was in application configuration. The **DONE** bit is set to 1 when the action is done.
Bit 14 Enable RTS/CTS modem control	Set to 1 when a DCE connected to the PLC requires hardware handshaking using RTS/CTS control. This bit may be used in conjunction with values contained in StartDly and EndDly. Start of transmission delay keeps RTS asserted for the time in StartDly (ms) before XXMIT sends a message out of PLC port. Likewise, end of transmission delay keeps RTS asserted for the time in EndDly (ms) after XXMIT has finished sending a message out of the PLC port. Once the end of transmission delay expires XXMIT de-assert RTS.
Bit 13 Enable RS485 mode	Set to 1 when the selected port should operate in RS485 mode. Otherwise it defaults to 0, which is RS232 mode. When using the Modbus port in RS485 mode with Modbus Messaging, make sure to use exactly the same parameters (baudrate, databits, stopbits, parity) for the XXMIT function block as configured for that port.
Bit 12	Reserved

Bit	Definition
Bit 11 Terminated ASCII input	Set to 1 to remove and discard all characters from FIFO until the starting string is matched, then these starting characters and subsequent characters are written into MsgIn until the terminator sequence is matched. The terminator string is also written into the MsgIn. Refer to *Terminated ASCII Input Function, page 438* for more details.
Bit 10 Simple ASCII input	Set to 1 to remove the ASCII characters from FIFO for writing into MsgIn array. Refer to *Simple ASCII Input Function, page 440* for more details.
Bit 9 Enable ASCII string messaging	Set to 1 when you want to send ASCII messages out of the PLC. XXMIT sends ASCII strings up to 1024 characters in length. You program the ASCII message into the MsgOut. Only use Bit 9 OR Bit 8, do not try to use both.
Bit 8 Enable Modbus messaging	Set to 1 when you want to send Modbus messages out of the PLC. Modbus messages may be in either RTU or ASCII formats. When data bits=8, XXMIT uses Modbus RTU format. When data bits=7, XXMIT uses Modbus ASCII format. Only use Bit 9 OR Bit 8, do not try to use both.
Bit 7 Enable ASCII receive FIFO	Set to 1 to allow the XXMIT function block to take control over the selected port (1) from the PLC. The block begins to receive ASCII characters into an empty 512 byte circular FIFO. Refer to *ASCII Receive FIFO, page 450* for more details.
Bit 6 Enable back space	Set to 1 to allow special handling of ASCII back space character (BS, 8Hex) when using either Simple ASCII Input (Bit 10) or Terminated ASCII Input (Bit 11). If Bit 6 is set, each back space character will NOT be stored into MsgIn. Refer to *Enable Back space, page 450* for more details.
Bit 5 Enable RTS/CTS flow control	Set to 1 to allow full duplex hardware flow control using the RTS and CTS handshaking signals for ASCII massaging. The RTS/CTS operates in both the input and output modes. Refer to *Enable RTS/CTS Flow Control, page 451* for more details.
Bit 4 Enable Xon/Xoff flow control	Set to 1 to allow full duplex software flow control using the ASCII Xon character (DC1, 11 Hex) and the ASCII Xoff character (DC3, 13 Hex). The Xon/Xoff operates in both the input and output modes. Refer to *Enable Xon/Xoff Flow Control, page 452* for more details.
Bit 3 Pulse dial modem	Set to 1 when using a Hayes compatible dial-up modem and you wish to pulse dial a telephone number. You program the phone number into the MsgOut. The length of the message must be in MsgLen. Pulse dialed numbers are sent to the modem automatically preceded by ATDT and with carriage return <CR> and line feed <LF> appended. Since the dial message is an ASCII string, bit 9 must be ON prior to sending the number to be dialed.
Bit 2 hangup modem	Set to 1 when using a Hayes compatible dial-up modem and you want to hangup the modem. You must use user logic to turn this bit ON. Since the hangup message is an ASCII string, bit 9 must be ON prior to sending the message. Hang up messages are sent to the modem automatically preceded by +++AT and with carriage return <CR> and line feed <LF> appended. XXMIT looks for a correct disconnect response from the modem before it turns ON the Done output signal, noting a successful completion.

Bit	Definition
Bit 1 Tone dial modem	Set to 1 when using a Hayes compatible dial-up modem and you wish to tone dial a telephone number. You program the phone number into the MsgOut. The length of the message must be in MsgLen. Tone dial numbers are sent to the modem automatically preceded by ATDT and with carriage return <CR> and line feed <LF> appended. Since the dial message is an ASCII string, bit 9 must be ON prior to sending the number to be dialed.
Bit 0 Initialize modem	Set to 1 when using a Hayes compatible dial-up modem and you want to initialize the modem. You program the initialization message into MsgOut and the length of the message into MsgLen. All messages are sent to the modem automatically preceded by AT and with a carriage return <CR> and line feed <LF> appended. Since the initialization message is an ASCII string, bit 9 must be ON prior to sending the message.

NOTE: In comparison with the Concept `XXMIT` Command word, the bits numeration of the Unity Command word has changed:

Command word conversion



In case of application from Concept to Unity, the new bit numeration has to be taken into account when the Command word is accessed by bits. This does **NOT** apply when communication word is accessed by word.

MsgOut

MsgOut contains the message data to be transferred, for example, ASCII characters for an ASCII transfer, definition of termination characters for terminated ASCII input or Modbus templates for Modbus master messages.

The data type that must be assigned to the parameter has to match the requirements of the function to be performed. In the case of Modbus operation, the Data Type of MsgOut and MsgIn must be equal.

NOTE: For all communication types (Modbus Master, ASCII Input / Output), MsgOut and MsgIn parameters must be assigned to a variable. The two variables must be either of Data Type Byte Array or Word Array. Other type may produce unpredictable results.

NOTE: For Modbus Messaging MsgOut must be a field of words. The minimum size of the array is WordArr9

MsgLen

You must enter the length of the current message according to the selected XXMIT function.

The following table gives an overview for Modbus and ASCII functions:

XXMIT function	Subfunction	Message Length
Modbus Messaging	01, 02, 03, 04, 05, 06, 08, 15, 16	5
Terminated ASCII Input		5
Simple ASCII Input		1...1024.
ASCII String Messaging		1...1024. The selected length must match the size of the array assigned to MsgOut. Otherwise you get error 129.

NOTE: For Modbus function codes, the value of MsgLen can be higher than five words but, only the first five words of the Modbus definition table are taken into account at RUN time.

Port

Port specifies the communications interface. The authorized values are 0 or 1.

Baudrate

XXMIT supports the following data rates: 50, 75, 110, 134, 150, 300, 600, 1200, 1800, 2000, 2400, 3600, 4800, 7200, 9600, 19200. To configure a data rate, enter its decimal number. When an invalid data rate is entered, the block displays an illegal configuration error (error code 127) in the XXMIT Status element.

Databits

XXMIT supports the following data bits: 7 and 8. To configure a data bit size, enter its decimal number into this element. Modbus messages may be sent in ASCII mode or RTU mode. ASCII mode requires 7 data bits, while RTU mode requires 8 data bits. When sending ASCII character message you may use either 7 or 8 data bits. When an invalid data bit is entered, the block displays an illegal configuration error (error code 127) in the XXMIT Status element. For more details on Modbus message formats refer to *Modicon Modbus Protocol Reference Guide* (www.modbus.org).

Stopbits

XXMIT supports one or two stop bits. Enter a decimal of either: 1 = one stop bit, or 2 = two stop bits. When an invalid stop bit is entered, the block displays an illegal configuration error (error code 127) in the XXMIT Status element.

Parity

XXMIT supports the following parity: none, odd and even. Enter a decimal of either: 0 = no parity, 1 = odd parity, or 2 = even parity. When an invalid parity is entered, the block displays an illegal configuration error (error code 127) in the XXMIT Status element.

RespTout

You enter the time value in milliseconds (ms) to determine how long XXMIT waits for a valid response message from a slave device (PLC, modem, etc.). In addition, the time applies to ASCII transmissions and flow control operations. When the response message is not completely formed within this specified time, XXMIT issues a fault. The valid range is 0... 32767 ms. The timeout is initiated after the last character in the message is sent.

RetryLmt

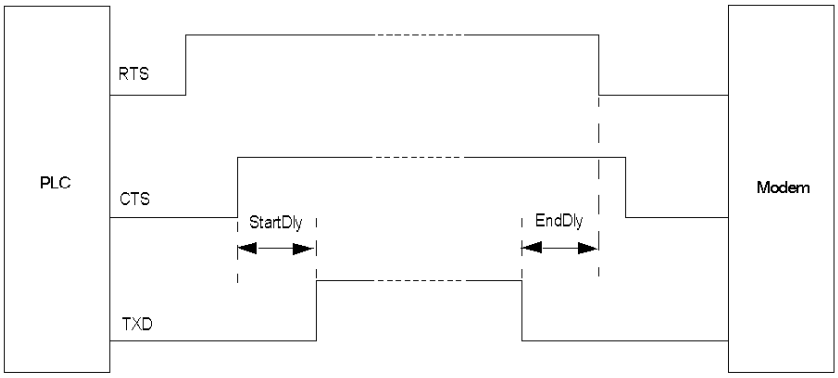
You enter the quantity of retries to determine how many times XXMIT sends a message to get a valid response from a slave device (PLC, modem, etc.). When the response message is not completely formed within this specified time, XXMIT issues a fault and a fault code. The valid range is 0... 32767 number of retries. This field is used in conjunction with RespTout.

StartDly

You enter the time value in milliseconds (ms) when RTS/CTS control is enabled, to determine how long XXMIT waits after CTS is received before it transmits a message out of the PLC port. Also, you may use this register even when RTS/CTS is NOT in control. In this situation, the entered time value determines how long XXMIT waits before it sends a message out of the PLC port. You may use this as a pre message delay timer. The valid range is 0... 32767 ms.

EndDly

You enter the time value in milliseconds (ms) when RTS/CTS control is enabled, to determine how long XXMIT keeps RTS asserted once the message is sent out of the PLC port. After the time expires, XXMIT dessert RTS. Also, you may use this register even when RTS/CTS is NOT in control. In this situation, the entered time value determines how long XXMIT waits after it sends a message out of the PLC port. You may use this as a post message delay timer. The valid range is 0... 32767 ms.



NOTE: On RS 485 communication the transmit signal is held to '1' during the EndDly time. On 2-wire connections any characters coming from the communication partner will be lost. Therefore set EndDly to 0 ms if the function is not needed.

Active

A value of 1 indicates that an XXMIT operation is in progress.

Done

A value of 1 indicates that the XXMIT operation has been completed successfully.

Error

A value of 1 indicates that an error has occurred or that the current XXMIT operation is terminated.

MsgIn

MsgIn contains the incoming message data, for terminated ASCII input or simple ASCII input.

In case of Modbus operation, the data type must be equal to the type of the MsgOut field.

RecCount

This element displays the number of received characters.

NOTE: When too many characters are received with no terminal character detected, an error is reported in the Status parameter (131) and the RecCount parameter is set to MsgLen-1.

Status

This element displays a fault code generated by the XXMIT.

A complete list is shown in the table below.

Fault Status

Fault Code	Fault Description
1	Modbus exception - Illegal function
2	Modbus exception - Illegal data address
3	Modbus exception - Illegal data value
4	Modbus exception - Slave device failure
5	Modbus exception - Acknowledge
6	Modbus exception - Slave device busy
7	Modbus exception -Negative acknowledge
8	Modbus exception -Memory parity error
9... 99	Reserved
100	Slave PLC data area cannot equal zero
101	Master PLC data area cannot equal zero
102	bit (%M) not configured
103	Master PLC memory word (%MW) not configured
104	Data length cannot equal zero
105, 106	Reserved
107	Transmit message time-out (This error is generated when the UART cannot complete a transmission in 10 seconds or less. This error bypasses the retry counter and will activate the error output on the first error, see Additional information for error 107 below).
108	Undefined error
109	Modem returned ERROR
110	Modem returned NO CARRIER
111	Modem returned NO DIALTONE
112	Modem returned BUSY
113	Invalid LRC checksum from the slave PLC, see 1) below
114	Invalid CRC checksum from the slave PLC, see 1) below
115	Invalid Modbus function code
116	Modbus response message time-out, see 2 below)
117	Modem reply time-out
118	XXMIT could not gain access to PLC communication port number 1
119	XXMIT could not enable PLC port receiver

Fault Code	Fault Description
120	XXMIT could not set PLC UART
121	Reserved
122	Invalid Port
123	Reserved
124	Undefined internal state
125	Broadcast mode not allowed with this Modbus function code
126	DCE did not assert CTS
127	• Illegal configuration (data rate, data bits, parity, or stop bits). • Execution in any other task than MAST. • Illegal Modbus function code. • Slave address greater than 247. • Data length parameter is greater than limits describe in this document. • Port number is not 1. • RespTout, StartDly, EndDly, RecCount, MsgLen, RetryLmt values are less than 0.
128	Unexpected response received from Modbus slave, see 1) below
129	Illegal command word setting
130	Command word changed while active
131	Invalid character count
132	Reserved
133	ASCII input FIFO overflow error
134	Invalid number of start characters or termination characters
135...149	Reserved
150	Either configured port already taken by another instance of the XXMIT or the configured port is not supported on that PLC
151	MsgOut is smaller than 12 Bytes while 'Modbus Master Messaging' function is selected. Also includes MsgLen parameter value less than 5
152	Variable connected to MsgOut is smaller than the value of the MsgLen parameter while 'ASCII String Messaging' is selected
153	Variable connected to MsgIn is smaller than the value of the MsgLen parameter while either 'Terminated ASCII Input' or 'Simple ASCII Input' is selected
154	XXMIT start bit set to 1 in a Quantum Hot Standby PLC that is not in the Primary mode.

NOTE: 1) This fault code does happen if the Modbus slave responds too fast.
In case the used Modbus slave is a Modicon PLC, please check the `Modbus Port Setup` of that PLC's configuration.
2) For the `XXMIT` function block in Modbus master mode, if error 116 persists after performing correct configuration using keypad or online modifications, rebuild and transfer application to the PLC.

Additional information for error 107

Error 107 appears when a discrepancy is detected on the `XXMIT` configuration. If `XXMIT` internal port setting is in RS232 mode and command word is in RS485 mode (#2100). The communication mode must be changed.

The following table shows how to change the communication mode if error 107 occurs:

Step	Action
1	Connect the PC to Unity
2	Put the PLC in STOP
3	Change Modbus port settings (RS232 or RS 485)
4	Change command word according to the requested mode
5	Rebuild application
6	Download application
7	Put the PLC in RUN

NOTE: Please note that:

- Switch from RS232 to RS485 mode takes 3 seconds when both internal ports settings and command words match. During those 3 seconds the error code 126 (DCE not asserted, only for RS232 mode) appears.
- Switch from RS485 to RS232 mode is immediate as it matches the default configuration.

Retry

The value displayed here indicates the number of retry attempts made by `XXMIT`. This element is read only.

This Output is updated one time only when the operation is complete.

For Modicon Quantum the Retry output is updated each time there is a new retry.

For Modicon Momentum the Retry output is updated once the operation is complete, with or without success.

XXMIT Communication Functions

XXMIT Command Word

The XXMIT performs six functions shown below. For each function certain bits of the Command word must be set.

Command Word Bits

Command Word Functions in Relation to Bits

Function	Command word bits that may be set to 1	Bits that MUST be set to = 0
Terminated ASCII input (Bit 11=1) (See note below)	14,7,6,5,4	10,9,8,3,2,1,0
Simple ASCII input (Bit 10=1) (See note below)	14,7,6,5,4	11,9,8,3,2,1,0
Simple ASCII output (Bit 9=1)	14,7,6,5,4	11,10,8,3,2,1,0
Modem output (Bit 9=1)	14,3,2,1,0	11,10,8,7,6,5,4 (plus one, but ONLY one, of the following bits is set to 1: 3,2,1 or 0, while the other three bits must be set to 0)
Modbus master messaging output (Bit 8=1)	14,	11,10,9,7,6,5,4,3,2,1,0

NOTE: When using either of these functions you MUST set Enable ASCII receive FIFO (Bit 7) to 1. Bit 15 (MSB) and Bits 13 and 12 are reserved. (See Table *Command, page 426*)

XXMIT ASCII Functions

At a Glance

XXMIT supports the following ASCII communication functions

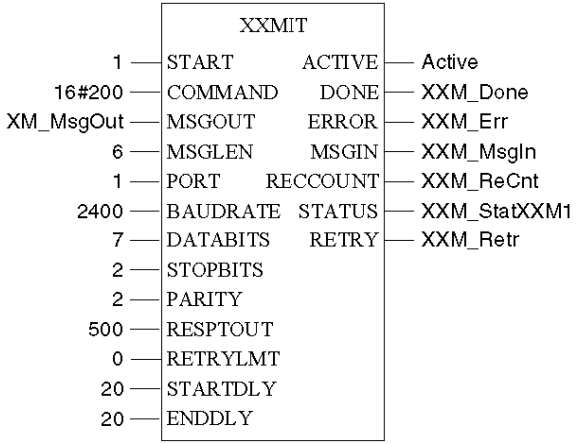
- Terminated ASCII Input
- Simple ASCII Input
- ASCII String Messaging

Modbus Port Configuration

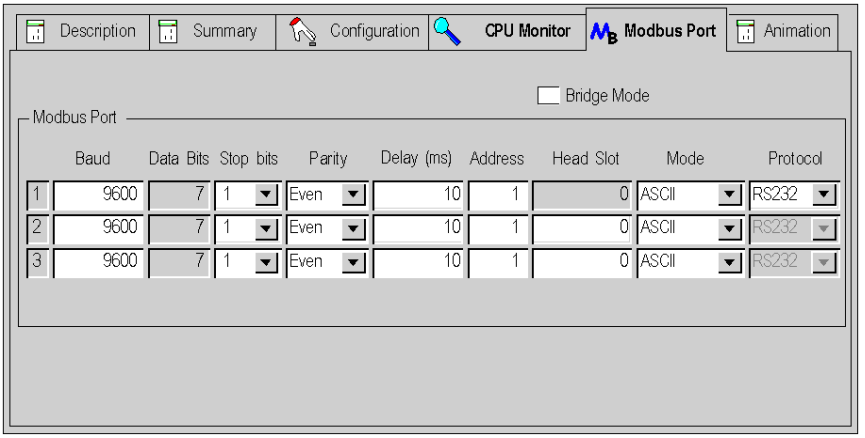
XXMIT function is configured through its Function Block. The Function Block initializes the Modbus port 1 every time it is active, with the defined parameters. The XXMIT parameters can be displayed in the controller's LCD display.

NOTE: The settings configured in the CPU configuration screen or via the controller's LCD display have no effect on the transmission values in master mode.

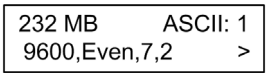
Example of XXMIT Function Block configuration:



Example of CPU Configuration screen with different value set:



Example of settings on controller's LCD display:



The 3 graphics above show different values for XXMIT function. Regardless of what is set or displayed in the CPU configuration screen or on the LCD display, the actual values used for transmission are the one from the XXMIT Function Block:

- Baudrate: 2400
- Parity: Even
- Data Bits: 7
- Stop Bits: 2

Terminated ASCII Input Function

When Bit 11 of the Command Word is activated for terminated ASCII Input messages, the MsgOut array has to contain the ASCII input definition table. Depending of which datatype you selected for MsgOut, the terminated ASCII definition table consists of three words or 6 bytes. The terminated ASCII input definition table is shown in the table below.

Terminated ASCII Input Definition Table (Datatype WordArray)

Word	High Byte	Low Byte
MsgOut[1]	Number of starting characters (allowed content = 0, 1, 2)	Number of terminator characters (allowed content = 1, 2)
MsgOut[2]	First starting character	Second starting character
MsgOut[3]	First terminator character	Second terminator character

Terminated ASCII Input Definition Table (Datatype ByteArray)

Byte	Function
MsgOut[1]	length of start string (0 or 1 or 2)
MsgOut[2]	length of termination string (1 or 2)
MsgOut[3]	1st start character
MsgOut[4]	2nd start character
MsgOut[5]	1st termination character
MsgOut[6]	2nd termination character

During the process, RecCount holds a running count of characters written into the MsgIn array. Once the terminated string is received the Done output on XXMIT goes ON and RecCount holds the total length of the received string including the starting and terminator strings. At this point XXMIT still owns the port and continues to save newly received characters into the ASCII receive FIFO, because the enable ASCII receive FIFO Command Word, Bit 7 is ON.

Using program logic, you can clear the simple ASCII input Bit before the next scan, while leaving the enable ASCII receive FIFO Bit ON. Thus, MsgIn is NOT over written by newer FIFO data, which is still collected in the FIFO.Using program logic, you can clear both bits for enable ASCII receive FIFO (Bit 7), and terminated ASCII input (Bit 11) to return port control back to the PLC.

When too many characters are written into the MsgIn array with NO terminator detected, or the MsgIn array is outside the allowed range for the configured PLC an error is reported in Status and the RecCount parameter is not significant. The character limit is the smaller of 1024 or two times the sizes of the MsgIn array.

Terminated ASCII Example

Assume that XXMIT is activated with the command word Bit 7 and 11 set. Enable ASCII FIFO and terminated ASCII. The following ASCII string is received by the port: "AMScrlf$weight = 1245 GRAMScrlf$wei". Refer to the ASCII Input Definition Table that shows the contents denoted by ( ) used in this example.

Terminated ASCII Input Definition Table (content Datatype Byte Array)

Byte	Content
MsgOut[1]	Number of starting characters (0x01)
MsgOut[2]	Number of terminator characters (0x02)
MsgOut[3]	First starting character ('$')
MsgOut[4]	Second starting character (Not Used)
MsgOut[5]	First terminator character ('cr')
MsgOut[6]	Second terminator character ('lf")

Terminated ASCII Input Definition Table Example (content for Datatype Word Array)

Word	High Byte	Low Byte
MsgOut[1]	Number of starting characters (0x01)	Number of terminator characters (0x02)
MsgOut[2]	First starting character ('$')	Second starting character (Not Used)
MsgOut[3]	First terminator character ('cr')	Second terminator character ('lf")

XXMIT becomes ACTIVE and then discards from the input FIFO the initial five characters, "AMScrlf", because they do not match the first starting character set to '$'. On the logic scan after the '$' is received, XXMIT remains ACTIVE and it copies the '$' and subsequent characters into the MsgIn array, updating RecCount with the count done so far, as the characters come in.

After the final termination character is received the output Done is activated and MsgLen contains the total length equal to 22 characters (0x0016). The MsgIn array contains: "$weight = 1245 GRAMScrlf" as Byte Array (or: "$w", "ei", "gh", "t ", "= ", "12", "45", " G", "RA", "MS", "crlf" if using a Word Array).

On the scan that the output Done is activated, the already received characters from the next message, "$wei", that came in after the termination string, remains in the ASCII input FIFO. This gives the program logic the opportunity to turn off the Terminated ASCII input before the next scan solve of XXMIT for this port, keeping those characters in the FIFO until the PLC completes processing the current message, that might take several scans.

Simple ASCII Input Function

All incoming characters are placed into the MsgIn array. If MsgIn is defined as Byte Array (as recommended), the incomming characters are simply stored first character into first array element, second character into second and so on.If MsgIn is defined as WordArray, two characters are stored in each element. The first character is stored in the low byte of the first element. The second character is stored in the high byte of the first element. The third character is stored in the high byte of the second element, and so on. The Message Length variable (MsgLen) contains the length of the message (1 ... 1024 characters).

NOTE: When Simple ASCII Input (Bit 10) and ASCII Receive FIFO (Bit 7) remain set, new characters are continuously transferred from FIFO into the same MsgIn array thus constantly over writing the previous characters stored into the MsgIn array.

ASCII String Messaging

When Command Word, Bit 9 is activated for String Messaging, the MsgOut array has to contain the ASCII information to be transmitted. The message length MsgLen has to be set to the length of the message to be transmitted.

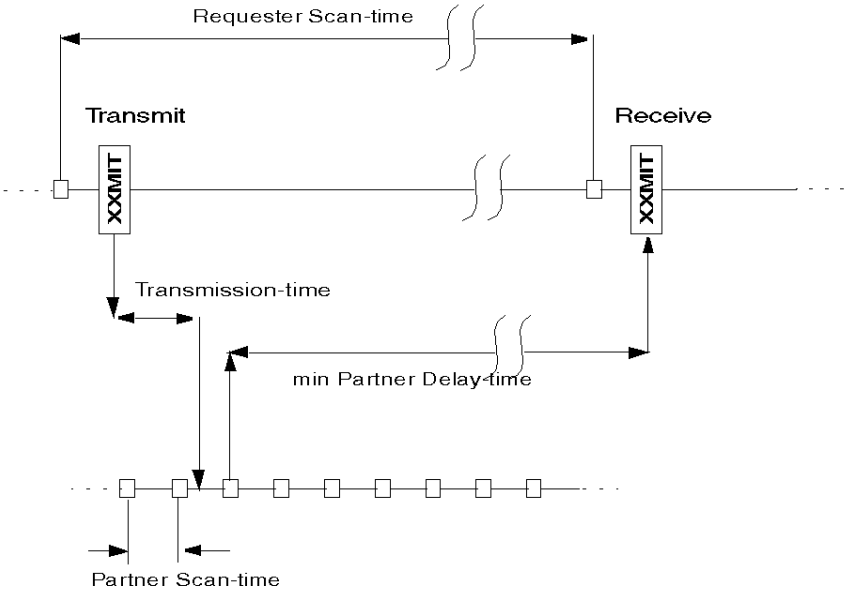
As mentioned in Detailed Parameter Description *(see page 426)*, MsgOut may be of any datatype. For ASCII String Messaging the type ByteArray reflects best the nature of strings: First Byte contains first character and so on. (For more information See Simple ASCII Send *(see page 456)*)

Transmit - Receive Transition

If your application requires to receive an answer from another device after transmitting a message (request - response), you need XXMIT to switch from transmit mode to receive mode in order to read the communication partner's response. The earliest point in time to switch XXMIT from transmit to receive is the cycle following the transmit operation. It is the responsibility of the user to ensure that the response is delayed by at least one cycle time of the requesting PLC to avoid communication failure.

The transmit delay on the communication partner's side is especially important in cases of long cycle times on the requester's side and fast communication partners.

Timing considerations for the Partner Delay-time:



From the above figure (not to scale) you can estimate the influence of the three different times Requester Scan-time, Transmission-time and Partner Scan-time on the required Partner Delay-time. As the requester's and partner's scans are asynchronous, the Partner Scan-time should not be taken into account. The transmission-time depends on telegram length and baud rate. A message with 18 characters at 9600 baud takes 14 ms. The main contribution obviously comes from the Requester Scan-time. So even the minimum Partner Delay-time could be less than the Requester Scan-time, we recommend to use the Requester Scan-time as the minimum Partner Delay-time to ensure a sound communication..

XXMIT Modem Functions

At a glance

XXMIT allows you to communicate to a Hayes compatible modem using the functions listed in the following table:

Modem Functions

Bit in Command Word	Function
Bit 3	Pulse dial modem
Bit 2	Hangup modem
Bit 1	Tone dial modem
Bit 0	Initialize modem

Pulse Dial Modem

Set Bit 3 of the command word to 1 when using a Hayes compatible dial-up modem and you wish to pulse dial a telephone number. You program the phone number into the MsgOut array. The length of the message must be in MsgLen. Pulse dialed numbers are sent to the modem automatically preceded by ATDP and with carriage return <CR> and line feed <LF> appended. Since the dial message is an ASCII string, bit 9 must be ON prior to sending the number to be dialed.

Hangup Modem

Set Bit 2 of the command word to 1 when using a Hayes compatible dial-up modem if you want to hangup the modem. You must use program logic to turn this bit ON. Since the hangup message is an ASCII string, bit 9 must be ON prior to sending the message. Hang up messages are sent to the modem automatically preceded by +++AT and with carriage return <CR> and line feed <LF> appended. XXMIT looks for a correct disconnect response from the modem before it turns ON the Done output signal, noting a successful completion.

Tone Dial Modem

Set Bit 1 of the command word to 1 when using a Hayes compatible dial-up modem and you wish to tone dial a telephone number. You program the phone number into the MsgOut array. The length of the message must be in MsgLen. Tone dialed numbers are sent to the modem automatically preceded by ATDT and with carriage return <CR> and line feed <LF> appended. Since the dial message is an ASCII string, bit 9 must be ON prior to sending the number to be dialed.

Initialize Modem

Set Bit 0 of the command word to 1 when using a Hayes compatible dial-up modem and you want to initialize the modem. You program the initialization message into the MsgOut array and the length of the message into MsgLen. All messages are sent to the modem automatically preceded by AT and with a carriage return <CR> and line feed <LF> appended. Since the initialization message is an ASCII string, bit 9 must be ON prior to sending the message.

XXMIT Modbus Functions

At a Glance

XXMIT supports the following Modbus function codes:

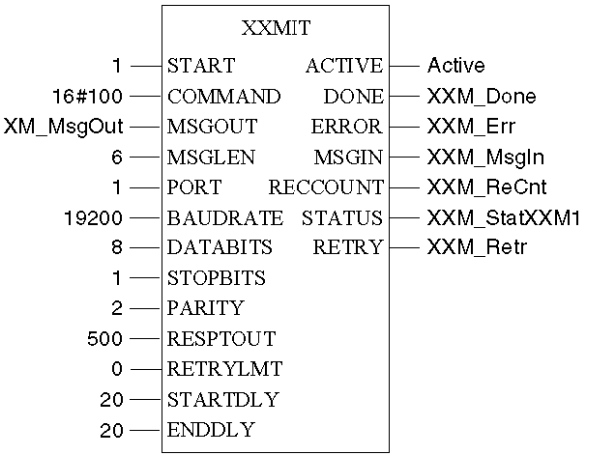
- 01 ... 06 and 15 ... 16
- 08

Modbus Port Configuration

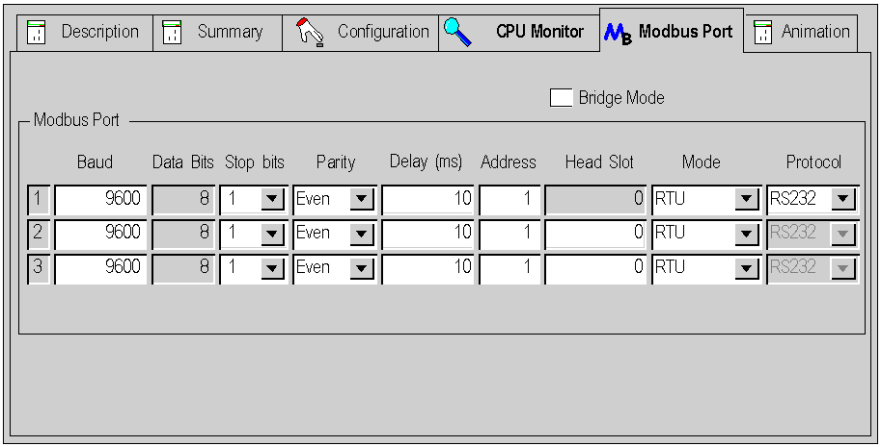
XXMIT function is configured through its Function Block. The Function Block initializes the Modbus port 1 every time it is active, with the defined parameters. The XXMIT parameters can be displayed in the controller's LCD display.

NOTE: The settings configured in the CPU configuration screen or via the controller's LCD display have no effect on the transmission values in master mode.

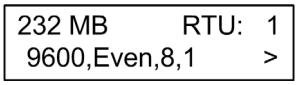
Example of XXMIT Function Block configuration:



Example of CPU Configuration screen with different value set:



Example of settings on controller's LCD display:



The 3 graphics above show different values for XXMIT function. Regardless of what is set or displayed in the CPU configuration screen or on the LCD display, the actual values used for transmission are the one from the XXMIT Function Block:

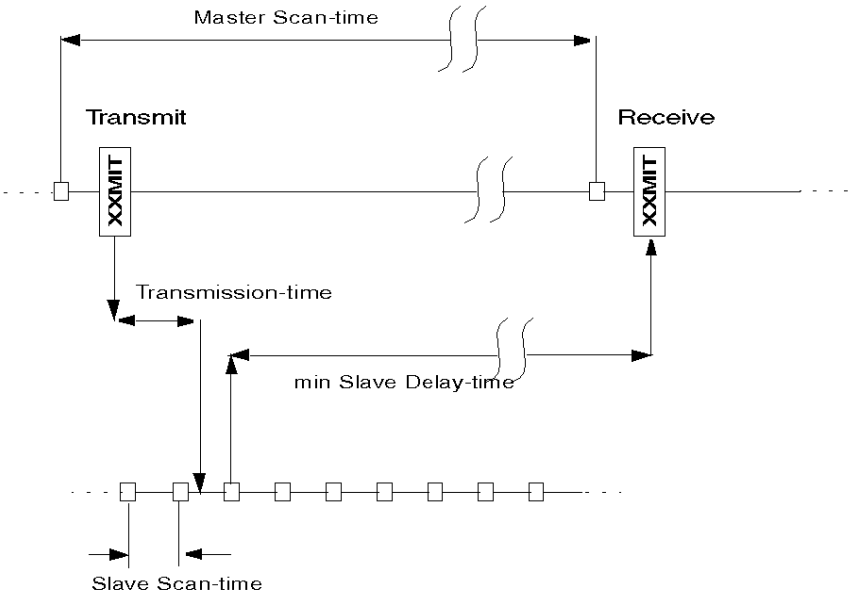
- Baudrate: 19200
- Parity: Even
- Data Bits: 8
- Stop Bits: 1

Transmit - Receive Transition

Except broadcast messages all Modbus functions require XXMIT to switch from transmit mode to receive mode in order to read the slave's response. XXMIT switches from transmit to receive in the cycle following the transmit operation. It is the responsibility of the user to ensure that the slave's response is delayed by at least one cycle time of the master to avoid communication failure.

The transmit delay on the slave side is especially important in cases of long master cycle times and fast slaves.

Timing considerations for the Slave Delay-time:



From the above figure (not to scale) you can estimate the influence of the three different times Master Scan-time, Transmission-time and Slave Scan-time on the required Slave Delay-time. As the master and slave scans are asynchronous, the Slave Scan-time should not be taken into account. The transmission-time depends on telegram type, baud rate and protocol. A standard Read request at 9600 baud using ASCII protocol for example takes 14 ms. The main contribution obviously comes from the Master Scan-time. So even the minimum Slave Delay-time could be less than the Master Scan-time, we recommend to use the Master Scan-time as the minimum Slave Delay-time to ensure a sound communication..

NOTE: For Quantum PLCs you can specify the delay time in the Modbus Port Settings dialog. The delay time can be specified between 10 and 1000 ms, which will automatically be rounded up to be divisible by 10.
You must enter the required delay-time **plus** 10 ms. For example to have a 110 ms delay you must enter 120 in this field.

NOTE: For 171 CBU 78090 Momentum PLCs, create an EF block instance for each port when using XMIT, L9_XMIT, or XXMIT EF blocks on the two serial ports of the PLC. **Do not** dynamically change the port number. Set the port number as an initial value for each instance because the block stores one single port configuration

Modbus Function Codes (01 ... 06, 15 and 16)

For Modbus messages, the MsgOut array has to contain the Modbus definition table. This has to be defined as a field of words. The Modbus definition table for Modbus function code: 01, 02, 03, 04, 05, 06, 15 and 16 is five words long and you must set MsgLen to 5 for successful XXMIT operation. The Modbus definition table is shown in the table below.

Modbus Definition Table Function Codes (01 ... 06, 15 and 16)

Content	Description
Modbus function code (MsgOut[1])	XXMIT supports the following function codes: 01 = Read multiple bits (%Q) 02 = Read multiple discrete bits (%I) 03 = Read multiple words (%MW) 04= Read multiple input words (%IW) 05 = Write single bit (%Q) 06 = Write single words (%MW) 15 = Write multiple bits (%Q) 16 = Write multiple words (%MW)
Quantity (MsgOut[2])	Enter the amount of data you want written to the slave PLC or read from the slave PLC. For example, enter 100 to read 100 words from the slave PLC or enter 32 to write 32 bits to a slave PLC. There is a size limitation on quantity that is dependent on the PLC model. Refer to Appendix A for complete details on limits.
Slave PLC address (MsgOut[3])	Enter the slave Modbus PLC address. Typically the Modbus address range is 1 ... 247. To send a Modbus message to multiple PLCs, enter 0 for the slave PLC address. This is referred to as Broadcast Mode. Broadcast Mode only supports Modbus function codes that writes data from the master PLC to slave PLCs. Broadcast Mode does NOT support Modbus function codes that read data from slave PLCs.
Slave PLC data area (MsgOut[4])	For a read command, the slave PLC data area is the source of the data. For a write command, the slave PLC data area is the destination for the data. For example, when you want to read bits (%I300 ... %I500) from a slave PLC, enter 300 in this field. When you want to write data from a master PLC and place it into words (%MW100) of a slave PLC, enter 100 in this field. Depending on the type of Modbus command (write or read), the source and destination data areas must be as defined in the Source and Destination Data Areas table below.
Master PLC data area (MsgOut[5])	For a read command, the master PLC data area is the destination for the data returned by the slave. For a write command, the master PLC data area is the source of the data. For example, when you want to write bits (%M16 ... %M32) located in the master PLC to a slave PLC, enter 16 in the field. When you want to read words (%IW1 ... %IW100) from a slave PLC and place the data into the master PLC data area (%MW100 ... %MW199), enter 100 in this field. Depending on the type of Modbus command (write or read), the source and destination data areas must be as defined in the Source and Destination Data Areas table below.

Source and Destination Data Areas for Function Codes (01 ... 06, 15 and 16)

Function Code	Master PLC Data Area	Slave PLC Data Area
03 (Read multiple 4x)	%MW (destination)	%MW (source)
04 (Read multiple 3x)	%MW (destination)	%IW (source)
01 (Read multiple 0x)	%M (destination)	%Q (source)
02 (Read multiple 1x)	%M (destination)	%I (source)
16 (Write multiple 4x)	%MW (source)	%MW (destination)
15 (Write multiple 0x)	%M (source)	%Q (destination)
05 (Write single 0x)	%M (source)	%Q (destination)
06 (Write single 4x)	%MW (source)	%MW (destination)

When you want to send 20 Modbus messages out of the PLC, you must transfer 20 Modbus definition tables one after another into MsgOut after each successful operation of XXMIT, or you may program 20 separate XXMITs and then activate them one at a time through user logic.

Modbus Function Code (08)

For Modbus messages, the MsgOut array has to contain the Modbus definition table. This has to be defined as a field of words. The Modbus definition table for Modbus function code: 08 is five words long and you must set MsgLen to 5 for successful XXMIT operation. The Modbus definition table is shown in the table below.

Modbus Definition Table Function Codes (08)

Content	Description
Modbus function code (MsgOut[1])	XXMIT supports the following function code: 08 = Diagnostics

Content	Description	
Diagnostics (MsgOut[2])	Enter the diagnostics subfunction code decimal value in this field to perform the specific diagnostics function desired. The following diagnostic subfunctions are supported:	
	Subfonction Code	**Description**
	00	Return query data
	01	Restart comm option
	02	Return diagnostic word
	03	Change ASCII input delimiter
	04	Force listen only mode
	05 ... 09	Reserved
	10	Clear counters
	11	(& diagnostics words in 384, 484)
	12	Return bus messages count
	13	Return bus comm error count
	14 ... 15	Return bus exception error count
	16	Not supported
	17	Return slave NAK count
	18	Return slave busy count
	19 ... 21	Return bus Char overrun count
Slave PLC address (MsgOut[3])	Enter the slave Modbus PLC address. Typically the Modbus address range is 1 ... 247. Function code 8 does NOT support Broadcast Mode (Address 0).	
Diagnostics function data field content (MsgOut[4])	You must enter the decimal value needed for the data area of the specific diagnostic subfunction:	
	Subfonction Code	**Description**
	02, 04, 10, 11, 12, 13, 16, 17 and 18	This value is automatically set to zero in the Modbus message sent to slave (this value is not reflected in the MsgOut buffer).
	00, 01, and 03	You must enter the desired data field value. For more details, refer to *Modicon Modbus Protocol Reference Guide (www.modbus.org)*.
Master PLC data area (MsgOut[5])	For all subfunctions, the master PLC data area is the destination for the data returned by the slave. You must specify a memory word %MW that marks the beginning of the data area where the returned data is placed. For example, to place the data into the master PLC data area starting at (%MW100), enter 100 in this field. Subfunction 04 does NOT return a response. For more details, refer to *Modicon Modbus Protocol Reference Guide (www.modbus.org)*.	

FIFO and Flow Control

At a glance

XXMIT allows the user to define the use of a receive FIFO buffer, flow control and the function of received back spaces.

ASCII Receive FIFO

Setting Bit 7 of the command word to 0 ends this function. When the FIFO receives 512 characters an internal overflow is set. When this occurs all subsequent characters are discarded, all ASCII input operations (simple and terminated) are ended, and the block returns an error until you toggle (Bit 7). When (Bit 7) is toggled, all data in the FIFO is discarded, both ASCII input control bits are ignored (Simple ASCII (Bit 10), Terminated ASCII (Bit 11)), and when no ASCII output controls are selected then the control of the serial port (1) is returned back to the PLC.

You need to set either Terminated ASCII (Bit 11) or Simple ASCII (Bit 10) to remove the ASCII characters from FIFO for processing. No more than one of the following three bits can be set simultaneously: Terminated ASCII (Bit 11), Simple ASCII (Bit 10), or ASCII String Messaging (Bit 9).

Full duplex operation may be achieved by setting both ASCII Receive FIFO (BIT 7), and ASCII String Messaging (Bit 9). This allows simple ASCII transmission out of the PLC while still receiving ASCII characters into FIFO. This is useful when working with dumb terminals. When ASCII Receive FIFO (Bit 7) is set none of the following ASCII output controls are allowed: Modbus Master Messaging (Bit 8), Pulse Dial Modem (Bit 3), Hangup Modem (Bit 2), Tone Dial Modem (Bit 1) and Initialize Modem (Bit 0).

Enable Back space

When a backspace (BS) is detected it is NOT stored into the MsgIn array, in fact it deletes the previous character and thus decreases the RecCount Character Counter. In contrast, when a regular ASCII character is detected it is stored in the MsgIn array and the RecCount Character Counter is increased.

NOTE: Back spaces CANNOT delete characters from an empty MsgIn array, thus the RecCount Character Counter never goes below 0.

This special back space functionality along with internal echo enabled at the terminal are very useful for dealing with dumb terminals. A single Terminated ASCII Input XXMIT searching for "cr" is activated with ASCII Receive FIFO (Bit 7) and back space (Bit 6) set. No additional program logic is required while you type and edit characters using the back space on the fly. When you type "cr" XXMIT activates the Done output, and the corrected data is all lined up properly in the MsgIn array.

Enable RTS/CTS Flow Control

The following pertains to the output mode. The XXMIT state goes to BLOCKED receiving when the receiving device indicates it cannot process additional characters by setting CTS to OFF. Likewise, The XXMIT state goes to UNBLOCKED when CTS is ON and the receiving devices indicates it CAN process additional characters.

When transmission is UNBLOCKED and Simple ASCII Output (Bit 9) and RTS/CTS Flow Control (Bit 5) are set then the transmit output data is sent out in 16 byte packets. After all output packets are sent then the Done output on XXMIT goes ON to indicate "Operation Successful".

If during a transmission it suddenly becomes BLOCKED, only the remaining characters in the current output packet are sent, never exceeding 16 characters, and XXMIT remains ACTIVE indefinitely. Only when the CTS in ON will the ASCII output resume sending all remaining output packets.

The following pertains to the input mode. Since RTS is an output signal, it can be used independently of the ASCII output transmit process, to BLOCK or UNBLOCK sending devices. When ASCII Receive FIFO (Bit 7) is set the RTS/CTS Flow Control works in the input mode. When ASCII Receive FIFO (Bit 7) is set and neither of the two ASCII inputs are set, Simple ASCII Input (Bit 10) or Terminated ASCII Input (Bit 11), the received characters will fill the FIFO in which they are inserted. In the mean time, the RTS Flow Control (Bit 5) is ON allowing the sending device to proceed.

When the FIFO (512 characters) is more than three quarters full with characters the RTS Control Flow (Bit 5 is cleared to BLOCK the sending device. The RTS Control Flow (Bit 5) remains cleared until either Simple ASCII Input (Bit 10) or Terminated ASCII Input (Bit 11) have removed enough characters from the FIFO whereby reducing it to less than one quarter full of characters at which point the RTS Control Flow (Bit 5) is turned ON.

⚠ CAUTION

TRANSMISSION LOCK-UP

If the CTS signal is forced indefinitely to OFF by the receiving device (or if there is a cable problem that forces the CTS signal to OFF), the XXMIT block never completes the transmission and the receiving device never receives the command. To prevent this kind of lock-up, it is advised to start a Timer when the XXMIT block is started and warn the application program when the Timer internal time is reached. It is also advised to reset the XXMIT Start bit.

Failure to follow these instructions can result in injury or equipment damage.

NOTE: The RTS/CTS Flow Control algorithm is different from RTS/CTS Modem Control. The former is related to full duplex receive buffer overflow. The latter deals with the transmit process gaining access to a shared transmission medium. Therefore, it is illegal to simultaneously request both of these RTS/CTS algorithms.

NOTE: You CANNOT select any type of RTS/CTS Flow Control (bit 5) handshaking when the port is in RS 485 mode (bit 13) because these signals do NOT exist in RS 485 mode.

Enable Xon/Xoff Flow Control

The following pertains to the output mode. The XXMIT state goes to BLOCKED when an Xoff character is received. Likewise the XXMIT state goes to UNBLOCKED when an Xon character is received. In neither case will Xon or Xoff be inserted into the FIFO.

When transmission is UNBLOCKED and Simple ASCII Output (Bit 9) and Xon/Xoff Flow Control (Bit 4) are set then the transmit output data is sent out in 16 byte packets. After all output packets are sent the Done output on the XXMIT block goes ON.

If during a transmission it suddenly becomes BLOCKED, only the remaining characters in the current output packet are sent, never exceeding 16 characters, and the XXMIT block remains ACTIVE indefinitely. Only when the next Xon character is received will the ASCII output resume sending all remaining output packets.

The following pertains to the input mode. Xon/Xoff may be used to BLOCK or UNBLOCK sending devices. When ASCII Receive FIFO (Bit 7) is set the Xon/Xoff Control Flow (Bit 4) works in the input mode. When ASCII Receive FIFO (Bit 7) is set and neither of the two ASCII inputs are set, Simple ASCII Input (Bit 10) or Terminated ASCII Input (Bit 11), the received characters will fill the FIFO in which they are inserted.

When the FIFO is more than three quarter full with characters and additional characters are received the FIFO state variable is set to send XOFF characters out the serial port after a delay of up to 16 character times BLOCKING the sender and clearing the FIFO state variable.

When all ASCII output functions (Bits 8,3,2,1, and 0) are OFF and the Xon/Xoff Flow Control (Bit 4) is ON the delay time defaults to 1 character time. In contrast, when all ASCII output functions (Bits 8,3,2,1, and 0) are ON and the Xon/Xoff Flow Control (Bit 4) is ON then the ASCII output is broken up into 16 byte packets. Thus, pending Xoff characters DO NOT have to wait more than 16 character times before BLOCKING the sender.

Once the sender has stopped transmission, the PLC eventually removes the characters from the FIFO using either Simple ASCII Input (Bit 10) or Terminated ASCII Input (Bit 11).

When FIFO becomes less than one quarter full with characters the FIFO state variable is set to send XON, thus, sending an Xon character out the serial port to UNBLOCK the sender.

NOTE: To prevent lockup due to a disconnected cable or other intermittent communication errors, when the sender is BLOCKED and did NOT receive the Xon character correctly we use the following algorithm. When FIFO becomes empty and no characters are subsequently received, then a steady stream of Xon characters are transmitted at the rate of once every 5 seconds.

NOTE: The Xon/Xoff Flow Control (Bit 4) is different from the RTS/CTS Control Flow (Bit 5). The former uses transmitted Xon and Xoff characters to prevent receive buffer overflow in full duplex mode. The latter uses hardware hand-shaking signals to accomplish the same goal. Therefore, it is illegal to simultaneously request both of these flow control algorithms because RTS/CTS Flow Control (Bit 5) Modem Control implies a half duplex network while Xon/Xoff Flow Control (Bit 4) implies a full duplex network.

Application Examples

Description

The following program is a short demo application with four instances of XXMIT showing the four main functions:

- Modbus Master
- Simple ASCII In
- ASCII Message Out
- Terminated ASCII In

Modbus Master

The following Modbus Master operation is a read request to a slave (address 4 for example) device connected to port 1 of the master:

- Read slave's %MW1 to %MW10
- into local %MW11 to %MW20

The Slave (address 4) must be set up with the following port parameters:

- 9600 baud
- 8 data bits
- 1 stop bit
- even parity (2)

The Master uses settings from XXMIT

Variable declaration for Modbus Master

The following table shows the variables used in the Modbus Master example:

Variable Name	Data Type	Initial Value	Comment
StartModbusMstr	BOOL		
ModbusMstrActive	BOOL		
ModbusMstrCommand	WORD	16#0100	Bit 8 set
ModbusMstrDone	BOOL		
ModbusMstrError	BOOL		
ModbusMstrSettings	WordArr9		
ModbusMstrSettings[1]		3	Modbus Code: Read multiple words
ModbusMstrSettings[2]		10	Amount of Words to read
ModbusMstrSettings[3]		4	Slave Modbus address
ModbusMstrSettings[4]		1	Source word
ModbusMstrSettings[5]		11	Destination Word
ModbusMstrSettings[6]		0	not used

Variable Name	Data Type	Initial Value	Comment
ModbusMstrSettings[7]			not used
ModbusMstrSettings[8]			not used
ModbusMstrSettings[9]			not used
ModbusMstrStatus	INT		
ModbusMstrNode	WORD	4	Enter Slave address
ModbusMstrErrorCounter	INT		
ModbusMstrDoneCounter	INT		

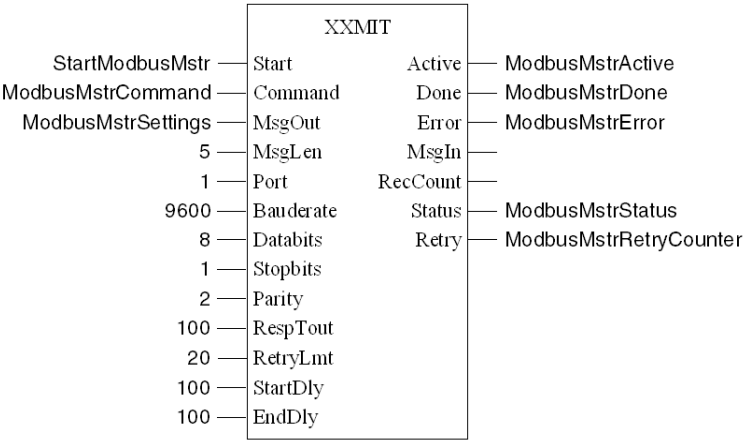
IEC Section for Modbus Master

Program the following in an FBD section:

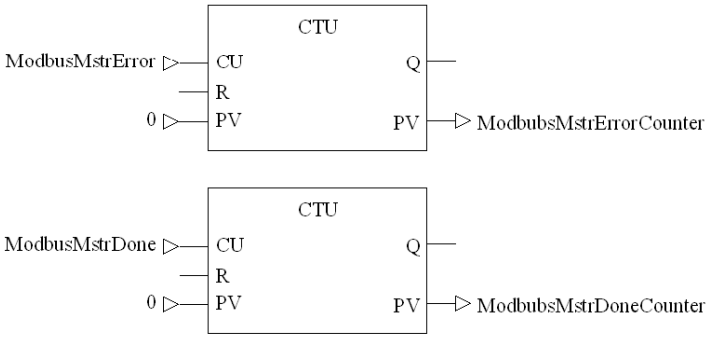
Slave node address assignment



Assignments to the XXMIT function block:



Count errors and successes



Simple ASCII Receive

Receives whatever comes into port 1. The receive buffer's length is assigned as 'SimpleReceive-Length', which has an initial value of 10.

Received characters are in MsgIn array, number of received characters in RecCount.

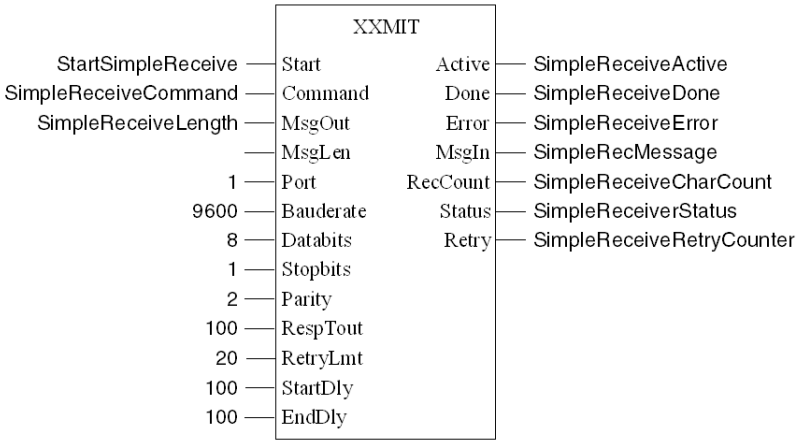
Variable declaration for Simple ASCII Receive

The following table shows the variables used in the Simple ASCII Receive example:

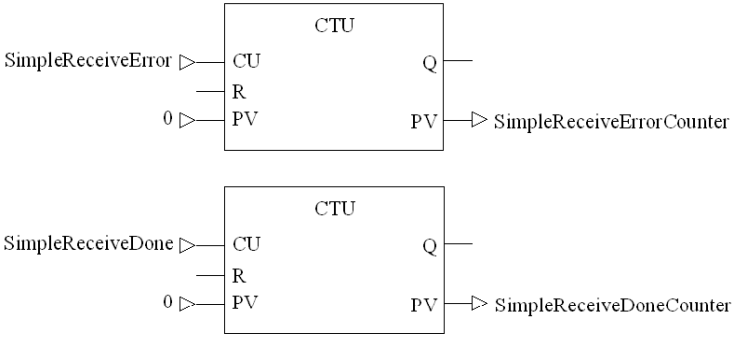
Variable Name	Data Type	Initial Value	Comment
StartSimpleReceive	BOOL		
SimpleReceiveActive	BOOL		
SimpleReceiveCharCounter	INT		
SimpleReceiveCommand	WORD	16#0480	Bits 7 and 10 set. FIFO enabled
SimpleReceiveDone	BOOL		
SimpleReceiveError	BOOL		
SimpleReceiveLength	INT	10	
SimpleReceiveRetryCounter	INT		
SimpleReceiveStatus	INT		
SimpleRecMessage	ByteArr12		
SimpleReceiveDoneCounter	INT		
SimpleReceiveErrorCounter	INT		

IEC Section for Simple ASCII Receive

Program the following in an FBD section:



Count errors and successes



Simple ASCII Send

Sends a simple ASCII message out off port 1, the message is 'Hello World!!'

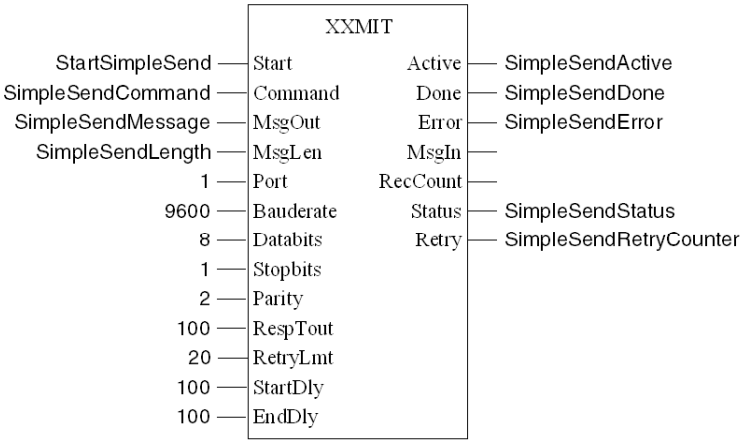
Variable declaration for Simple ASCII Send

The following table shows the variables used in the Simple ASCII Send example:

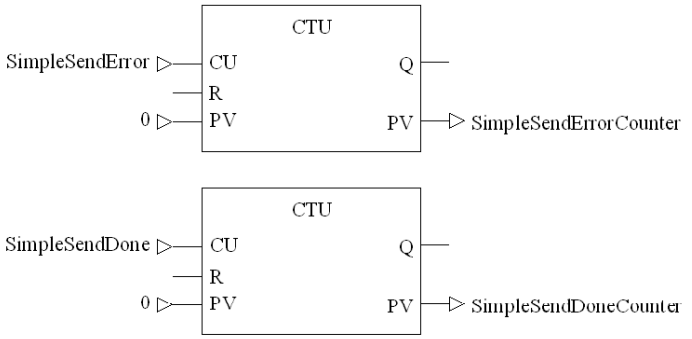
Variable Name	Data Type	Initial Value	Comment
StartSimpleSend	BOOL		
SimpleSendActive	BOOL		
SimpleSendCommand	WORD	16#0200	Bit 9 set
SimpleSendDone	BOOL		
SimpleSendError	BOOL		
SimpleSendLength	INT	14	Number of characters to send
SimpleSendMessage SimpleSendMessage[1] SimpleSendMessage[2] SimpleSendMessage[3] SimpleSendMessage[4] SimpleSendMessage[5] SimpleSendMessage[6] SimpleSendMessage[7] SimpleSendMessage[8] SimpleSendMessage[9] SimpleSendMessage[10] SimpleSendMessage[11] SimpleSendMessage[12] SimpleSendMessage[13] SimpleSendMessage[14]	ByteArr36	 16#48 16#65 16#6C 16#6C 16#6F 16#20 16#57 16#6F 16#72 16#6C 16#64 16#20 16#21 16#21	'Hello World !!'
SimpleSendRetryCounter	INT		
SimpleSendStatus	INT		
SimpleSendDoneCounter	INT		
SimpleSendErrorCounter	INT		

IEC Section for Simple ASCII Send

Program the following in an FBD section:



Count errors and successes



Terminated ASCII Receive

After receiving the 'starting characters' "AB", the function block puts all received characters into the receive buffer MsgIn. The receiver will stop when the 'finishing characters' "CD" are received, whereby the "Done" output will be set, to indicate the successfull completion. The max. length of the receive buffer is assigned as "TermReceiveLength", which is set to an initial value of 20 in this example.

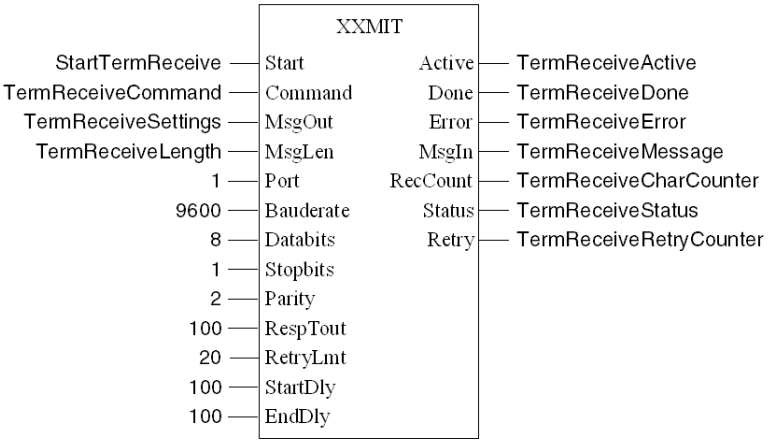
Variable declaration for Terminated ASCII Receive

The following table shows the variables used in the Terminated ASCII Receive example:

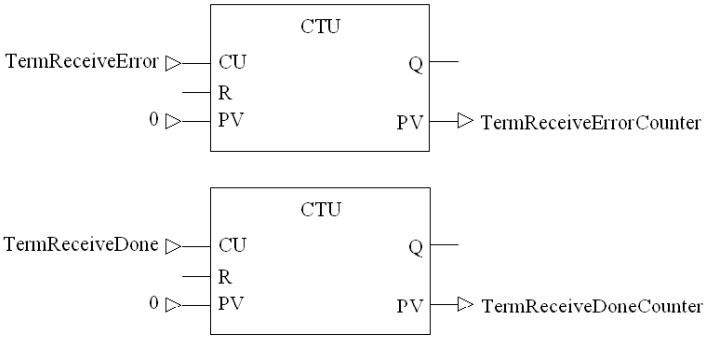
Variable Name	Data Type	Initial Value	Comment
StartTermReceive	BOOL		
TermReceiveActive	BOOL		
TermReceiveCharCounter	INT		
TermReceiveCommand	WORD	16#0880	Bits 11 and 7 set. FIFO enabled
TermReceiveDone	BOOL		
TermReceiveError	BOOL		
TermReceiveLength	INT	20	
TermReceiveMessage	ByteArr36		Received characters
TermReceiveRetryCounter	INT		
TermReceiveSettings TermReceiveSettings[1] TermReceiveSettings[2] TermReceiveSettings[3] TermReceiveSettings[4] TermReceiveSettings[5] TermReceiveSettings[6]	ByteArr36	 16#02 16#02 16#41 16#42 16#43 16#44	 length of termination string (1 or 2) length of start string (0, 1 or 2) 1st start character 2nd start character 1st termination character 2nd termination character
TermReceiveStatus	INT		
TermReceiveDoneCounter	INT		
TermReceiveErrorCounter	INT		

IEC Section for Terminated ASCII Receive

Program the following in an FBD section:



Count errors and successes



Entering Strings as initial values

The Data Editor of Unity Pro allows you to easily enter Strings as initial values into byte arrays.

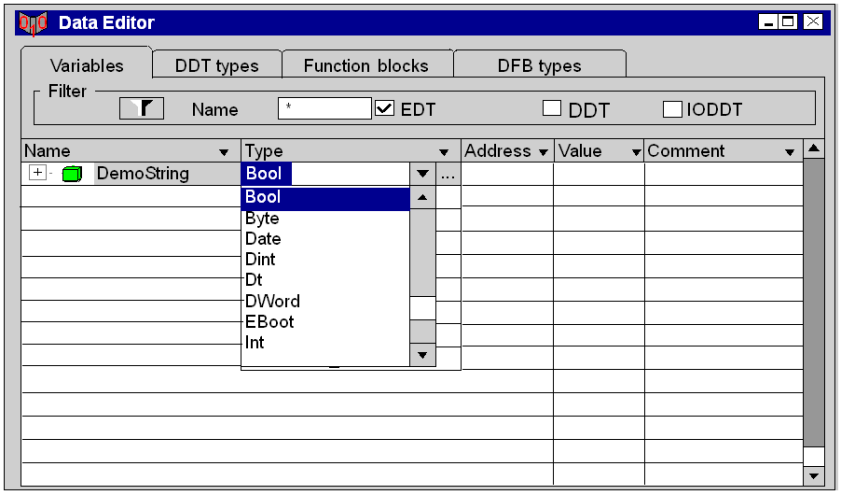
The following part gives a short describtion of how to define a variable 'DemoString' as 'ByteArr36' and enter a string 'My Text ! ' as initial value.

Open the Data Editor

From the main menu select:

```
Tools -> Data Editor.
```

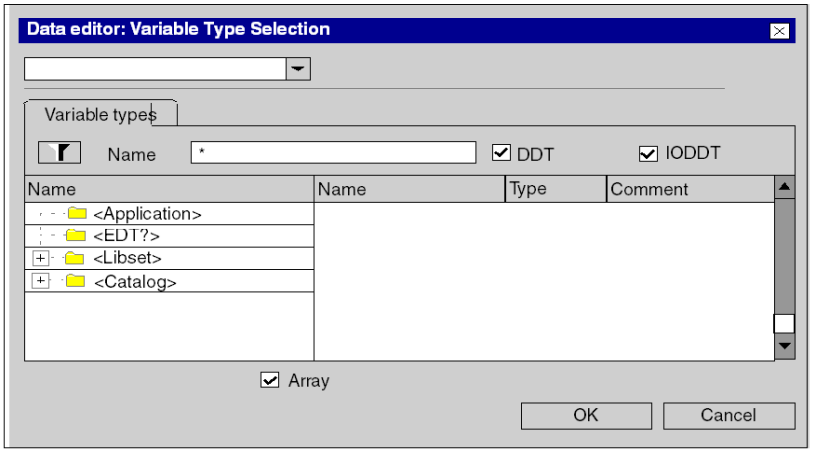
Data Editor



Declaring new String name

Declare stringname for Array by clicking on  next to type (in this case bool) This will then open a variable type selection window.

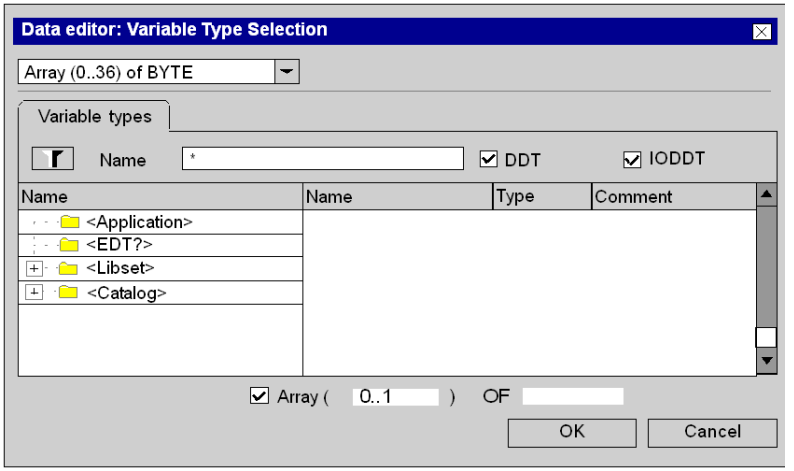
Define Element Type 1



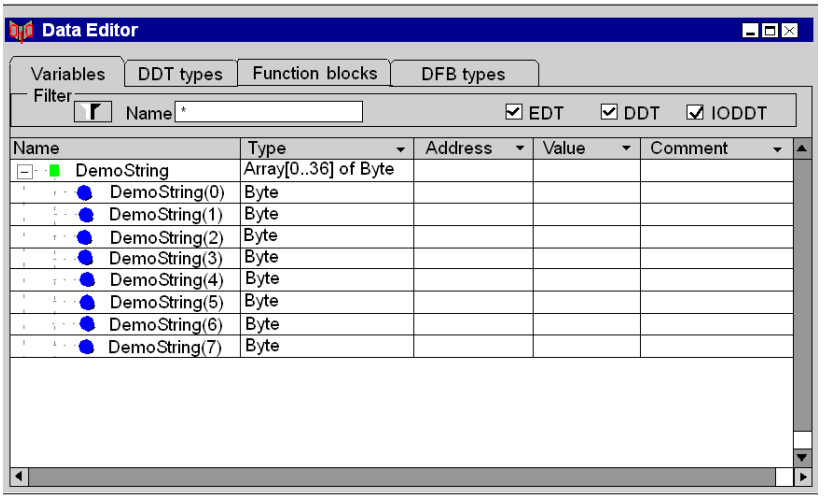
Define Element Type

Check the Array checkbox and define array element type and number of elements. It is now possible to see and modify values for expanded byteArr36.

Define Element Type 2



Definition ByteArr36

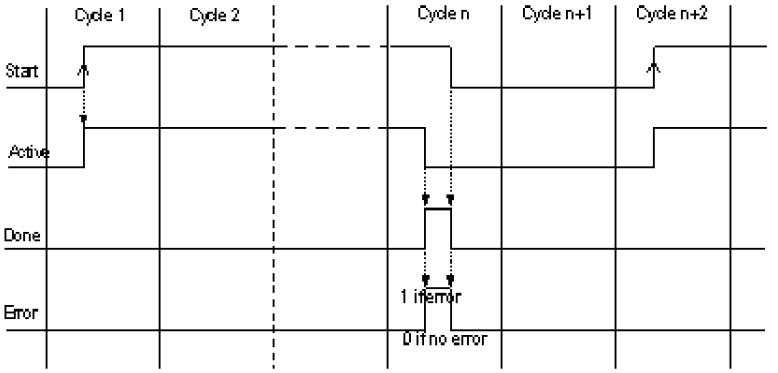


Section 43.3
XXMIT: Programming Rules

XXMIT Programming Rules

Programming Rules Description

It may take several PLC cycles to send a Modbus request or character string. The Start, Active, Done and Error bits operate in the following way:



The start signal is written by the application. Active, Done and Error signals are read by the application.

⚠ CAUTION

MULTIPLE TRANSMISSION

If the Start signal is maintained to 1 after the completion, the XXMIT block is restarted. This will generate multiple transmission of the same Modbus or ASCII message to the receiving device. To prevent this multiple transmission, it is advised to reset the Start bit as soon as the Done bit is seen at 1.

Failure to follow these instructions can result in injury or equipment damage.

If the start signal is maintained to 1 after the completion, XXMIT is restarted

Several XXMITs can not be activate at the same time. If more than one XXMIT is activated, the first scanned block will operate and prevent operation of all other XXMITs until it has completed. The next block in the scan that is called will generate an immediate fault code 150.

The XXMIT function block can only be used in MAST task. There is an immediate fault code (127) if it is started in the FAST / AUX or EVENT tasks (there is no control performed by Unity Pro at build time).

NOTE: The XXMIT input parameter must be initialized before setting the START input. They must not be changed during the function block activity. If the START bit is reset to 0 before completion, the function block is stopped (active bit to 0). To have a complete block execution, the value of 1 must be applied on the START bit until the operation has finished or until an error has occured.

Section 43.4
Technical References for XXMIT

Overview

This section describes the Technical References for XXMIT.

What Is in This Section?

This section contains the following topics:

Topic	Page
Modbus Query/Response Parameter Limits	467
XXMIT Configuration using Hayes Compatible Dial-Up Modems (Only)	468
Hayes Application Example	472

Modbus Query/Response Parameter Limits

Parameter Limits Based on Function Code

The query/response parameters are limited based upon the type of Function Code you are using. Refer to the applicable table below.

Quantum PLC Maximum Parameters table:

Function Code	Description	Query	Response
1	Read Multiple Bits	2000 bits	2000 bits
2	Read Multiple Bits	2000 bits	2000 bits
3	Read Multiple Words	125 words	125 words
4	Read Multiple Words	125 words	125 words
5	Write Single Bit	1 bit	1 bit
6	Write Single Word	1 word	1 word
15	Write Multiple Bits	800 bits	800 bits
16	Write Multiple Word	100 words	100 words

XXMIT Configuration using Hayes Compatible Dial-Up Modems (Only)

Description

There are three commands that you need to become familiar with when interfacing dial-up modems to XXMIT.

These commands are:

- Initialize modem
- Dial modem
- Hangup modem.

Before an ASCII message or a Modbus message goes through the modem, you must first send an initialization string and then a dial string to the modem. Once the modem has dialed the telephone number and made a connection to the remote modem, you may send an unlimited number of ASCII messages or Modbus messages through the modem. To send multiple messages, you increment the message pointer to the next message after each successful XXMIT operation. When all messages are sent, you may then send the hangup string to the modem.

Initialization Message

The initialization message is just like any other ASCII message and may be a maximum of 512 characters long, although 50 characters is usually more than enough to initialize a modem. You may implement any Hayes AT command as part of the initialization string. We recommend the following commands when initializing a modem for use with XXMIT.

Initialization Message for Dial-Up Modem

Initialization Message =	AT&F&K0&Q0&D0V1Q0X0E1
AT=	Self-calibrate Modem [1]
&F=	Recall factory configuration as active configuration [1]
&K0=	Disable local flow control [2]
&Q0=	Communicate in asynchronous mode [2]
&D0=	Ignore status of DTR signal [1]
V1=	Display result codes as words [1] If V1 is not used or if modem is not capable of returning verbose responses XXMIT returns error 117 (modem replay time out).
Q0=	Return result codes [1]
X0=	Provide basic call progress result codes: Connect, No Carrier, and Ring [1]
E1=	Echo characters from the keyboard to the screen in command state [1]
1 These parameters must always be part of the initialization string for XXMIT to function properly. **2** These parameters should be part of the initialization string for XXMIT to transmit a message to remote modem properly. Only a experienced modem user should change or not use these parameters.	

NOTE: While some modem manufacturers state full compatibility with Hayes, they may still be slightly different. Therefore, we recommend using only those commands that have the same definition as those stated above.

The initialization message must always start with Hayes standard AT command. XXMIT automatically precedes modem command messages with AT and appends the message with carriage return (0x0D) and line feed (0x0A) characters since these are required by all modem control messages. Other (non controlling) ASCII messages do not have to end with a carriage return and line feed.

For example, a typical initialization message that XXMIT sends to the modem.

Message	Length
(AT)&F&K0&Q0&D0V1X0Q0 (<CR><LF>) [1]	17 characters
1 Characters within parentheses are automatically sent.	

For example, the initialization message may also be used to set S-registers of the modem.

Message	Length
(AT)S0=1 (<CR><LF>) [1]	4 characters
1 Characters within parentheses are automatically sent.	

To have XXMIT send an initialization message to the modem, bit 9 and bit 0 of the command word must be ON. When bit 0 is ON, bits 1 and 2 must not be ON or XXMIT will not complete the operation successfully. To actually send the message, Start input of XXMIT must come ON and stays ON until the operation is complete or an error occurs. When XXMIT determines the message was successfully sent to the modem, it turns ON the Done output. When an error occurs, the Error output comes ON. The Active output is ON while the message is being sent to the modem.

NOTE: To eliminate some user logic programming, you may initialize the modem with parameters via a terminal program and not use XXMIT. Once the parameters are in the modem memory they may be saved to non-volatile memory with an AT command, usually &W.

Dial Message

The dial message is used to send a telephone number to the modem. Only AT commands related to dialing a number should be included with the message. Examples of typical dial messages used with XXMIT are shown below.

For example, dial telephone number using tone dialing.

Message	Length
(AT)DT)6800326 (<CR><LF>)[1]	7 characters
1 Characters within parentheses are automatically sent.	

For example, dial telephone number using pulse dialing.

Message	Length
(AT)DP)6800326 (<CR><LF>)[1]	7 characters
1 Characters within parentheses are automatically sent.	

For example, dial telephone number using tone dialing, wait to hear dial tone before dialing number, and pause before dialing the rest of the number.

Message	Length
(AT)DT)W,6800326 (<CR><LF>)[1]	9 characters
1 Characters within parentheses are automatically sent.	

To have XXMIT send a tone dial message to the modem, bit 9 and bit 1 of the command word must be ON. When bit 1 is ON, bits 0 and 2 must not be ON or XXMIT will not complete the operation successfully. To actually send the message, the Start input of XXMIT must come ON and stays ON until the operation is complete or an error occurs. When XXMIT determines the message was successfully sent to the modem, it turns ON the Done output. When an error occurs, the Error output comes ON. The Active output is ON while the message is being sent to the modem.

NOTE: Because it takes so long for a local modem to make a connection to a remote modem, the timeout value, in RespTout should be quite long when sending a dial message to a modem. For example, set the timeout for 30,000 mS when sending a dial message. When the timeout value is too short, XXMIT issues a message timeout. You may have to try several settings before finding the optimal time.

Hangup Message

The hangup message is used to hangup the modem. Only AT commands related to hanging up the modem should be used in this message. An example of a typical hangup message is shown below.

For example, hangup modem message.

Message	Length
(+++AT)H0 (<CR><LF>)[1]	2 characters
1 Characters within parentheses are automatically sent.	

When the hangup message is sent to a modem that is already connected to a remote modem, XXMIT must first set the local modem in command mode. XXMIT does this by sending a escape sequence +++ to the modem. XXMIT assumes that +++ sets the modem in command mode. Some modem manufactures let the owner change this default escape sequence. For XXMIT to function properly the modem should be set to accept the +++ escape sequence.

To have XXMIT send a hangup message to the modem, bit 9 and bit 2 of the command word must be ON. When bit 2 is ON, bits 0 and 1 must not be ON or XXMIT will not complete the operation successfully. To actually send the message, the Start input of XXMIT must come ON and stays ON until the operation is complete or an error occurs. When XXMIT determines the message was successfully sent to the modem, it turns ON the Done output. When an error occurs, the Error output comes ON. The Done output is ON while the message is being sent to the modem.

NOTE: Expert: Because it takes so long for a local modem to hangup once it receives the hangup command, the timeout value, in RespTout should be quite long when sending a dial message to a modem. For example, set the timeout for 30,000 mS when sending a dial message. When the timeout value is too short, XXMIT issues a message timeout. You may have to try several settings before finding the optimal time.

Hayes Application Example

Description

The following program is a short demo application to send the "Hello World" message through a Hayes compatible Modem (Com One, Deskline 56K).

To start the application, the "stage" variable has to be set to 1 in a Unity Pro animation table.

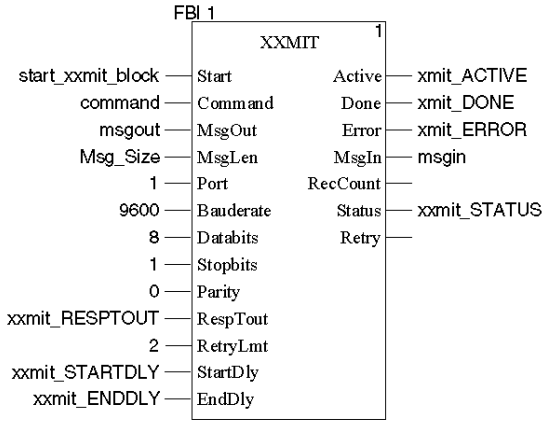
Variable Data Types

start_xxmit_block	BOOL
command	WORD
msgout	ARRAY[0..40] OF BYTE
Msg_Size	INT
xmit_DONE	BOOL
xmit_ERROR	BOOL
xmit_ACTIVE	BOOL
xxmit_RESPTOUT	INT
xxmit_STARTDLY	INT
xxmit_ENDDLY	INT

xmit_STATUS_mem	INT
counter	INT
stage	WORD

IEC section for XXMIT

Program the following in an FBD section:



IEC Section for Modem commands

Program the following in an ST section:

```
(* %S6 is used to generate a 2 second delay *) %m6 := %s6;
```

```
if xxmit_STATUS_Mem = 0 then
if not(xxmit_STATUS = 0) then
xxmit _STATUS_Mem := xxmit_STATUS;
end_if; end_if;
```

```
(* If XXMIT error, the function block is stopped *) if xmit_ERROR=1 then
stage := 0;
counter := 0;
start_xxmit_block:=0; end_if;
```

```
if stage = 1 then
(* Initialization message*)
xxmit_STATUS_Mem := 0;
counter := 0;
xxmit_ENDDLY := 700;
xxmit_STARTDLY := 600;
xxmit_RESPTOUT := 700;
(* Command word init *)
command:=2#0000001000000001;
msgout[0] := 16#26;(*&->26*)
msgout[1] := 16#46;(*F->46*)
msgout[2] := 16#26;(*&->26*)
msgout[3] := 16#4B;(*K->46*)
```

```
msgout[4] := 16#30;(*0->30*)
msgout[5] := 16#26;(*&->26*)
msgout[6] := 16#44;(*D->44*)
msgout[7] := 16#30;(*0->30*)
 msgout[8] := 16#56;(*V->56*)
msgout[9] := 16#31;(*1->31*)
msgout[10] := 16#51;(*Q->51*)
msgout[11] := 16#30;(*0->30*)
msgout[12] := 16#58;(*X->58*)
msgout[13] := 16#30;(*0->30*)
msgout[14] := 16#45;(*E->45*)
msgout[15] := 16#31;(*1->31*)
Msg_Size := 16;
start_xxmit_block:=1;
stage := 2; end_if;
```

```
if stage = 3 then
(* Dial message *)
xxmit_STARTDLY := 100;
xxmit_ENDDLY := 100;
xxmit_RESPTOUT := 32000;
(* Command word init *)
command:=2#0000001000000010;
(* Extension number *)
msgout[0] := 16#32;(*2*)
msgout[1] := 16#35;(*5*)
msgout[2] := 16#37;(*7*)
msgout[3] := 16#34;(*4*)
start_xxmit_block:=1;
Msg_Size := 4;
stage := 4; end_if;
```

```
if (stage = 5)then
if RE(%m6) then
counter := counter + 1;
end_if;
```

```
(* Two seconds delay *)
if stage = 5 and counter = 2 then
counter := 0;
stage := 7;
end_if; end_if;
```

```
if stage = 7 then
(* ASCII message to be send *)
 xxmit_STARTDLY := 300;
xxmit_ENDDLY := 400;
xxmit_RESPTOUT := 32000;
(* Command word init *)
command:2=#0100001000000000;
msgout[0] := 16#48; (*H*)
msgout[1] := 16#65; (*e*)
msgout[2] := 16#6C; (*l*)
msgout[3] := 16#6C; (*l*)
msgout[4] := 16#6F; (*o*)
msgout[5] := 16#20; (* *)
msgout[6] := 16#57; (*W*)
msgout[7] := 16#6F; (*o*)
msgout[8] := 16#72; (*r*)
msgout[9] := 16#6C; (*l*)
msgout[10] := 16#64; (*d*)
msgout[11] := 16#20; (* *)
msgout[12] := 16#21; (*!*)
msgout[13] := 16#21; (*!*)
Msg_Size := 14;
start_xxmit_block:=1;
stage := 8; end_if;
```

```
if stage = 100 then
(* Hangup message *)
command:=2#0000001000000100;
xxmit_STARTDLY := 300;
xxmit_ENDDLY := 400;
xxmit_RESPTOUT := 500;
msgout[0] := 16#48;(* *)
msgout[1] := 16#30;(*CR*)
Msg_Size := 2;
start_xxmit_block:=1; end_if;
```

```
(* Change of state after each XXMIT operation *) if xmit_DONE = 1 then
start_xxmit_block:=0;
if stage = 2 then
stage := 3;
end_if;
```

```
if stage = 4 then
ounter := 0;
stage := 5;
end_if;
```

```
if stage = 8 then
stage := 99;
end_if;
```

```
if stage = 99 then
stage :=100;
end_if; end_if;
```

Section 43.5
Cabling Information

Overview

This section describes cables and pinouts for the hardware components used with XXMIT.

What Is in This Section?

This section contains the following topics:

Topic	Page
Cable Pinouts	478
Cable Adapter Kits	489

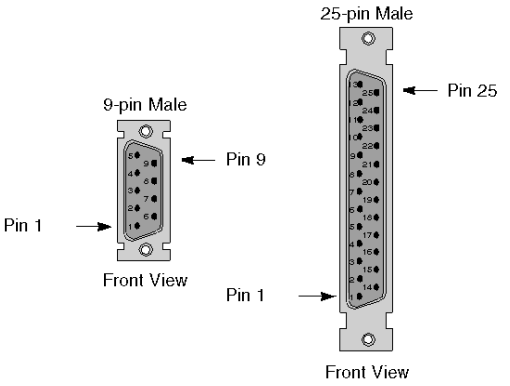
Cable Pinouts

Interface Cable Pinouts

You need to build an interface cable between your PLC and the modem or printer. The actual cable is connected to the Port which is supported by the PLC and to the RS232 port of the modem or printer, or direct to another PLC's Modbus port. Because the XXMIT supports many modems and printers the pinouts are going to vary. Some pinouts are provided below.

9-pin (RS-232) to 25-pin (Modem) with no RTS/CTS Control

Refer to the figure for Front Views of Connectors.

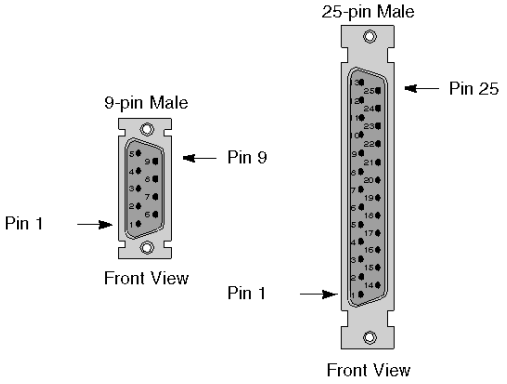


Refer to the Connector Pinouts table.

Connector Pinouts				
9-Pin Connector			25-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	2	Yes	3	RXD
TXD	3	Yes	2	TXD
RTS	7 Jumpered 8 Jumpered		4 Jumpered 5 Jumpered	RTS
CTS				CTS
DSR	4 Jumpered 6 Jumpered		6 Jumpered 20 Jumpered	DSR
DTR				DTR
GND	5	Yes	7	GND

9-pin (RS-232) to 25-pin (Modem) with RTS/CTS Control

Refer to the figure for Front Views of Connectors.

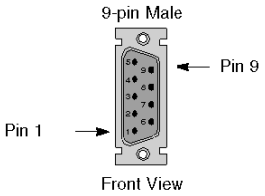


Refer to the Connector Pinouts table.

Connector Pinouts				
9-Pin Connector			25-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	2	Yes	3	RXD
TXD	3	Yes	2	TXD
RTS	7	Yes	4	RTS
CTS	8	Yes	5	CTS
DSR	4 Jumpered 6 Jumpered		6 Jumpered 20 Jumpered	DSR
DTR				DTR
GND	5	Yes	7	GND

9-pin to 9-pin (Null Modem)

Refer to the figure for Front Views of Connectors.

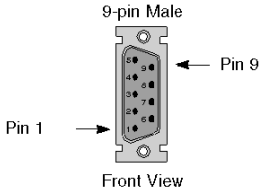


Refer to the Connector Pinouts table.

Connector Pinouts				
9-Pin Connector			9-Pin Connector	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	2	Yes	3	TXD
TXD	3	Yes	2	RXD
RTS	7 Jumpered 8 Jumpered		7 Jumpered 8 Jumpered	RTS
CTS				CTS
DSR	4 Jumpered 6 Jumpered		4 Jumpered 6 Jumpered	DSR
DTR				DTR
GND	5	Yes	5	GND

9-pin to 9-pin (Modem)

Refer to the figure for Front Views of Connectors.

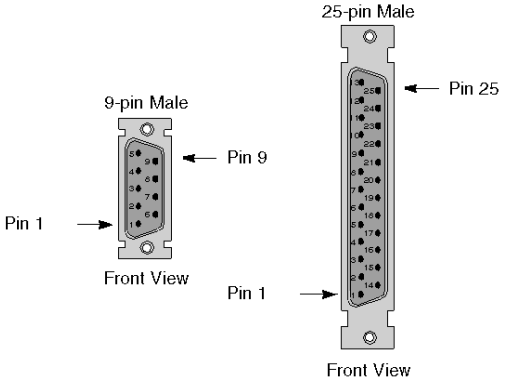


Refer to the Connector Pinouts table.

Connector Pinouts				
9-Pin Connector			9-Pin Connector	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
TXD	2	Yes	2	TXD
RXD	3	Yes	3	RXD
RTS	7	Yes	7	RTS
CTS	8	Yes	8	CTS
DSR	4 Jumpered 6 Jumpered		4 Jumpered 6 Jumpered	DSR
DTR				DTR
GND	5	Yes	5	GND

9-pin to 25-pin (Null Modem)

Refer to the figure for Front Views of Connectors.

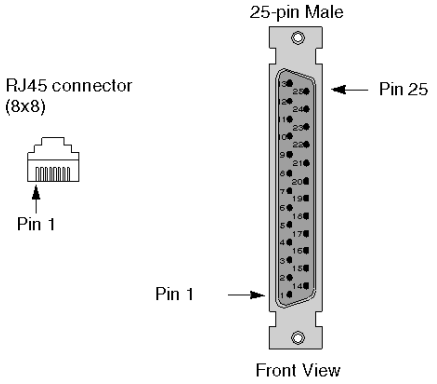


Refer to the Connector Pinouts table.

Connector Pinouts				
9-Pin Connector			25-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Siignal Name
RXD	2	Yes	2	TXD
TXD	3	Yes	3	RXD
RTS	7 Jumpered 8 Jumpered		4 Jumpered 5 Jumpered	RTS
CTS				CTS
DSR	4 Jumpered 6 Jumpered		6 Jumpered 20 Jumpered	DSR
DTR				DTR
GND	5	Yes	7	GND

RJ45-(8x8) to 25-pin (Null Modem) 110XCA20401

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

Connector Pinouts				
RJ45 Connector			25-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	2	TXD
TXD	3	Yes	3	RXD
RTS	6 Jumpered 7 Jumpered		4 Jumpered 5 Jumpered	RTS
CTS				CTS
GND	5	Yes	7	GND
DSR	2	Yes	6 20	DSR
				DTR
Chassis Ground	8	Yes	1	Chassis Ground

⚠ CAUTION

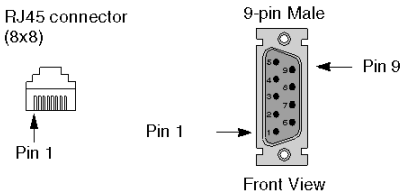
RISK OF 5V SHORT CIRCUIT

Pin1 of the RJ45 receives 5V from the PLC.

Failure to follow these instructions can result in injury or equipment damage.

RJ45-(8x8) to 9-pin (Null Modem) 110XCA20301

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

Connector Pinouts				
RJ45 Connector			9-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	3	TXD
TXD	3	Yes	2	RXD
RTS	6 Jumpered 7 Jumpered		7 Jumpered 8 Jumpered	RTS
CTS				CTS
GND	5	Yes	5	GND
DSR	2	Yes	4 6	DTR
				DSR
Chassis Ground	8	Yes		Case of the Connector

⚠ CAUTION

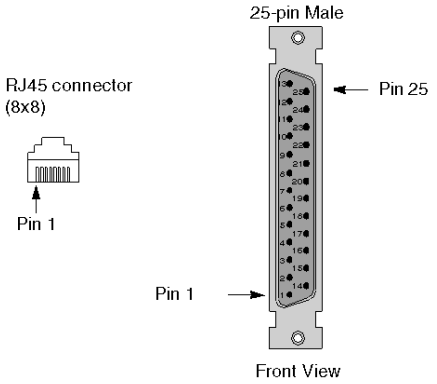
RISK OF 5V SHORT CIRCUIT

Pin1 of the RJ45 receives 5V from the PLC.

Failure to follow these instructions can result in injury or equipment damage.

RJ45-(8x8) to 25-pin (Modem) 110XCA20401

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

Connector Pinouts				
RJ45 Connector			25-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	3	RXD
TXD	3	Yes	2	TXD
RTS	6 Jumpered 7 Jumpered		4 Jumpered 5 Jumpered	RTS
CTS				CTS
GND	5	Yes	7	GND
DSR	2	Yes	6 20	DSR
				DTR
Chassis Ground	8	Yes	1	Chassis Ground

⚠ CAUTION

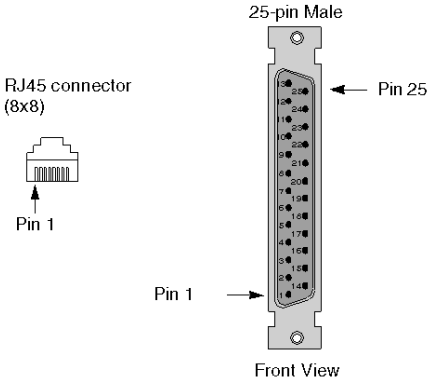
RISK OF 5V SHORT CIRCUIT

Pin1 of the RJ45 receives 5V from the PLC.

Failure to follow these instructions can result in injury or equipment damage.

RJ45-(8x8) to 25-pin (Modem) 110XCA20401

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

Connector Pinouts				
RJ45 Connector			25-Pin D-shell	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	3	RXD
TXD	3	Yes	2	TXD
RTS	6	Yes	4	RTS
CTS	7	Yes	5	CTS
GND	5	Yes	7	GND
			6 Jumpered 20 Jumpered	DSR
				DTR
Chassis Ground	8	Yes	1	Chassis Ground

⚠ CAUTION

RISK OF 5V SHORT CIRCUIT

Pin1 of the RJ45 receives 5V from the PLC.

Failure to follow these instructions can result in injury or equipment damage.

RJ45-(8x8) to RJ45-(8x8) (Modem)

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

Connector Pinouts				
RJ45 Connector			RJ45 Connector	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	4	RXD
TXD	3	Yes	3	TXD
RTS	6	Yes	6	RTS
CTS	7	Yes	7	CTS
GND	5	Yes	5	GND
DSR	2	Yes	2	DSR
Chassis Ground	8	Yes	8	Chassis Ground

⚠ CAUTION

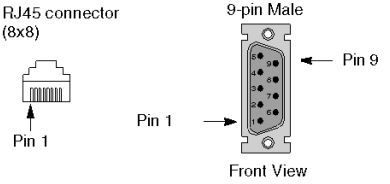
RISK OF 5V SHORT CIRCUIT

Pin1 of the RJ45 receives 5V from the PLC.

Failure to follow these instructions can result in injury or equipment damage.

9-pin to RJ45-(8x8) (Modem) 110XCA20301

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

Connector Pinouts				
RJ45 Connector			9-Pin Connector	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	2	RXD
TXD	3	Yes	3	TXD
RTS	6 Jumpered 7 Jumpered		7 Jumpered 8 Jumpered	RTS
CTS				CTS
GND	5	Yes	5	GND
DSR	2	Yes	6 4	DSR
				DTR
Chassis Ground	8	Yes	Case of the Connector	

⚠ CAUTION

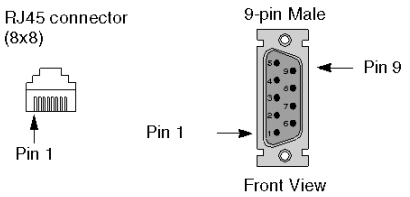
RISK OF 5V SHORT CIRCUIT

Pin1 of the RJ45 receives 5V from the PLC.

Failure to follow these instructions can result in injury or equipment damage.

9-pin to RJ45-(8x8) (Modem) 110XCA20301

Refer to the figure for Front Views of Connectors.



Refer to the Connector Pinouts table.

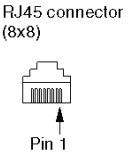
Connector Pinouts				
RJ45 Connector			9-Pin Connector	
Signal Name	Pinout	Pinout Connected To ...	Pinout	Signal Name
RXD	4	Yes	2	RXD
TXD	3	Yes	3	TXD

RTS	6	Yes	7	RTS
CTS	7	Yes	8	CTS
GND	5	Yes	5	GND
			6 Jumpered 4 Jumpered	DSR
				DTR
Chassis Ground	8	Yes	Case of the Connector	

⚠ CAUTION
RISK OF 5V SHORT CIRCUIT Pin1 of the RJ45 receives 5V from the PLC. **Failure to follow these instructions can result in injury or equipment damage.**

RJ 45 High end CPU RS 422/RS 485

Refer to the figure for front view of the CPU Connector.



Pinouts table for the RJ 45 High end Quantum RS 422/RS 485 connector.

Pin	RS-422 signal	RS-485 signal
1	RX -	D -
2	RX +	D +
3	TX +	
4	Nc	Nc
5	GND	GND
6	TX -	
7	Nc	Nc
8	Optional PE	Optional PE

NOTE: For RS-485 protocol pins 1 and 6 must be short circuited as well as pins 2 and 3.

Cable Adapter Kits

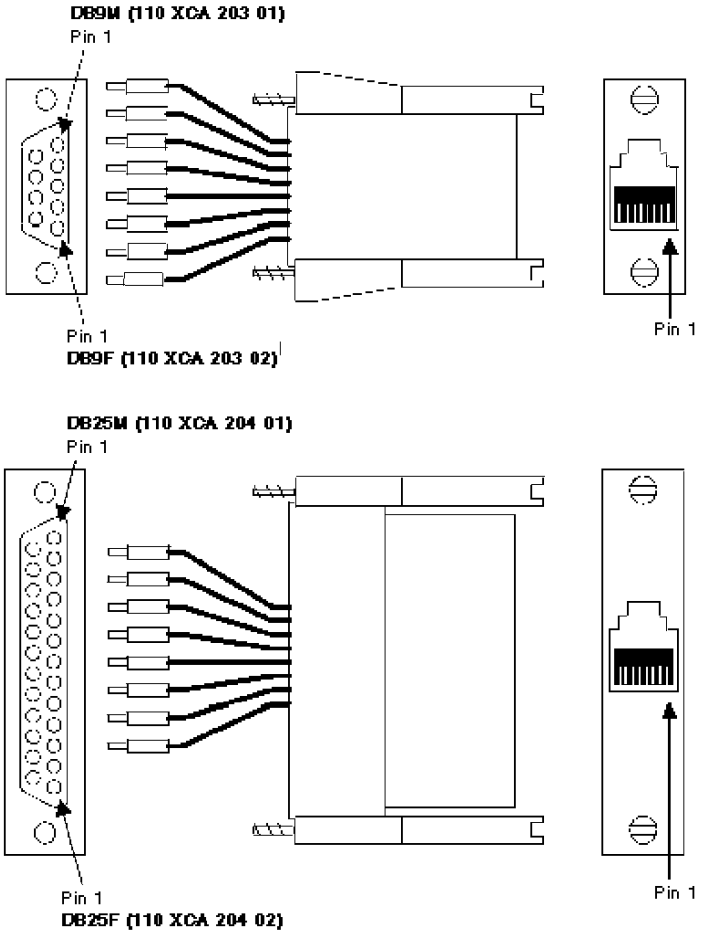
Cable Adapter Kits for RJ45

You may want to purchase Cable Adapter Kits for your RJ45 (8x8) requirements rather than make them. The table below provides a list of available kits.

Available Cable Adapter Kids

Description	Part Number
RJ45-(8x8) to 25-Pin (Male)	110XCA20401
RJ45-(8x8) to 9-Pin (Male)	110XCA20301
RJ45-(8x8) to 9-Pin (Female)	110XCA20302
RJ45-(8x8) to 25-Pin (Female)	110XCA20402

DB to RJ45 Adapter Kits



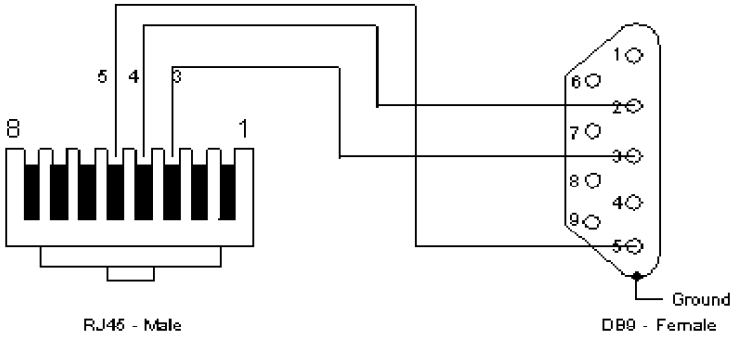
Cable Connection Example 1

For standard terminal operating at low speeds or using software flow control, a simple 3-pins connection can be used.

RJ45 to DB9

PLC Quantum		Modem Connector	
RJ 45		DB9	
RJ45-3	TXD	RXD	DB25-2

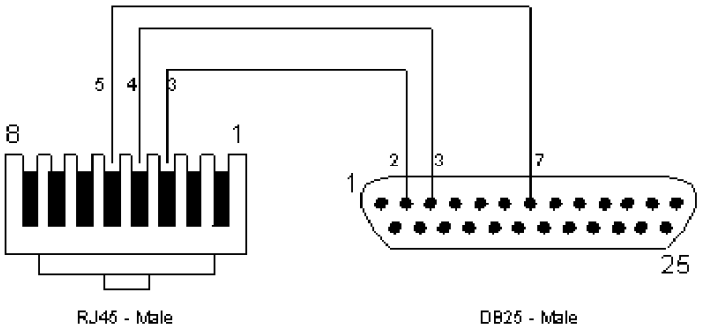
PLC Quantum		Modem Connector	
RJ45-4	RXD	TXD	DB-25-3
RJ45-5	GND	GND	DB25-5



Connection Example 2

RJ45 to DB25

PLC Quantum		Modem Connector	
RJ 45		DB25	
RJ45-3	TXD	RXD	DB25-3
RJ45-4	RXD	TXD	DB-25-2
RJ45-5	GND	GND	DB25-7



Appendices



Appendix A
EFB Error Codes and Values

Introduction

The following tables show the error codes and error values created for the EFBs of the Library.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Tables of Error Codes for the Communication Library	496
Common Floating Point Errors	497
Error Codes of EFBs with STATUS parameter	498
Details of STATUS error code from 31ss to 37ss	501
Details of EFB TCP/IP Ethernet Error Codes 5mss	508
Details of EFB Modbus Plus Error Codes 6mss	512
Quantum EFB SY/MAX Specific Error Codes	513
EtherNet/IP Detected Error Codes	515
CANopen SDO Abort Code	518

Tables of Error Codes for the Communication Library

Introduction

When an error occurs, the FFBs of the Communication Library generate and error code in the STATUS parameter *(see page 498)* (for EFBs with a `STATUS` output parameter), an error information in the diag buffer (if the system diagnostic is enabled) and an error code in %SW125 *(see Unity Pro, System Bits and Words, Reference Manual)*. The error codes list is given in the following table.

Extended

Table of error codes and errors values created for EFBs of the `Extended` family.

Error code	ENO state in error	Error value in Dec	Error value in Hex	Error description
E_EFB_MSTR_ERROR	F	-30191	16#8A11	MSTR communication error
E_EFB_NOT_STATE_RAM_4X	F	-30531	16#88BD	Variable not mapped to % MW (4x) area
E_EFB_MSTR_ERROR	F	-30191	16#8A11	MSTR communication error
E_EFB_OUT_OF_RANGE	F	-30192	16#8A10	Internal error: EFB has detected a violation e.g. write exceeds %MW (4x) boundaries
E_EFB_NOT_STATE_RAM_4X	F	-30531	16#88BD	Variable not mapped to %MW (4x) area
W_WARN_OUT_OF_RANGE	F	30110	16#759E	Parameter out of range
E_EFB_NOT_STATE_RAM_4X	F	-30531	16#88BD	Variable not mapped to %MW (4x) area
E_EFB_MSTR_ERROR	F	-30191	16#8A11	MSTR communication error
W_WARN_OUT_OF_RANGE	F	30110	16#759E	Parameter out of range
E_EFB_NOT_STATE_RAM_4X	F	-30531	16#88BD	Variable not mapped to %MW (4x) area
E_EFB_MSTR_ERROR	F	-30191	16#8A11	MSTR communication error

NOTE: To know the meaning of `STATUS` output parameter codes, see error codes *(see page 498)*.

Common Floating Point Errors

Introduction

The following table shows the common error codes and error values created for floating point errors. These error information are displayed in the Diagnostic Viewer and the error code values are written in %SW125 *(see Unity Pro, System Bits and Words, Reference Manual)*.

Common Floating Point Errors

Table of common floating point errors

Error codes	Error value in Dec	Error value in Hex	Error description
FP_ERROR	-30150	16#8A3A	Base value (not apearing as an error value)
E_FP_STATUS_FAILED_IE	-30151	16#8A39	Illegal floating point operation
E_FP_STATUS_FAILED_DE	-30152	16#8A38	Operand is denormalized - not a valid REAL number
E_FP_STATUS_FAILED_ZE	-30154	16#8A36	Illegal divide by zero
E_FP_STATUS_FAILED_ZE_IE	-30155	16#8A35	Illegal floating point operation / Divide by zero
E_FP_STATUS_FAILED_OE	-30158	16#8A32	Floating point overflow
E_FP_STATUS_FAILED_OE_IE	-30159	16#8A31	Illegal floating point operation / Overflow
E_FP_STATUS_FAILED_OE_ZE	-30162	16#8A2E	Floating point overflow / Divide by zero
E_FP_STATUS_FAILED_OE_ZE_IE	-30163	16#8A2D	Illegal floating point operation / Overflow / Divide by zero
E_FP_NOT_COMPARABLE	-30166	16#8A2A	Internal error

Error Codes of EFBs with STATUS parameter

Form of the Function Error Code

`STATUS` parameter error codes appear as **Mmss**, where:

- **M** is the high code
- **m** is the low code
- **ss** is a subcode

Common Error Codes

Hexadecimal error codes description:

Hex. Error Code	Description
1001	Abort by user.
1002	Warm Start initiated abort.
2001	An operation type that is not supported has been specified in the control block.
2002	One or more control block parameters were modified while the `MSTR` element was active (this only applies to operations which require several cycles for completion). Control block parameters may only be modified in inactive `MSTR` components.
2003	Invalid value in the length field of the control block.
2004	Invalid value in the offset field of the control block.
2005	Invalid value in the length and offset fields of the control block.
2006	Unauthorized data field on slave.
2007	Unauthorized network field on slave.
2008	Unauthorized network routing path on slave.
2009	Routing path equivalent to their own address.
200A	Attempt to get more global data words than available.
200B	Peer cop conflict on WR/RD global.
200C	Bad pattern for change address request.
200D	Bad address for change address request.
200E	The control block or data buffer is not assigned, or (incorrect size, parts of the control block or data buffer are located outside of the %MW (4x) range.
200F	Space for response in data buffer is too small.
2010	Control buffer length invalid.
2011	Invalid parameter.
2012	Syntax error in “rail.slot.chan” string.
2013	Module missing, not detected, or not configured.
2015	No channel data (channel out of bound).

Hex. Error Code	Description
2016	Abort on timeout.
2017	Invalid task context.
2018	Ethernet security system service error.
2019	Invalid answer data (received data do not correspond to the expected answer).
201A	Invalid answer checksum.
201B	Compatibility issue (for example, EF or DDT version not compatible with firmware version).
30ss	Exceptional response by the Modbus slave with specific **ss** exception code *(see page 500)*.
31ss	Exceptional Unity protocol error response by the Modbus slave with specific **ss** error code *(see page 501)*.
32ss	Exceptional Unity protocol IO request error acknowledgement by the Modbus slave with specific **ss** error code *(see page 502)*.
33ss	UNI-TE specific report.
34ss	Generic communication report *(see page 503)* (correspond to Communication Report field of Premium/M340 EF Management Parameters).
35ss	Generic Operation Report in case of correct exchange *(see page 503)* (correspond to Operation Report field of Premium/M340 EF Management Parameters when Communication Report = 16#00).
36ss	Generic Operation Report in case of refused message *(see page 503)* (correspond to Operation Report field of Premium/M340 EF Management Parameters when Communication Report = 16#FF).
37ss	CIP General Status Code. *(see page 505)*
4001	Inconsistent response by the Modbus slave.
4002	Inconsistent Modbus Umas response.
4003	Inconsistent UNI-TE response (depending on the module).
4004	Read request for the status words not accepted by the module channel.
4005	Command parameters not accepted by the module channel.
4006	Adjustment parameters not accepted by the module.
4007	SDO abort code (4 bytes) may be found in data field if data field is available.
5mss	TCP/IP Ethernet specific error codes *(see page 508)*.
6mss	Modbus Plus specific routing path error *(see page 512)*. The subfield m shows where the error occurred (a 0 value means local node, 2 means 2nd device in route, and so on).
7mss	SY/MAX specific error codes *(see page 513)*.
8mss	EtherNet/IP detected error codes *(see page 515)*.

Hex. Error Code	Description
F001	Wrong destination node was specified for the MSTR operation. Referenced S985 option not present or in reset mode.
F002	Component not fully initialized.

Modbus Specific Exception Function Codes (30ss)

This table lists the ss hexadecimal values in 30ss error codes:

Hex. Error Code	Description
3001	Slave does not support requested operation.
3002	Non-existing slave registers were requested.
3003	An unauthorized data value was requested.
3004	Unrecoverable error detected in the slave.
3005	Slave has accepted a lengthy program command.
3006	Function cannot currently be carried out: lengthy command running.
3007	Slave has rejected lengthy program command.
300A	Gateway unable to allocate an internal communication path.
300B	No response obtained from target device.
30FF	Modbus extended exception. Additional data available in data field (if data field provided): ● Exception length: represents the length of the extended exception response, not including these 2 bytes. ● Exception data: error information specific to the function code involved.

The ss value corresponds to the Modbus Exception code returned by the Modbus slave device in case of error (second byte of the Modbus exception PDU):

- exception-function_code = request function code + 0x80: 1 byte
- exception_code: 1 byte (returned as ss in 16#30ss error code

Details of STATUS error code from 31ss to 37ss

Unity Protocol Error Codes (31ss)

This table lists the ss hexadecimal value in 31ss error code:

Hex. Error Code	Description
3100	Generic Unity Protocol error.
3180	Generic communication error.
3181	PLC is reserved by someone else.
3182	You must reserve the PLC.
3183	Unknown request or subcode.
3184	Unknown object (for example: %Z not implemented).
3185	The response could not be built.
3186	Request has invalid parameters (for example: the request is badly built, or has few/many parameters or has a bad Csa command, and so on.)
3187	Bad sequence (for example: EndDownload executed before BeginDownload).
3188	The size of the response exceed the size of the available buffer.
3189	Module not configured (may be bad address).
318A	Action is not permitted on this object.
318B	Busy state: the previous operation is still working, or all internal resources are busy for the I/O request, or too much parallel upload, and so on.
3190	Generic error: something wrong in application.
3191	Access violation: write read-only block, or read-only variable, or download while memory is protected, and so on.
3192	Object not accessible because it is in use.
3193	Out of bounds: exceeds the range of %MW, or too many breakpoints, or call stack too large, and so on.
3194	Invalid length.
3195	Reference to a non-existent resource or task, or variable address is not in the DFB data area, and so on.
3196	The object or resource is already defined. For example: trying to start an already started, or a breakpoint-ID already used, and so on.
3197	Data inconsistent or data state not allowed. For example: wrong data or bad value when writing an object.
3198	The object exists but is not initialized.
3199	Channel out of bounds in an I/O request.
319A	Request not yet implemented.

Hex. Error Code	Description
31A0	Incompatible application, wrong target, or platform.
31A1	Signature check failed.
31A2	Wrong PCMCIA memory configuration.
31B0	PLC is not in the right mode: download with PLC in RUN mode, or debug while PLC in NOCONF, try to step a task, not break-pointed, upload operation aborted by download or OnlineChange, and so on.
31B1	Mode cannot be changed: an I/O forces premium to stop.
31B2	An internal timeout occurred.
31B3	The watchdog time is elapsed.
31FF	Undefined general error.

Unity Protocol IO Request Error Acknowledgement (32ss)

This tables lists the ss hexadecimal values in 32ss error code:

Hex. Error Code	Description
3202	Exchange error.
3207	Another explicit exchange is in progress.
3209	Operation is impossible.
320A	Data is rejected by the IOB.
320B	Writing is not authorized.
320C	Maximum number of exchanges.
3284	Unknown object.
3286	Invalid read buffer.
328A	Unknown or invalid action.
328B	All the buffers are in use.
3293	Object out of range.
3297	Object value forbidden (write operations only).
3299	Channel out of range.

Generic Communication Report (34ss)

This table lists the ss hexadecimal value in 34ss error code:

Hex. Error Code	Description
3401	Exchange stop on timeout.
3402	Exchange stop on user request (CANCEL).
3403	Incorrect address format.
3404	Incorrect destination address
3405	Incorrect management parameter format.
3406	Incorrect specific parameters.
3407	Error detected in sending to the destination.
3409	Reserved.
340A	Insufficient receive buffer size.
340B	No processor system resources.
340C	Incorrect exchange number.
340D	No telegram received.
340E	Incorrect length.
340F	Telegram service not configured.
3410	Network module missing.
3411	Request missing.
3412	Application server already active.
3413	UNI-TE V2 transaction number incorrect.

The ss value corresponds to the Communication Report code *(see page 43)* returned by existing communications EFs on Premium/Atrium/Mxxx platforms.

Generic Operation Report (35ss and 36ss)

This report byte is specific to each function and specifies the result of the operation on the remote application.

Generic operation report in case of correct exchange is coded under the form 16#35ss. If the message is refused by the remote device, the report has the form 16#36ss.

This table lists the ss hexadecimal value in 35ss error code:

Hex. Error Code	Description
3501	Request not processed.
3502	Incorrect response.

The ss value in 35ss codes corresponds to operation report field of Premium/M340 EF Management Parameters when Communication Report *(see page 44)*= 16#00.

This table lists the ss hexadecimal value in 36ss error code:

Hex. Error Code	Description
3601	No resources towards the processor.
3602	No line resources.
3603	No device or device without resources.
3604	Line error.
3605	Length error.
3606	Faulty communication channel.
3607	Addressing error.
3608	Application error.
360B	No system resources.
360C	Communication function not active.
360D	Destination missing.
360F	Intra-station routing problem or channel not configured.
3611	Address format not managed.
3612	No destination resources.
3614	Non-operational connection (example: Ethernet TCP/IP).
3615	No resource on the local channel.
3616	Access not authorized (example: Ethernet TCP/IP).
3617	Inconsistent network configuration (example: Ethernet TCP/IP).
3618	Connection temporarily unavailable.
3621	Application server stopped.
3630	Transmission error.

The ss value in 36ss codes corresponds to operation report field of Premium/M340 EF Management Parameters when Communication Report *(see page 44)*= 16#FF.

CIP General Status Code (37ss)

The following table lists the status codes that may be present in the general status code field of a detected CIP error response message:

General Status Code (in hex)	Status Name	Description of Status
3701	Connection unsuccessful	A connection-related service was unsuccessful along the connection path.
3702	Resource unavailable	Resources needed for the object to perform the requested service were unavailable.
3703	Invalid parameter value	See status code 0x20, which is the preferred value to use for this condition.
3704	Path segment error	The path segment identifier or the segment syntax was not understood by the processing node. Path processing stops when a path segment error is detected.
3705	Path destination unknown	The path is referencing an object class, instance, or structure element that is not known or is not contained in the processing node. Path processing stops when a path destination unknown error is detected.
3706	Partial transfer	Only part of the expected data was transferred.
3707	Connection lost	The messaging connection was lost.
3708	Service not supported	The requested service was not implemented or was not defined for this object class/instance.
3709	Invalid attribute value	Invalid attribute data detected.
370A	Attribute list error	An attribute in the `Get_Attribute_List` or `Set_Attribute_List` response has a non-zero status.
370B	Already in requested mode/state	The object is already in the mode/state being requested by the service.
370C	Object state conflict	The object cannot perform the requested service in its current mode/state.
370D	Object already exists	The requested instance of object to be created already exists.
370E	Attribute not settable	A request to modify a non-modifiable attribute was received.
370F	Privilege violation	A permission/privilege check was unsuccessful.
3710	Device state conflict	The mode/state of the device prohibits the execution of the requested service.
3711	Reply data too large	The data to be transmitted in the response buffer is larger than the allocated response buffer.
3712	Fragmentation of a primitive value	The service specified an operation that is going to fragment a primitive data value, that is, half a REAL data type.

General Status Code (in hex)	Status Name	Description of Status
3713	Not enough data	The service did not supply enough data to perform the specified operation.
3714	Attribute not supported	The attribute specified in the request is not supported.
3715	Too much data	The service supplied more data than was expected.
3716	Object does not exist	The object specified does not exist in the device.
3717	Service fragmentation sequence not in progress	The fragmentation sequence for this service is not currently active for this data.
3718	No stored attribute data	The attribute data of this object was not saved prior to the requested service.
3719	Store operation unsuccessful	The attribute data of this object was not saved due to an unsuccessful attempt.
371A	Routing unsuccessful, request packet too large	The service request package was too large for transmission on a network in the path to the destination. The routing device was forced to abort the service.
371B	Routing unsuccessful, response packet too large	The service response packet was too large for transmission on a network in the path from the destination. The routing device was forced to abort the service.
371C	Missing attribute list entry data	The service did not supply an attribute in a list of attributes that was needed by the service to perform the requested behavior.
371D	Invalid attribute value list	The service is returning the list of attributes supplied with status information for those attributes that were invalid.
371E	Embedded service error	An embedded service resulted in a detected error.
371F	Vendor-specific error	A vendor-specific error has been detected. The additional code field of the error response defines the particular error encountered. Use this general code only when none of the codes presented in this table or within an object class definition accurately reflect the detected error.
3720	Invalid parameter	A parameter associated with the request was invalid. This code is used when a parameter does not meet the requirements of this specification and/or the requirements defined in an application object specification.
3721	Write-once value or medium already written	An attempt was made to write to a write-once medium (for example, WORM drive, PROM) that has already been written, or to modify a value that cannot be changed once established.
3722	Invalid reply received	An invalid reply is received (for example, reply service code does not match the request service code, or reply message is shorter than the minimum expected reply size). This status code a serve for other causes of invalid replies.

General Status Code (in hex)	Status Name	Description of Status
3723	Buffer overflow	The message received is larger than the receiving buffer can handle. The entire message was discarded.
3724	Message-format error	The format of the received message is not supported by the server.
3725	Key failure in path	The key segment that was included as the first segment in the path does not match the destination module. The object-specific status indicates which part of the key check was unsuccessful.
3726	Path size invalid	The size of the path that was sent with the service request is either not large enough to allow the request to be routed to an object or too much routing data was included.
3727	Unexpected attribute in list	An attempt was made to set an attribute that is not able to be set at this time.
3728	Invalid member ID	The member ID specified in the request does not exist in the specified class/instance/attribute.
3729	Member not settable	A request to modify a non-modifiable member was received.
372A	Group 2 only server — general error	This detected error code may only be reported by DeviceNet group 2 only servers with 4 Kb or less code space and only in place of service not supported, attribute not support, or attribute not settable.
372B	Unknown Modbus error	A CIP to Modbus translator received an unknown Modbus exception code.
372C	Attribute not gettable	A request to read a non-readable attribute was received.
372D...37CF	–	Reserved by CIP for future extensions.
37D0...37FF	Reserved for object class and service errors	This range of detected error codes is used to indicate object class-specific detected errors. Use this range only when none of the codes presented in this table accurately reflect the error that is detected.

NOTE: Taken by permission from *The CIP Networks Library, Volume 1*, Common Industrial Protocol (CIP™), Edition 3.6, April 2009.

Details of EFB TCP/IP Ethernet Error Codes 5mss

TCP/IP Ethernet Network Specific Error Codes (5mss)

NOTE: Ethernet errors are managed by the Ethernet modules or the Ethernet coprocessor except for error code 5050 (hex).

Hex. Error Code	Meaning
5001	Inconsistent response by the network
5004	Interrupted system call
5005	I/O error
5006	No such address
5009	Socket descriptor is invalid
500C	Not enough memory
500D	Authorization denied
5011	Entry exists
5016	Argument is invalid
5017	Internal table has run out of space
5020	Connection is lost
5023	This operation was blocked and the socket is non-blocking
5024	The socket is non-blocking and the connection can not be closed
5025	The socket is non-blocking and a previous connection attempt has not been finalized
5026	Socket operation performed on a non-socket
5027	The destination address is not valid
5028	Message too long
5029	Wrong type of protocol for the socket
502A	Protocol not available
502B	Protocol not supported
502C	Socket type not supported
502D	Operation not supported on a socket
502E	Protocol family not supported
502F	Address family not supported
5030	Address already in use
5031	Address not available
5032	Network is out of order

Hex. Error Code	Meaning
5033	Network is unreachable
5034	Network dropped connection on reset
5035	Connection aborted by the peer
5036	Connection reset by the peer
5037	An internal buffer is required but can not be assigned
5038	Socket already connected
5039	Socket not connected
503A	Cannot send after socket shutdown
503B	Too many references, cannot splice
503C	Connection timed out (see note below)
503D	Connection refused
5040	Host is out of order
5041	The destination host could not be reached from this node
5042	Directory not empty
5046	NI_INIT returned '-1'
5047	MTU is not valid
5048	Hardware length is not valid
5049	Route specified cannot be found
504A	Collision in Select call: these conditions have already been selected by another task
504B	Task ID is invalid
5050	No network resource
5051	Length error
5052	Addressing error
5053	Application error
5054	Client can not process the request
5055	No network resource
5056	Non-operational TCP connection
5057	Incoherent configuration
51ss	SMTP service error codes *(see page 510)*
53ss	Modbus client service error codes *(see page 510)*

NOTE:

- Error code 5055 (hex) can occur before an error code 503C (hex).
- No remote device takes precedence over a timeout.

SMTP Service Error Codes (51ss)

Hexadecimal SMTP service error codes:

Hex. Error Code	Meaning
5100	Internal error
5101	SMTP component not operational
5102	Mail header not configured
5103	Invalid mail header value
5104	Cannot connect to the SMTP server
5105	Error in transmitting content of email body to the SMTP server
5106	Closing SMTP connection with the server returned an error
5107	SMTP **HELO** request failed
5108	SMTP **MAIL** request failed. SMTP server may require authentication
5109	SMTP **RCPT** request failed
510A	No recipient has been accepted by the SMTP server
510B	SMTP **DATA** request failed
510C	Send email request contains an invalid length
510D	Authentication failed
510E	A reset component request has been received while the connection was open

Modbus Client Service Error Codes (53ss)

Hexadecimal Modbus client service error codes:

Hex. Error Code	Meaning
5300	Not used (reserved for future use)
5301	The component has run out of resources
5302	The IP address provided is not appropriate. For example: 0.0.0.0, or broadcast address, or multicast address, etc.
5303	The transaction has timed-out. The remote server accepted a request and has not answered within 2 minutes.
5304	All connections are currently used
5305	Access denied
5306	Network can not be reached
5307	Host is down
5308	Network dropped connection on reset

Hex. Error Code	Meaning
5309	Network is down
530A	Connection refused
530B	Connection timed out
530C	Wrong MBAP header

Details of EFB Modbus Plus Error Codes 6mss

Modbus Plus Specific Error Codes (6mss)

NOTE: The **m** field in **6mss** error code is an `Index` in the routing information that shows where an error has been detected. **m** = 0 means that the error has been detected in the local node, **m** = 2 means it has been detected in the second device in the route, and so on.

Hex. Error Code	Description
6m01	No response reception
6m02	Access to program denied
6m03	Node out of service and unable to communicate
6m04	Unusual response received
6m05	Router-node data path busy
6m06	Slave out of order
6m07	Wrong destination address
6m08	Unauthorized node type in routing path
6m10	Slave has rejected the command
6m20	Slave has lost an activated transaction
6m40	Unexpected master output path received
6m80	Unexpected response received
F001	Wrong destination node was specified for the `MSTR` operation

Quantum EFB SY/MAX Specific Error Codes

SY/MAX-Specific Error Codes

When using SY/MAX Ethernet, three additional types of errors may appear in the `CONTROL[1]` register of the control block.

The error codes have the following meaning:

- 71xx Error: Errors found by the SY/MAX remote device
- 72xx Error: Errors found by the server
- 73xx Error: Errors found by the Quantum translator

SY/MAX-Specific Hexadecimal Error Codes

SY/MAX-specific hexadecimal error codes:

Hex. Error Code	Description
7101	Invalid opcode found by the SY/MAX remote device
7103	Invalid address found by the SY/MAX remote device
7109	Attempt to write to a write protected register found by the SY/MAX remote device
F710	Receiver overflow found by the SY/MAX remote device
7110	Invalid length found by the SY/MAX remote device
7111	Remote device not active, no connection (occurs when retry attempts and time-out have been used up), found by the SY/MAX remote device
7113	Invalid parameter in a read operation found by the SY/MAX remote device
711D	Invalid route found by the SY/MAX remote device
7149	Invalid parameter in a write operation found by the SY/MAX remote device
714B	Invalid drop number found by the SY/MAX remote device
7101	Invalid opcode found by the SY/MAX server
7203	Invalid address found by the SY/MAX server
7209	Attempt to write to a write protected register found by the SY/MAX server
F720	Receiver overflow found by the SY/MAX server
7210	Invalid length found by the SY/MAX server
7211	Remote device not active, no connection (occurs when retry attempts and time-out have been used up), found by the SY/MAX server
7213	Invalid parameter in a read operation found by the SY/MAX server
721D	Invalid route found by the SY/MAX server
7249	Invalid parameter in a write operation found by the SY/MAX server
724B	Invalid drop number found by the SY/MAX server

Hex. Error Code	Description
7301	Invalid opcode in an MSTR block request from the Quantum translator
7303	Read/Write QSE module status (200 route address out of range)
7309	Attempt to write to a write protected register when a status write is carried out (200 route)
731D	Invalid route found by the Quantum translator. Valid routes: ● dest_drop, 0xFF ● 200, dest_drop, 0xFF ● 100+drop, dest_drop, 0xFF ● All other routing values produce an error
734B	One of the following errors occurred: ● No CTE (configuration extension table) has been configured ● No CTE table entry has been made for the QSE model slot number ● No valid drop has been specified ● The QSE module has not been reset after the creation of the CTE. Note: After writing and configuring the CTE and downloading to the QSE module, the QSE module must be reset for the modifications to become effective. ● When using an MSTR instruction no valid slot or drop has been specified

EtherNet/IP Detected Error Codes

EtherNet/IP Detected Error Codes

EtherNet/IP hexadecimal detected error codes include:

Detected Error Code (hex)	Description
800D	Timeout on the explicit message request
8012	Bad device
8015	Either: ● Nor resources to handle the message, or ● Internal detected error: no buffer available, no link available, impossible to send to the TCP task
8018	Either: ● Another explicit message for this device is in progress, or ● TCP connection or encapsulation session in progress
8030	Timeout on the Forward_Open request
Note: The following 81xx detected errors are Forward_Open response detected errors that originate at the remote target and are received via the CIP connection.	
8100	Connection in use or duplicate Forward_Open
8103	Transport class and trigger combination not supported
8106	Ownership conflict
8107	Target connection not found
8108	Invalid network connection parameter
8109	Invalid connection size
8110	Target for connection not configured
8111	RPI not supported
8113	Out of connections
8114	Vendor ID or product code mismatch
8115	Product type mismatch
8116	Revision mismatch
8117	Invalid produced or consumed application path
8118	Invalid or inconsistent configuration application path
8119	Non-Listen Only connection not opened
811A	Target object out of connections
811B	RPI is smaller than the production inhibit time
8123	Connection timed out
8124	Unconnected request timed out

Detected Error Code (hex)	Description
8125	Parameter detected error in unconnected request and service
8126	Message too large for unconnected_send service
8127	Unconnected acknowledge without reply
8131	No buffer memory available
8132	Network bandwidth not available for data
8133	No consumed connection ID filter available
8134	Not configured to send scheduled priority data
8135	Schedule signature mismatch
8136	Schedule signature validation not possible
8141	Port not available
8142	Link address not valid
8145	Invalid segment in connection path
8146	Detected error in Forward_Close service connection path
8147	Scheduling not specified
8148	Link address to self invalid
8149	Secondary resources unavailable
814A	Rack connection already established
814B	Module connection already established
814C	Miscellaneous
814D	Redundant connection mismatch
814E	No more user-configurable link consumer resources: the configured number of resources for a producing application has reached the limit
814F	No more user-configurable link consumer resources: there are no consumers configured for a producing application to use
8160	Vendor specific
8170	No target application data available
8171	No originator application data available
8173	Not configured for off-subnet multicast
81A0	Detected error in data assignment
81B0	Optional object state detected error
81C0	Optional device state detected error

Detected Error Code (hex)	Description
Note: All 82xx detected errors are register session response detected errors.	
8200	Target device does not have sufficient resources
8208	Target device does not recognize message encapsulation header
820F	Reserved or unknown detected error from target

CANopen SDO Abort Code

Overview

The SDO abort code is used when the SDO command is not successful.

SDO Abort Code

SDO Abort Code Value (hex)	Parameter
0503 0000	Toggle bit not alternated.
0504 0000	SDO protocol timed out.
0504 0001	Client/server command specifier not valid or unknown.
0504 0002	Invalid block size (block mode only).
0504 0003	Invalid sequence number (block mode only).
0504 0004	CRC error (block mode only).
0504 0005	Out of memory.
0601 0000	Unsupported access to an object.
0601 0001	Attempt to read a write only object.
0601 0002	Attempt to write a read only object.
0602 0002	Object does not exist in the object dictionary.
0604 0041	Object cannot be mapped to the PDO.
0604 0042	The number and length of the objects to be mapped would exceed PDO length.
0604 0043	General parameter incompatibility reason.
0604 0047	General internal incompatibility in the device.
0606 0000	Access failed due to an hardware error
0607 0010	Data type does not match, length of service parameter does not match
0607 0012	Data type does not match, length of service parameter too high
0607 0013	Data type does not match, length of service parameter too low
0609 0011	Sub-index does not exist.
0609 0030	Value range of parameter exceeded (only for write access).
0609 0031	Value of parameter written too high.
0609 0032	Value of parameter written too low.
0609 0036	Maximum value is less than minimum value.
0800 0000	General error
0800 0020	Data cannot be transferred or stored to the application.
0800 0021	Data cannot be transferred or stored to the application because of local control.

SDO Abort Code Value (hex)	Parameter
0800 0022	Data cannot be transferred or stored to the application because of the present device state.
0800 0023	Object dictionary dynamic generation fails or no object dictionary is present (for example, object dictionary is generated from file and generation fails because of an file error).

Glossary

A

ADDM_TYPE

This predefined type is used as an output for the ADDM function. This is an ARRAY[0..8] OF Int. You can find it in the library, in the same family as the EFs that use it.

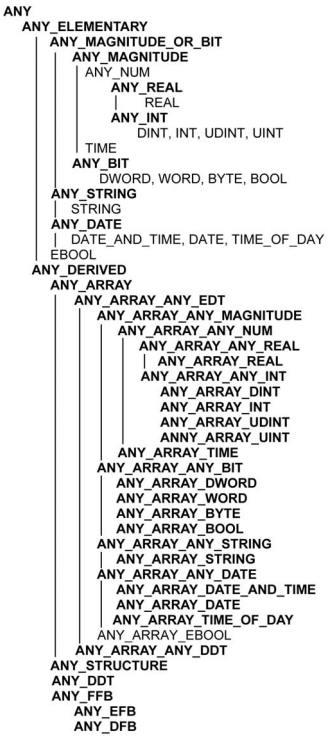
ADDR_TYPE

This predefined type is used as an output for the ADDR function. This is an ARRAY[0..5] OF Int. You can find it in the library, in the same family as the EFs that use it.

ANY

There is a hierarchy among the various data types. In the DFBs, it is sometimes possible to declare variables that can contain several types of values. In that case we use `ANY_xxx` types.

The figure below describes this hierarchical structure:



ARRAY

An ARRAY is a table containing elements of a single type.

The syntax is as follows: ARRAY [<limits>] OF <Type>

Example:

ARRAY [1..2] OF BOOL is a one-dimensional table with two elements of type BOOL.

ARRAY [1..10, 1..20] OF INT is a two-dimensional table with 10x20 elements of type INT.

Assigned variables

A variable whose position in the PLC memory can be known. For example, the Water_pressure variable is associated with %MW102. Water_pressure is said to be assigned.

B

BOOL

BOOL is the abbreviation for the Boolean type. This is the basic data type in computing. A BOOL variable can have either of the following two values: 0 (FALSE) or 1 (TRUE).

A bit extracted from a word is of type BOOL, for example: %MW10.4.

BYTE

When 8 bits are grouped together, they are called a BYTE. You can enter a BYTE either in binary mode or in base 8.

The BYTE type is encoded in an 8 bit format which, in hexadecimal format, ranges from 16#00 to 16#FF.

D

DINT

DINT is the abbreviation of Double INTeger (encoded in 32 bits).

The upper/lower limits are as follows: -(2 to the power of 31) to (2 to the power of 31) - 1.

Example:

-2147483648, 2147483647, 16#FFFFFFFF.

E

EN

EN stands for **EN**able; it is an optional block input. When the EN input is enabled, an ENO output is set automatically.

If EN = 0, the block is not enabled; its internal program is not executed, and ENO is set to 0.

If EN = 1, the block's internal program is run and ENO is set to 1. If an error occurs, ENO is set to 0.

If the EN input is not connected, it is set automatically to 1.

ENO

ENO stands for **E**rror **NO**tification; this is the output associated with the optional input EN.

If ENO is set to 0 (because EN = 0 or in case of an execution error):

- the status of the function block outputs remains the same as it was during the previous scanning cycle that executed correctly;
- the output(s) of the function, as well as the procedures, are set to "0".

F

Function

See EF.

I

INT

INT is the abbreviation of single INTeger (encoded in 16 bits).

The upper/lower limits are as follows: -(2 to the power of 15) to (2 to the power of 15) - 1.

Example:

-32768, 32767, 2#1111110001001001, 16#9FA4.

S

STRING

A STRING variable is a series of ASCII characters. The maximum length of a string is 65,534 characters.

W

WORD

The type `WORD` is encoded in a 16 bit format and is used to perform processing on series of bits.

This table shows the upper/lower limits of each of the bases that can be used:

Base	Lower limit	Upper limit
Hexadecimal	16#0	16#FFFF
Octal	8#0	8#177777
Binary	2#0	2#1111111111111111

Examples of representation

Data	Representation in one of the bases
0000000011010011	16#D3
1010101010101010	8#125252
0000000011010011	2#11010011

Index

A

ADDM, *71*
ADDM_TYPE, *71*, *77*
ADDMX, *77*
ADDR, *83*
ADDR_TYPE, *83*
asynchronous execution
    ETH_PORT_CTRL, *119*
availability of the instructions, *29*

C

CANCEL, *85*
communication - instructions
    ADDM, *71*
    ADDMX, *77*
    ADDR, *83*
    CANCEL, *85*
    CREAD_REG, *89*
    CWRITE_REG, *97*
    DATA_EXCH, *105*
    ETH_PORT_CTRL, *119*
    EthPort_Control_MX, *123*
    EXCH_QX, *125*
    general information for Premium and Atrium, *48*
    INPUT_BYTE, *131*
    INPUT_CHAR, *137*
    INPUT_CHAR_QX, *149*
    MBP_MSTR, *153*
    ModbusP_ADDR, *199*
    OUT_IN_CHAR, *205*
    OUT_IN_MBUS, *213*
    PRINT_CHAR, *251*
    PRINT_CHAR_QX, *263*
    RCV_TLG, *267*
    READ_ASYN, *273*
    READ_GDATA, *277*
    READ_REG, *279*
    READ_REG_QX, *287*
    READ_SDO, *291*
    READ_VAR, *295*
    SEND_EMAIL, *315*
    SEND_REQ, *319*
    SEND_TLG, *337*
    SYMAX_IP_ADDR, *343*
    TCP_IP_ADDR, *349*
    UNITE_SERVER, *355*
    WRITE_ASYN, *361*
    WRITE_GDATA, *365*
    WRITE_REG, *367*
    WRITE_REG_QX, *375*
    WRITE_SDO, *379*
    WRITE_VAR, *383*
    XXMIT, *419*
CREAD_REG, *89*
CWRITE_REG, *97*

D

Data Structure
    XMIT_CFG, *407*
    XMIT_SET, *406*
DATA_EXCH, *105*

E

error codes, *495*
  CREAD_REG, *498*
  CWRITE_REG, *498*
  EXCH_QX, *498*
  GET_TS_EVT_M, *498*
  GET_TS_EVT_Q, *498*
  INPUT_CHAR_QX, *498*
  PRINT_CHAR_QX, *498*
  PWS_CMD, *498*
  PWS_DIAG, *498*
  READ_PARAM_MX, *498*
  READ_REG, *498*
  READ_REG_QX, *498*
  READ_SDO, *498*
  READ_STS_MX, *498*
  READ_STS_QX, *498*
  RESTORE_PARAM_MX, *498*
  SAVE_PARAM_MX, *498*
  WRITE_PARAM_MX, *498*
  WRITE_REG, *498*
  WRITE_REG_QX, *498*
  WRITE_SDO, *498*
ETH_PORT_CTRL, *119*, *119*
EthPort_Control_MX, *123*
EXCH_QX, *125*
execution type
  ETH_PORT_CTRL, *119*

F

FTP/TFTP services
  enable/disable, *115*, *187*
function block
  ETH_PORT_CTRL, *119*

H

HTTP services
  enable/disable, *115*, *187*

I

INPUT_BYTE, *131*
INPUT_CHAR, *137*
INPUT_CHAR_QX, *149*
instructions
  availability, *29*

M

MBP_MSTR, *153*
ModbusP_ADDR, *199*

O

OUT_IN_CHAR, *205*
OUT_IN_MBUS, *213*

P

Parameter Description, *404*
PRINT_CHAR, *251*
PRINT_CHAR_QX, *263*

R

RCV_TLG, *267*
READ_ASYN, *273*
READ_GDATA, *277*
READ_REG, *279*
READ_REG_QX, *287*
READ_SDO, *291*
READ_VAR, *295*

S

security
  ETH_PORT_CTRL, *119*
SEND_EMAIL, *315*
SEND_REQ, *319*
SEND_TLG, *337*
string management - instructions
  INPUT_CHAR, *137*
  OUT_IN_CHAR, *205*
  PRINT_CHAR, *251*
SYMAX_IP_ADDR, *343*
synchronous execution
  ETH_PORT_CTRL, *119*

T

TCP_IP_ADDR, *349*

U

UNITE_SERVER, *355*

W

WRITE_ASYN, *361*
WRITE_GDATA, *365*
WRITE_REG, *367*
WRITE_REG_QX, *375*
WRITE_SDO, *379*
WRITE_VAR, *383*

X

XMIT, *399*
XMIT_CFG, *407*
XMIT_SET, *406*
XXMIT, *419*